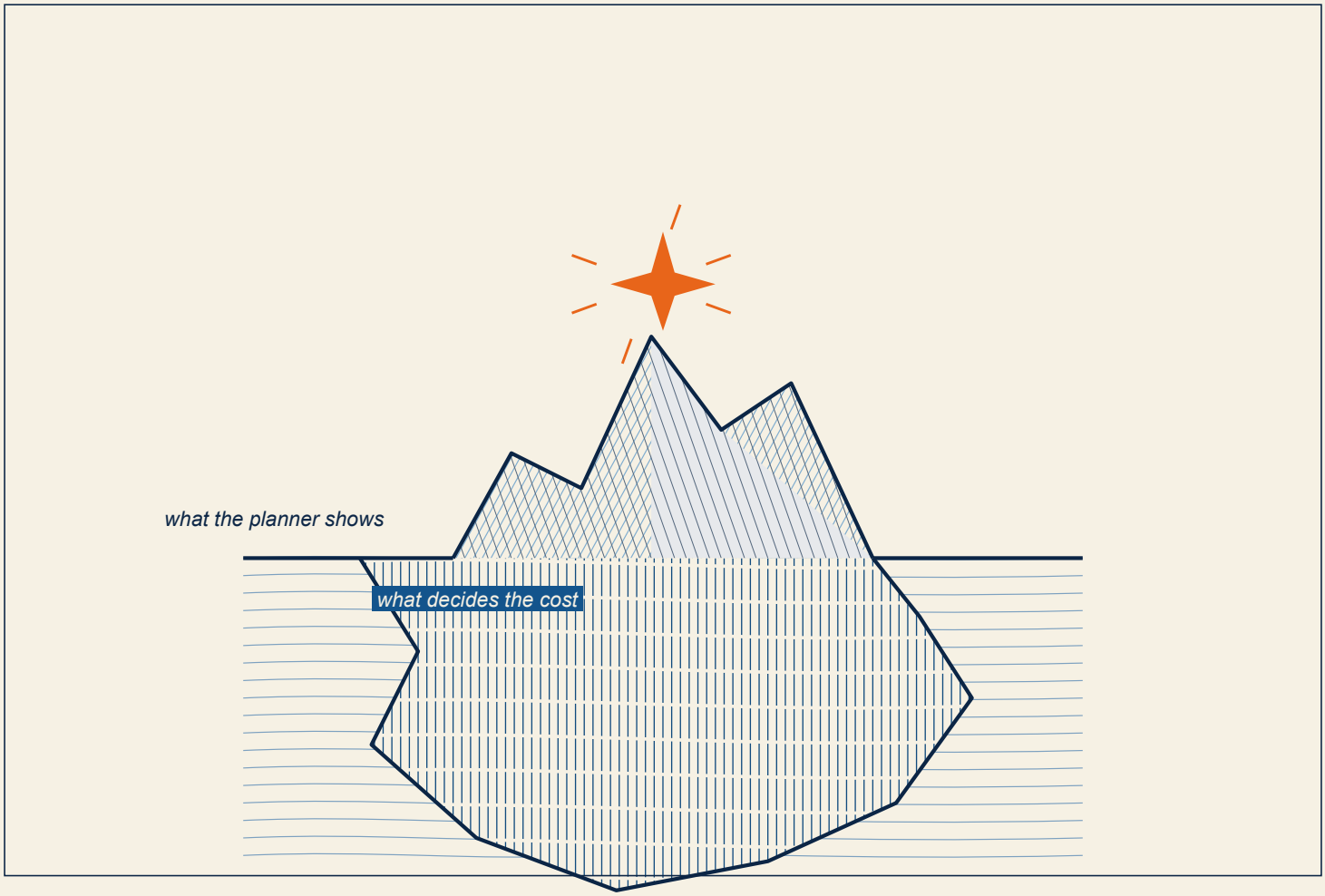
A PRACTITIONER'S DEEP DIVE

Spark Query Optimization

and the Iceberg Lakehouse

Why your queries run the way they do --- from Catalyst to the shuffle to the metadata layer --- and how to change it



Alan Phan

Apache Spark 3.5.2
Apache Iceberg v2

Spark Query Optimization and the Iceberg Lakehouse

A PRACTITIONER'S DEEP DIVE

Spark Query Optimization

and the Iceberg Lakehouse

Why your queries run the way they do
— from Catalyst to the shuffle to the metadata layer —
and how to change it

Alan Phan

Spark Query Optimization and the Iceberg Lakehouse
A Practitioner's Deep Dive

Copyright © 2026 Alan Phan. All rights reserved.

Edition. First draft, 2026. Every behavioral claim is written against Apache Spark 3.5.2 and Apache Iceberg (format version 2) on the stack described in Appendix A. Behavior on other versions may differ; where this book knows of a difference, it says so.

Source and lab. The manuscript, the figures, and the lab scripts that produced every measured number are kept in one repository, `github.com/AlanPhan04/spark-101-to-pro`. The measurements were taken on a single machine (Section A.3); absolute timings will differ on your hardware, and the book says which of its claims are ratios, which are counts, and which are only claims.

Trademarks. Apache, Apache Spark, Spark, Apache Iceberg, Iceberg, Apache Hive, and Apache Parquet are trademarks of the Apache Software Foundation. All other names are the property of their owners. This book is an independent work and is not affiliated with or endorsed by any of them.

Disclaimer. The information in this book is provided as is. Test every configuration change and every maintenance procedure on a copy of your data before you run it against production tables, particularly anything that deletes files.

Typeset with XeLaTeX. Figures are vector PDF, drawn for this book.

Preface

This book began as a study journal. I run Spark SQL on an Apache Iceberg lakehouse every working day — Spark 3.5.2 and Iceberg v2 — and I kept noticing the same thing about how I, and the people around me, fixed slow queries. We changed a setting, reran the query, kept the change if it helped, and moved on without knowing why it had. That works while a query takes a minute. It stops working when each trial takes an hour and costs real money, and it stops working entirely when the setting that helped last quarter makes this quarter's query slower.

The notes that became this book were an attempt to replace that trial loop with a model. The model is one sentence, and the rest of the book is its consequences:

> *Every Spark performance decision is a choice about where and when data moves — between operators, across the network, and between metadata and storage — and you can predict and change those movements once you can see them.*

A scan reads files; a shuffle moves rows between executors so that rows with the same key meet; a join decides which side to move; adaptive execution changes its mind once it has seen real sizes; Iceberg's planner reads a few kilobytes of metadata so it does not have to read gigabytes of data. These look like separate subjects. They are one subject, and once you see them as one, the question you ask of a slow query changes from "which config should I try?" to "what is moving, how much, and does it have to?"

What makes this book different

There are good books on Spark and good material on Iceberg. This one is organized around two commitments.

Mechanism before settings. This is not a list of `spark.sql.*` keys with recommended values. A configuration appears only after the mechanism it controls has been explained, and always with what it costs. A reader who finishes should be able to derive most of the "recommended" values from first principles, and to recognize the case where the recommendation is wrong. When a rule of thumb is useful — and some are — the book shows where it comes from.

The seam between Catalyst and Iceberg. Most of the cost of a lakehouse query is decided at the boundary between the engine and the table format: what the planner can push into the scan, what the manifests let it skip, how the file layout written last night shapes the tasks run this morning. Treating Spark and Iceberg as two separate subjects hides that boundary. This book is built around it: Part II ends with pruning and bucketing on the Spark side, and Part III picks the same threads up on the Iceberg side.

A running specimen, and numbers you can reproduce

The book follows one query, introduced in Chapter 1: a week of orders joined to a product dimension and aggregated to an on-time delivery rate. On the production-sized cluster it takes about four minutes. We do not try to make it faster in the first chapter; we learn to *read* it — the plan, the stages, the shuffle, the one task that runs longer than the rest — and then every chapter changes one thing about it and shows the consequence.

Wherever I could, I measured rather than asserted. The experiments run in a local Docker container with Spark 3.5.2 and Iceberg 1.6.1 against synthetic data generated to the same shape as the production tables (Section A.3), and the scripts live alongside the manuscript. Several times the measurement contradicted what I had written from memory: a default that I believed applied did not fire, a threshold compared a different quantity than I had assumed, a mechanism needed a second setting before it did anything.

Those corrections are in the text, where they belong, because the gap between what a setting is documented to do and what it does on your data is precisely where tuning goes wrong.

Some claims I have not tested. These are marked "claim — not tested" (or "verify on your cluster") rather than smoothed over. Treat them as hypotheses.

How the book is organized

Part I, How Spark Executes a Query, builds the machinery that moves data and teaches you to see it: the driver, executors, jobs, stages and tasks (Chapter 1); the Catalyst pipeline from SQL text to physical plan (Chapter 2); and the executor's memory and CPU budget (Chapter 3).

Part II, Making Queries Fast, treats each chapter as one lever on data movement: the shuffle (Chapter 4), joins (Chapter 5), adaptive query execution (Chapter 6), pruning, partitioning, and bucketing (Chapter 7), and caching and reuse (Chapter 8).

Part III, The Iceberg Lakehouse, extends the model below the file: metadata is data that moves too. It covers the table format itself (Chapter 9), catalogs and the commit protocol (Chapter 10), partitioning and layout (Chapter 11), reading efficiently (Chapter 12), and writing and maintaining tables (Chapter 13).

Part IV, Operating Spark, applies the model under real constraints: sizing and configuration (Chapter 14), debugging and profiling (Chapter 15), and streaming into the lakehouse (Chapter 16).

Part V, Internals, goes one level deeper on the same machinery: Tungsten and code generation (Chapter 17), the shuffle framework and scheduler (Chapter 18), and Data-Source V2 (Chapter 19). It is optional, and the earlier parts do not depend on it.

Five appendices follow: the reference environment and running example (Appendix A), a configuration reference (Appendix B), a guide to reading query plans (Appendix C), a glossary (Appendix D), and suggestions for further reading (Appendix E).

A note on scope

The book is written against one stack, and it says so every time a claim depends on it. Spark and Iceberg both change quickly, and behavior that is true on 3.5.2 may not be true on 4.0. I would rather give you a model you can re-test against your version than a table of values that quietly goes stale.

Acknowledgments

This book stands on the Apache Spark and Apache Iceberg projects — their source code, their specifications, and the documentation and talks of the people who maintain them. Every number measured here was possible because those projects are open and can be run on a laptop. Errors that remain are mine, and corrections are welcome through the repository.

Alan Phan

2026

Who This Book Is For

This book is written for a working data engineer or analytics engineer, two to six years in, who runs Spark SQL on an Iceberg lakehouse most days and has a nagging sense that the optimizer and the table format are black boxes. You may recognize some of this:

- You own pipelines that are fine until they are not — a job that doubled in runtime after the data grew, a join that spills to disk, a query that scans the whole table when it should scan a day.
- You have opened the Spark UI, but you read it like a dashboard rather than a diagnostic.
- You treat `spark.sql.*` settings as spells found on a forum, and you could not say which mechanism a given one touches.
- You know the JVM exists and that garbage collection and memory matter, without a working model of either.
- You think of Iceberg as "Parquet with time travel."

If that describes you, the book is aimed at the gap between where you are and where Table 1 says you can be.

What you should already know

The book assumes, and does not teach:

- **SQL**: joins, aggregation, window functions, subqueries.

Table 1: Where the book starts you and where it takes you

Topic	**Today**	**After the book**
Query plans	"There is an `EXPLAIN` somewhere."	Predicts the physical plan and its shuffles before running; reads `EXPLAIN FORMATTED` fluently.
Slowness	Guesses, adds memory, retries.	Localizes the cost to a stage and operator from the UI and metrics.
The shuffle	"The slow part."	Knows the write and read paths, sizes partitions, removes unnecessary exchanges.
Joins	Lets Spark decide.	Predicts the strategy, forces it when statistics are wrong, fixes skew.
Iceberg	"Parquet with snapshots."	Reads the metadata tree and predicts what a query will prune.
Configuration	Copies snippets.	Knows which knob touches which mechanism, and what it costs.

- **The relational model** and enough relational algebra to read a plan as a tree of operators.
- **Columnar versus row storage**, at the level of knowing what Parquet is for. Appendix A adds the background Part III needs.
- **Kubernetes as an operator**: pods, resource limits, and why a pod is OOM-killed.
- Reading a stack trace, and basic command-line and Git use.

You do *not* need to know Scala or Java. A few examples use Scala where SQL cannot express the idea, and they are short and explained line by line.

What this book will not teach

To keep the focus on query performance on a lakehouse, the book leaves out:

- Scala or Java as languages;
- MLlib, GraphX, and machine-learning workloads;
- provisioning a cluster, or administering Kubernetes beyond what the executor sizing chapters require;
- the pandas API on Spark;
- Delta Lake and Apache Hudi, except where a comparison with Iceberg clarifies a design choice.

Who might want a different book

If you are brand new to Spark and have never run a query on it, begin with an introductory text and come back at Chapter 1; the chapter is gentle, but it does not teach what a DataFrame is. If you administer the cluster rather than query it, Parts I and IV will serve you, and Part III will explain what your users' tables are doing to your object store. And if you already read query plans for a living, the Iceberg chapters (Part III) and the measured experiments are where this book is most likely to tell you something new.

How to Use This Book

Five parts, one dependency order

The book has five parts. Part I is the foundation and everything else assumes it. Parts II and III are the core; they can be read in either order after Part I. Part IV applies the model operationally, and Part V is optional depth. Figure 1 shows the dependencies.

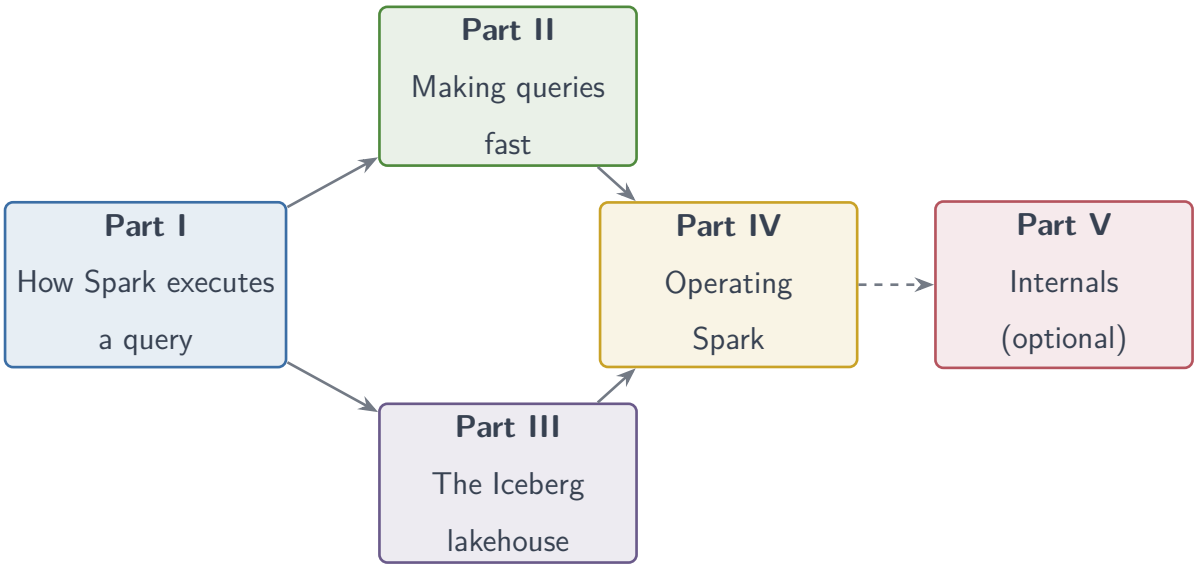


Figure 1: Reading order. Part I first; Parts II and III in either order; Part IV once you have the model; Part V when you want to know how it is built.

Within a part, chapters build on their predecessors, but each states its prerequisites at the start so you can jump in.

Four ways to read it

Straight through. The intended path. Each chapter ends with a summary and exercises, and the running specimen carries from one to the next.

"My query is slow right now." Read Chapter 1 for the vocabulary, then the `EXPLAIN` sections of Chapter 2 and Appendix C to read the plan, then Chapter 15 to localize the cost. Each chapter's symptom table (see below) then points you to the mechanism that explains it.

"I run Iceberg and want the lakehouse side first." Chapter 1 and Chapter 7 for the Spark-side vocabulary of pruning, then Part III in order. Return to Part II when a chapter in Part III refers back to it.

"I size and operate clusters." Chapter 3, Chapter 4, and Part IV. Skip the join and Iceberg chapters until a symptom sends you there.

The shape of a chapter

The chapters share one pattern, so you know where to look:

1. an **opening** that states the question the chapter answers, using the running example;
2. the **mechanism**, explained before any setting is mentioned;
3. a **measured demonstration** — the plan, the metric, or the file listing — usually from the lab (Section A.3);
4. a **symptom** → **evidence** → **knob** → **cost** table, which is the chapter's field guide: what you will observe, where to see it, what to change, and what the change costs;
5. a **summary and exercises**.

The tables are deliberately last. If you start with them, you get the configs without the model, which is the failure mode this book is written against.

The running example

One dataset runs through every chapter: an online marketplace's `orders` table, an Iceberg table partitioned by day, with a small star schema around it (`dim_product`, `dim_customer`, `dim_date`). It is defined in Appendix A. Using the same tables everywhere means a change in one chapter can be seen in the plans of another, and you never have to learn a new toy schema.

Running the examples yourself

The book does not require a cluster. The `lab/` directory of the repository contains a Docker-based environment with Spark 3.5.2, the Iceberg runtime, a generator for the synthetic `orders` data, and one script per experiment; Section A.3 describes the setup and its limits. The lab is a single machine, so it reproduces the *mechanisms* and the *ratios* reliably and the absolute timings not at all. Where a chapter reports a time, it says which environment produced it.

If your stack differs from the reference — another Spark version, a different catalog or object store — the mechanisms in the book still hold, but defaults, config names, and some behaviors will not. The text flags the places where it knows the version matters; for everything else, re-run the relevant experiment on your stack before you rely on a number.

Exercises

Exercises at the end of each chapter ask you to predict first and run second: write down the plan you expect, the number of shuffles, the number of files scanned, and only then run the query. The learning is in the gap between the prediction and the result.

Conventions Used in This Book

One stack, and what "true" means

Every behavioral claim is written against the reference stack in Appendix A: Spark 3.5.2 and Iceberg format version 2. Three rules follow from that.

- A behavior is stated as fact only if it holds on that stack.
- When behavior changed across versions, a Note names the version (for example, that adaptive coalescing is on by default from Spark 3.2).
- Defaults are quoted as of Spark 3.5.2. A number — a threshold, a size, a speedup — always comes with a source, or with "measured" and the environment that measured it.

Measured, derived, and claimed

The book distinguishes three kinds of statement, because they deserve different trust.

Measured. Produced by a lab script and shown with its output. These are the numbers and plans in listings and figures. They are reproducible on the lab setup; on your cluster, expect the same mechanism and a different magnitude.

Derived. Follows from a formula or from the source code of Spark or Iceberg, and is checked against a measurement where one exists.

Claimed. Stated from documentation or experience, but not tested here. These are marked "claim — not tested" or "claim — verify on your cluster."

Listings

Code and output appear in listings with a language tag: `sql`, `scala`, or `text` for plans, UI excerpts, and shell output. Query plans are lightly trimmed; an ellipsis comment such as `-- ... 3 Filter nodes elided` marks a cut. Wide plans use a smaller type and wrap long lines rather than running off the page. Configuration keys in running text are typeset as `spark.sql.adaptive.enabled`, and may break at the dots.

A listing that produces output is followed by that output, labelled, so you can compare what you expect with what happened.

Terminology

Each term is defined once, in italics, at its first use, and used plainly after that. Where two names circulate for the same thing, the book picks one:

- *stage boundary*, not "shuffle boundary";
- *unified memory*, not "managed memory";
- *manifest list*, *manifest*, and *data file*, exactly as the Iceberg specification spells them;
- an *executor* is a process, a *task* is a thread, and a *slot* is one core.

The glossary (Appendix D) lists every term with the chapter that defines it.

Figures and cross-references

Figures are vector graphics, so they stay sharp at any zoom, and share one color palette. Figures that show a measured result say so in the caption. Cross-references are links; a reference to a chapter, section, figure, or listing is always followed by its number.

Callouts

Four callouts mark the text that is not part of the main line of the argument. At most one appears on a typical page.

> **Note** Marks a version gotcha, a clarification, or a pointer to a later chapter.

> **Warning** Marks a default or a pattern that silently costs money, correctness, or data.

> **Tip** Marks a diagnostic shortcut — a UI panel to check, a one-line command.

> **Analogy** Marks a plain-language comparison used to build intuition for a mechanism — never a substitute for the mechanism itself, and always pushed only as far as it stays accurate. Skippable if it doesn't click for you; the surrounding prose never depends on it.

The running example

The `orders` table and its star schema are introduced in Appendix A and reused in every chapter. Table and column names are typeset in `monospace`.

Contents

Preface iv

Who This Book Is For viii

How to Use This Book xi

Conventions Used in This Book xiv

I How Spark Executes a Query 1

1 The Shape of a Spark Application 2

1.1 A query, end to end 2

1.2 Driver, executors, and the cluster manager 3

1.3 Deploy modes and Spark on Kubernetes 8

1.4 Jobs, stages, and tasks 9

1.5 Narrow versus wide dependencies 11

1.6 Reading the UI for the first time 13

1.7 Summary 14

2 From SQL to Execution: the Catalyst Pipeline 16

2.1 The pipeline, overview 16

2.2 Unresolved to resolved: the catalog 18

2.3 Logical optimization: the rules that matter 19

2.4 What the rules are worth, measured 20

2.5 Physical planning and strategies 21

2.6 Whole-stage code generation 23
2.7 Reading `EXPLAIN` 25
2.8 Summary 27

3 How an Executor Uses Memory and CPU 29
3.1 The JVM in one page: bytecode and just-in-time compilation 29
3.2 Heap layout and object representation 31
3.3 The executor memory model 32
3.4 Garbage collection: Parallel, G1, ZGC 36
3.5 Serialization and its cost 38
3.6 Summary 40

II Making Queries Fast 41

4 The Shuffle 42
4.1 What a partition is: input partitions versus shuffle partitions 42
4.2 The shuffle write path 44
4.3 The shuffle read path 44
4.4 Sizing shuffle partitions 46
4.5 Shuffle compression 48
4.6 Push-based shuffle 50
4.7 Finding and removing unnecessary exchanges 51
4.8 Summary 52

5 Joins 53
5.1 Broadcast hash join 53
5.2 Shuffle hash join 56
5.3 Sort-merge join 56
5.4 Broadcast nested loop join 57
5.5 How the planner chooses 57
5.6 What the strategies cost, measured 59
5.7 Forcing a strategy 60

5.8 Storage-partitioned joins for bucketed Iceberg tables 61
5.9 Summary 62

6 Adaptive Query Execution 63
6.1 Why runtime beats planning time 63
6.2 The feedback loop 64
6.3 Coalescing shuffle partitions 66
6.3.1 The target size, and what parallelism does to it 67
6.3.2 Seeing it in the plan 71
6.4 Converting sort-merge to broadcast at runtime 71
6.4.1 What the conversion saves, and what it cannot 72
6.4.2 The two broadcast thresholds 73
6.5 Skew-join handling: split and replicate 76
6.5.1 The split and the replication 76
6.5.2 Reproducing the straggler, and why the defaults did not fire . . 77
6.6 Reading an AQE plan 80
6.7 What AQE cannot fix 82
6.8 Summary 84

7 Partitioning, Bucketing, and Pruning 86
7.1 Reading a scan node 86
7.2 Partition pruning and the pushdown killers 88
7.3 Dynamic partition pruning 90
7.3.1 When DPP applies 91
7.4 Bucketing to avoid a shuffle 94
7.4.1 Sizing the bucket count 95
7.5 `repartition` versus `coalesce` 96
7.6 Column pruning and `ReadSchema` 99
7.7 Summary 100

8 Caching and Reuse 103
8.1 When caching pays 103
8.2 Lazy `cache()` versus eager `CACHE TABLE` 104

8.3 Storage levels and the GC trade-off . . . 105
8.4 How execution and storage memory borrow at runtime . . . 108
8.5 Exchange and subquery reuse . . . 110
8.6 Unpersist and cache thrashing . . . 112
8.7 Summary . . . 113

III The Iceberg Lakehouse 115

9 Table Formats and the Iceberg Spec 116
9.1 From Hive tables to table formats . . . 116
9.2 The Iceberg spec, format version 2 . . . 117
9.2.1 The tree, measured . . . 118
9.3 Snapshots and the metadata tree . . . 121
9.3.1 The commit as an atomic pointer swap . . . 123
9.4 Position deletes versus equality deletes . . . 123
9.5 Iceberg, Delta Lake, and Hudi . . . 125
9.6 Summary . . . 126

10 Catalogs and Commits 127
10.1 What a catalog does . . . 127
10.2 Service-backed catalogs . . . 128
10.3 The atomic pointer swap . . . 129
10.4 Optimistic concurrency and retries . . . 130
10.5 Catalog configuration in Spark . . . 134
10.6 Summary . . . 134

11 Iceberg Partitioning and Layout 136
11.1 Hidden partitioning and partition transforms . . . 136
11.2 Partition evolution . . . 139
11.3 Sort orders and Z-ordering . . . 141
11.4 Target file size and the small-files problem . . . 142
11.5 Write distribution modes . . . 143

11.6 Summary 144

12 Reading Iceberg Efficiently 147

12.1 Manifest pruning and metadata filtering 147

12.1.1 Counting what each layer skipped 148

12.2 Column statistics and predicate pushdown into Iceberg 151

12.3 Metrics modes: `full`, `truncate`, `counts`, `none` 153

12.4 Bloom filters 154

12.5 Vectorized reads 155

12.6 Split planning 155

12.7 The metadata tables as a diagnostic 156

12.8 Summary 157

13 Writing and Maintaining Iceberg 160

13.1 Copy-on-write versus merge-on-read 160

13.2 `MERGE`, `UPDATE`, and `DELETE` 163

13.3 Write distribution and skew on write 165

13.4 Compacting data files 166

13.5 Rewriting manifests 166

13.6 Snapshot expiration and orphan files 167

13.7 Metadata management for streaming tables 168

13.8 Summary 170

IV Operating Spark 172

14 Tuning Resources and Configuration 173

14.1 Executor sizing from the mechanism 173

14.2 Parallelism: partitions against slots 177

14.3 Dynamic allocation 178

14.4 The external and push-based shuffle service 181

14.5 Configuration precedence and profiles 181

14.6 From symptom to lever 182

14.7 Summary 182

15 Debugging and Profiling 185

15.1 The Spark UI as a diagnostic 185

15.2 Event logs and the history server 187

15.3 Reading task, shuffle, spill, and GC metrics 188

15.4 Common failure signatures 189

15.5 async-profiler and flame graphs 192

15.6 Debugging query plans 193

15.7 From symptom to lever 193

15.8 Summary 193

16 Streaming into the Lakehouse 196

16.1 The micro-batch model and the unbounded table 196

16.2 Sources and sinks 197

16.3 Event time and watermarking 198

16.4 Stateful operations and the state store 199

16.5 Output modes and triggers 200

16.6 Exactly-once and the commit 201

16.7 Writing to Iceberg: cadence, small files, maintenance 203

16.8 From symptom to lever 204

16.9 Summary 204

V Internals 207

17 Tungsten and Code Generation 208

17.1 The Tungsten binary row format 208

17.2 Off-heap memory management 210

17.3 Cache-aware computation 211

17.4 Whole-stage code generation internals 211

17.5 Expression code generation and its limits 214

17.6 Columnar processing and vectorization 215

17.7 From symptom to lever . . . 217
17.8 Summary . . . 217

18 The Shuffle Framework and the Scheduler 218
18.1 The shuffle manager and shuffle writers . . . 218
18.2 The map output tracker . . . 220
18.3 The DAG scheduler and the task scheduler . . . 221
18.4 RPC and block transfer . . . 221
18.5 Locality and speculation . . . 222
18.6 From symptom to lever . . . 225
18.7 Summary . . . 226

19 DataSource V2 and Extensions 227
19.1 The DataSource V2 read and write API . . . 227
19.2 Pushdown negotiation . . . 230
19.3 How Iceberg reports splits and statistics . . . 233
19.4 Custom Catalyst rules and SQL extensions . . . 234
19.5 Plugins . . . 235
19.6 From symptom to lever . . . 236
19.7 Summary . . . 237

A The Reference Environment and the Running Example 238
A.1 The stack . . . 238
A.2 The cluster shape . . . 238
A.3 The lab: where the captured output comes from . . . 239
A.4 The running example: `orders` . . . 240
A.5 A columnar-format primer . . . 241

B Configuration Reference 243
B.1 Memory (Chapter 3) . . . 243
B.2 Shuffle (Chapter 4, Chapter 18) . . . 244
B.3 Adaptive Query Execution (Chapter 6) . . . 245
B.4 Joins (Chapter 5) . . . 245

B.5 Partitioning and pruning (Chapter 7) . . . 246
B.6 Iceberg catalog and commits (Chapter 10) . . . 247
B.7 Iceberg table properties (Chapter 11, Chapter 12, Chapter 13) . . . 247
B.8 Streaming (Chapter 16) . . . 248
B.9 Dynamic allocation and scheduling (Chapter 14, Chapter 18) . . . 248

C Reading Query Plans 250
C.1 The six `EXPLAIN` modes . . . 250
C.2 Reading a physical plan . . . 251
C.3 Scan operators . . . 252
C.4 Exchange operators . . . 253
C.5 Sort operators . . . 254
C.6 Join operators . . . 255
C.7 Aggregation operators . . . 256
C.8 Filter and Project . . . 257
C.9 Whole-stage codegen markers . . . 257
C.10 AQE nodes in a final plan . . . 258
C.11 Predicate-pushdown killers . . . 258

D Glossary 260

E Further Reading 264

References 267

List of Tables

1 Where the book starts you and where it takes you ix

1.1 The specimen at a glance, matching Figure 1.3. Task counts were captured on the lab of Section A.3, one week of `orders`; the reference cluster's counts are larger. 11

1.2 Narrow versus wide, by operation. `coalesce` is the one surprise: it looks like `repartition` but avoids a shuffle by only ever merging partitions, never splitting or reshuffling keys (Section 7.5). 13

2.1 The analogy above, mapped exactly onto the pipeline in Figure 2.1. . . 18

2.2 The `EXPLAIN` modes. Start with the default for a quick check and move to `FORMATTED` when you need each node's detail. 25

3.1 The unified pool against the formula, on four heaps (Spark 3.5.2). . . . 34

3.2 The three collectors, at a glance. 37

4.1 Written once, read immediately (shuffle) wants decompression speed; written once, read many times (storage) can afford to spend more on ratio. 50

6.1 The three runtime rewrites, each a planning-time decision revisited with measured evidence. 65

6.2 Coalescing a shuffle on 40 slots, with `parallelismFirst` true and false. Partition counts are approximate: AQE merges greedily in partition order, so real counts differ slightly. With `parallelismFirst` on, a small shuffle keeps as many partitions as there are slots; with it off, the same shuffle collapses to a handful of tasks and leaves most of the cluster idle. 68

6.3 Measured on the lab. Each prediction uses only the formula and the shuffle bytes the reduce stage read (`pF` is `parallelismFirst`). The advisory size changes nothing until it falls below the parallelism term ($512.1/40 \approx 12.8$ MB), exactly as the formula says. 69

6.4 The two thresholds guard against different failures. The planning-time one must stay conservative because its input is a guess; the adaptive one sees a measurement, but a measurement of the shuffled rows, not of the broadcast relation that gets built from them. 74

6.5 The adaptive threshold decides on the shuffle stage's size, not on the relation that gets built: a 4 MB threshold promoted a join whose broadcast relation was 20.0 MiB. 74

6.6 Both the relative check ($5\times$ median) and the absolute floor (256 MB) must pass before AQE calls a partition skewed. The 220 MB partition shows why the floor matters: it clears the relative bar but is still an ordinary amount of data for one task, not a straggler. 76

6.7 Skew handling splits only the side whose rows can be partitioned without changing the join result. A full outer join cannot be skew-split at all. . . 78

6.8 Measured on the lab (advisory size 8 MB, skew floor 32 MB). Skew handling turned 42 tasks into 51 and cut the slowest task from 5115 ms to 844 ms, 6.1 times, while the median barely moved. 79

6.9 Where AQE does nothing. Each row names the earlier decision that still matters. 83

6.10 The AQE levers: each symptom with the evidence that confirms it, the knob that moves it, and what the knob costs. 85

7.1 The scan-node fields that report pruning. An empty list on a partitioned table is the first thing to look for when a scan reads too much. 87

7.2 One day of `orders`, five ways. Spark rewrites a date comparison on a timestamp column into a range on the bare column, and the range prunes. A string cast or arithmetic on the column leaves nothing to push, and the scan reads the whole month. 89

7.3 Dynamic partition pruning on the lab. It cuts the scan from 31 tasks to 4, an 87% reduction, only when the fact table is partitioned by the join column itself and the dimension side is a broadcast. The last two rows show filters that exist and prune nothing. 92
7.4 Conditions for dynamic partition pruning, each with the lab row that shows it failing. 93
7.5 The ways to set a partition count. Only the forms that name a count fix it against AQE's coalescing. 97
7.6 Fourteen days of `orders` (490.6 MB read), then 10 output partitions. `coalesce` merged the scan itself down to 10 tasks and shuffled nothing. `repartition` kept the scan at 14 tasks, wrote and then read 1001.2 MB of shuffle data (rows are larger once deserialized), and made each scan task about twice as slow because it now also writes shuffle files. 98
7.7 The pruning and layout levers: each symptom, the evidence in the plan, the fix, and what it costs. 101

8.1 Wall-clock seconds for N aggregations over one week of `orders`, single runs on a warm lab JVM. The cache costs about 4.2 s to build and saves about 0.4 s per read, so it only catches up near the tenth read. 104
8.2 What the Storage tab holds after each step. Only `CACHE TABLE` caches before any query reads it. 106
8.3 One week of `orders` at three storage levels. Serializing saved 27%, not the 2–3$\times$ that holds for caches of JVM objects. 106
8.4 Reuse on the lab. Disabling it doubles the scan and costs about a third more time. The last row is the fragile case: two aggregates over the same table and key differ in their aggregate function, so the exchanges differ and nothing is reused. 111
8.5 The caching and reuse levers: symptom, evidence, lever, and cost. . . . 114

9.1 Deleting one row, measured. Copy-on-write rewrote the whole 34.8 MB data file; merge-on-read wrote a 1.5 KB file that records the file path and the row position. 124

9.2 Where the three table formats put their defaults 125

10.1 Twelve writers, one table, 48 commits, measured. With the default, 34 commits needed a second attempt and one needed four; with no retries, 7 failed outright. In both runs the row count equals 20,000 times the commits that succeeded: a lost swap loses no data, only the attempt. . . 131
10.2 Iceberg commit-retry table properties 132

11.1 Common Iceberg partition transforms 137
11.2 Day against hour for a one-week filter. Both read 6.8 million rows and about 47 MB; the hour layout reads 24 times the files and plans 50% slower, and its 1.4 MB files were packed four to a task only because the split planner combined them (Section 12.6). 138
11.3 The layout levers: symptom, evidence, lever, and cost. 146

12.1 The pruning layers a scan passes through, in order 148
12.2 The week query on two layouts. Both read the same 7 files and 46.8 MB; the layer that did the skipping differs. With a single manifest the 24 files were excluded by their per-file partition values, one layer down. With one manifest per commit, 24 whole manifests were never opened. . . . 149
12.3 One `customer_id` equality lookup, measured on four copies of the same rows. Sorting lets the manifest skip 199 of 200 files before any is opened. The bloom filter helps inside each file, but all 10 are still opened. Storing only counts removes the min/max the manifest needs, so all 200 files are opened, and Parquet's own row-group statistics rescue most of the bytes. 152
12.4 The same scan with vectorized reads on and off. The bytes read are identical; decoding row by row made each task about seven times slower. 155
12.5 Scan tasks for ten 35 MB files. Adaptive sizing hides `read.split.target-size` for a small scan. With it off, the 128 MB target packs three files per task and a 512 MB target packs all ten into one. An 8 MB target cannot split a file that has no split offsets, so it leaves 10 tasks. 156
12.6 The read-path levers: symptom, evidence, lever, and cost. 158

13.1 Choosing copy-on-write versus merge-on-read 161

13.2 Ten five-row updates, measured. Copy-on-write rewrote a 34.8 MB data file each time; merge-on-read wrote one tiny data file and one tiny delete file. The price shows up in the read: 25% slower with ten delete files outstanding, and only partly recovered by compaction. 162
13.3 The write and maintenance levers: symptom, evidence, lever, and cost. . 170

14.1 The unified pool read from Spark's memory manager at start-up (it equals $(\text{heap} - 300\ \text{MiB}) \times 0.6$), and the share each concurrent task gets. Spilling cells in Figure 14.1 are exactly the ones below roughly 300–450 MB. . 175
14.2 What dynamic allocation did in each lab setup. The first two rows run the same job; the difference, 0.2 s against 25.6 s, is the price of letting shuffle-holding executors go. 180
14.3 The value a session reported after each layer was added. Each layer overrides the one above it in the table; static settings such as executor memory cannot be changed once the application has started. 182
14.4 The tuning levers of this chapter: symptom, evidence, lever, cost. 183

15.1 Three OOM signatures, reproduced on the lab. The distinguishing evidence is the exception class (or its absence), not the words "out of memory." 190
15.2 The debugging levers: symptom, evidence, lever, cost. 194

16.1 `Trigger.AvailableNow` over a directory that grows between runs. The first run processes everything, the second finds nothing and commits nothing (no empty snapshot), and the third processes only the new files. 201
16.2 The lab's measurements and their extrapolation to a day ($\times 864$). Each trigger produced one file per commit because the table has one partition for the day; the lab's rate (about 1.7 bytes per row after compression) is far lower than a production stream, so the shape of the relationship is the transferable part, not the absolute sizes. 204
16.3 The streaming levers: symptom, evidence, lever, cost. 205

17.1 The Tungsten levers: symptom, evidence, lever, cost. 216

18.1 Which shuffle writer a given shuffle gets, in the order Spark checks . . 219

18.2 Locality level of tasks recorded in the lab's event logs. No task was `RACK_LOCAL` or `ANY`: the lab has one machine, so the cross-node levels cannot occur. 224
18.3 The scheduler levers: symptom, evidence, lever, cost. 225

19.1 The DSv2 interfaces implemented by Iceberg 1.6.1's `SparkCatalog`, `SparkTable`, `SparkScanBuilder` and `SparkBatchQueryScan`, read by reflection from a running session. Each row is a capability the planner can use, and each chapter that shows its effect is cited. 228
19.2 What `SupportsReportStatistics` reported to the optimizer. The per-row figure is the same for every row count, and the data-column filter changed nothing — the source has no selectivity information. 233
19.3 The reported size is rows times the default width of the columns the scan projects. 234
19.4 With the extension removed, reads and metadata tables still work (they are catalog and DSv2 features); the Iceberg-specific *syntax* does not, because it is parser rules the extension injects. 235
19.5 The DSv2 levers: symptom, evidence, lever, cost. 236

A.1 The reference stack 239
A.2 `orders` schema 240

List of Figures

1 Reading order. Part I first; Parts II and III in either order; Part IV once you have the model; Part V when you want to know how it is built. . . . xi

1.1 One application: a single driver process that plans and coordinates, and a set of executor processes that run tasks and cache data. The cluster manager — here the Kubernetes API — hands out the containers the processes run in. 4

1.2 The specimen query from submission to result. The driver compiles and schedules but never touches the data; executors do all the reading, shuffling, and computing; the cluster manager only supplies the pods. 6

1.3 The specimen as five stages, one per side of each shuffle. Stages 0 and 1 scan `orders` and `dim_product` and shuffle by `product_id`; stage 2 joins and partially aggregates; stages 3 and 4 finish the aggregate and sort. Rose arrows are the four shuffles. One task in stage 2 runs far longer than its peers — the first sighting of skew. 10

1.4 A narrow dependency: each output partition is built from one input partition, so the work pipelines in place. A wide dependency: each output partition draws from every input partition, so the data must be redistributed — a shuffle — and the stage ends. 12

2.1 The compilation pipeline. Parsing produces a tree that is pure syntax. Analysis resolves names against the catalog. Logical optimization rewrites the tree with rules. Physical planning picks concrete operators and shuffle points. Code generation emits Java that the JIT compiles. The side notes name what each stage decides. 17

2.2 Bytes read and wall clock with one optimizer rule excluded. Without column pruning the scan read 8× the bytes (281 against 34 MB); without predicate pushdown it read 321 MB and 32 million rows instead of 4.9 million; with all three rules excluded together, 1,262 MB and 9.1 s against 34 MB and 2.0 s. Excluding constant folding changed neither bytes nor time here: the folded expression was cheap next to the scan. . 22

3.1 One query, eight consecutive runs in a fresh JVM. The first run pays for class loading, interpretation and compilation; later runs run compiled code. 30

3.2 An executor's memory: the JVM heap (reserved / unified / user), the optional off-heap pool, and the overhead the process needs outside both. The cluster manager's container limit must cover all three. Numbers are worked for an 8 GB heap with 2 GB overhead. 35

3.3 A stop-the-world pause freezes every task slot on the executor at once, not one at a time. If the pause outlasts the heartbeat interval, the driver concludes the executor is gone and re-runs its work elsewhere — so a GC problem shows up as lost executors and repeated stages, not just as slow ones. 37

3.4 One stage under three collectors. Wall clock is close for G1 and Parallel; the longest pause and the per-task GC metric are not. 38

3.5 Left: one million integers as a primitive array and as boxed objects. Right: bytes per five-field record under Java and Kryo serialization, one record at a time and as one array. 39

4.1 The shuffle write and read paths. The map task never talks to the reduce task directly: it registers its file's location with the driver's map output tracker, and the reduce task asks the tracker where to fetch from. 45

4.2 For a fixed shuffle size, size per task is exactly (shuffle bytes) / (partition count) — an exact relationship, not a rule of thumb. For a 100 GB shuffle the default 200 partitions puts 512 MB in every task, well past where it spills; 512–800 partitions lands each task in the 128–200 MB band. . . 47
4.3 Shuffle bytes and wall clock by codec. Compression cut the bytes written from 1,080 MB to 119–259 MB and the time from 10.6 s to 7.4–8.1 s, on a local disk. `zstd` wrote half the bytes of `lz4` and was not slower. . 49

5.1 The four join strategies as plan shapes. Broadcast hash join shuffles nothing on the large side; shuffled hash join shuffles both sides but sorts neither; sort-merge join shuffles and sorts both; broadcast nested loop join broadcasts one side to evaluate a condition that is not an equality. 54
5.2 The two cheapest-to-compare strategies side by side. A sort-merge join shuffles and sorts both sides before merging; a broadcast hash join leaves the big side in place and ships the small side to every executor. 55
5.3 The planner stops at the first rule that applies. A hint wins outright; otherwise the estimated size decides, and sort-merge is the default for two large sides. 58
5.4 Seconds for the same join under each strategy. Broadcast was fastest on all three dimensions (3.8–4.8 s), shuffled hash in between (5.5–7.8 s) and sort-merge slowest (9.3–11.3 s). Each run has two exchanges for broadcast and three for the shuffled strategies. 59

6.1 AQE inserts a measurement between stages: when a shuffle finishes, the driver reads the exact size of every output partition and re-optimizes only the part of the plan that has not run. Blue steps run on the driver, green on the executors; the rose dashed box is the stage boundary where the measurement happens. 65
6.2 2000 shuffle partitions of 3 MB each carry 6 GB; AQE merges adjacent partitions toward the 64 MB advisory size, so the next stage runs about 94 tasks instead of 2000. Rose marks the cost, green the corrected state. 67

6.3 The formula against the lab. For seven settings the bars pair the task count the formula predicts (blue) with the one the reduce stage ran (green); the log scale shows how far apart the settings are and how close each pair is. 69

6.4 The same join before and after the runtime rewrite. Both inputs were already shuffled when AQE measured the filtered dimension at 3.8 MiB, so the big side is read locally instead of fetched and neither side is sorted. The relation built for the broadcast is 20.0 MiB in memory. 73

6.5 Without skew handling, one 3.2 GB partition runs as a single task long after the rest of the stage has finished. AQE splits it into pieces near the advisory size and copies the small side's slice to each piece, turning one straggler into about 50 ordinary tasks. 77

6.6 The 200 reduce partitions of the skewed join as measured on the lab, one bar per task. The viral product is 47 times the median, far above the relative test and the lab floor, yet the default 256 MB floor is off the scale above it and the 64 MB split target sits barely under it. 79

7.1 A filter that resolves against the partition value keeps 7 of 31 daily partitions of `orders` and skips the other 24 before any file is opened. Green marks what is read; dashed gray marks what is skipped. 88

7.2 The same one day of `orders`, five filters, measured. Three forms reach the scan as a range and read 6.7 MB in one task; the two that do not read the whole month, 31 times as many tasks and 31 times the bytes. 90

7.3 Without DPP, pruning happens last: the whole fact table is scanned before the join discards most of it. With DPP, the small dimension side runs first and its matching keys are injected as a filter *before* the fact scan, so pruning happens first. 91

7.4 Without bucketing, a join shuffles both sides to line up the keys. With both tables bucketed on the join key, the matching rows are already in the same-numbered bucket on each side, and each task joins one bucket pair with no `Exchange`. 95

7.5 What each choice does to the stage graph for ten output partitions of two weeks of `orders`. `coalesce` folds the merge into the scan and adds no shuffle; `repartition` keeps the scan wide and pays for a 1001 MB shuffle. 98

8.1 Total time for N aggregations over the same week, with and without `cache()`. The shaded gap is time the cache costs; the lines converge near ten reads because each cached read still scans 563 MB of cached batches against 58 MB of Parquet columns. 105

8.2 Borrowing is symmetric; reclaiming is not. Storage can spread into idle execution space, but the moment a join or sort needs that space back, execution evicts the borrower's cached blocks to reclaim it — storage has no equivalent power over a running task. 109

8.3 A cached month of `orders` before and after a join ran. All partitions stay cached, but 235 MB more of them sit on disk once execution claimed the memory it needed. 110

8.4 The same self-join of an aggregate planned two ways. With reuse, the scan and exchange run once and the join reads the result twice (`ReusedExchange`); without it, the scan and the shuffle write run twice. 111

8.5 Three caches that do not fit together. Each new cache evicts the least recently used blocks of an older one, and re-reading the damaged cache evicts its neighbour: LRU has no notion of which DataFrame you still need. 112

9.1 The metadata tree for one snapshot of `orders`. The catalog resolves to one table metadata file, which points to one manifest list, which points to the manifests that make up this snapshot, which list the actual data and delete files. Every level is immutable; a new commit writes new files at whichever levels changed and leaves the rest untouched. 117

9.2 The metadata tree of `t_meta` after three commits, as the lab's metadata tables report it. The catalog points to `v3.metadata.json`; the snapshot-3 manifest list names three manifests, one written by each commit, which list five data files. 119

9.3 Three snapshots of one table. Each commit writes one new manifest (orange) and a new manifest list, and the list also names every earlier manifest unchanged (green). Older snapshots keep their own lists, so time travel reads them untouched. 121
9.4 A position delete at sequence number 5 applies to the data files at 1, 2, and 4 and not to a later append at 6. File `C` carries number 4 because a copy-on-write delete rewrote it; sequence numbers record when a file was *added*, not when its rows were first written. 122
9.5 Bytes written to delete a single row: the whole data file under copy-on-write, one tiny delete file under merge-on-read. The log scale spans more than four orders of magnitude. 124

10.1 Three catalogs, one job. Each keeps a single pointer per table and swaps it conditionally; a writer that loses sees a rejection from whichever mechanism guards the pointer. Only the file-based catalog was run on the lab; the other two are described from the Iceberg documentation. 128
10.2 How many attempts each of the 48 commits needed. The default retries absorbed almost every lost race; with retries off, a lost race is a failed job. 131
10.3 Snapshots committed over time for the two runs. Twelve writers still produce a single chain of commits: the catalog pointer admits one writer at a time, so the table's write rate is bounded by commit latency, not by writer count. 132
10.4 Schematic: two writers racing for the same base version. The first writer's swap lands first and succeeds; the INSERT session's swap against the same stale base is rejected, so it re-reads the new current version, rebuilds its manifest on top of it, and retries — succeeding one version later, with no error surfaced to either caller. 133

11.1 The same month of `orders` at two partition granularities. The hour spec has 24 times the files at one twenty-fourth the size, and a week-long query opens 168 files instead of 7 for the same bytes. 138

11.2 After evolving `orders` from `month(order_ts)` to `day(order_ts)`, both specs coexist under the same current snapshot. Old manifests keep describing month-partitioned files; new manifests describe day-partitioned files. One scan reads both, pruning each manifest against its own recorded spec. 139
11.3 One day of old data lives in the 517 MB month-partition file and decodes 11.3 million rows; one day of new data lives in its own 34.8 MB file and decodes 0.97 million. Evolution kept the old data readable, not as fast. . 140
11.4 Data files opened by three lookups on three layouts of the same rows. Sorting by `(customer_id, product_id)` makes customer lookups nearly free (1 file) and leaves product lookups reading 43 of 43. Z-ordering gives up the best case for the other lookup: 24 files for a customer, 3 for a product, 2 for both. 142
11.5 The same ten days written under each distribution mode. With no shuffle, each of the ten tasks wrote one file per day it held, 100 files of 3.6 MB; with `hash` or `range` each day's rows were gathered first, giving 10 files of 34.8 MB. The inserts took 25.1, 24.3, and 23.8 s. 144
11.6 Bytes per bucket for `bucket(16, product_id)` on 9.7 million orders. Hashing sends every row of the viral product to the same bucket, so one file and one write task carry 7.1 times the median. 145

12.1 The four pruning layers, narrowing top to bottom. Each layer only ever has to consider what the layer above it did not already exclude — a file skipped by the manifest list is never even checked against a bloom filter or row-group statistics. 149
12.2 The scan counters for one week of `orders` on a table with one manifest per daily commit. The partition summaries in the manifest list excluded 24 of 31 manifests before any was opened; the 7 survivors held 7 files, all needed; the tasks then read 46.8 MB, the four columns the query names. 150

12.3 Per-file `customer_id` ranges, from the manifest, for the same 9.7 million rows. Unsorted, each of 10 files spans the whole key range and the lookup key falls inside all of them. Sorted by `customer_id`, the 200 files have disjoint ranges and the key falls inside exactly one. 152

12.4 Bytes read for one `customer_id` lookup, by table layout. Sorting wins by two orders of magnitude; the bloom filter halves the read; storing only counts throws the sorted layoutś advantage away at the manifest and leaves Parquet footers to recover most of it. 153

13.1 One small `UPDATE`, two paths. Merge-on-read is nearly three times faster and writes about nine thousand times fewer bytes. 162

13.2 The same `UPDATE` under copy-on-write and merge-on-read. CoW pays once, at write time, and every later read is plain data with no merge; MoR pays a small write-time cost and pushes the rest onto every read until the next compaction. 163

13.3 The same `MERGE`, with and without a static range on the partition column in the `ON` clause. The predicate cut the bytes the scan read by 5.7 times. 165

13.4 Sixty tiny commits, then compaction, then a manifest rewrite. Compaction fixed the files and the scan time but *added* a manifest; the manifest rewrite then merged all 61 into one, with no further change to this small scan. 166

13.5 Table directory size through the two cleanup procedures. Expiring snapshots frees what superseded history held; a stray file from a failed write adds 5 MB until `remove_orphan_files` deletes it. 168

13.6 The four maintenance procedures in the usual order. The first two make reads cheaper and leave old files behind; the third frees the disk of expired history; the fourth removes files no snapshot named, which is a disjoint set. 169

14.1 The same stage under twelve executor shapes. The spill pattern follows a diagonal: it appears whenever the execution memory per concurrent task falls below a threshold, whichever of heap or cores moved it there. Each cell is one run. 174
14.2 Spill against execution memory per task. The twelve runs separate cleanly at a single threshold, not by heap size and not by core count. 175
14.3 More slots buy time; more slots in the same heap cost GC. The rings mark runs that spilled. The three heaps lie almost on top of each other in the left panel: for this stage, spilling to a fast disk cost less than the run-to-run noise one would expect between single runs. 176
14.4 The same sweep on two stages. When the scan dominates (left) the shuffle partition count is nearly invisible until it reaches the thousands. When the shuffle stage does the work (right) too few partitions leave slots idle and too many pay a per-task cost: a hyperbola with a flat bottom. 178
14.5 Executors over time, three configurations. Left: job 1 leaves shuffle files, and tracking keeps all four executors through the idle period. Middle: the same, with a tracking timeout of 8 s; the executors holding the files are released and the second job has to ask for them again. Right: a job that leaves no shuffle files, so every executor is released after its idle timeout. 179

15.1 The UI read top to bottom: each tab narrows the search before the next is worth opening. 186
15.2 Bytes in the event log by event type. `TaskEnd` events, which carry every task's metrics, are 79% of the file at about 6.4 KB each; the environment snapshot is the second-largest entry at 80 KB. 187
15.3 The 40 tasks of the join stage. One task is 9.7 times the median. 188
15.4 Median task against the slowest task of the same stage. 188
15.5 Task attempts without and with speculation. The speculative copy (orange) started at 8.1 s and won; the original (red) was killed when it did. 191

15.6 The stage attempts of the second run. The final stage failed at once with a `FetchFailedException`; the map stage re-ran for the missing output (3 tasks); a middle stage failed with a second `FetchFailedException`; the map stage ran again (4 tasks); then the middle and final stages succeeded. Three tasks failed with `FetchFailed`, 7.7 s in all. 192

16.1 One iteration of the micro-batch loop. The offset log is written before the batch runs and the commit log after it, which is what makes a restart safe: a replay re-runs exactly the range that either never committed or already did, never anything in between. 197

16.2 State rows against time. Without a watermark nothing tells Spark a window is complete, so state grows by about 1,000 rows every 5-second window and was still climbing (19,121 rows, 5.0 MB) when the run stopped. With a 10-second watermark it plateaus at 4,000 rows (1.4 MB) as closed windows are evicted — 16,000 rows removed over the run. 199

16.3 Left: reported state memory against state rows for the default and RocksDB providers. Right: micro-batch duration over time. RocksDB held the same rows in one sixth of the reported memory and paid with slower commits. 200

16.4 Rows emitted per micro-batch for the same query. `append` emits nothing until a window closes (0 or 3 rows); `update` emits the windows that changed (3 to 6 rows); `complete` re-emits the whole result table, which grew from 3 to 27 rows over 15 batches and keeps growing. 201

16.5 Records per snapshot across a kill and restart. The table ended with 29 snapshots whose epoch ids run 0 to 28 without a gap under one query id; the first batch after the restart (epoch 19) carried 20,000 records, catching up on the downtime. The table held 176,000 rows, all distinct, covering sequence numbers 0 to 175,999 with no gap. 202

16.6 The trigger interval sets the commit count, and with it the file count, the file size and the manifest count. A 2-second trigger produced 49 files of 66 KB and 49 manifests in 100 seconds; a 30-second trigger produced 3 files of 1.16 MB and 3 manifests. 203

17.1 One `UnsafeRow`: a null bitset, one 8-byte slot per field, and a variable-length region past the last fixed slot. A fixed field's slot holds its value directly; a variable field's slot holds an (offset, length) pointer into the variable region of the same buffer. 209
17.2 The same stage on-heap and with a 1 GB off-heap pool added. GC time did not fall (0.65 to 0.72 s in all four), spill was identical (417 MB to disk), and wall-clock time was 6–20% higher with off-heap enabled. One run each. 211
17.3 The same sort with radix sort on (the default) and off. Radix sort made the stage 1.7× faster; the comparison sort the setting falls back to is the prefix-based one described above. 212
17.4 Whole-stage codegen on and off. A filter-and-aggregate and an arithmetic chain ran 1.4–1.5× faster with it; a shuffle-bound group-by did not change (0.99×), because its time is in the exchange, not in the fused loop. 213
17.5 Whole-stage codegen holds up to exactly 100 columns and is abandoned at 101. The interesting part is the time: the 99- and 100-column stages took about 43 s, ten times the 4 s of the 101-column interpreted fallback. One run each, log scale. 214
17.6 The same arithmetic as a built-in expression (0.29 s), a pandas UDF evaluated in Arrow batches (2.5 s) and a row-at-a-time Python UDF (9.2 s). The plan shows `ArrowEvalPython` for the second and `BatchEvalPython` for the third; neither is inside the whole-stage-codegen loop. 216

18.1 The writer each SQL shuffle got, in Spark 3.5.2. The choice flips at exactly 200 partitions: up to the bypass threshold every shuffle used the bypass writer; at 201 and above, the serialized (`UnsafeShuffleWriter`) path. The sort-merge join's third exchange, the single-partition one under the final count, stayed on bypass. No SQL shuffle used the general sort writer. 220
18.2 Driver-side cost of a stage of empty tasks. Per-task cost is flat at about 0.04 ms up to 20,000 tasks and rises sixfold at 80,000. 222

18.3 The scheduler's five components. The DAG scheduler hands the task scheduler a set of runnable tasks and is only reinvoked when the map output tracker reports a lost executor's output as gone; the task scheduler reads fetch locations from the tracker and issues its own small control messages over RPC, kept separate from the large shuffle bytes on the block-transfer path. 223

19.1 The write protocol, with the failure path measured. A task that raises an error stops the job before `commit()`: the table had one snapshot and two data files before the failed `INSERT` and the same afterwards. 229

19.2 Rows produced by the scan for thirteen predicates. Only two — a range that equals exactly one day partition, and the `date()` form that Iceberg rewrites to that range — leave no `Filter` above the scan. Every other predicate keeps one. 230

19.3 The same negotiation, three outcomes. Only a predicate that equals the partition boundary is fully handled; one hour inside the partition is pushed for pruning but re-checked by Spark; a `CAST` is not pushed. . . . 231

19.4 `COUNT(*)` on the whole table, on one day, `MIN`/`MAX` of `order_ts` and `MAX(order_total)` were answered from metadata in 0.09–0.17 s. `COUNT(*)` with a data-column filter and `SUM` had to scan the data files (0.6–1.0 s). . 232

Part I

How Spark Executes a Query

Chapter 1

The Shape of a Spark Application

A query you would actually write — a week of `orders` joined to a product dimension, aggregated to an on-time delivery rate — runs in four minutes. This chapter does not make it faster. It uses it as a specimen: you run it, open the Spark UI, and learn to name every moving part — the driver, the executors, the one job, its stages, the shuffle between them, and the one task doing more than the rest. By the end you can read a running job instead of watching a progress bar.

1.1 A query, end to end

Here is the specimen. It asks, for each product category, what fraction of orders in the first week of March were delivered within their promised window. It runs against the `orders` table and the `dim_product` dimension introduced in Appendix A; `orders` is an Iceberg table partitioned by `day(order_ts)`, holding a few hundred million rows for the month.

```
SELECT p.category,
       AVG(CASE WHEN o.delivery_days <= o.promised_days THEN
          1.0 ELSE 0.0 END) AS on_time_rate
FROM orders o
JOIN dim_product p ON o.product_id = p.product_id
WHERE o.order_ts >= TIMESTAMP '2026-03-01 00:00:00'
```

```
  AND o.order_ts <  TIMESTAMP '2026-03-08 00:00:00'
GROUP BY p.category
ORDER BY on_time_rate;
```

Listing 1.1: The specimen query

Submit it and Spark does not simply "run the SQL." The text is parsed into a tree, resolved against the catalog, rewritten by the optimizer, turned into a plan of physical operators, and compiled to Java bytecode. That whole compilation is Chapter 2's subject; for now, take it as given that by the time anything executes, Spark holds a *physical plan*: a tree of operators such as *scan*, *filter*, *exchange* (a shuffle), *sort-merge join*, and *hash aggregate*.

Executing that plan is a distributed affair. The plan is split into a *job*, the job into *stages*, and each stage into *tasks* that run in parallel on *executors*. The result flows back to the process that submitted the query, the *driver*. On the reference cluster this specimen runs as five stages and a few hundred tasks, and it takes about four minutes.

Why four minutes and not one, or forty? That question is what the rest of the book answers. This chapter's job is narrower: to give you the vocabulary and the tools to *see* the structure of the run, so that later chapters have something concrete to change. Everything that follows — joins, the shuffle, adaptive execution, the Iceberg metadata layer — is a way of moving less data, moving it less often, or moving it more evenly. You cannot decide to move data differently until you can see where it moves now.

1.2 Driver, executors, and the cluster manager

A Spark *application* is one driver plus a set of executors, all working on your queries until the application exits (Figure 1.1).

The **driver** is the process that runs your program. It holds the `SparkSession`, compiles each query into a plan, breaks the plan into stages and tasks, hands tasks to executors, tracks their progress, and gathers results. It is a single process and a single point of failure: if the driver dies, the application dies. It is also where a `collect()` or a `toPandas()` lands every row it pulls back, which is the most common way to run the driver out of memory (Chapter 3, Section 15.3).

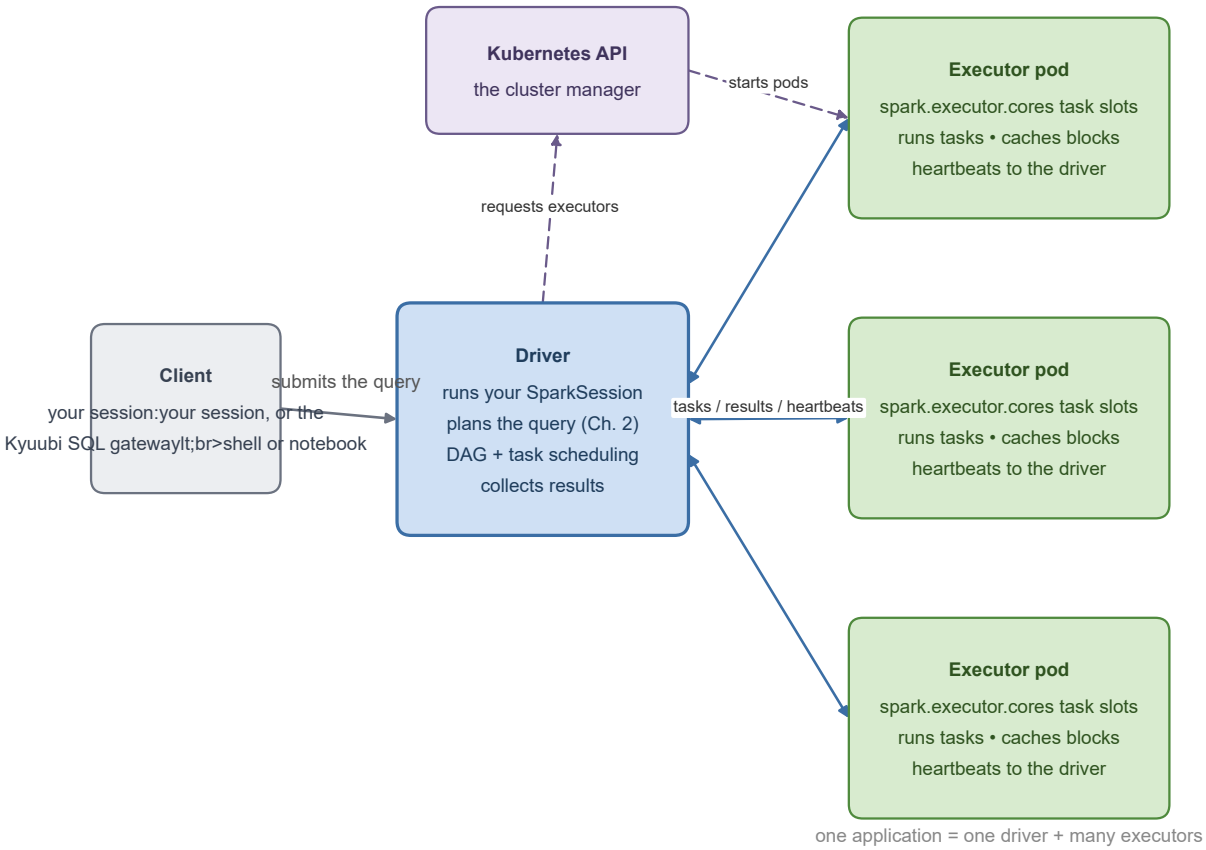


Figure 1.1: One application: a single driver process that plans and coordinates, and a set of executor processes that run tasks and cache data. The cluster manager — here the Kubernetes API — hands out the containers the processes run in.

An **executor** is a JVM process that does the actual work. Each executor has a fixed number of *task slots* — one per core, set by `spark.executor.cores` — and runs one task per slot at a time. Executors also hold cached blocks and the map-side output of shuffles, and they send regular heartbeats to the driver so it knows they are alive. An application typically has many executors and exactly one driver.

The **cluster manager** owns the machines and hands out the containers that the driver and executors run in. Spark supports several: its own standalone manager, Hadoop YARN, and Kubernetes (Apache Mesos support was removed in Spark 3.2). The reference stack uses Kubernetes, so "container" means "pod" and the cluster manager is the Kubernetes API server.

> **Note** The driver–executor split is why two very different failures look different. An executor lost to an out-of-memory kill takes its tasks down, and Spark simply reruns them elsewhere; the job slows but survives. The driver running out of memory — usually from pulling too much data back — kills the whole application at once, with no retry.

> **Analogy** A busy restaurant kitchen has the same three roles. The **driver** is the head chef at the pass: reading each ticket, deciding how the dish comes together, calling out instructions — and never touching a pan. The **executors** are the line cooks, each at their own station with their own pans (task slots), doing the actual cutting and cooking and calling back to the pass when a plate is ready. The **cluster manager** is the staffing agency: when the chef needs more hands on a busy night, it sends cooks over, but it never runs a station itself. Push it further than that and it breaks — no kitchen survives a chef who cannot cook a single plate — but the division of labour is exactly right: one planner, many workers, and an outside party that only ever supplies the workers.

Following the specimen from submission to result

Putting the three roles together, here is what happens end to end when you run the query from Listing 1.1, step by step (Figure 1.2).

1. **Submit.** Your SQL client — a `spark-sql` shell or a notebook session — sends

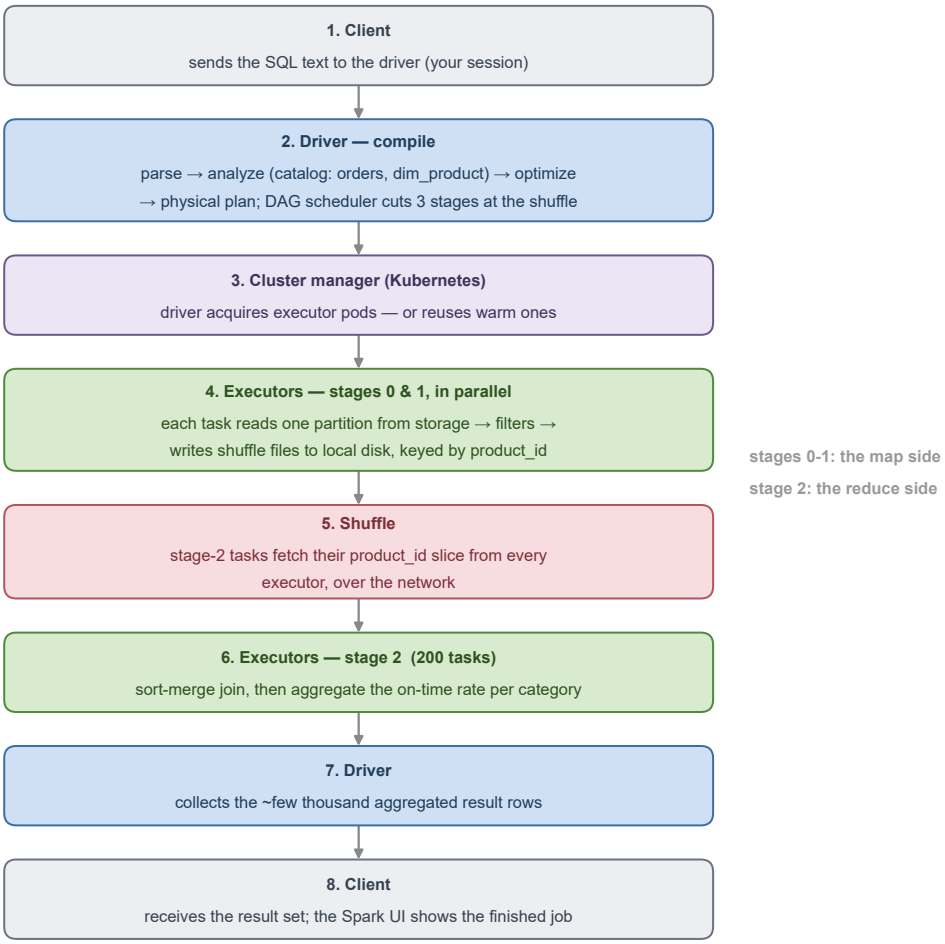


Figure 1.2: The specimen query from submission to result. The driver compiles and schedules but never touches the data; executors do all the reading, shuffling, and computing; the cluster manager only supplies the pods.

the query *text* to the driver. Nothing is parsed on your side.

2. **Compile (driver).** The driver parses the text, resolves `orders` and `dim_product` against the catalog, runs the optimizer, and produces a physical plan (Chapter 2). Its DAG scheduler then cuts that plan into five stages, one cut per `Exchange` node, and builds a set of tasks for each — one task per input partition.

3. **Acquire executors (cluster manager).** If the driver does not already have enough executors, it asks Kubernetes for more pods. In a long-lived session the executors from the previous query are usually still running, so this step does nothing and the query starts at once.

4. **Run the map stages (executors).** The driver sends stage 0 and stage 1 tasks to free task slots, each task carrying its plan fragment and the id of the one partition it must process. Stage 0's roughly 180 tasks each read one file split of the March `orders` partitions from storage, apply the `order_ts` filter, and write small files to the executor's *local* disk, bucketed by `product_id`. Stage 1 does the same for `dim_product`. The two stages run concurrently, and every task reports its metrics to the driver as it finishes.

5. **Shuffle.** Once both map stages are done the driver launches stage 2. The plan calls for 200 tasks, one per value of `spark.sql.shuffle.partitions`; AQE may merge them into fewer (Chapter 6). Each stage-2 task connects to *every* executor that ran a map task and fetches the shuffle files for its slice of `product_id`. This all-to-all transfer over the network is the shuffle, and it is why the map side (stages 0–1) and the reduce side (stage 2) are separate stages.

6. **Run the reduce stages (executors).** Each stage-2 task now holds every row for its products, from both tables. It sorts and merges them, performs the join, computes a partial aggregate per category, and writes shuffle files again, this time keyed by `category`. Stage 3 fetches those, finishes the aggregate, and writes range-partitioned output for the `ORDER BY`; stage 4 sorts it.

7. **Collect (driver).** The result is one row per category — a handful of rows — so the driver gathers them from the last stage's tasks. A query returning millions of rows

would overwhelm the driver at exactly this step (Chapter 3).

8. **Return.** The driver sends the result set to the client and marks the query complete. The Spark UI now shows the finished query, its stages, and every task's metrics — the view you learn to read in Section 1.6.

Throughout, the driver is also doing bookkeeping the diagram leaves out: tracking which tasks have finished, re-sending any that failed to a different executor, and updating the UI. It never reads or writes the data itself — only the plan, the schedule, and the final small result.

1.3 Deploy modes and Spark on Kubernetes

Where the *driver* runs is the *deploy mode*. In **client mode** the driver runs in whatever process launched the application — your shell, a notebook kernel, or a long-lived gateway. In **cluster mode** the launcher asks the cluster manager to start the driver as just another container, then disconnects. Client mode is convenient for interactive work because the driver is local to you; cluster mode is standard for scheduled jobs because nothing on your side has to stay running.

On Kubernetes the sequence for a cluster-mode job is: `spark-submit` calls the Kubernetes API to create a *driver pod*; the driver pod starts, and its Spark code calls the same API to create *executor pods*; each executor pod starts, connects back to the driver, and registers its task slots. From then on the driver schedules tasks onto executors directly, and the Kubernetes API is only involved again when pods are added or removed — for example under dynamic allocation (Section 14.1).

Interactive work does not usually run `spark-submit` per query. A long-lived driver (a persistent `SparkSession` in a shell or notebook) accepts SQL one statement after another. The engine, stages, and tasks are all still there; the session just keeps the driver and its executors warm so you are not paying startup cost on every query.

1.4 Jobs, stages, and tasks

Spark transformations are *lazy*: `filter`, `join`, `groupBy`, and the rest build up a plan but run nothing. Work starts only when an *action* asks for a result — a write, a `count`, a `collect`, or the implicit collect that returns rows to a SQL client. Each action triggers one **job**. (With Adaptive Query Execution, Chapter 6, one SQL query shows up as *several* jobs, because Spark submits each query stage as its own job and later jobs list the earlier stages as skipped. Count stages, not jobs.)

Within a job, the driver's *DAG scheduler* — named for the **d**irected **a**cyclic **g**raph of stages it builds, since a plan's dependencies only ever point forward, never back into a cycle — cuts the plan into **stages**. The cut always falls at a shuffle: a stage is the longest run of work that can be pipelined without moving data between executors. Everything up to a shuffle goes in one stage; the shuffle ends it; what comes after starts a new one. A stage that reads a shuffle cannot begin until the stage that writes it has finished, which is why stage boundaries are also the points where a job can stall.

A stage is executed as a set of **tasks** — one per partition of the stage's input, each doing identical work on its own slice of the data. A task is the unit Spark schedules onto an executor's task slot; one task occupies one core for its duration. The number of tasks in a stage is therefore the number of partitions, and it sets how much parallelism the stage can use: 8 tasks on a cluster with 200 free slots leaves 192 idle.

Figure 1.3 shows the specimen's shape. The planner chose a sort-merge join because `dim_product`, a catalog of millions of products, is about 170 MB: far over the 10 MB broadcast threshold, so both inputs are shuffled so that rows with the same `product_id` land on the same executor (Chapter 5). The aggregate groups by a different key and the `ORDER BY` needs a global order, so two more shuffles follow. Four exchanges give five stages:

- **Stage 0** — scan the March partitions of `orders`, apply the timestamp filter, and write shuffle files keyed by `product_id`. One task per input file split: roughly 180 on the reference cluster, 7 on the lab's one-week slice.
- **Stage 1** — scan `dim_product` and write shuffle files keyed by `product_id`. A few dozen tasks (14 on the lab).

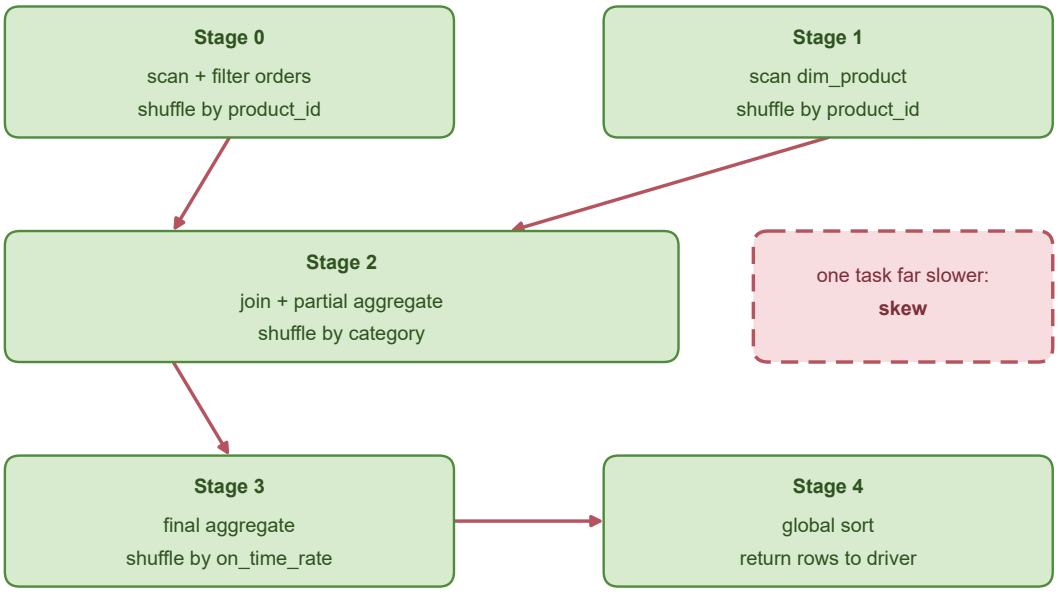


Figure 1.3: The specimen as five stages, one per side of each shuffle. Stages 0 and 1 scan `orders` and `dim_product` and shuffle by `product_id`; stage 2 joins and partially aggregates; stages 3 and 4 finish the aggregate and sort. Rose arrows are the four shuffles. One task in stage 2 runs far longer than its peers — the first sighting of skew.

- **Stage 2** — fetch both sides' shuffle files, merge them on `product_id`, join, and compute a partial aggregate per category. 200 tasks, because `spark.sql.shuffle.partitions` defaults to 200 (Chapter 4); AQE merged them to 40 on the lab (Chapter 6).
- **Stage 3** — fetch the per-category partial aggregates and finish the aggregate. One task on the lab: twenty rows is little work.
- **Stage 4** — sort the twenty rows and return them.

Stage	Reads	Tasks (lab)	Writes / does
0	`orders`, March partitions, filtered	7	shuffle files, keyed by `product_id`
1	`dim_product`	14	shuffle files, keyed by `product_id`
2	stage 0 + stage 1 shuffle output	40	sort-merge join, partial aggregate
3	stage 2 shuffle output	1	final aggregate
4	stage 3 shuffle output	1	global sort

Table 1.1: The specimen at a glance, matching Figure 1.3. Task counts were captured on the lab of Section A.3, one week of `orders`; the reference cluster's counts are larger.

The `ORDER BY` on twenty rows is cheap, but it is still a stage: a global sort needs a range-partitioned shuffle, and Spark runs a small extra sampling job to choose the range boundaries.

1.5 Narrow versus wide dependencies

The reason a shuffle ends a stage is the kind of dependency it creates between partitions (Figure 1.4).

A **narrow** dependency is one where each output partition depends on a bounded, known set of input partitions — usually exactly one. `filter`, `select`, and `map` are narrow: partition 3 of the output is partition 3 of the input with a function applied. Narrow operations chain together with no data movement, which is why an entire run of them collapses into a single stage and executes as one pipelined pass per partition.

A **wide** dependency is one where an output partition can depend on *every* input partition. `groupBy`, `join` (unless one side is broadcast), `distinct`, `repartition`, and window functions are wide: to put all rows for `product_id = 47` in one place, Spark must look at every input partition, because any of them might hold such a row. That redistribution is the shuffle, and it is the wall between stages.

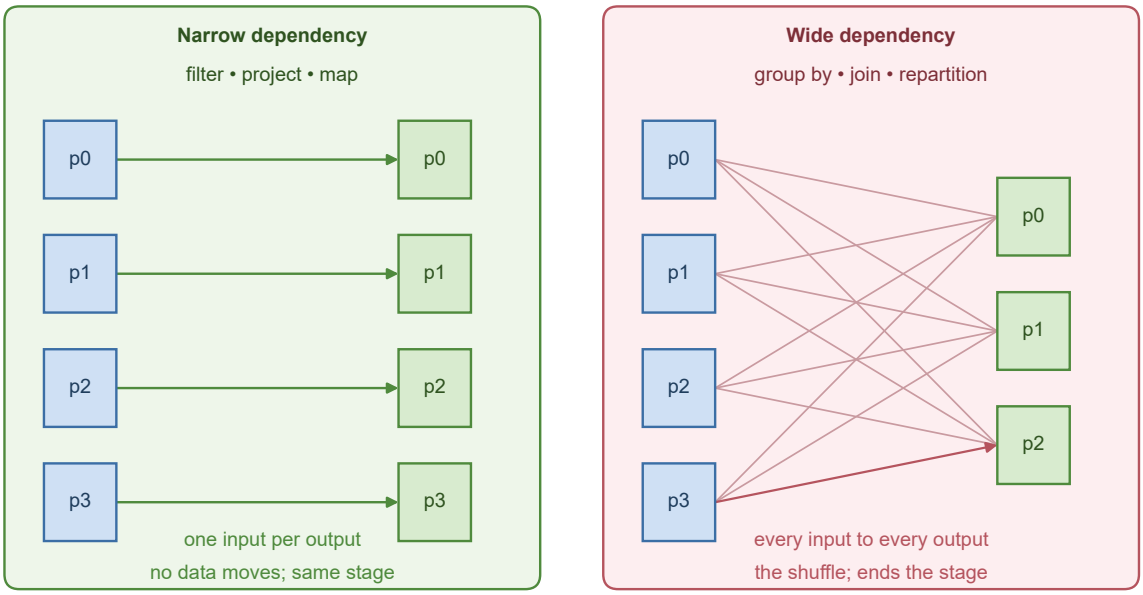


Figure 1.4: A narrow dependency: each output partition is built from one input partition, so the work pipelines in place. A wide dependency: each output partition draws from every input partition, so the data must be redistributed — a shuffle — and the stage ends.

Operation	Dependency	Why
`filter`, `select`, `withColumn`	narrow	one input partition maps to one output partition
`union`	narrow	partitions are concatenated, no keys compared
`coalesce(n)` (fewer partitions)	narrow	merges adjacent partitions without moving keys
`groupBy`, `join` (no broadcast), window functions	wide	equal keys from anywhere must co-locate
`distinct`	wide	compiles to an aggregation
`repartition(n)`, `orderBy` (global)	wide	explicit or ordering-driven redistribution

Table 1.2: Narrow versus wide, by operation. `coalesce` is the one surprise: it looks like `repartition` but avoids a shuffle by only ever merging partitions, never splitting or reshuffling keys (Section 7.5).

This is the single most useful idea in the book: **cost lives at stage boundaries**. Narrow work is cheap and scales cleanly. Wide work writes data to disk, moves it over the network, and re-reads it, and it forces the whole cluster to wait at a barrier. Almost every technique in Chapter 4 through Chapter 8 is about having fewer wide dependencies, making the ones you keep move less data, or making them move it more evenly.

1.6 Reading the UI for the first time

The Spark UI — served by the driver, on port 4040 by default, and preserved after the fact by the history server (Section 15.3) — has a tab for each level of the model.

SQL / DataFrame The query as a plan tree, annotated with the number of rows and the time spent at each operator after execution. This is where you confirm which join strategy ran and count the *Exchange* nodes; each Exchange is one shuffle.

Jobs One row per job. With AQE the specimen shows several — one per query stage — all belonging to one SQL query.

Stages The stages of each job, with a timeline and, for every stage, a *Summary Metrics* table: the minimum, 25th percentile, median, 75th percentile, and maximum across the stage's tasks for duration, shuffle read size, records, spill, and GC time.

Executors One row per executor: task slots, memory used, disk spilled, and total GC time. A cluster where one executor shows far more GC than the others is usually holding skewed data.

Storage Cached datasets and how much of each is actually resident.

For the specimen, open the Stages tab and look at stage 2's Summary Metrics. The median task duration is a few seconds; the maximum is close to a minute. One task is doing a large multiple of the work of the typical task. That is *data skew*: one `product_id` — a heavily-discounted item that went briefly viral — holds far more orders than the others, so the task that got that product's partition runs alone while the rest of the stage has finished. You cannot fix it yet. Chapter 6 and Chapter 5 will. But you can now *see* it, and seeing it is the prerequisite for everything else.

> **Tip** Four numbers tell you most of what is wrong with a slow stage: max versus median *task duration* (skew), *shuffle read/write* size (how much data the wall is moving), *spill* (not enough execution memory, Chapter 3), and *GC time* (too much, Section 3.4). Check them in that order.

1.7 Summary

You can now describe what happens when a query runs. The *driver* plans it and coordinates; *executors* run *tasks*, one per core, on partitions of the data; the *cluster manager* (Kubernetes here) provides the containers. An action triggers a *job* (under AQE, several small ones per query); the plan is cut into *stages* at every shuffle; a stage runs one task per partition. A *narrow* dependency keeps work in place; a *wide* dependency forces a shuffle and ends the stage, and that is where a query spends most of its time. In the Spark UI you can find the job, its stages, and their tasks, identify a stage boundary as a shuffle from the Exchange nodes in the SQL tab, and spot an uneven stage from the gap between its maximum and median task time.

The specimen still takes four minutes. Chapter 2 opens up how Spark decided on this plan; Chapter 4 onward is how you change it.

Exercises

1. Run two queries against `orders`: `SELECT count(*) FROM orders` and `SELECT product_id, count(*) FROM orders GROUP BY product_id`. Open the Stages tab for each. Which produced a second stage, and why?

2. In the SQL tab for the specimen query, count the *Exchange* nodes. Match each one to a stage boundary in Figure 1.3.

3. In stage 2's Summary Metrics, record the median and maximum task duration and the maximum shuffle-read size. Which of the four numbers from the tip is the outlier?

Chapter 2

From SQL to Execution: the Catalyst Pipeline

A SQL string does not run. It is parsed into a tree, resolved against the catalog, rewritten by rules, turned into a physical plan by choosing strategies, and compiled to JVM bytecode. Every change you can make to a query's performance is a change to one of those stages. This chapter traces the specimen query from Chapter 1 through all of them and teaches you to read `EXPLAIN FORMATTED` well enough to predict the physical plan before you run it.

2.1 The pipeline, overview

Catalyst is the optimizer inside Spark SQL. It is a library for building and rewriting trees, plus a fixed sequence of passes that turn a query into an executable plan (Figure 2.1). Every DataFrame operation and every SQL statement goes through the same pipeline; the DataFrame API is just a second front end that builds the same tree the SQL parser does.

> **Analogy** Recall the kitchen from Section 1.2. The order goes through the same five stages there as a query does here. A waiter writes the spoken order onto a kitchen

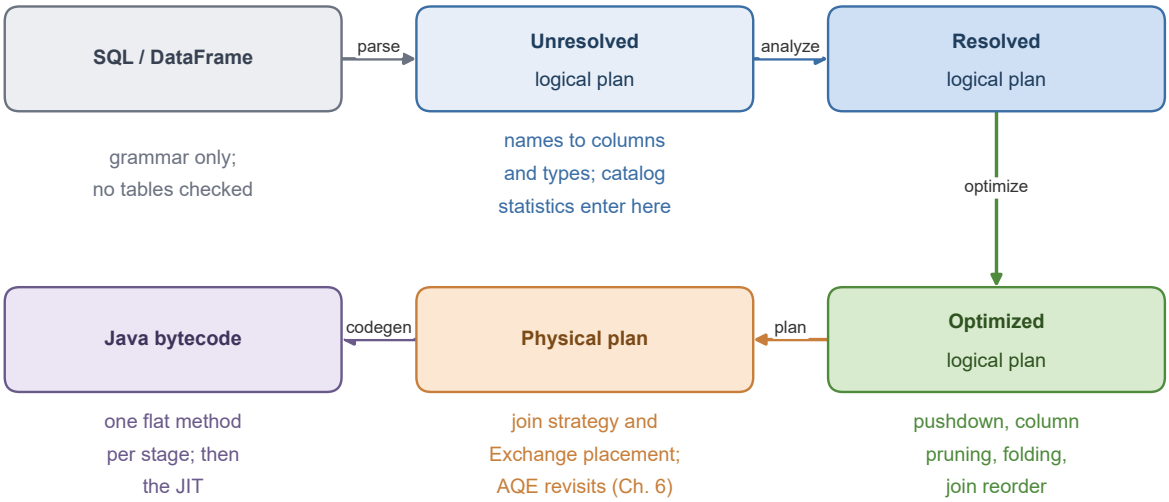


Figure 2.1: The compilation pipeline. Parsing produces a tree that is pure syntax. Analysis resolves names against the catalog. Logical optimization rewrites the tree with rules. Physical planning picks concrete operators and shuffle points. Code generation emits Java that the JIT compiles. The side notes name what each stage decides.

ticket in shorthand, without checking whether the kitchen actually has those ingredients tonight — that is *parsing*: pure syntax. The expediter then checks the ticket against the pantry and the menu, catching an order for a dish that is not served or an ingredient that is out — that is *analysis*, against the catalog. The head chef mentally simplifies the prep — "we're already chopping onions for table 4, chop extra for this ticket too" — without changing what gets served: that is *logical optimization*. Stations and techniques are then assigned — grill or oven, this cook or that one — based on what is actually fastest tonight: that is *physical planning*. And a dish the kitchen has made a thousand times gets cooked by hands that no longer think through each step: that is *whole-stage code generation* (Section 2.6).

In the kitchen	In Catalyst	What changes
waiter writes the ticket	parse	text → syntax tree; nothing checked
expediter checks the pantry & menu	analyze	names resolved against the catalog
chef simplifies the prep	logical optimization	rule-based rewrites (Section 2.3)
stations & techniques assigned	physical planning	operators & shuffle points chosen
practiced hands, no thinking	whole-stage codegen	generated Java, then the JIT

Table 2.1: The analogy above, mapped exactly onto the pipeline in Figure 2.1.

Four representations matter, and you can print all of them (Section 2.7):

1. the **unresolved logical plan** — the parse tree; relations and columns are just names.
2. the **resolved (analyzed) logical plan** — names bound to real tables and typed columns.
3. the **optimized logical plan** — the same query, rewritten to do less work.
4. the **physical plan** — a tree of executable operators, with the shuffles made explicit as *Exchange* nodes.

The physical plan is what Chapter 1 called "the plan." Reading it, and knowing which pass produced each part of it, is what lets you look at a query and say in advance how many stages it will have and where they will stall.

2.2 Unresolved to resolved: the catalog

The parser only checks grammar. `SELECT p.category FROM orders o JOIN dim_product p` … parses whether or not those tables exist; the result is a tree whose relation and attribute nodes are still marked *unresolved* — `orders` and `p.category` are held as names, not yet bound to anything.

The *analyzer* resolves them. It asks the **catalog** what `orders` and `dim_product` are, gets back their schemas, and replaces each unresolved node with a concrete one:

`category` becomes a `string` column belonging to `dim_product`, `delivery_days` becomes an `int` on `orders`, the `JOIN` condition is type-checked, and `AVG(…)` is bound to the aggregate function. If a name does not resolve, this is where the query fails — before any work is scheduled.

For Iceberg tables the catalog is a Spark *catalog plugin* (Chapter 19); Chapter 10 covers what it stores and how commits move through it. What matters here is that the catalog is also where *table statistics* come from — row counts, column value ranges, distinct counts. Spark's cost-based optimizer uses them to order joins and to decide what is small enough to broadcast (Chapter 5). An Iceberg table exposes a total file size and row count from its metadata, but not per-column statistics unless you have run `ANALYZE TABLE …COMPUTE STATISTICS`. This is why so many lakehouse joins pick the wrong strategy at planning time and have to be rescued by a hint or by AQE.

2.3 Logical optimization: the rules that matter

The optimizer applies *rules* — functions from a logical plan to a better logical plan — grouped into *batches* that run repeatedly until the plan stops changing. Spark ships around a hundred rules. A handful account for most of the speed-up, and all of them do the same thing: move work earlier, or delete it.

Predicate pushdown. A filter is pushed as close to the scan as the semantics allow, so fewer rows enter the rest of the plan. In the specimen the `order_ts` range moves into the `orders` scan, where it becomes a *partition filter* (Iceberg skips whole days) and a *pushed filter* (Parquet skips row groups). Chapter 7 is about keeping this rule working; a function or a `CAST` on the filtered column defeats it.

Column pruning. The scan reads only the columns the query references. The specimen touches `product_id`, `delivery_days`, `promised_days`, and `order_ts` from `orders` and `product_id`, `category` from `dim_product` — so a fifty-column `orders` table is read four columns wide. A `SELECT *` anywhere in a subquery turns this off.

Constant folding and simplification. Expressions with no runtime input are evaluated once at planning time; the `TIMESTAMP '2026-03-01 …'` literals are computed here, not per row. Boolean and null simplifications (`x AND true`, `x = x`, `IS NOT NULL` on a non-nullable column) are folded away.

Join reordering. With reliable statistics and cost-based optimization enabled (`spark.sql.cbo.enabled` and `spark.sql.cbo.joinReorder.enabled`), a multi-way join is reordered to keep intermediate results small — join the pair that produces the fewest rows first. Without statistics Spark keeps the written order, which is one more reason Iceberg tables benefit from `ANALYZE`.

> **Note** Most optimizer rules are *heuristic*: they rewrite the plan into a form that is almost always better, using no cost model. Only join selection and join reordering are cost-based, and only when statistics exist. When people say "the optimizer is smart," they usually mean the heuristic rules, which are reliable; the cost-based part is only as good as the catalog's numbers.

2.4 What the rules are worth, measured

The lab ran one query — a three-day filter, a join of `orders` to `dim_product`, a constant expression `(1 + 0.05 + 0.02)` in the aggregate — and printed its plan at every stage with `EXPLAIN EXTENDED`. Three rewrites are visible between the analyzed and the optimized plan:

```
== Analyzed Logical Plan ==
Aggregate [category], [category, sum(order_total * cast((1 + 0.05) + 0.02 as
double)) AS gross]
+- Filter (order_ts >= 2026-03-05 AND order_ts < 2026-03-08 AND (1 = 1))
   +- Join Inner, (product_id = product_id)
      :- RelationV2[order_id, order_ts, product_id, customer_id,
      promised_days,
                    delivery_days, order_total] lab.db.orders
      +- RelationV2[product_id, category, list_price, title, brand,
      description] lab.db.dim_product
```

```
== Optimized Logical Plan ==
Aggregate [category], [category, sum(order_total * 1.07) AS gross]
+- Join Inner, (product_id = product_id)
   :- Filter isnotnull(product_id)
   :  +- RelationV2[order_ts, product_id, order_total] lab.db.orders
   +- Filter isnotnull(product_id)
      +- RelationV2[product_id, category] lab.db.dim_product
```

Listing 2.1: The same query before and after optimization (Spark 3.5.2; lines shortened)

Read the differences line by line. The constant expression became the literal `1.07` (constant folding), and `1 = 1` vanished. Both scans shrank from seven and six columns to three and two (column pruning). The date predicate left the `Filter` — it was pushed into the scan, where Section 19.2 shows it is fully handled because three whole days align with the partitions — and the optimizer added `isnotnull(product_id)` filters, inferred from the inner-join condition, which cannot change the result and let the scan discard null keys early.

To price each rule, the lab re-ran the query with one rule excluded at a time through `spark.sql.optimizer.excludedRules` (Figure 2.2). The result is identical in every run; only the work changes.

Two cautions from the experiment. Excluding the Iceberg scan-pushdown rule by name changed nothing — bytes and rows were identical to the baseline — so that rule is not removable this way, and its effect is the one measured in Figure 19.2 instead. And wall-clock times of 1–2 seconds carry run-to-run noise of the order of the differences between the constant-folding and baseline bars; the byte and row counts are the reliable column.

2.5 Physical planning and strategies

The optimized *logical* plan still says only *what* to compute. The planner turns each logical operator into one or more *physical* operators by applying *strategies*:

- a logical `Join` becomes `BroadcastHashJoin`, `ShuffledHashJoin`, or `SortMergeJoin`, chosen from the estimated sizes of its inputs and `spark.sql.autoBroadcastJ`

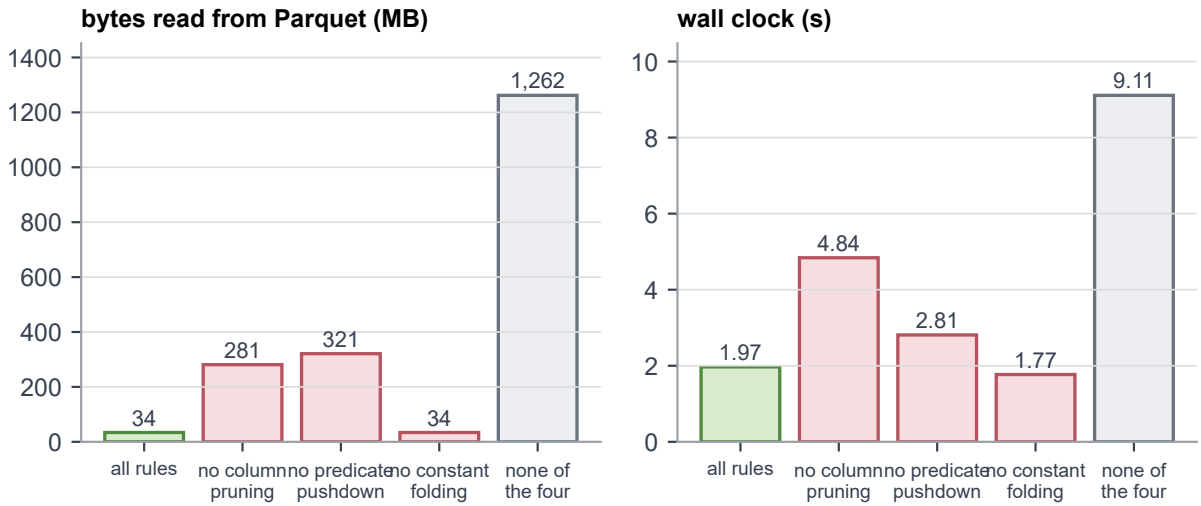


Figure 2.2: Bytes read and wall clock with one optimizer rule excluded. Without column pruning the scan read 8× the bytes (281 against 34 MB); without predicate pushdown it read 321 MB and 32 million rows instead of 4.9 million; with all three rules excluded together, 1,262 MB and 9.1 s against 34 MB and 2.0 s. Excluding constant folding changed neither bytes nor time here: the folded expression was cheap next to the scan.

`oinThreshold` (Chapter 5).

- a logical `Aggregate` becomes a partial `HashAggregate` on each partition, an `Exchange` to bring equal keys together, and a final `HashAggregate`.
- a logical `Sort` that needs a global order becomes a range-partitioning `Exchange` followed by a local `Sort`.

The two-phase aggregate is worth tracing through with actual row counts, because "partial then final" only pays off if the partial step actually shrinks the data. Say the specimen's `orders` scan is split across 200 partitions, and category `electronics` has rows scattered across all of them. The *partial* `HashAggregate` runs once per partition, independently, before any data moves: partition 17 might hold 40,000 `electronics` rows and collapse them to a single `(electronics, count=40000, sum=…)` row; partition 84 does the same for its own share. Every partition now emits at most one row per category it happened to contain — typically a few dozen rows instead of tens of thousands. The `Exchange` then shuffles only *those* small partial rows by category, and the *final* `HashAggregate` combines the (now few hundred, not tens of millions) partial

results into one row per category. The shuffle moves kilobytes of pre-aggregated rows instead of gigabytes of raw ones — that reduction is the entire reason the planner inserts a partial stage at all rather than shuffling raw rows straight to one final aggregate.

The planner also inserts `Exchange` (shuffle) and `Sort` nodes wherever an operator's required input distribution is not already satisfied — and, crucially, *removes* them where it is. If two joins share a key and the data is already partitioned by it, the second join reuses the first's exchange. This is why the *number of Exchange nodes in the physical plan* is the single best predictor of a query's cost, and why Chapter 1 told you to count them in the UI.

For the specimen, the planner produces (captured on the lab of Section A.3): two scans, each followed by an `Exchange hashpartitioning(product_id, 200)`; a `SortMergeJoi` because `dim_product` is about 170 MB, far over the 10 MB broadcast threshold — Iceberg reports a table's size from its snapshot metadata, so the planner knows this without any column statistics; a partial `HashAggregate`; a second exchange, `hashpartitioning(categor 200)`, because the aggregate groups by a different key than the join, so the join's partitioning does not carry over; the final `HashAggregate`; and a fourth exchange, `rangepartitioning(on 200)`, for the global `ORDER BY` before the final `Sort`. Four exchanges, so five stages, as Figure 1.3 shows.

2.6 Whole-stage code generation

A physical plan could be executed by walking the operator tree and calling `next()` on each node once per row — the "Volcano" iterator model. For a chain of `Filter` → `Project` → `HashAggregate`, every row passes through three virtual method calls, and the boxing and the short-lived row objects between operators usually cost more than the actual filtering and arithmetic.

Spark does not do this. For a chain of operators within a stage it *generates Java source* for the whole chain — one method that scans, filters, projects, and feeds the aggregate in a tight loop, with no virtual call, no intermediate row object, and field access straight out of the packed binary row (Chapter 17). That method is compiled to bytecode and then by the JVM's JIT to native code.

In a printed plan you see this as a numbered marker. Operators fused into one generated function are wrapped: `*(2) HashAggregate`, `*(2) Project`, `*(2) Filter`. The `*(n)` means "part of generated stage `n`"; the number is just a sequential id, not an execution order. An operator *without* a star is a boundary where fusion stopped and Spark falls back to row-at-a-time.

A stage stops being generated for a handful of reasons, and a plan missing stars where you expect them is usually hitting one:

- an `Exchange` or `BroadcastExchange` — you cannot fuse operators across a network boundary, so codegen always restarts on the far side of a shuffle;
- a stage whose input or output schema has more columns than `spark.sql.codegen.maxFields` (100 by default);
- a generated method that would exceed the JVM's 64 KB bytecode limit for a single method (`spark.sql.codegen.hugeMethodLimit`); Spark logs `Found too long generated codes` and drops that stage to the interpreter;
- a compile failure in the generated Java, after which Spark silently falls back (`spark.sql.codegen.fallback`) — the only trace is a `WARN` in the executor log;
- an imperative (non-declarative) aggregate or UDAF. Spark's built-in aggregates (`SUM`, `AVG`, `COUNT`) are *declarative*: each is expressed as a small set of Catalyst expressions (an update expression, a merge expression) that the codegen framework can inline directly into the generated method, the same as any other expression. A UDAF written against the older imperative `Aggregator`/`UserDefinedAggregateFunction` API instead hands Spark opaque Scala/Java method bodies to call — there is no expression tree to inline, so Spark can only invoke those methods the ordinary virtual-call way, and fusion stops at that node.

A plan missing a star where you expected one confirms which of these fired. Compare a healthy plan, `*(2) HashAggregate(keys=[category], …)`, against the same query with an imperative UDAF substituted in for the `AVG`: the node becomes plain `HashAggregate(keys=[category], …)` with no leading `*`, and every operator that

would otherwise have fused with it in that stage loses its star too, since fusion is all-or-nothing for the chain that touches the un-fusable node.

`EXPLAIN CODEGEN` prints the generated Java for every whole-stage stage, which is how you confirm what actually got fused.

2.7 Reading `EXPLAIN`

Now that you know what the `*(n)` markers and `Exchange` nodes mean, here is how to read the rest of a plan systematically. `EXPLAIN` prints the plans without running the query. There are five modes, selected by a keyword in SQL or a `mode` argument to `df.explain(…)`; Table 2.2 lists them.

Mode	Shows
(default)	the physical plan only — the quick check for join strategy, shuffle count, codegen
`EXTENDED`	all four plans: parsed, analyzed, optimized logical, and physical — use to see what a rule did
`FORMATTED`	the physical plan as a compact tree *plus* a numbered list expanding each node's detail — the one to learn
`COST`	the optimized logical plan annotated with the planner's row-count and size *estimates* — use to see why it did not broadcast
`CODEGEN`	the physical plan plus the generated Java for each whole-stage stage

Table 2.2: The `EXPLAIN` modes. Start with the default for a quick check and move to `FORMATTED` when you need each node's detail.

`EXPLAIN FORMATTED` has two parts: a compact tree, then a numbered list where each node's details are spelled out. Read the tree **bottom-up** — the leaves are scans, the root is the final operator.

```
== Physical Plan ==
AdaptiveSparkPlan (17)
+- Sort (16)                          -- global ORDER BY
   +- Exchange (15)                   -- range(on_time_rate): shuffle #4
      +- HashAggregate (14)           -- final aggregate
```

```
         +- Exchange (13)            -- hash(category): shuffle #3
            +- HashAggregate (12)    -- partial aggregate
               +- Project (11)
                  +- SortMergeJoin Inner (10)   -- dim_product: too big
                     :- Sort (5)                -- local sort per input
                     :  +- Exchange (4)         -- hash(product_id): #1
                     :     +- Project (3)
                     :        +- Filter (2)
                     :           +- BatchScan lab.db.orders (1)
                     +- Sort (9)
                        +- Exchange (8)         -- hash(product_id): #2
                           +- Filter (7)
                              +- BatchScan lab.db.dim_product (6)

(1) BatchScan lab.db.orders
    Output [4]: [order_ts, product_id, promised_days, delivery_days]
    filters=order_ts IS NOT NULL, order_ts >= 1772323200000000,
            order_ts < 1772928000000000, product_id IS NOT NULL
```

Listing 2.2: `EXPLAIN FORMATTED` on the specimen, before it runs (trimmed)

The `*(n)` markers appear only after execution, in the final plan; the plan before it runs has none. The root is `AdaptiveSparkPlan` because AQE is on (Chapter 6).

The nodes you will meet most — a full catalogue is Appendix C:

Scan, at every leaf. `BatchScan` for an Iceberg or other DataSource V2 table (Spark's newer, pluggable table-source API, covered in Chapter 19 — Iceberg tables always go through it); `FileScan parquet` for a Hive-catalogued table; `InMemoryTableScan` for cached data. Its fields are where pruning is confirmed or denied (an Iceberg `BatchScan` prints its pushed predicates in one `filters=` list instead, as above): `PartitionFilters` (whole partitions skipped — directories on object storage), `PushedFilters` (row groups skipped inside the files that are read, using column min/max and bloom filters), `DataFilters` (everything, including the part evaluated row-by-row after reading), and `ReadSchema` (the columns actually read — should be only those the query needs). `Batched: true` means vectorized columnar reads are on.

Exchange, at every stage boundary. Each `Exchange` is one shuffle: serialize, write to local disk, transfer, re-read. Count them — it is the best single predictor of cost. `hashpartitioning(key, n)` co-locates equal keys (joins, `GROUP BY`); `rangepartitioning` produces globally ordered partitions (a global `ORDER BY`); `SinglePartition` funnels everything to one task (a global aggregate, `coalesce(1)`). The `ENSURE_REQUIREMENTS` tag means the planner inserted this exchange because a downstream operator needs that distribution. `BroadcastExchange` is a different move: it ships one small side whole to every executor, and it appears only below the build side of a `BroadcastHashJoin`.

Sort. `Sort [key ASC NULLS FIRST], false, 0` — the boolean is `true` for a global sort, `false` for a local per-partition one. A `SortMergeJoin` always has two local sorts, one per input, between the exchanges and the join.

> **Tip** The fast scan of any plan, in order: (1) the join operator — `SortMergeJoin` on a table you expected to be small means its size estimate is over the broadcast threshold (Chapter 5); (2) the number of `Exchange` nodes; (3) `PartitionFilters` non-empty on a partitioned table (Chapter 7); (4) `PushedFilters` non-empty; (5) `ReadSchema` listing only needed columns.

After the query runs, the SQL tab shows the same tree with *actual* row counts and, if AQE was active, the adapted plan rooted at `AdaptiveSparkPlan isFinalPlan=true` — which is the plan that really ran, not the one `EXPLAIN` printed beforehand (Chapter 6).

2.8 Summary

A query becomes an execution in fixed stages: parse to a syntax tree, resolve names and types against the catalog, rewrite the logical plan with mostly heuristic rules (pushdown, column pruning, folding; join reordering only with statistics), choose physical operators and shuffle points with strategies, and generate Java for each pipelined chain. The physical plan — readable with `EXPLAIN FORMATTED` — tells you the join strategy, the number of shuffles, and whether pruning and column selection are working, which together determine how many stages the query has and where it will spend its time. The recurring lakehouse failure is missing catalog statistics, which makes the cost-based decisions (join

strategy and order) unreliable until you `ANALYZE` or override.

Exercises

1. Run `EXPLAIN FORMATTED` on the specimen query. Confirm the two Exchange nodes, the `SortMergeJoin`, and a non-empty `PartitionFilters` on the `orders` scan.

2. Write the same result three ways: with an explicit subquery, with a CTE, and as one flat statement. Compare the *optimized* logical plans. Which differ, and which are identical?

3. Add `WHERE CAST(o.product_id AS STRING) = '47'` to the query and re-run `EXPLAIN FORMATTED`. What happened to `PushedFilters`?

Chapter 3

How an Executor Uses Memory and CPU

Spark runs on the Java Virtual Machine: the driver and every executor is a JVM process, and the engine's memory behaviour, its failure modes, and much of its internal design follow directly from how the JVM manages memory and runs code. This chapter builds the memory map of a single executor — which regions exist, which a query can use, and which are consumed no matter what — then adds the two costs layered on top of it: garbage collection and serialization. By the end you can size a pod from its configuration, explain a container kill that left no Java error, and read GC time in the UI. How execution and storage memory borrow from each other while a job runs is picked up alongside caching in Chapter 8; how to size these regions for a workload is Chapter 14.

3.1 The JVM in one page: bytecode and just-in-time compilation

Spark is written in Scala and runs on the Java Virtual Machine, and two features of that platform shape how it performs.

The first is *just-in-time compilation*. Scala compiles to JVM *bytecode*, a portable instruction set the JVM does not run directly. Instead it interprets bytecode at first and

watches which methods run often; a *hot* method is then compiled to native machine code by the JIT, with optimizations — inlining, loop unrolling, dead-code elimination, escape analysis — that depend on what the method actually did the first few thousand times it ran. Two consequences follow. A Spark process is slowest in its first seconds, before the JIT has warmed up, which is one reason interactive sessions keep a driver alive rather than starting one per query (Section 1.3) and why a micro-benchmark that runs a snippet once tells you almost nothing. And the JIT compiles *tight, predictable* code far better than code full of virtual calls and branches — which is exactly why Spark's execution engine generates a single flat Java method for a chain of operators (Section 2.6, Chapter 17) instead of walking an operator tree.

The lab measured the warm-up directly: the same filter-and-aggregate query run eight times in one fresh JVM took 6.5 s the first time and 0.8–1.6 s from the second run on (Figure 3.1) — a factor of about eight, with nothing changed but how much of the code the JIT had already compiled.

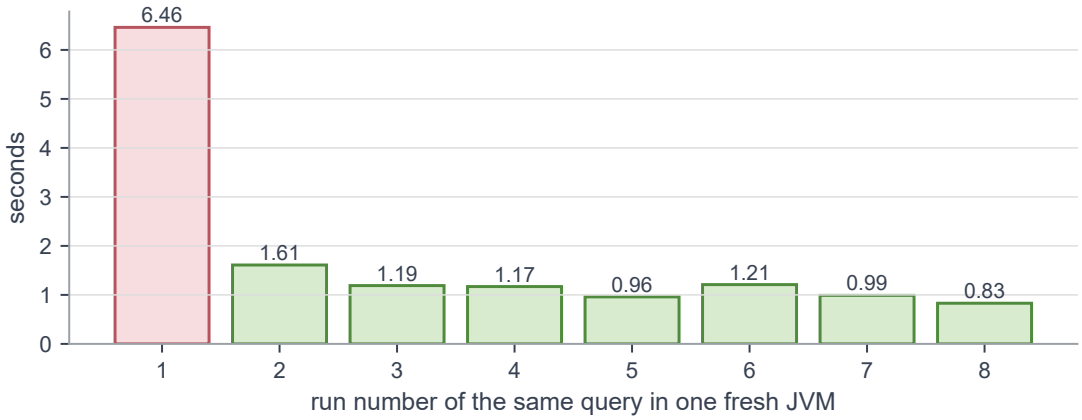


Figure 3.1: One query, eight consecutive runs in a fresh JVM. The first run pays for class loading, interpretation and compilation; later runs run compiled code.

The second is *automatic memory management*: the program allocates objects and never frees them, and a *garbage collector* reclaims what is no longer reachable. This is a convenience with a running cost, and for a data engine moving billions of rows the cost is large enough that Spark works hard to keep most of its data *out* of the managed heap entirely. The rest of this chapter is about where an executor's memory goes, and Section 3.4 about what the garbage collector does to it.

3.2 Heap layout and object representation

The JVM manages a single contiguous region of memory, the *heap*, from which every object a program allocates is drawn. A Spark executor is one JVM process, and the value of `spark.executor.memory` is precisely the maximum size of that heap. Everything a task does — decoding a Parquet page into rows, building a hash table for a join, accumulating a group-by result — competes for space in it, and when it fills, the task must either spill to disk or fail.

Two properties of the heap drive most of Spark's decisions about memory. The first is *per-object overhead*. Every object carries a header — a mark word and a pointer to its class — before any field is stored, and the allocator pads every object to an eight-byte boundary.

```
java.lang.Integer
  object header   12 bytes   (mark word + class pointer)
  int value        4 bytes
  ----------------------
  total           16 bytes   to carry 4 bytes of data
```

Listing 3.1: Cost of a boxed integer on a 64-bit JVM

Measured with Spark's `SizeEstimator`, one million integers cost 4.0 MB as an `int[]` and 20.6 MB as a `List<Integer>`: 5.2 times more (left panel of Figure 3.5). A ten-character string costs 56 bytes for 10 bytes of text. A row represented as an array of boxed Java objects (an `Integer`, a `Double`, a `String` with its own header and backing character array) can be several times larger in the heap than the same row encoded as raw bytes. This is the amplification you see when a compressed Parquet file expands to many times its on-disk size once decoded into Java objects, and it is why Spark's execution engine stores rows off the object heap in a packed binary format rather than as JVM objects (Chapter 17).

The second property is that the heap is *managed*: the program never frees memory explicitly, and reclamation is the job of the garbage collector (Section 3.4). Spark does not hand the whole heap to a query; it subdivides it into regions with different owners and different rules, described next.

3.3 The executor memory model

Configuring Spark's memory means deciding how one executor process divides the memory it is allowed to use. That process holds three separately configured pools, and confusing them is the most common source of both wasted memory and unexplained container kills.

> **Analogy** Picture the heap as an apartment building with a fixed number of floors (`spark.executor.memory`). One utility closet is permanently locked for building systems (*reserved*) — no tenant ever gets a key. A shared common floor (*unified memory*) is reconfigured on demand: some nights it hosts a big event that needs the whole floor (*execution*), other nights it is rented out as storage lockers (*storage*) — and management can always cancel a storage booking to make room for a scheduled event, but never the reverse. The remaining floors are private apartments (*user memory*): whatever a tenant does inside their own apartment is invisible to the building manager, and if a tenant packs it past capacity, nobody bails them out — the apartment simply collapses (`OutOfMemoryError`). A separate storage unit down the street, rented by some tenants but not others, is the *off-heap* pool; and the building's own infrastructure — elevators, wiring, hallway lighting (*overhead*) — draws power regardless of occupancy. The whole lot only fits inside the property line the landlord (the cluster manager) enforces: exceed it, and the building is condemned, tenants and all.

The heap. `spark.executor.memory` sets the maximum size of the JVM heap, the region every object the executor allocates comes from. Spark carves it into three parts:

- **Reserved memory** — a fixed 300 MB Spark holds back for its own internal objects. It is not configurable and never available to a query.
- **Unified memory** — by default 60% of (heap − 300 MB), set by `spark.memory.fraction`. This is the only pool Spark accounts for byte by byte and the only one that spills to disk gracefully when it runs out. It is shared between *execution memory* (the working memory of a running operator: hash tables for joins and aggregations, sort buffers, the shuffle write buffer) and *storage memory* (cached

DataFrame and RDD blocks, and broadcast data held on the executor). `spark.memory.storageFraction` (default 0.5) is a soft boundary between the two: either side may borrow the other's idle space, but execution can always evict borrowed cached blocks to reclaim its half, whereas storage cannot push execution below the border. How this plays out while a job runs is covered in Chapter 8.

- **User memory** — the remaining 40%. Everything Spark does not manage lives here: the objects your UDFs and RDD closures create, the structures a `mapPartitions` builds up. Spark does not track this pool and cannot spill it; overflowing it throws `OutOfMemoryError` and kills the executor.

Off-heap memory. When `spark.memory.offHeap.enabled` is set, `spark.memory.offHeap.size` adds a second unified-memory pool that lives outside the heap, addressed directly by the engine and invisible to the garbage collector (Section 3.4). Execution and storage draw on it in addition to their on-heap pool. It is not taken out of `spark.executor.memory`; it is additional memory the process reserves, and it must be counted into the container's limit.

Concretely: an executor already running `spark.executor.memory=8g` that adds `spark.memory.offHeap.enabled=true` and `spark.memory.offHeap.size=4g` does not shrink its 8 GB heap down to make room for those 4 GB — the 4 GB is layered on top, for a combined 12 GB of heap plus off-heap unified memory before overhead is even added. The container or pod limit has to be resized to cover all 12 GB (plus overhead) or the extra off-heap pool simply causes the kind of container kill Section 15.4 describes, with nothing in the JVM's own heap metrics hinting at why.

Overhead. `spark.executor.memoryOverhead` (default the larger of 384 MB and 10% of the heap) is memory the process needs that is neither heap nor off-heap pool: JVM thread stacks, class metadata (Metaspace), buffers for network transfer and compression codecs, and — for PySpark — the Python worker processes, which can each take hundreds of megabytes. It is not a safety margin; it is space that is actually consumed, and its default is frequently too small for PySpark or codec-heavy workloads, in which case the container is killed with no Spark-level error.

The pool, measured. The lab read the unified pool from Spark's memory manager for four heap sizes (Table 3.1); the formula $(\text{heap} - 300\ \text{MiB}) \times 0.6$ matches to the decimal. The reserved 300 MiB also sets a floor: asked to start with a 400 MB heap, Spark refused with `[INVALID_DRIVER_MEMORY] System memory 405471232 must be at least 471859200` — 450 MiB, which is 1.5 times the reserved amount.

Heap (`-Xmx`)	**Unified pool, measured (MiB)**	$(\textbf{heap} - 300) \times 0.6$ **(MiB)**
512 MiB	127.2	127.2
1 GiB	434.4	434.4
2 GiB	1048.8	1048.8
4 GiB	2277.6	2277.6

Table 3.1: The unified pool against the formula, on four heaps (Spark 3.5.2).

What the cluster manager enforces. The pod or container is sized as the sum of all three pools (Figure 3.2). If the process's real usage crosses that limit — most often because overhead was underestimated or user memory grew unbounded — the kernel kills it (Section 15.4). The heap merely filling up, by contrast, makes unified memory spill to disk and only throws when user memory or a single un-spillable allocation is the cause.

Per task, not per executor. Execution memory is shared equally across the tasks an executor runs at once, which is `spark.executor.cores`. A larger heap with proportionally more cores gives no single task more room: memory per task is the execution pool divided by the core count. A 4 GB unified pool on a 4-core executor gives each of its 4 concurrently-running tasks about 1 GB to build a hash table or sort buffer in; keep the same 4 GB pool but raise the executor to 8 cores, and each task's share drops to about 512 MB — more parallelism, but less room per task, on the very same heap. The number of cores per executor is therefore a memory decision as much as a parallelism one, and an executor with too many cores spills on work a narrower one finishes in memory. Figure 14.1 measured exactly this on twelve executor shapes (Section 14.1).

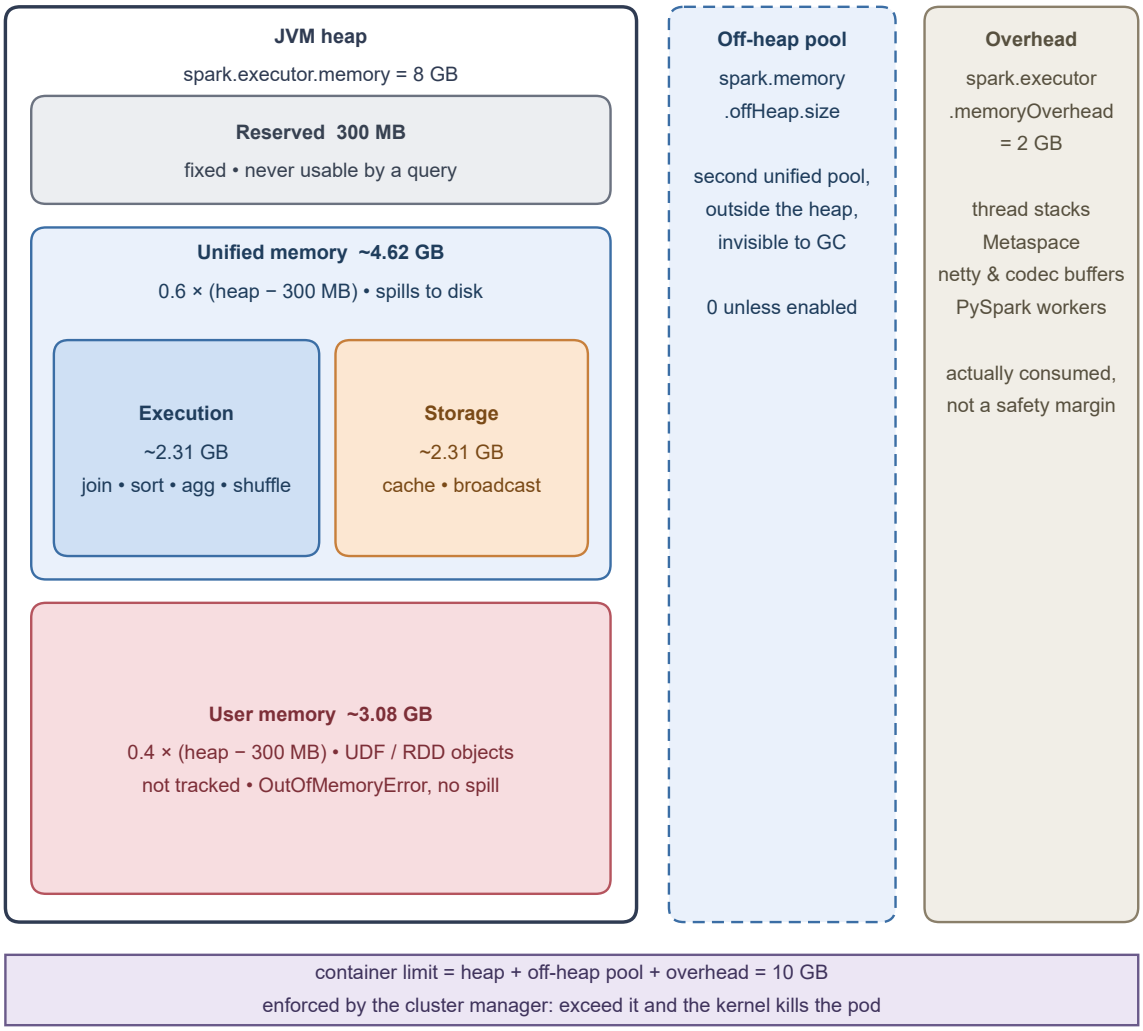


Figure 3.2: An executor's memory: the JVM heap (reserved / unified / user), the optional off-heap pool, and the overhead the process needs outside both. The cluster manager's container limit must cover all three. Numbers are worked for an 8 GB heap with 2 GB overhead.

3.4 Garbage collection: Parallel, G1, ZGC

Because the heap is managed, the JVM must periodically find objects that are no longer reachable and reclaim their space. This work is the *garbage collector* (GC), and for a data-processing workload it is never free: the collector competes with the application for CPU, and some of its phases are *stop-the-world*, meaning every application thread in the JVM is frozen until the phase completes. On an executor a long stop-the-world pause stalls every running task at once, and if it exceeds the heartbeat timeout the driver declares the executor lost and re-runs its work elsewhere (Section 15.4).

Collectors differ mainly in how they trade pause length against throughput and CPU cost.

> **Analogy** Picture three ways to clean a large office building overnight. Parallel GC is an all-lights-off deep clean: every employee is sent home, the whole crew cleans the whole floor at once, and the doors reopen only when they are finished — the fastest total clean, but nobody works while it happens. G1 cleans one wing at a time while the rest of the building stays open, closing only the wing being cleaned right now, briefly and often. ZGC has cleaners who tidy around the working employees continuously and almost invisibly, closing nothing at all — at the cost of moving more carefully and needing a bigger crew.

- **Parallel GC** uses multiple threads but collects the whole heap in one stop-the-world pause. It gives the highest raw throughput when pauses do not matter, but pause time grows with heap size, so a large executor can freeze for seconds at a time.

- **G1** (Garbage-First) divides the heap into equal-sized regions and collects a subset of them per cycle, doing most of its marking concurrently with the application. Pause times are shorter and roughly bounded regardless of heap size, at the cost of more bookkeeping and CPU. It is the default at the heap sizes a Spark executor typically runs and the collector most Spark tuning assumes.

- **ZGC** does essentially all of its work concurrently and keeps pauses under a millisecond even on very large heaps, at a higher constant CPU and memory overhead.

It is worth choosing only for executors with tens of gigabytes of heap where G1 pauses are still a problem.

Collector	Pause	Throughput / CPU	Best for
Parallel	long, grows with heap	highest throughput, lowest overhead	batch jobs where pauses don't matter
G1 (default)	short, roughly bounded	moderate bookkeeping cost	most Spark executors
ZGC	sub-millisecond	higher constant CPU & memory	very large heaps where G1 still pauses

Table 3.2: The three collectors, at a glance.

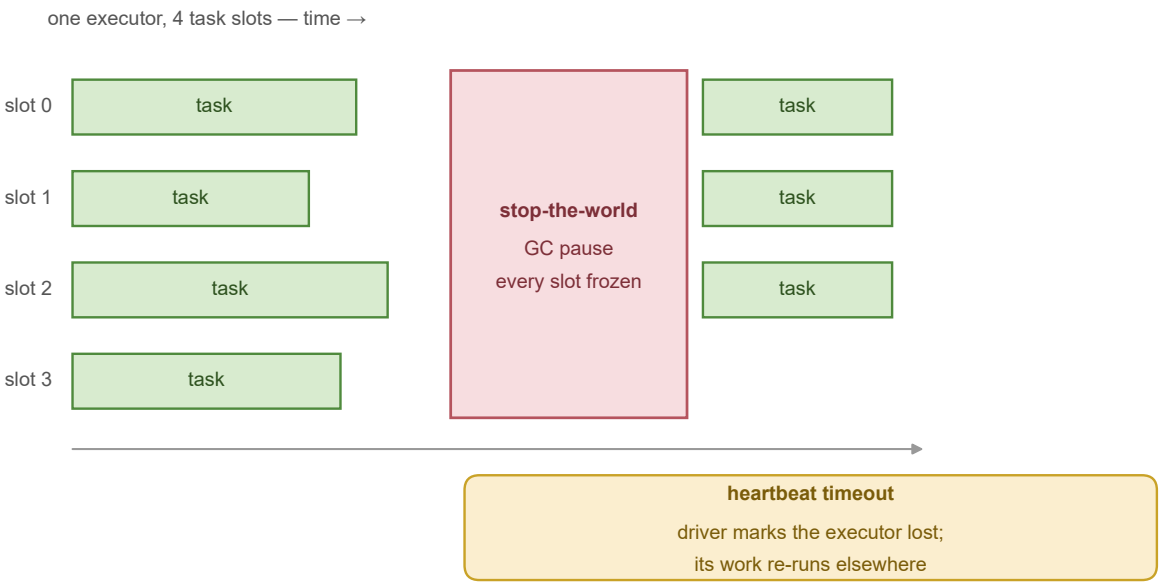


Figure 3.3: A stop-the-world pause freezes every task slot on the executor at once, not one at a time. If the pause outlasts the heartbeat interval, the driver concludes the executor is gone and re-runs its work elsewhere — so a GC problem shows up as lost executors and repeated stages, not just as slow ones.

The lab ran the same sort-heavy stage (4 GB heap, 8 cores) under each collector (Figure 3.4). Wall-clock time was within 1.3 seconds for G1 and Parallel (22.8 and 23.1 s) and 19% worse for ZGC (27.2 s). The pauses differ sharply: G1 paused 118 times for

574 ms in total, its longest pause 47 ms; Parallel paused 38 times for 3.4 s in total, its longest 835 ms. The *GC time* column of the task metrics shows something the pause log does not: 1.8 s for G1 but 25.4 s for Parallel, because the metric is accumulated *per task*, so one stop-the-world pause is counted once for every task that was running through it — 835 ms across eight slots is nearly seven seconds. (ZGC's 34.5 s is concurrent collector work charged to tasks; the lab's `-Xlog:gc` format for ZGC carries no pause lines, so its pause column is not available.) On this stage, at a 4 GB heap, G1's shorter pauses cost no throughput and ZGC cost throughput.

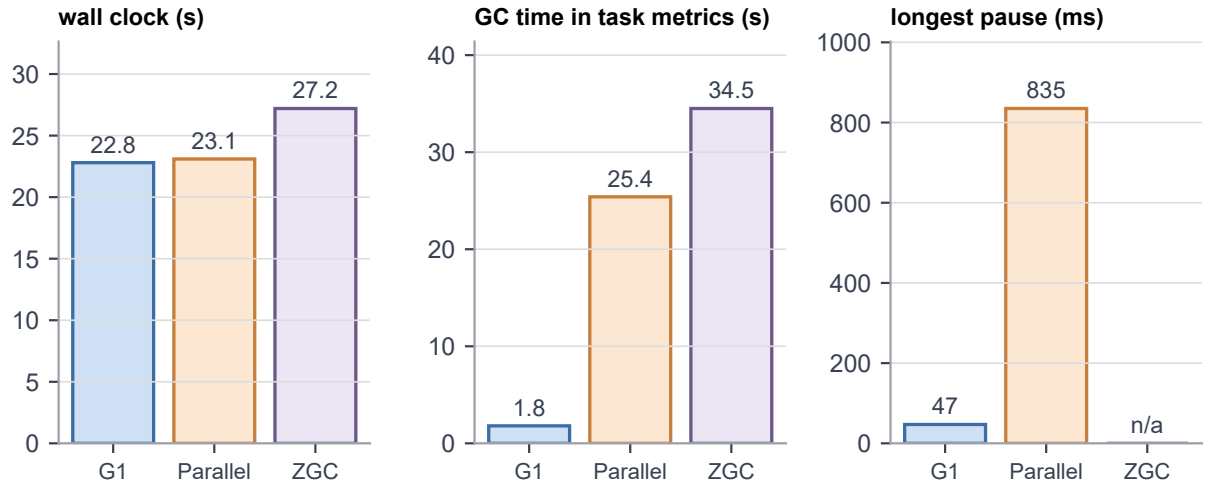


Figure 3.4: One stage under three collectors. Wall clock is close for G1 and Parallel; the longest pause and the per-task GC metric are not.

The generational hypothesis — that most objects die young — underlies all three: they collect a small "young" area cheaply and often, and the full-heap collection rarely. A workload that holds a large amount of long-lived data in cache violates that assumption and pushes the collector toward the expensive path, which is one more reason to cache deliberately (Chapter 8). Monitoring GC time is covered in Section 15.3; the executor JVM options that select and tune a collector are in Chapter 14.

3.5 Serialization and its cost

The engine keeps a join's hash table or a block of cached rows as one large off-heap byte buffer rather than millions of small Java objects, which is why that data is invisible to the garbage collector and costs nothing to collect (Section 3.3, Chapter 17). Getting data

into and out of that flat form is serialization.

Serialization is the conversion of an in-memory object graph into a flat sequence of bytes, and *deserialization* is its inverse. Spark serializes whenever data leaves a JVM: across the network during a shuffle, to disk when unified memory spills or a block is cached in serialized form, and when a broadcast variable is shipped to executors. The cost is both CPU and size.

The two options for arbitrary Java objects differ sharply. The built-in Java serializer writes the full class name and every field name and type alongside the values, repeated for every object, so most of the output is self-description rather than data. *Kryo* writes a compact encoding — a short numeric class identifier and the field values. The lab serialized 20,000 five-field records both ways (right panel of Figure 3.5): one record at a time, Java wrote 299 bytes per record and Kryo 42 — seven times smaller; as one array the gap shrinks to 58 against 41 bytes per record, because the class description is written once. The advantage is therefore largest for many small separately serialized objects, which is what a shuffle of RDD records is.

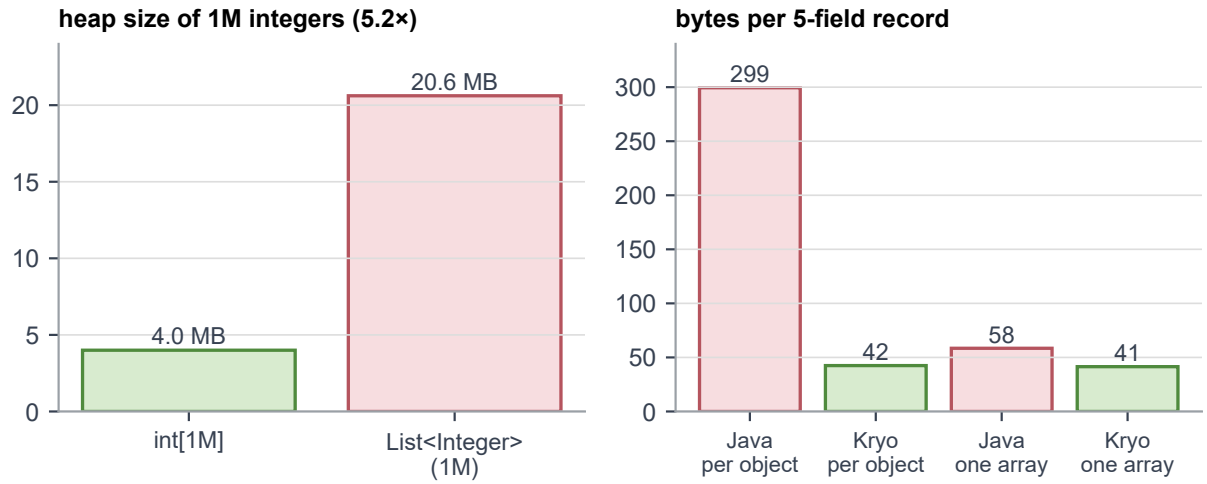


Figure 3.5: Left: one million integers as a primitive array and as boxed objects. Right: bytes per five-field record under Java and Kryo serialization, one record at a time and as one array.

The smaller output directly reduces shuffle volume, spill size and broadcast footprint (Chapter 14).

DataFrame and SQL workloads sidestep both serializers for most operations: rows already live in the engine's binary format (Chapter 17), so moving them is a memory

copy with no object graph to walk. The Java-versus-Kryo choice still governs RDD code, user-defined functions, and anything that puts arbitrary Java objects into a shuffle or a broadcast — the same user-memory objects that Section 3.3 noted Spark does not manage.

3.6 Summary

An executor is one JVM process, and its memory is three separately configured pools: the heap (`spark.executor.memory`), an optional off-heap pool, and overhead. The heap is carved into a fixed 300 MB reserve, *unified memory* (the only pool Spark accounts for and the only one that spills), and *user memory* (untracked, un-spillable, throws on overflow). The container limit the cluster manager enforces is the sum of all three; cross it and the process is killed with no Java error, whereas merely filling the heap makes unified memory spill. Execution memory is shared across an executor's cores, so cores-per-executor is a memory decision. On top of the map sit two costs: garbage collection, whose stop-the-world pauses freeze every task at once and can cost you an executor; and serialization, which Kryo makes several times cheaper than the Java default for RDD and UDF code, though DataFrame rows bypass it entirely. You can now size an executor from its workload and explain the two kinds of memory failure by which pool overflowed.

Exercises

1. Given a pod memory limit and a workload description, propose `spark.executor.memory`, `spark.executor.cores`, and `spark.executor.memoryOverhead`, and justify each number from the memory map in Figure 3.2.

2. An executor is killed by Kubernetes with `OOMKilled` and no Spark exception in the log. A second run fails with a Java `OutOfMemoryError` in a task. Which pool overflowed in each case, and which knob addresses each?

3. In the Executors tab, one executor shows 40% of task time in GC while the others show 5%. What is the most likely cause, and which chapter covers the fix?

Part II

Making Queries Fast

Chapter 4

The Shuffle

Every `JOIN`, `GROUP BY`, `DISTINCT`, window function, and `repartition` puts a wall between two stages: the shuffle. It writes every partition to disk, moves it across the network, and re-reads it. It is where most of a query's time goes, and it is the mechanism behind the wide dependency from Section 1.5. This chapter opens the wall: the write path, the read path, how to size shuffle partitions from data volume instead of the default 200, and how to find and remove exchanges the query does not need.

4.1 What a partition is: input partitions versus shuffle partitions

Spark never processes a table as one unit; it splits it into **partitions**, and one task handles exactly one partition (Section 1.4). Two different mechanisms create them, and mixing them up is a common source of confusion when reading a plan.

An **input partition** is created when a stage reads from storage. Spark decides how many by dividing the bytes to read by a target size: roughly `spark.sql.files.maxPartitionBytes` (128 MB) for Hive-catalogued files, or the split size Iceberg's planner reports from its manifests (`read.split.target-size`, also 128 MB by default, Chapter 12). One data file is not one partition either way — small files are combined and large ones split so that each input partition is close to the target size.

Note Both configs measure the file's *on-disk* (compressed) length, never the decompressed size — neither Spark nor Iceberg opens a file just to plan splits. For a compressed columnar file this is the same amplification as Section 3.2, one step earlier: a "128 MB" partition can decode to several times that in a task's execution memory, a quiet cause of spill unrelated to shuffle-partition sizing.
Splittability is a separate question from ratio: Parquet and ORC stay splittable at any codec, since Spark cuts at independently-decodable row-group or stripe boundaries. A `.gz`-compressed text/CSV/JSON file cannot — gzip's stream has no resume point, so the whole file is one partition regardless of size.

A **shuffle partition** is created when a wide operator redistributes data. Its count is set directly: by `spark.sql.shuffle.partitions` (default 200) for a join or aggregation, or by an explicit `repartition(n)`. Spark decides which shuffle partition a row belongs to with a hash:

```
partition_id = hash(key) mod num_shuffle_partitions
```

The same key always hashes to the same partition, which is exactly what a join or a `GROUP BY` needs — it is how rows that must be compared end up on the same executor. Note what this does *not* guarantee: a shuffle partition holds every row for potentially many different keys, and nothing about the hash keeps that mixture even. Two keys can collide into the same partition, and one key alone can dominate a partition if enough rows share it. That is the seed of data skew (Chapter 6); for now, the point is that shuffle-partition count is a number *you* choose, not one Spark infers from the data the way it infers input-partition count.

Analogy A shuffle is a citywide moving day: many offices, each organized by building, have to be reorganized by department into new, single-department buildings. Every current building (a map task) has its people first check which new building they've been assigned to (compute `partition_id`), sort themselves into that order, and box up their belongings by destination — each building keeping one manifest of what it packed and where each box is going (the one data file plus its index). The moving company's dispatcher (the map output tracker) is the only party who ever

> holds the complete picture of which old building currently has which department's boxes. When a new department building is ready (a reduce task), it does not go door to door itself — it asks the dispatcher, then sends trucks to every old building that packed something for it (the all-to-all fetch), and once everything arrives it unpacks and arranges the furniture in the new layout (the merge). If a truck can only carry so much in one trip, the extra boxes wait on the sidewalk until the next one — a spill to disk.

4.2 The shuffle write path

Figure 4.1 follows one shuffle from a map task in stage 0 of the specimen to a reduce task in stage 2. On the write side, every map task:

1. computes `partition_id = hash(product_id) mod 200` for each of its output rows;
2. sorts its rows by that partition id (Spark has used only this *sort-based* shuffle writer since 1.6 — there is no separate hash-shuffle path to choose between any more);
3. spills the sort to local disk if the shuffle write buffer (bounded by Section 3.3's execution memory) fills before the task's data does;
4. writes the result as *one* data file plus *one* index file for the whole task, with the index recording where each partition's byte range begins.

One file per map task, regardless of how many shuffle partitions there are, is why raising `spark.sql.shuffle.partitions` does not multiply the number of files on disk — it only changes how the one file's byte ranges are cut up. The driver's *map output tracker* records, for every map task, which executor holds that file and the offsets for each partition inside it.

4.3 The shuffle read path

A reduce task starts by asking the map output tracker for the location of its partition's data across *every* map task in the previous stage — for stage 2 of the specimen, that is

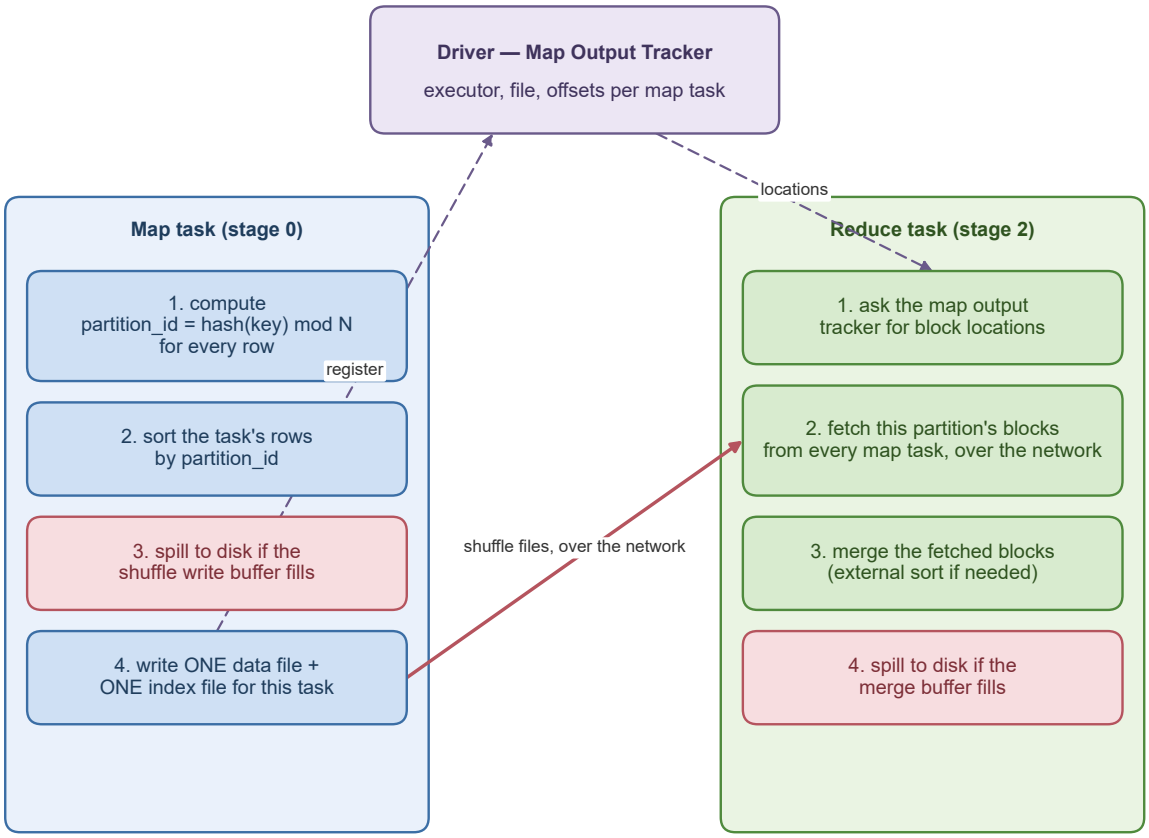


Figure 4.1: The shuffle write and read paths. The map task never talks to the reduce task directly: it registers its file's location with the driver's map output tracker, and the reduce task asks the tracker where to fetch from.

every one of stage 0's and stage 1's tasks. It then fetches those blocks over the network via the block transfer service, several fetches in flight at once up to `spark.reducer.maxSizeInFlight`, and merges them — an external (disk-backed) merge if the combined size does not fit in the task's share of execution memory.

This is the step that makes a shuffle expensive relative to a narrow operation: it is genuinely all-to-all. With M map tasks and R reduce tasks, up to $M \times R$ blocks are produced and each of the R reduce tasks may fetch from each of the M executors that produced them. For the specimen's stage 2, that is the roughly 180 map tasks from stage 0's `orders` scan (Figure 1.3) against 200 reduce tasks: up to $180 \times 200 = 36{,}000$ distinct map-output blocks, and in the worst case every one of the 200 reduce tasks opens a connection to every one of the executors that ran a stage-0 task — not 200 fetches, but up to 36,000 individual block transfers, which is why this step, not the CPU work in either stage, is usually the wall-clock bottleneck. Section 4.6 covers the mechanism Spark added specifically to make this fetch pattern cheaper at scale; the scheduler machinery behind both sides is Chapter 18.

4.4 Sizing shuffle partitions

Because the write path always produces the same one file per task, the only real lever on shuffle cost is *how big each shuffle partition ends up*, which is set by `spark.sql.shuffle.partitions` against the total volume being shuffled:

```
target partitions = total shuffle bytes / target task size
   (128-200 MB)
```

The default of 200 is not tuned to your data; it is just a constant. For a 100 GB shuffle it is badly wrong in one direction, as Figure 4.2 makes exact: 200 partitions puts 512 MB in every task, several times past the point where the task's share of execution memory holds it and it spills to disk. A query that shuffles only a few gigabytes has the opposite problem — 200 partitions of a few megabytes each, so the fixed per-task scheduling overhead dominates the actual work.

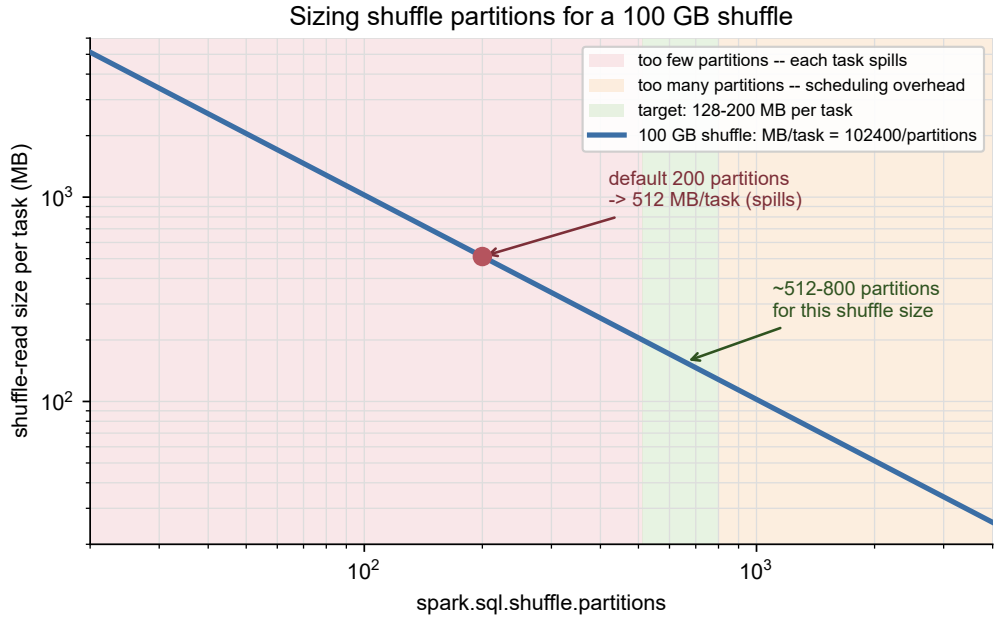


Figure 4.2: For a fixed shuffle size, size per task is exactly (shuffle bytes) / (partition count) — an exact relationship, not a rule of thumb. For a 100 GB shuffle the default 200 partitions puts 512 MB in every task, well past where it spills; 512–800 partitions lands each task in the 128–200 MB band.

> **Warning** If you set `spark.sql.shuffle.partitions` too low, Spark does *not* raise it for you. Partitions simply grow past the point where they fit in memory and spill, silently, until the job either finishes slowly or runs out of disk. There is no auto-correction on the low side.

The practical answer is to set the count generously as a ceiling and let Adaptive Query Execution shrink it back down once it has seen the real data size, rather than trying to predict the exact right number up front:

```
spark.sql.shuffle.partitions                        = 2000
spark.sql.adaptive.enabled                          = true
spark.sql.adaptive.coalescePartitions.enabled       = true
spark.sql.adaptive.coalescePartitions.parallelismFirst = false
spark.sql.adaptive.advisoryPartitionSizeInBytes = 134217728
   -- 128 MB, twice the 64 MB default
```

Over-partitioning is cheap to fix at runtime — AQE merges small adjacent partitions after the shuffle completes and it knows their true size (Chapter 6); under-partitioning,

which spills, is not something a later stage can undo. Set high and let AQE come down, not the other way around.

Partition count versus the cluster's task slots. The formula above sizes a partition in bytes; it says nothing about how many can run at once, which is set separately by the cluster's total task slots:

```
total task slots = executor count x spark.executor.cores
```

Partition count against *that* number, not against the shuffle size, is what Section 1.4 meant by "8 tasks on a cluster with 200 free slots leaves 192 idle" — too few partitions wastes capacity you are already paying for. The reverse failure is subtler: if partition count is only a little more than a multiple of the slot count — 41 partitions on 40 slots, say — the first wave uses every slot, and the second wave runs on a single core while the other 39 sit idle, costing nearly as much wall-clock time as the first wave for a fraction of the work. Prefer a partition count that divides the slot count cleanly over one that merely clears it.

Executor count is not a fixed input here either. Under dynamic allocation (Section 14.3), Spark requests more executors when it sees more pending tasks than running ones, so a higher partition count can grow the cluster rather than the other way around — the causal arrow runs from partitions to executors, not from a fixed executor count down to how many partitions to choose. What executor *shape* does constrain is partition *size*: execution memory per task is the execution pool divided by `spark.executor.cores` (Section 3.3), so more cores packed into the same executor leaves less room per task and calls for smaller partitions to avoid spill.

4.5 Shuffle compression

Shuffle data is compressed by default (`spark.shuffle.compress=true`) using `spark.io.compression.codec`, which defaults to `lz4`. That default is deliberate: shuffle bytes are written once and read back almost immediately by the very next stage, so *decompression speed* dominates the cost, not compression ratio. `lz4` and `snappy` both compress and decompress in a small fraction of the time `zstd` or `gzip` take, at a some-

what worse ratio. That trade-off is the opposite of Parquet storage on Iceberg, where a file is written once but may be read by hundreds of later queries, so a slower, denser codec such as `zstd` pays for itself (Chapter 11). Using the same codec for both because "it worked for storage" is a common way to leave shuffle performance on the table.

The lab measured the claim instead of assuming it. It shuffled four columns of all 30 million `orders` rows by `customer_id` under each codec and with compression off (Figure 4.3); the codec is a launch-time setting, and `spark.conf.set` at runtime is rejected.

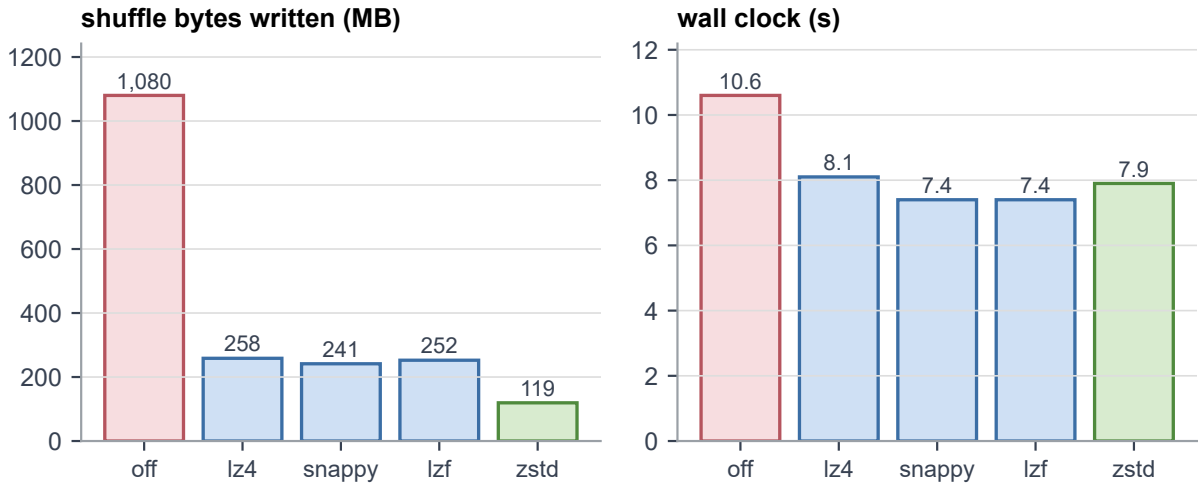


Figure 4.3: Shuffle bytes and wall clock by codec. Compression cut the bytes written from 1,080 MB to 119–259 MB and the time from 10.6 s to 7.4–8.1 s, on a local disk. `zstd` wrote half the bytes of `lz4` and was not slower.

Compression is clearly worth its CPU — 4.2× fewer bytes with `lz4` and a 24% shorter run, even before any network is involved. The more surprising result is that `zstd` was not slower than `lz4` here (7.9 against 8.1 s) while writing 9× fewer bytes than the uncompressed shuffle: on this stage the time went to other work, so the denser codec's extra CPU did not show. That does not make `zstd` the default recommendation — the stage was not CPU-saturated, and a CPU-bound one would see the codec's cost — but it weakens the folklore that only the fastest codec is acceptable for a shuffle. Where the network or disk is the bottleneck, bytes saved matter more than in this lab.

Codec	Ratio	Compress	Decompress	Best for
lz4	low–medium	very fast	very fast	shuffle (the default)
snappy	low–medium	very fast	very fast	shuffle, an alternative to lz4
zstd	high (9× in the lab)	medium	fast	Parquet storage; worth testing for shuffle
gzip	highest	slow	slow	archival only — avoid for shuffle

Table 4.1: Written once, read immediately (shuffle) wants decompression speed; written once, read many times (storage) can afford to spend more on ratio.

4.6 Push-based shuffle

The read path in Section 4.3 has a structural cost at scale: many small, random fetches, one per (map task, reduce partition) pair, which stresses the block transfer service and, if a map executor is lost, forces its tasks to be recomputed before the shuffle can finish. *Push-based shuffle* (stable since Spark 3.2) restructures this: after writing its file, each mapper also *pushes* its blocks to shuffle services, which merge blocks from many mappers that belong to the same reduce partition into a small number of larger merged files. A reduce task then does a handful of large sequential fetches instead of many small ones.

> **Note** Push-based shuffle needs an external, out-of-executor shuffle service capable of receiving and merging pushed blocks, which is a natural fit for YARN's long-lived Node Managers. Its support on Kubernetes is narrower and depends on running a compatible external shuffle service alongside your executors — confirm it is available on your cluster (Appendix B) before turning on `spark.shuffle.push.enabled`; without it, the setting has nothing to push to.

4.7 Finding and removing unnecessary exchanges

Every `Exchange` in a physical plan is a shuffle you are paying for (Section C.4), and not every one is necessary. Four patterns account for most avoidable ones.

A manual `repartition()` right before an Iceberg write. If the target table has `write.distribution-mode=hash` (Chapter 13), Iceberg already shuffles the write by partition key. Calling `repartition()` yourself first adds a *second*, usually redundant, shuffle. Let the table's distribution mode do it, or set the mode to `none` only when you are certain the data is already correctly distributed.

A `DISTINCT` on data that is already unique. `DISTINCT` compiles to an aggregation, which shuffles. If the upstream source already guarantees uniqueness — a primary key, or a prior `GROUP BY` on the same columns — the `DISTINCT` is pure cost.

An `ORDER BY` inside a subquery. Only the outermost `ORDER BY` in a query is guaranteed to survive to the result; SQL does not promise row order through an intermediate view or CTE. An inner sort still costs a range-partitioning exchange even though nothing downstream can rely on its effect.

Repeated aggregation on the same key. Two separate `GROUP BY product_id` computations in the same query — one for the on-time rate, one for total orders — shuffle twice on the same key when a single `GROUP BY` with two aggregate functions would shuffle once. Watch for `ReuseExchange` in the plan (Section C.4): the planner catches this only when the two exchanges are byte-for-byte the same partitioning, so a small difference (a different partition count, an added column) silently defeats it.

> **Tip** Run `EXPLAIN FORMATTED` and count the `Exchange` nodes before touching anything else. Every one you remove is a shuffle the job no longer pays for; every one you cannot explain is worth a second look.

4.8 Summary

A shuffle partition is not the same thing as an input partition: input partitions come from splitting storage by size, shuffle partitions come from a count you set and a hash that routes rows into it. The write path always produces one file per map task, indexed by partition; the read path is genuinely all-to-all, which is what makes a shuffle costly. Size `spark.sql.shuffle.partitions` from the actual volume being shuffled — 128–200 MB per task is the target — set it generously and let AQE coalesce down rather than risk under-partitioning, which spills and does not self-correct. `lz4` trades ratio for decompression speed, which is the right trade for data read once, immediately. Push-based shuffle turns many small fetches into few large ones where an external shuffle service supports it. And the cheapest shuffle is the one you remove: count the `Exchange` nodes and look for a redundant `repartition`, `DISTINCT`, inner sort, or duplicated aggregation before reaching for any other fix.

Exercises

1. From a stage's shuffle-read size in the UI, compute a target partition count for a 128 MB task size, then verify against the observed task sizes.

2. Run the specimen query with `spark.sql.shuffle.partitions` set to 4000 instead of 200. Confirm in the UI that stage 2 now has 4000 tasks, and that Figure 4.2's formula predicts their size correctly.

3. Find a query in your own workload with two `Exchange` nodes that partition on the same column. Check whether `ReuseExchange` fired; if not, find the difference between the two exchanges that prevented it.

Chapter 5

Joins

Spark has four ways to run a join and picks one from a size estimate. When the estimate is right, the choice is right. The estimate comes from table size, and a table's size says nothing about how much a filter will shrink it: Iceberg knows `dim_product` *is 170 MB but not that a* `category` *filter leaves 4 MB of it, so a join that should have broadcast a 4 MB slice shuffles a billion rows instead. This chapter covers the four strategies, how the planner chooses between them, and how to override it when you know better than the statistics do.*

Figure 5.1 shows the shape each strategy leaves in a plan, and Figure 5.2 contrasts the two ends of the cost range. The rest of this chapter explains why each shape exists and what it costs; the identifiers to spot them by are catalogued in Section C.6.

5.1 Broadcast hash join

If one side of a join is small enough, Spark skips shuffling the large side entirely: it collects the small side to the driver, packages it once, and ships a full copy to every executor (a `BroadcastExchange`), where it is built into an in-memory hash table keyed on the join column. Each task on the large side then probes that local hash table row by row — no shuffle, no sort, and the large side is read exactly once, in place.

This is the cheapest strategy whenever it applies, which is exactly why the planner tries it first. Two costs are easy to underestimate. First, the *driver* holds the whole broad-

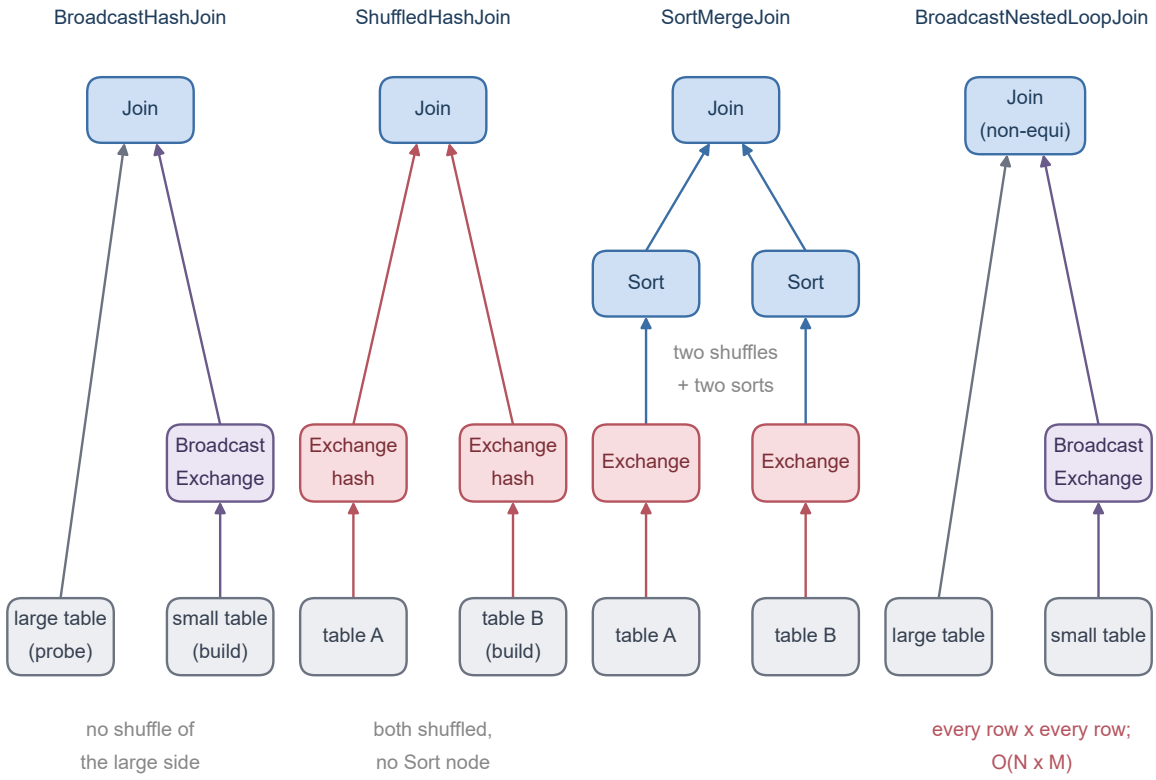


Figure 5.1: The four join strategies as plan shapes. Broadcast hash join shuffles nothing on the large side; shuffled hash join shuffles both sides but sorts neither; sort-merge join shuffles and sorts both; broadcast nested loop join broadcasts one side to evaluate a condition that is not an equality.

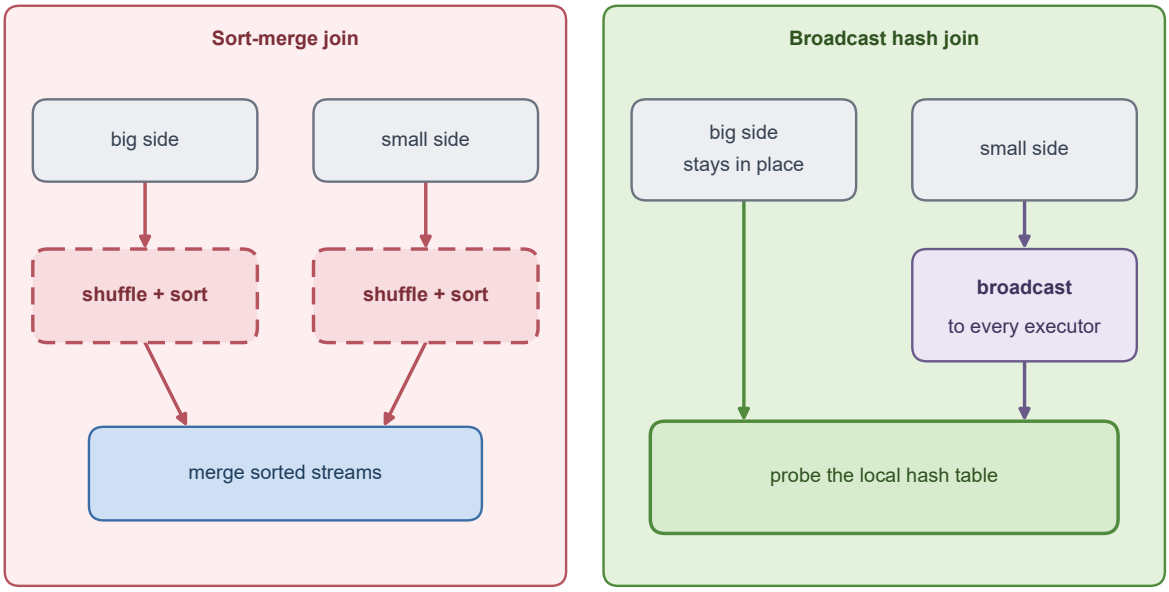


Figure 5.2: The two cheapest-to-compare strategies side by side. A sort-merge join shuffles and sorts both sides before merging; a broadcast hash join leaves the big side in place and ships the small side to every executor.

cast table in memory while assembling it, at roughly twice the table's serialized size; a broadcast large enough to hurt the driver is a self-inflicted wound distinct from anything happening on the executors. Second, the size used to decide broadcastability is the table's *on-disk* size, and a compressed columnar file routinely decodes to several times that once it is rows in memory (Section 3.2) — so a "100 MB" table can be genuinely 500 MB–1 GB once broadcast and decoded, which is why the planning-time threshold is conservative by design. Spark also enforces a hard ceiling (roughly 8 GB serialized, and a few hundred million rows) past which a table cannot be broadcast at all, regardless of the threshold.

Put numbers on it, from the lab (Section A.3). Filter `dim_product` to one category and 100 000 rows remain. Spark's own statistics for that side's shuffle read `3.8 MiB`; the broadcast relation built from those same rows measured `20.0 MiB` in memory, more than five times larger (Listing 6.4). A threshold compared against the first number is therefore a weak guard for the second, and the driver holds roughly two copies of that relation while it collects the rows and builds the payload to ship out. A dimension ten times larger inflates the same way, which is why the comfortable-looking 10 MB threshold still leaves real headroom before it threatens a driver's heap.

5.2 Shuffle hash join

When neither side is small enough to broadcast but one side is still meaningfully smaller than the other, Spark can shuffle both sides on the join key and build an in-memory hash table from the smaller (build) side on each executor, then probe it with the larger (probe) side — without sorting either. This is a `ShuffledHashJoin`: two shuffles, like sort-merge, but no `Sort` nodes, which makes it cheaper than sort-merge whenever the build side reliably fits in memory per partition.

That condition is also its weakness: unlike sort-merge, a shuffled hash join has no graceful way to spill an oversized build-side partition to disk. If a single partition's build side is larger than expected — most often because of skew — the task fails outright rather than slowing down. Spark's planner is correspondingly conservative about choosing it: by default it prefers sort-merge whenever both sides have sortable keys (`spark.sql.join.preferSortMergeJoin=true`), and only considers a shuffled hash join when that preference is turned off and the build side is estimated at at least three times smaller than the probe side. Concretely: with the preference off, an estimated 500 MB `dim_customer` against a 2 GB `orders` slice qualifies (2 GB is $4\times$ larger), and the plan shows `ShuffledHashJoin` with no `Sort` node on either input; the same 500 MB against only 1 GB of `orders` does not qualify ($2\times$, short of the $3\times$ bar), and the planner falls back to sort-merge instead, adding a `Sort` node to each side of the plan that was absent in the qualifying case.

5.3 Sort-merge join

Sort-merge join is the default for two large, roughly comparable inputs, and it is what the specimen query uses in Chapter 2, because `dim_product` (about 170 MB) is far over the broadcast threshold: both sides are shuffled on the join key, each partition is then sorted locally, and the join is produced by walking both sorted streams together, advancing whichever key is behind. It is the most expensive of the equi-join strategies — two shuffles and two sorts — but it is also the most robust: a sort spills to disk cleanly under memory pressure the way a hash table cannot, so it scales to inputs of any size on

either side.

5.4 Broadcast nested loop join

An equi-join needs a hash or a sort; a join whose condition is not an equality — `ON o.order_total > c.credit_limit`, for instance — needs neither, and Spark falls back to comparing every row of one side against every row of the other. If one side is small enough to broadcast, this is a `BroadcastNestedLoopJoin`: no shuffle, but $O(N \times M)$ comparisons inside each task. If neither side is small enough to broadcast, Spark falls all the way back to a `CartesianProduct` (Section C.6), which is rarely what you want on two large tables and is usually a sign that a join condition is missing.

> **Note** A full outer join cannot use broadcast hash join, even with an explicit hint: there is no way to detect an unmatched row on the broadcast side without scanning it, which the strategy is built to avoid. A `FULL OUTER` join falls back to sort-merge regardless of size.

5.5 How the planner chooses

For an equi-join, the planner tries strategies in a fixed order, stopping at the first one that applies (Figure 5.3):

1. an explicit join hint, if the query has one (Section 5.7);
2. **broadcast hash join**, if either side's estimated size is at or below `spark.sql.autoBroadcastJoinThreshold` (10 MB by default);
3. **sort-merge join**, if `spark.sql.join.preferSortMergeJoin` is `true` (the default) and the keys are sortable;
4. **shuffled hash join**, if that preference is off and the build side is estimated at least $3\times$ smaller than the probe side and fits per-partition memory;
5. **sort-merge join**, as the fallback for any remaining equi-join with sortable keys.

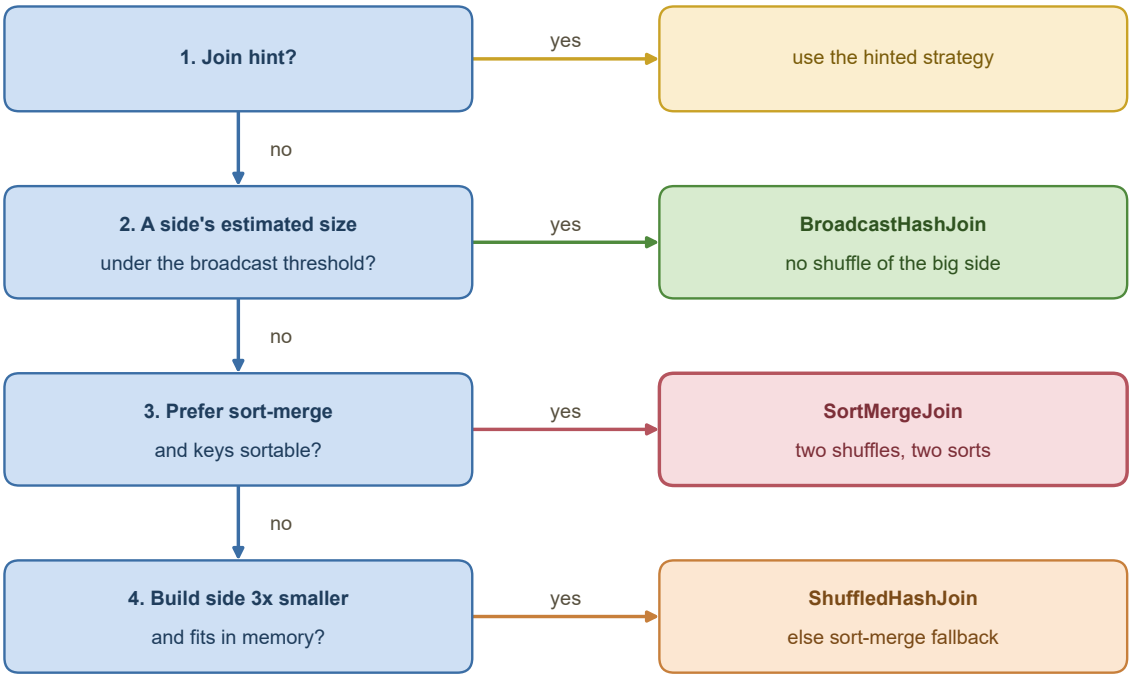


Figure 5.3: The planner stops at the first rule that applies. A hint wins outright; otherwise the estimated size decides, and sort-merge is the default for two large sides.

Every one of these decisions, except the explicit hint, is made from *estimated* sizes — from the catalog if statistics exist, or from on-disk file size if they do not (Section 2.2). This is the single most common way real joins go wrong: not a bug, but a plan built on a wrong number.

Trace one query through the ordered list to see how the choice actually falls out. For `orders JOIN dim_customer`, with `dim_customer` estimated at 500 MB and no hint present: step 1 does not apply (no hint); step 2 does not apply (500 MB is far past the 10 MB threshold); step 3 *does* apply — `preferSortMergeJoin` is `true` by default and both join keys are plain strings, sortable — so the planner stops right there and emits a sort-merge join. Step 4 (the shuffled-hash-join check) is never even evaluated, because the search stops at the first rule that matches; a shuffled hash join only becomes reachable at all if someone first turns `preferSortMergeJoin` off, which is why Section 5.2 described it as the strategy the planner reaches for last, not by default.

5.6 What the strategies cost, measured

The lab forced each equi-join strategy with a hint on three dimensions of different shape and joined each to the 30-million-row `orders` table, with AQE off and automatic broadcast disabled so that only the hint decides (Figure 5.4). The plan operator was checked in every run: each hint produced the operator it names.

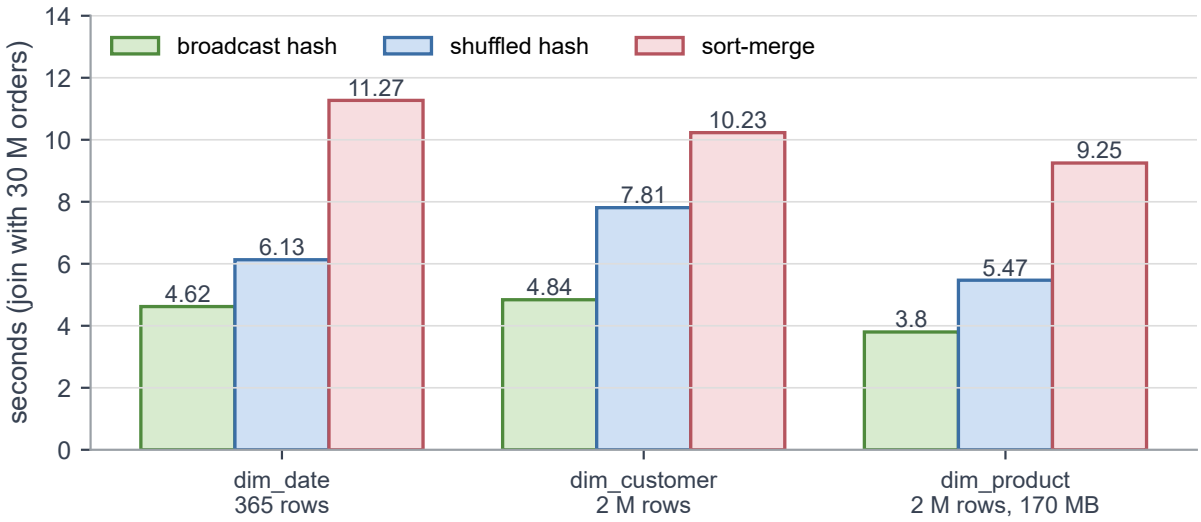


Figure 5.4: Seconds for the same join under each strategy. Broadcast was fastest on all three dimensions (3.8–4.8 s), shuffled hash in between (5.5–7.8 s) and sort-merge slowest (9.3–11.3 s). Each run has two exchanges for broadcast and three for the shuffled strategies.

Two readings. First, the ordering follows the work each strategy skips: broadcast avoids shuffling the large side; shuffled hash shuffles both sides but builds a hash table instead of sorting; sort-merge shuffles and sorts both. Second, broadcast won even for the 2-million-row, 170 MB `dim_product`, which the default planner would *not* broadcast (its estimate of 216 MB exceeds the 10 MB threshold, Table 19.2): the threshold is a safety rail against driver and executor memory, not the point where broadcast stops being faster. A non-equi condition leaves two choices (Section 5.4); with a `BROADCAST` hint the plan was a `BroadcastNestedLoopJoin` (8.1 s for 200,000 rows against 365), and a `MERGE` hint on the non-equi condition was ignored in favour of a `CartesianProduct`.

> **Warning** A join hint names the *alias* when the table is aliased. The first lab run wrote `BROADCAST(dim_date)` against `JOIN dim_date d`: no error, no warning, and a sort-merge join in every plan. If a hint seems ignored, read the operator in `EXPLAIN` before concluding the optimizer overruled it.

5.7 Forcing a strategy

When the estimate is wrong, a hint overrides it directly. The planner sees `dim_product` at its full 170 MB and will not broadcast it, but you know the `category` filter leaves a few megabytes:

```
SELECT /*+ BROADCAST(p) */ o.product_id, p.category
FROM orders o
JOIN (SELECT * FROM dim_product WHERE category = 'books') p
  ON o.product_id = p.product_id;
```

Listing 5.1: Forcing a broadcast of a filtered dimension

The hints, in priority order, are `BROADCAST` (or `BROADCASTJOIN`/`MAPJOIN`), `SHUFFLE_MERGE` (or `MERGE`) for sort-merge, and `SHUFFLE_HASH` for a shuffled hash join. A hint is an instruction, not a suggestion: Spark will honor it even when its own estimate disagrees, so a stale hint on a table that has since grown past a sane broadcast size is a real production hazard, not a theoretical one.

The more durable fix is to give the planner real numbers instead of overriding its guess:

```
ANALYZE TABLE db.dim_product COMPUTE STATISTICS FOR ALL COLUMNS
   ;
```

Listing 5.2: Giving the CBO real statistics

Iceberg tables carry file-level size and row counts in their metadata for free, so the planner's size estimate for an *unfiltered* table is accurate; `dim_product`'s 170 MB is why the specimen shuffled it in Chapter 2. What Iceberg does not carry is the column-level statistics the cost-based optimizer uses to estimate how much a *filter* shrinks a table, so a filtered scan is still estimated at full size. `ANALYZE` supplies those statistics, and running

it once after a table's data changes materially is usually cheaper, and safer over time, than hard-coding a hint. Whether it changes a particular join is something to measure, not assume.

> **Tip** Adaptive Query Execution catches many of these mistakes without any of the above: it re-evaluates a join's strategy *after* its inputs have shuffled, against their real post-shuffle size, and promotes a sort-merge join to a broadcast if the data turns out small enough (Chapter 6). Hints and `ANALYZE` matter most for the joins AQE cannot rescue — most often because the small side is broadcastable only before a shuffle changes its size, not after.

5.8 Storage-partitioned joins for bucketed Iceberg tables

Every strategy above pays the same structural cost sort-merge join pays most visibly: two shuffles, one per side, every single time the query runs, even though the join key never changes between runs. The next strategy exists to avoid paying that cost at query time at all, by doing the equivalent work once, up front, when the tables are written. Every strategy above assumes the two sides arrive unaligned and have to be shuffled into agreement. If both tables are already *bucketed* on the join key with the same bucket count (Chapter 11), that assumption is false: rows that must be compared are already co-located file by file, and Spark can join corresponding buckets directly with no exchange on either side — a *storage-partitioned join*. This turns the most expensive part of a sort-merge join, the two shuffles, into work done once at write time instead of on every query that joins the tables.

The condition is strict: both tables need the identical bucket transform and bucket count on the join column, and Spark needs `spark.sql.sources.v2.bucketing.enabled` set to consider it at all. It is a design decision made when the tables are created, not a query-time tuning knob — worth it specifically for a join pattern repeated often enough that paying the shuffle cost once, at write time, beats paying it on every read.

5.9 Summary

Four strategies, in the order the planner tries them: broadcast hash join (cheapest, but bounded by the small side's real decoded size, not its file size), sort-merge join (the robust default for two large sides), shuffled hash join (cheaper than sort-merge when it applies, but unable to spill), and broadcast nested loop or Cartesian for a non-equi condition. Every automatic choice rests on a size estimate, and Iceberg tables commonly lack the column statistics that estimate needs — `ANALYZE TABLE` and join hints exist specifically to correct that, and AQE corrects a real subset of it for free at runtime. Bucketing both sides of a frequent join on the same key removes the shuffle from the decision entirely.

Exercises

1. Given four table sizes and the broadcast threshold, predict each join's strategy. Then change one config value and predict which join flips.

2. Run the specimen query and read the join strategy. Then raise `spark.sql.autoBroadcastJoinThreshold` above `dim_product`'s size and run it again. Predict the number of `Exchange` nodes before and after, and check against the plan.

3. Write a join with a non-equi condition on two tables, neither of which is small. Confirm in `EXPLAIN FORMATTED` that it becomes a `CartesianProduct`, and rewrite it to avoid one.

Chapter 6

Adaptive Query Execution

The specimen's join stage shuffles into 200 partitions, a number the planner chose before it had seen a single byte. In the week of the viral product, one of those partitions holds far more than its share, and the stage finishes only when the task reading that one partition does. Raise `spark.sql.shuffle.partitions` *to 2000 for the other stages and the opposite problem appears: thousands of tasks, each moving a few megabytes. Adaptive Query Execution (AQE) fixes both by measuring after each shuffle and re-planning what has not yet run. This chapter covers what AQE re-decides, from what evidence, how to read the plan that results, and the places where it cannot help. Every plan and task count in it was captured on the lab described in Section A.3.*

6.1 Why runtime beats planning time

Every decision in Chapter 2 and Chapter 5 — how many shuffle partitions, which join strategy, whether to broadcast — is made from an *estimate*: a file size, a row count, a number the catalog may or may not have. The estimate is wrong in predictable ways. A filter's selectivity is guessed, a join's output cardinality is guessed from guesses, and a Parquet file's on-disk size says little about its decoded size (Section 3.2). The optimizer cannot do better, because the data has not moved yet.

Adaptive Query Execution is Spark inserting a measurement after every shuffle. Once a shuffle has run, Spark knows the exact size of every partition it produced — no estima-

tion error, no missing statistics. AQE uses that certainty to re-plan the stages that have not run. It has been on by default since Spark 3.2 (`spark.sql.adaptive.enabled`), and there is little reason to turn it off.

> **Analogy** Planning without AQE is plotting a road trip from a map: you pick the route once, before leaving, from the distances printed on it. AQE adds a checkpoint at every toll booth where you read the real traffic and may take a different exit. The analogy breaks in one place that matters: you can change only the road *after* the toll booth. The miles already driven, and the booths you decided to pass through, stay as they were.

That limit is the chapter's second theme. AQE cannot change a stage that has already run, and it cannot remove a shuffle that planning decided was necessary. It optimizes what happens after the measurement, not before it.

6.2 The feedback loop

Figure 6.1 shows the mechanism that all three optimizations share. Without AQE, Spark turns the physical plan into stages once, at the start, and runs them in order. With AQE, the driver cuts the plan at every `Exchange` into *query stages*: a query stage is a piece of the plan that is materialized (run to completion, its output written as shuffle files or collected for a broadcast) before anything downstream of it is planned in detail. The wrapper node that owns this loop is `AdaptiveSparkPlan`.

The loop has three steps. First, a query stage finishes and reports its statistics: the exact byte size of every shuffle partition, and the total rows and bytes it produced. Second, the driver substitutes those measured numbers for the estimates in the logical plan that remains, and runs the optimizer again over only that unfinished part. Third, it plans the physical operators for the next query stage from the result and launches it. Then the loop repeats, once per shuffle, until no exchange remains.

Three rewrites fall out of this one loop, and the rest of the chapter takes them in turn. Table 6.1 names each next to its planning-time counterpart: AQE does not add new kinds of decision; it revisits decisions Catalyst already made and gives them evidence.

Before a query runs, `EXPLAIN` shows the plan Spark *intends* to run. Listing 6.1 is the

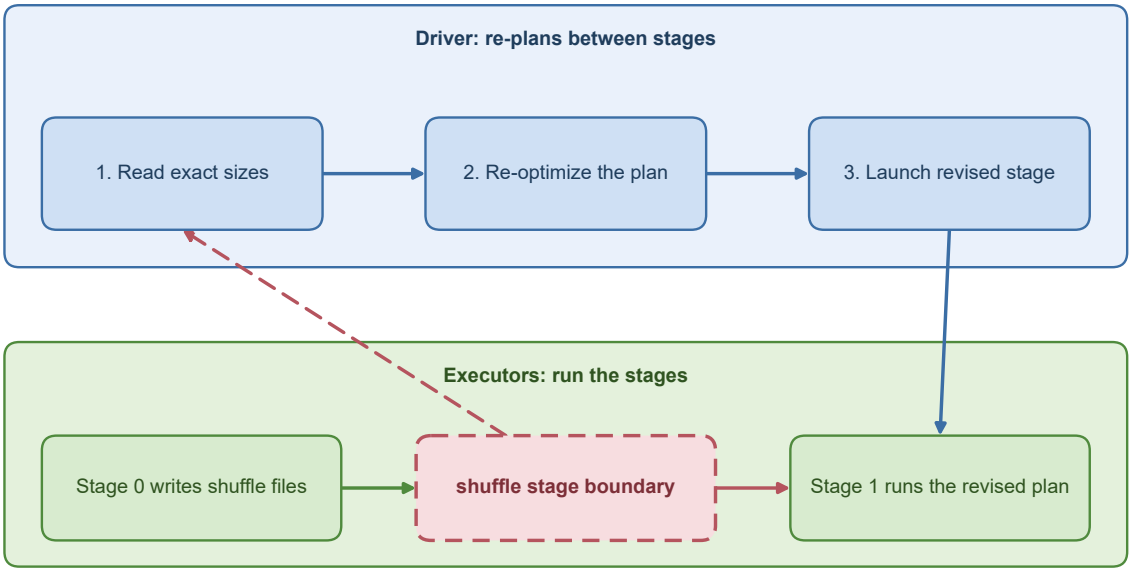


Figure 6.1: AQE inserts a measurement between stages: when a shuffle finishes, the driver reads the exact size of every output partition and re-optimizes only the part of the plan that has not run. Blue steps run on the driver, green on the executors; the rose dashed box is the stage boundary where the measurement happens.

AQE rewrite	**Planning-time decision it revisits**	**Evidence it uses**	**Where it shows up in the plan**
Coalesce partitions	shuffle partition count (Section 4.4)	byte size of every partition	`AQEShuffleRead coalesced`
Broadcast promotion	join strategy (Section 5.5)	total size of one shuffled side	`BroadcastHashJoin`, `BroadcastQueryStage`
Skew split	one partition per key range	partition size against the median	`SortMergeJoin(skew=true)`, `AQEShuffleRead … skewed`

Table 6.1: The three runtime rewrites, each a planning-time decision revisited with measured evidence.

specimen's, captured on the lab; the root reads `isFinalPlan=false`, the flag that says this plan may still change.

```
AdaptiveSparkPlan isFinalPlan=false
+- Sort [on_time_rate ASC NULLS FIRST], true, 0
   +- Exchange rangepartitioning(on_time_rate ASC NULLS FIRST, 200)
      +- HashAggregate(keys=[category], functions=[avg(...)])
         +- Exchange hashpartitioning(category, 200)
            +- HashAggregate(keys=[category], functions=[partial_avg(...)])
               +- Project [promised_days, delivery_days, category]
                  +- SortMergeJoin [product_id], [product_id], Inner
                     :- Sort [product_id ASC NULLS FIRST]
                     :  +- Exchange hashpartitioning(product_id, 200)
                     :     +- ... Filter <- BatchScan lab.db.orders[...]
                     +- Sort [product_id ASC NULLS FIRST]
                        +- Exchange hashpartitioning(product_id, 200)
                           +- Filter <- BatchScan lab.db.dim_product[...]
```

Listing 6.1: The specimen's plan before it runs (`EXPLAIN`, trimmed)

Notice what the planner chose. `dim_product` is about 170 MB, and Iceberg reports a table's size from its snapshot metadata, so the planner knows this without column statistics; it is far over the 10 MB threshold, and the join is planned as a `SortMergeJoin` with four exchanges in all. That is the right call for the whole table. It is the wrong call for a query that filters `dim_product` down to one category, and the planner cannot see that, because the metadata gives it the size of the table, not how much a filter removes. That gap is what AQE's join rewrite exists for, and the rest of the chapter shows each of the three rewrites on this plan.

6.3 Coalescing shuffle partitions

Section 4.4 recommended setting `spark.sql.shuffle.partitions` generously and letting AQE bring the count back down. This is the mechanism that does it. After a shuffle, AQE reads the exact size of every partition and merges *adjacent* ones until each is close to a target size (Figure 6.2). The count you set is only a ceiling: Spark starts from `spark.sql.shuffle.partitions` partitions, or from `spark.sql.adaptive`

`.coalescePartitions.initialPartitionNum` if you set that instead, and merges down from there.

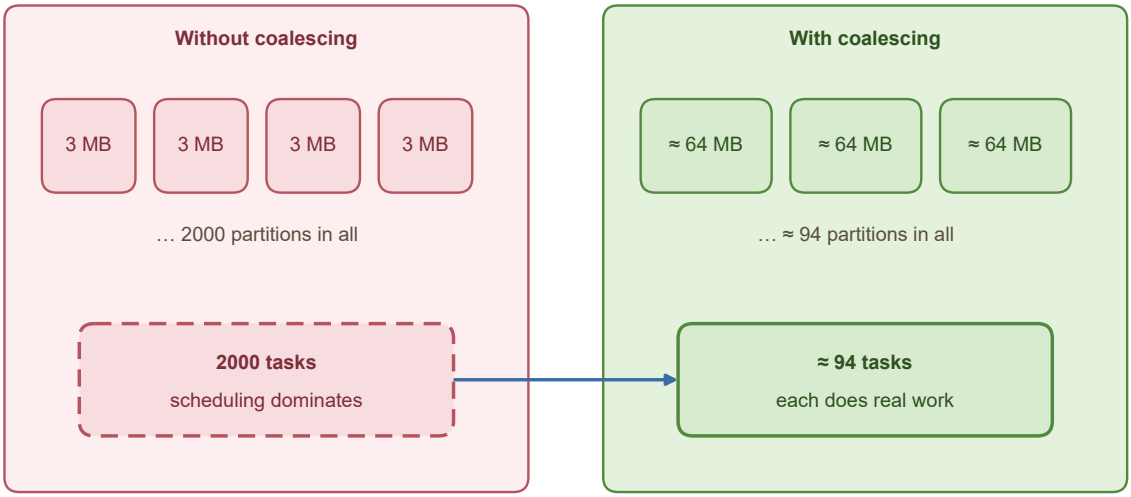


Figure 6.2: 2000 shuffle partitions of 3 MB each carry 6 GB; AQE merges adjacent partitions toward the 64 MB advisory size, so the next stage runs about 94 tasks instead of 2000. Rose marks the cost, green the corrected state.

6.3.1 The target size, and what parallelism does to it

The target is where most readers go wrong, because two keys influence it. `spark.sql.adaptive.advisoryPartitionSizeInBytes` defaults to 64 MB. `spark.sql.adaptive.coalescePartitions.minPartitionSize` (1 MB) is a floor that stops AQE from producing absurdly small partitions. And `spark.sql.adaptive.coalescePartitions.parallelismFirst` (default `true`) tells Spark not to merge so far that the cluster goes idle. With `parallelismFirst` on, the effective target is

$$\text{target} \;=\; \max\Big(\min\big(\tfrac{\text{total shuffle bytes}}{P},\ \text{advisory size}\big),\ \text{min partition size}\Big),$$

where P is the cluster's default parallelism, which on the reference cluster is its 40 task slots (Appendix A). In words: the advisory size is a *ceiling* on the target, and the parallelism term pulls it lower whenever the shuffle is small enough that 64 MB partitions would leave slots idle. Turn `parallelismFirst` off and the formula collapses to the advisory size alone. The Spark documentation describes the setting as ignoring the

advisory size; that is true when the shuffle is small, as in the first rows below, and not when it is large.

Table 6.2 puts numbers on the formula for the reference cluster.

Shuffle	P	**Target (`true`)**	**Parts.**	**Target (`false`)**	**Parts.**
6 GB	40	$\min(150, 64) = 64$ MB	≈ 94	64 MB	≈ 94
1.5 GB	40	$\min(37.5, 64) = 37.5$ MB	40	64 MB	≈ 24
200 MB	40	$\min(5, 64) = 5$ MB	40	64 MB	≈ 4
20 MB	40	$\max(\min(0.5, 64), 1) = 1$ MB	≈ 20	64 MB	1

Table 6.2: Coalescing a shuffle on 40 slots, with `parallelismFirst` true and false. Partition counts are approximate: AQE merges greedily in partition order, so real counts differ slightly. With `parallelismFirst` on, a small shuffle keeps as many partitions as there are slots; with it off, the same shuffle collapses to a handful of tasks and leaves most of the cluster idle.

The first row is the case Figure 6.2 draws: 6 GB divided over 40 slots is 150 MB per slot, far above the 64 MB advisory size, so the advisory size wins and the stage runs about $6000/64 \approx 94$ tasks. The second and third rows are where `parallelismFirst` earns its default: 1.5 GB at 64 MB would give 24 tasks on 40 slots, with 16 cores idle, while the parallelism term keeps all 40 busy. The fourth row shows the floor: with only 20 MB to move, nothing smaller than 1 MB is worth a task.

Checking the formula on the lab

The formula is a claim about Spark's source, so the lab tests it directly. The lab pins `spark.default.parallelism` to 40 so that P matches the reference cluster, then runs the same group-by with shuffle partitions set to 2000 and changes one setting at a time. Table 6.3 lists what the reduce stage actually ran, against what the formula predicts from the shuffle bytes the stage read.

Figure 6.3 plots the same comparison. All seven rows agree to within a few tasks (AQE merges greedily, so the observed count runs slightly above the quotient), which is what makes the ceiling reading of the advisory size more than a guess: a 16 MB advisory

Setting changed	Shuffle read	Advisory	pF	Predicted	Observed tasks
AQE off	94.1 MB	—	—	2000	2000
baseline	94.1 MB	64 MB	true	40	40
`parallelismFirst` off	94.1 MB	64 MB	false	2	2
bigger shuffle	512.1 MB	64 MB	true	≈ 40	41
smaller advisory	512.1 MB	16 MB	true	≈ 40	41
tiny advisory	512.1 MB	4 MB	true	≈ 128	134
bigger shuffle, pF off	512.1 MB	64 MB	false	≈ 8	9

Table 6.3: Measured on the lab. Each prediction uses only the formula and the shuffle bytes the reduce stage read (pF is `parallelismFirst`). The advisory size changes nothing until it falls below the parallelism term ($512.1/40 \approx 12.8$ MB), exactly as the formula says.

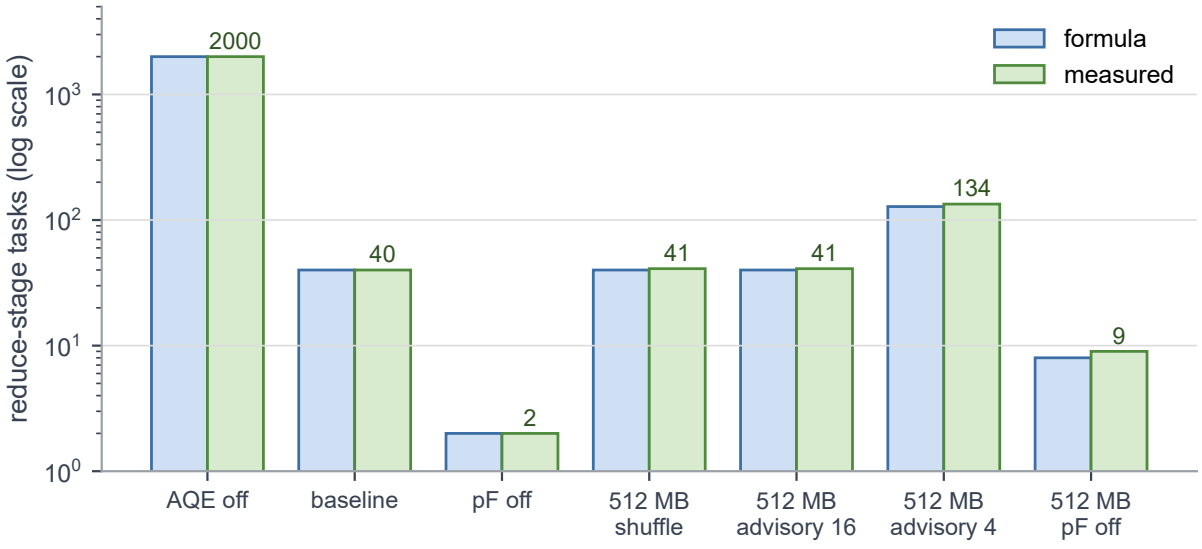


Figure 6.3: The formula against the lab. For seven settings the bars pair the task count the formula predicts (blue) with the one the reduce stage ran (green); the log scale shows how far apart the settings are and how close each pair is.

and a 64 MB advisory produce the same 41 tasks, because $512.1/40$ is below both. The cost of ignoring the formula shows in the task times. The last row's 9 tasks each read about 64 MB and took a median of 8.2 s; the 41-task baseline took a median of 0.6 s per task.

One more observation from these runs, because it will mislead you otherwise. The final plan prints a `Statistics` annotation on every `ShuffleQueryStage`, and for the 94.1 MB shuffle above it says `sizeInBytes=245.9 MiB`. That number is the *uncompressed* data size of the shuffled rows. The coalescing decision tracked the 94.1 MB that the reduce tasks actually read: the compressed shuffle files. (`parallelismFirst=false` gave 2 tasks, not the 4 that 245.9 MiB would give.) The Spark UI reports the second number; the plan annotation reports the first.

> **Warning** P is Spark's *default parallelism*, which is the number of cores the application holds *right now* unless you set `spark.default.parallelism`. Under dynamic allocation (Chapter 14) a query that starts while the application holds few executors sees a small P, and AQE coalesces harder than the cluster you eventually get could use. The lab checked this on a standalone cluster with dynamic allocation: the first query ran with a default parallelism of 2, the second, eight seconds later, with 4. Both reduce stages ran 5 tasks. The probable reason is timing: AQE reads P when the map stage *finishes*, and by then the first query's own map tasks had already pulled all four executors in (they registered between 3 and 6 seconds after the job started). The lab's first query therefore did not see the small P; to provoke the effect one would have to run a query whose map stage finishes before the executors arrive. Treat the warning as a possible effect on short queries against a cold application, not a measured one.

When you want AQE to respect the advisory size exactly — say, you are sizing partitions for a downstream write and the 40-slot term only gets in the way — set `parallelismFirst` to `false`:

```
SET spark.sql.adaptive.coalescePartitions.parallelismFirst=false;
SET spark.sql.adaptive.advisoryPartitionSizeInBytes=134217728;   -- 128 MB
```

Listing 6.2: Making the advisory size binding

Section 4.4's recommended settings use exactly this pairing; the first line is what makes the second one do what the second line says.

6.3.2 Seeing it in the plan

When coalescing fires, the final plan carries an `AQEShuffleRead` node whose argument is `coalesced`, over the shuffle's query stage. The statistics AQE saw sit on the stage:

```
AdaptiveSparkPlan (13)
+- == Final Plan ==
   * HashAggregate (8)
   +- AQEShuffleRead (7)                  -- Arguments: coalesced
      +- ShuffleQueryStage (6)            -- 245.9 MiB, 5.37E+6 rows
         +- Exchange (5)                  -- hash(customer_id, 2000)
            +- * HashAggregate (4)
               +- ... scan and filter ...
```

Listing 6.3: A coalesced shuffle read in a final plan (`EXPLAIN FORMATTED`, trimmed)

The `Exchange` still says 2000, because that is what was planned and what the map side wrote. The coalescing is a property of the *read*: the reduce tasks fetch several adjacent map-side partitions as one. No data is rewritten. That is why coalescing is free: the shuffle files are already on disk, and AQE only changes how many reduce tasks read them.

> **Tip** Merging is one-directional. AQE only ever combines *adjacent* small partitions into fewer, larger ones; it never splits an oversized partition this way. An oversized partition is handled by a different mechanism (Section 6.5), and coalescing does nothing for it.

6.4 Converting sort-merge to broadcast at runtime

Section 5.5 chooses a join strategy from an estimate, and an estimate can land on the wrong side of the threshold: a filter on `dim_product` shrinks it to a few megabytes, and the planner, which sees only the 170 MB table, cannot know. AQE gets a second chance after the join's inputs have shuffled. It reads the runtime statistics of each side's shuffle

stage, and if one is below the adaptive threshold, it rewrites the sort-merge join into a broadcast hash join before the join runs. The final plan shows `BroadcastHashJoin` where the plan had `SortMergeJoin`, with a `BroadcastQueryStage` holding the small side.

6.4.1 What the conversion saves, and what it cannot

The conversion arrives after both sides of the join have shuffled, because AQE cannot measure a shuffle that has not run. That fixes what the rewrite is able to save. The big side's map-side shuffle *write* has already happened and is not undone; the savings are the sort-merge join's costs that come after it: the network fetch of the big side, and sorting both sides. To avoid the fetch, AQE reads shuffle output with a *local shuffle reader* (`spark.sql.adaptive.localShuffleReader.enabled`, default `true`): each reduce task reads shuffle files from its own executor instead of pulling a slice from every executor. The plan marks it with the argument `local`. Figure 6.4 shows the join before and after.

```
+- * BroadcastHashJoin Inner BuildRight
   :- AQEShuffleRead                        -- Arguments: local
   :  +- ShuffleQueryStage                  -- 155.1 MiB, 6.78E+6 rows
   :     +- Exchange                        -- big side shuffled anyway
   +- BroadcastQueryStage                  -- 20.0 MiB, 1.00E+5 rows
      +- BroadcastExchange
         +- AQEShuffleRead                 -- Arguments: local
            +- ShuffleQueryStage           -- 3.8 MiB, 1.00E+5 rows
               +- Exchange                 -- small side, now a broadcast
```

Listing 6.4: A runtime broadcast promotion in a final plan (`EXPLAIN FORMATTED`, trimmed)

Compare this to a join Catalyst broadcast at planning time. There, the big side is never shuffled at all, and Spark saves the map-side write too. A runtime promotion is the consolation prize, not the equal: you still pay the shuffle you planned. When a side is reliably small, a hint or `ANALYZE` (Section 5.7) is still the better fix, because it prevents the shuffle instead of salvaging half of it.

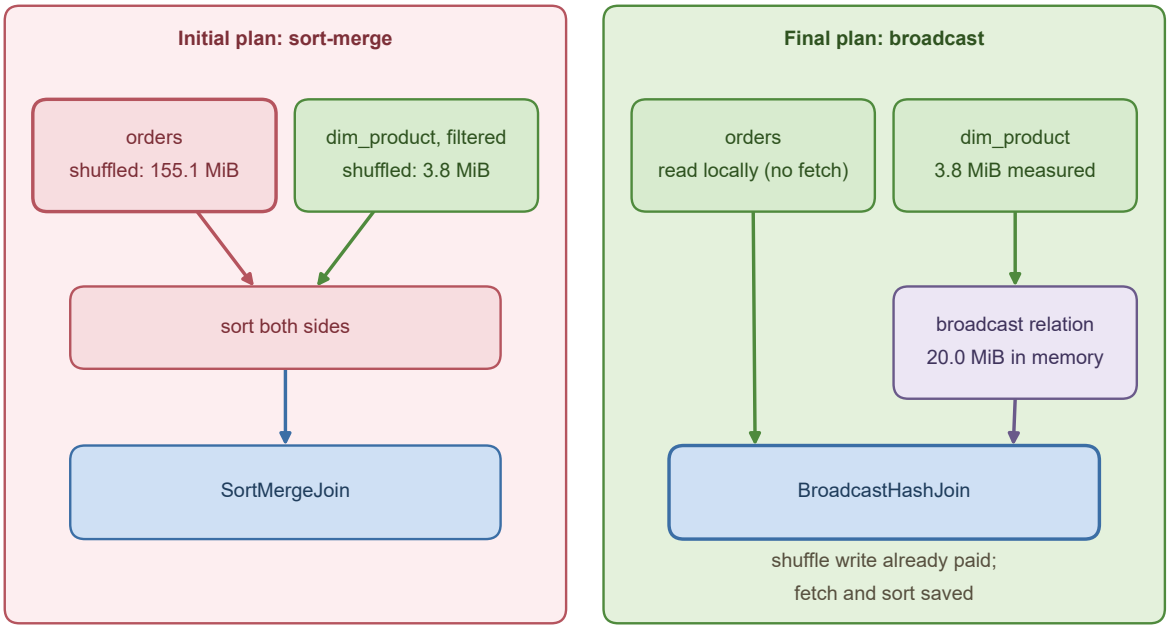


Figure 6.4: The same join before and after the runtime rewrite. Both inputs were already shuffled when AQE measured the filtered dimension at 3.8 MiB, so the big side is read locally instead of fetched and neither side is sorted. The relation built for the broadcast is 20.0 MiB in memory.

6.4.2 The two broadcast thresholds

`spark.sql.autoBroadcastJoinThreshold` and `spark.sql.adaptive.autoBroadcastJoinThreshold` are two separate keys. Confusing them is easy because the second has no default of its own: when unset, it takes the value of the first, 10 MB. They compare against different evidence, as Table 6.4 sets out.

The lab shows the gap between those last two quantities directly. In Listing 6.4 the filtered `dim_product`'s shuffle stage measures 3.8 MiB, and the `BroadcastQueryStage` built from it measures 20.0 MiB: the in-memory hash relation is more than five times larger than the number the threshold was compared against. Table 6.5 tests the comparison by moving the adaptive threshold across 3.8 MiB. The compressed shuffle files for that side were only 0.9 MB, which would have cleared the 1 MB threshold; the join was not promoted, so the comparison is against the larger, uncompressed figure.

That makes the threshold a weaker memory guard than its name suggests. It bounds the size of the shuffled data, and the relation built from that data can exceed it several

	Planning-time threshold	**Adaptive threshold**
Key	`spark.sql.autoBroadcastJoinThreshold`	`spark.sql.adaptive.autoBroadcastJoinThreshold`
Default	10 MB	unset, so the same as the planning key
Compared against	an estimate: table metadata, catalog statistics, or file sizes	the `Statistics(sizeInBytes)` of the shuffle stage, measured
What can go wrong	the estimate is low and the real table is large; the driver or executors run out of memory	the measured size understates the in-memory relation, because the built hash relation is larger
Decided	before any stage runs	after the side's shuffle stage finishes

Table 6.4: The two thresholds guard against different failures. The planning-time one must stay conservative because its input is a guess; the adaptive one sees a measurement, but a measurement of the shuffled rows, not of the broadcast relation that gets built from them.

Adaptive threshold	**Shuffle stage measures**	**Final join**
1 MB	3.8 MiB	`SortMergeJoin`
4 MB	3.8 MiB	`BroadcastHashJoin` (relation is 20.0 MiB)
unset (= 10 MB, from the planning key)	3.8 MiB	`BroadcastHashJoin`

Table 6.5: The adaptive threshold decides on the shuffle stage's size, not on the relation that gets built: a 4 MB threshold promoted a join whose broadcast relation was 20.0 MiB.

times over; size the driver for the relation, not for the threshold.

> **Warning** Broadcasting collects the small side to the driver before shipping it to every executor. Raising either threshold to hundreds of megabytes spends driver memory (`spark.driver.memory`) on every such join in the application, and Spark refuses to broadcast a relation above 8 GB regardless. Raise the adaptive threshold for a table you know is stable in size, not as a blanket setting.

AQE also declines to broadcast a side whose shuffle output is mostly empty partitions. `spark.sql.adaptive.nonEmptyPartitionRatioForBroadcastJoin` (default 0.2) sets the minimum share of non-empty partitions a side needs; below it, AQE keeps the sort-merge join however small the total is. The reason is that a mostly-empty shuffle usually means the data lives in a few hot partitions, and broadcasting that is a different bet than the total suggests.

Nothing needs configuring to see the promotion: the query below ran with default settings and was promoted, because the filtered side's 3.8 MiB is under the 10 MB default. The experiment in Table 6.5 changes one key:

```
-- SET spark.sql.adaptive.autoBroadcastJoinThreshold = 1048576;  -- 1 MB: no promotion
SELECT p.category, COUNT(*) AS orders
FROM orders o
JOIN dim_product p ON o.product_id = p.product_id
WHERE o.order_ts >= TIMESTAMP '2026-03-01 00:00:00'
  AND o.order_ts <  TIMESTAMP '2026-03-08 00:00:00'
  AND p.category = 'books'
GROUP BY p.category;
```

Listing 6.5: A filtered dimension that AQE promotes to a broadcast

Before the query runs, `EXPLAIN` prints a `SortMergeJoin`: the planner sees 170 MB. After it runs, the SQL tab shows the shape in Listing 6.4. Uncomment the first line and the `SortMergeJoin` survives into the final plan. That pair of runs is the whole argument for treating the two keys as separate.

6.5 Skew-join handling: split and replicate

A shuffle partition holds every row whose key hashes to it (Section 4.1), and no amount of repartitioning shrinks a single key that is larger than the others. In the specimen's week, the viral product's rows all land in one partition of the join's shuffle. AQE detects that after the shuffle runs. It looks at the sizes of all the partitions on a join side and flags one as skewed only if it passes *two* tests: it is larger than `spark.sql.adaptive.skewJoin.skewedPartitionFactor` (5.0) times the median partition size, *and* larger than `spark.sql.adaptive.skewJoin.skewedPartitionThresholdInBytes` (256 MB). Requiring both avoids flagging a partition as skewed just because the median happened to be tiny. Table 6.6 applies the check to a stage with a 40 MB median, so the relative bar is 200 MB and the absolute floor is 256 MB.

Partition	**Size**	**vs. 5× median (200 MB)**	**Flagged skewed?**
ordinary product key	55 MB	below	no
mildly popular key	220 MB	above, but below the 256 MB floor	no
viral product key	3.2 GB	above, and above the 256 MB floor	**yes**

Table 6.6: Both the relative check ($5\times$ median) and the absolute floor (256 MB) must pass before AQE calls a partition skewed. The 220 MB partition shows why the floor matters: it clears the relative bar but is still an ordinary amount of data for one task, not a straggler.

6.5.1 The split and the replication

Figure 6.5 shows the fix. AQE splits the oversized partition into several sub-partitions and *replicates* the matching partition from the other side of the join across all of them. Each sub-partition then runs as its own task, in parallel, instead of one task carrying the whole key while every other task in the stage sits idle.

The split target is the larger of the advisory size (64 MB) and the average size of the *non-skewed* partitions. With a 40 MB median, that is 64 MB, so the 3.2 GB partition

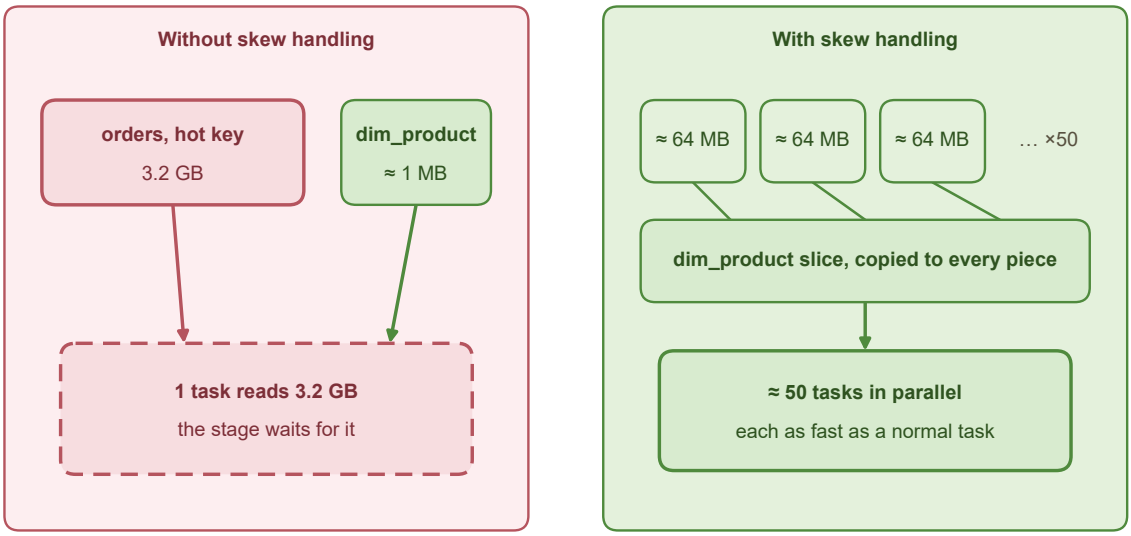


Figure 6.5: Without skew handling, one 3.2 GB partition runs as a single task long after the rest of the stage has finished. AQE splits it into pieces near the advisory size and copies the small side's slice to each piece, turning one straggler into about 50 ordinary tasks.

becomes about $3200/64 = 50$ sub-partitions. The split happens along *map-task boundaries*: each sub-partition is a range of map outputs for that reducer, so a stage with only 8 map tasks can split a partition into at most 8 pieces, whatever the target says. On the replicated side, the cost is reading its slice 50 times: for a 1 MB slice of `dim_product`, 50 MB of extra reads, which is nothing.

The cost is real when the other side is large. If *both* sides are skewed on the same key, AQE splits both, and the join runs every pair of pieces: $m \times n$ tasks over $m + n$ pieces of data. Skew handling is a good trade only because, in the common case, one side of a skewed join is a small dimension.

Which side can be split depends on the join type, because splitting the wrong side of an outer join changes the result:

6.5.2 Reproducing the straggler, and why the defaults did not fire

To study skew on the specimen, remove the escape hatch that would otherwise broadcast `dim_product`, and compare skew handling off and on over the whole month:

Join type	Left side split?	Right side split?
Inner, cross	yes	yes
Left outer, left semi, left anti	yes	no
Right outer	no	yes
Full outer	no	no

Table 6.7: Skew handling splits only the side whose rows can be partitioned without changing the join result. A full outer join cannot be skew-split at all.

```
SET spark.sql.adaptive.skewJoin.skewedPartitionThresholdInBytes = 33554432;
-- 32 MB (lab scale)
SET spark.sql.adaptive.advisoryPartitionSizeInBytes = 8388608;
-- 8 MB (lab scale)
SET spark.sql.adaptive.skewJoin.enabled = false;      -- run 1: no skew
handling
-- run the month-wide join, then open Stages -> the join stage -> Summary
Metrics

SET spark.sql.adaptive.skewJoin.enabled = true;       -- run 2
-- run it again and compare max vs median task time
```

Listing 6.6: Isolating skew handling on the specimen's join

On the lab's 30 million rows, with one product taking 30% of them, the hot partition is about 73 MB of compressed shuffle data against ordinary partitions of about 1 MB. That is skewed by any sane reading, and the defaults did *not* split it: the 256 MB absolute floor is a number tuned for production data, and the lab's hot partition never reaches it. Lowering the floor to 32 MB was still not enough. The split target is the 64 MB advisory size, and a 73 MB partition sits barely above it, so splitting it would leave a remainder too small to keep as its own piece. The plan showed no skew handling in either case. (The split heuristic is the explanation consistent with the observation; confirm it against `OptimizeSkewedJoin` in the 3.5 source.) Figure 6.6 plots all 200 partitions against these thresholds.

It split once the target dropped below the partition, with the advisory size at 8 MB

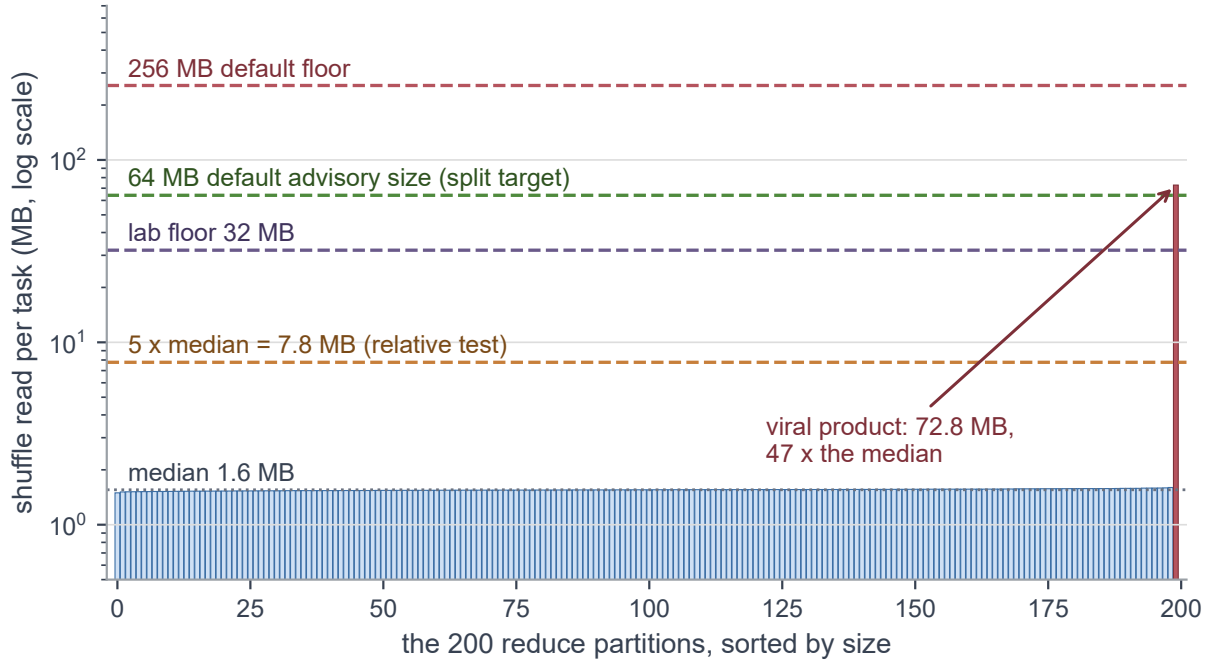


Figure 6.6: The 200 reduce partitions of the skewed join as measured on the lab, one bar per task. The viral product is 47 times the median, far above the relative test and the lab floor, yet the default 256 MB floor is off the scale above it and the 64 MB split target sits barely under it.

and the floor at 32 MB. Table 6.8 shows the stage.

Run	Join-stage tasks	Max task shuffle read	Median task	Max task
skew handling off	42	72.8 MB	775 ms	5115 ms
skew handling on	51	9.5 MB	647 ms	844 ms

Table 6.8: Measured on the lab (advisory size 8 MB, skew floor 32 MB). Skew handling turned 42 tasks into 51 and cut the slowest task from 5115 ms to 844 ms, 6.1 times, while the median barely moved.

The straggler is gone: the largest task reads 9.5 MB instead of 72.8 MB, and the stage ends when its slowest task, now close to the median, ends. The final plan marks the handling on the join node and on the skewed side's read:

```
AdaptiveSparkPlan isFinalPlan=true
+- ...
   +- *(5) SortMergeJoin(skew=true) [product_id], [product_id], Inner
```

```
   :- *(3) Sort [product_id ASC NULLS FIRST]
   :  +- AQEShuffleRead coalesced and skewed
   :     +- ShuffleQueryStage 0
   +- *(4) Sort [product_id ASC NULLS FIRST]
      +- AQEShuffleRead coalesced
         +- ShuffleQueryStage 1
```

Listing 6.7: A skew-split join in a final plan (`EXPLAIN`, trimmed)

Two details are easy to misread. The argument is `coalesced and skewed`: one `AQEShuffleRead` applies both rewrites, splitting the hot partition and merging the small ones around it. And only the split side is marked; the replicated side reads as plain `coalesced`, because replication is a property of which map outputs each task reads, not a separate node.

If the skew is worse than the defaults catch, lower the two thresholds — `skewedPartitionFactor` to 3.0 and `skewedPartitionThresholdInBytes` to 128 MB, say — and check that the advisory size is below the hot partition. Each false positive spends extra tasks and extra reads of the replicated side. By default AQE also skips skew handling if applying it would add an extra shuffle elsewhere in the plan; `spark.sql.adaptive.forceOptimizeSkewedJoin` (default `false`, since Spark 3.3) overrides that. (Claim — verify the default on 3.5.2.)

6.6 Reading an AQE plan

The plan `EXPLAIN` prints before an action runs is not the plan that ran. The one that matters is the plan after execution, which you get from the SQL tab or by calling `explain` on the same DataFrame after the action. It is rooted at `AdaptiveSparkPlan isFinalPlan=true`, and `EXPLAIN FORMATTED` prints it as two trees, `Final Plan` and `Initial Plan`, so the rewrites are visible as the difference between them. Listing 6.8 is the skew-split join of the previous section, with the decisions numbered for the walk-through that follows.

```
AdaptiveSparkPlan isFinalPlan=true                                  -- (1)
+- == Final Plan ==
   *(6) HashAggregate(keys=[category], functions=[avg(...)])
   +- AQEShuffleRead coalesced                                      -- (2)
```

```
      +- ShuffleQueryStage 2                                        -- (3)
         +- Exchange hashpartitioning(category, 200)
            +- *(5) HashAggregate(keys=[category], ...partial_avg)
               +- *(5) SortMergeJoin(skew=true) [product_id], ... -- (4)
                  :- *(3) Sort [product_id ASC NULLS FIRST]
                  :  +- AQEShuffleRead coalesced and skewed       -- (5)
                  :     +- ShuffleQueryStage 0
                  +- *(4) Sort [product_id ASC NULLS FIRST]
                     +- AQEShuffleRead coalesced
                        +- ShuffleQueryStage 1
+- == Initial Plan ==                                              -- (6)
   HashAggregate(keys=[category], functions=[avg(...)])
   +- Exchange hashpartitioning(category, 200)
      +- ... SortMergeJoin [product_id], [product_id], Inner ...
```

Listing 6.8: A final AQE plan, annotated (`EXPLAIN`, trimmed)

Read it from the root down:

1. **The root.** `isFinalPlan=true` confirms this is the plan as executed; `isFinalPlan=false` is the compile-time plan from `explain()` before an action.

2. **Coalescing.** `AQEShuffleRead coalesced` over the aggregation's shuffle: AQE merged its many small partitions before the final aggregate ran (Section 6.3).

3. **Query stage numbers.** The number after `ShuffleQueryStage` identifies one materialization point. The `Exchange` under it is what Catalyst planned, still reading 200, because coalescing changes the read, not the write.

4. **The join flag.** `SortMergeJoin(skew=true)` marks that skew handling applied to *this* join, not just somewhere upstream.

5. **Skewed reads.** `coalesced and skewed` on the split side; the replicated side shows only `coalesced`.

6. **The Initial Plan.** What Catalyst planned before any stage ran. Compare it with the Final Plan to see what AQE changed.

One query, two unrelated runtime corrections, each visible as its own marker in the same final plan. A promoted join looks different: the `SortMergeJoin` node is gone, replaced by a `BroadcastHashJoin` as in Listing 6.4. A join cannot show both a skew split and a broadcast promotion on the same side: a promoted side has no shuffle partitions left to measure for skew. The reference for every AQE node, including the ones this chapter does not need, is Section C.10.

> **Tip** `EXPLAIN FORMATTED` prints a `Statistics(sizeInBytes=…, rowCount=…)` annotation on every query stage of a final plan. These are the numbers AQE replanned from, so they are the first thing to read when a rewrite did not fire: compare them with the thresholds in this chapter before touching a config.

6.7 What AQE cannot fix

AQE is the safety net under the decisions the earlier chapters taught, not a replacement for them. Table 6.9 lists the places it does nothing, and why. Two rows were checked on the lab: a query with no exchange printed a plain physical plan with no `AdaptiveSparkPlan` root, and `repartition(200)` and `repartition(200, customer_id)` both ran exactly 200 reduce tasks, while `repartition(customer_id)` with no count was coalesced to 47.

The pattern in the table is the chapter's closing rule: **AQE turns a guess into a measurement, but only for the stages that have not run yet.** That is why Chapter 4's advice to set partitions generously, Chapter 5's advice to give the planner real statistics, and Chapter 7's advice to bucket a repeated join all still stand.

> **Note** This chapter is pinned to Spark 3.5.2, and every default quoted comes from the 3.5 documentation. AQE keeps changing between releases — defaults and rules have moved across 3.0, 3.2, and 3.3 — so recheck each key against the migration guide of the version you actually run before relying on a number here.

Situation	Why AQE cannot help	What to do instead
The first stage: scans and file splits	nothing has run, so there is nothing to measure	size input splits and files (Section 4.1)
A query with no exchange and no subquery	no query stage boundary exists, so AQE does not wrap the plan	nothing to fix; a single-stage plan has no shuffle to re-plan
An explicit `repartition(n)`	the user asked for exactly n partitions, and AQE honors it	use `repartition` by column, or drop the count
Skew in an aggregation, not a join	AQE's skew rule applies to joins only	partial aggregation, or salting the key (Chapter 15)
Structured Streaming micro-batches	AQE is not applied to streaming queries in 3.5 (claim — verify)	size partitions explicitly (Chapter 16)
A shuffle planning already committed to	AQE re-plans around the exchange, not through it	bucketing or storage-partitioned joins (Section 5.8)
Both sides of a join skewed on the same key	splitting both multiplies tasks rather than shrinking them	filter or pre-aggregate the hot key

Table 6.9: Where AQE does nothing. Each row names the earlier decision that still matters.

6.8 Summary

Planning happens before any data moves; AQE runs after every shuffle, when partition sizes are exact, and re-plans the stages that have not run. The loop (Figure 6.1) makes three rewrites. It coalesces small partitions toward a target that is the advisory size, pulled lower by the cluster's parallelism when the shuffle is small (Table 6.3); it promotes a sort-merge join to a broadcast when one side's measured shuffle size clears the adaptive threshold, saving the fetch and the sort but not the shuffle write, and without bounding the memory of the relation it builds (Table 6.5); and it splits a partition that is both far above the median and above an absolute floor, replicating the other side (Table 6.8). Reading the final plan confirms which of these fired, and Table 6.10 turns each symptom into the evidence and the knob.

Chapter 7 turns from deciding how data moves to deciding how much of it has to be read at all.

Exercises

1. **Run and observe.** Run Listing 6.6 on the specimen with the advisory size at 8 MB and the skew floor at 32 MB (scale both to your data). Record the max and median task duration of the join stage with skew handling off, then on, and find `SortMergeJoin(skew=true)` in the second run's final plan.

2. **Read a plan.** In Listing 6.8, which operators carry a runtime decision, and which two nodes would you check first to confirm the aggregation's shuffle was coalesced? Rewrite the plan as it would look if `dim_product` were promoted to a broadcast.

3. **Predict, then check.** A stage's shuffle produces 20 partitions of 1 MB each on a 16-core cluster. Using the target formula in Section 6.3.1, predict how many partitions AQE leaves with `parallelismFirst` on, and with it off. Then run it and compare against the reduce stage's task count.

Symptom	Evidence	Knob	Cost
thousands of tiny tasks after a shuffle	no `AQEShuffleRead coalesced` in the final plan; tiny shuffle-read sizes in **Stages**	check `coalescePartitions.enabled`; raise the initial partition count	none: coalescing reads existing files
too few tasks for the cluster after coalescing	task count below slot count in the stage	keep `parallelismFirst=true`; lower the advisory size	more, smaller tasks
a sort-merge join on a small table	no `BroadcastQueryStage`; small `Statistics(sizeInBytes)` on the stage	raise the adaptive broadcast threshold	driver memory; relation larger than the threshold
one straggler task, max ≫ median	**Stages** Summary Metrics	lower the skew floor and the advisory size	extra tasks and replicated reads
AQE seems to do nothing	no `isFinalPlan=true` root, or a row of Table 6.9 applies	fix the earlier decision	none

Table 6.10: The AQE levers: each symptom with the evidence that confirms it, the knob that moves it, and what the knob costs.

Chapter 7

Partitioning, Bucketing, and Pruning

The cheapest data to move is data you never read. The specimen asks about one week of a month of `orders`*; a plan that reads all 31 days and filters afterwards has done four times the work, and no later stage can undo it. This chapter covers the ways Spark skips data before it moves any: partition pruning skips whole directories, dynamic partition pruning does it using values discovered from the other side of a join, bucketing lays data out so a join needs no shuffle, and column pruning reads only the columns the query names. Each is easy to defeat — a function on the partition column, a* `CAST`*, a stray* `SELECT *` *— and the plan tells you when you have. Every plan and byte count below was measured on the lab of Section A.3, and two of them overturned what the first draft of this chapter assumed.*

7.1 Reading a scan node

Every pruning mechanism in this chapter reports itself in one place: the scan node at the bottom of the plan. Learn its fields first, because the rest of the chapter is about making them say the right thing. Table 7.1 lists them.

The names depend on the reader. A Hive-catalogued Parquet table, read through

Field	What it says	An empty value means
`PartitionFilters`	predicates resolved against partition values, so whole partitions are skipped	the filter did not resolve against the partition column; every partition is read
`PushedFilters`	predicates handed to the file reader, which skips row groups by their min/max statistics	nothing was pushed; rows are filtered after they are decoded
`ReadSchema`	the columns the scan actually reads	(never empty) a schema wider than the query needs is a pruning failure
`RuntimeFilters`	filters injected at run time, such as dynamic partition pruning's (Section 7.3)	no runtime filter was added

Table 7.1: The scan-node fields that report pruning. An empty list on a partitioned table is the first thing to look for when a scan reads too much.

Spark's built-in file source, prints `PartitionFilters`, `PushedFilters`, and `ReadSchema` on a `FileScan` node. An Iceberg table is read through the DataSource V2 API, and its `BatchScan` node prints the pushed predicates inside a single `filters=[…]` list instead, the columns it reads in square brackets after the table name, and `RuntimeFilters` beside them. The meaning is the same; the spelling differs.

```
BatchScan lab.db.orders[order_ts, product_id, promised_days, delivery_days]
  lab.db.orders (branch=null)
  [filters=order_ts IS NOT NULL, order_ts >= 1772323200000000,
           order_ts < 1772928000000000, groupedBy=]
  RuntimeFilters: []
```

Listing 7.1: The specimen's Iceberg scan node (`EXPLAIN`, one line wrapped)

Read it field by field. The four columns in brackets are the `ReadSchema` equivalent: the specimen names `order_ts`, `product_id`, `promised_days`, and `delivery_days`, and nothing else, so `order_id`, `customer_id`, and `order_total` are never read. The two `order_ts` comparisons are the week's bounds, already converted to microseconds since the epoch; they are the predicates that decide which partitions survive. `RuntimeFilters`

is empty because nothing in this query adds one.

7.2 Partition pruning and the pushdown killers

A table physically partitioned by a column — for a Hive-catalogued table, one subdirectory per distinct value; for Iceberg, the hidden partitioning covered in Chapter 11 — lets the planner skip whole partitions before any file inside them is opened, whenever a filter can be resolved against the partition value alone. Figure 7.1 shows the specimen's case: `orders` is partitioned by day, the week's filter matches 7 of the 31 daily partitions, and the other 24 are discarded at plan time, without opening a file or reading a footer.

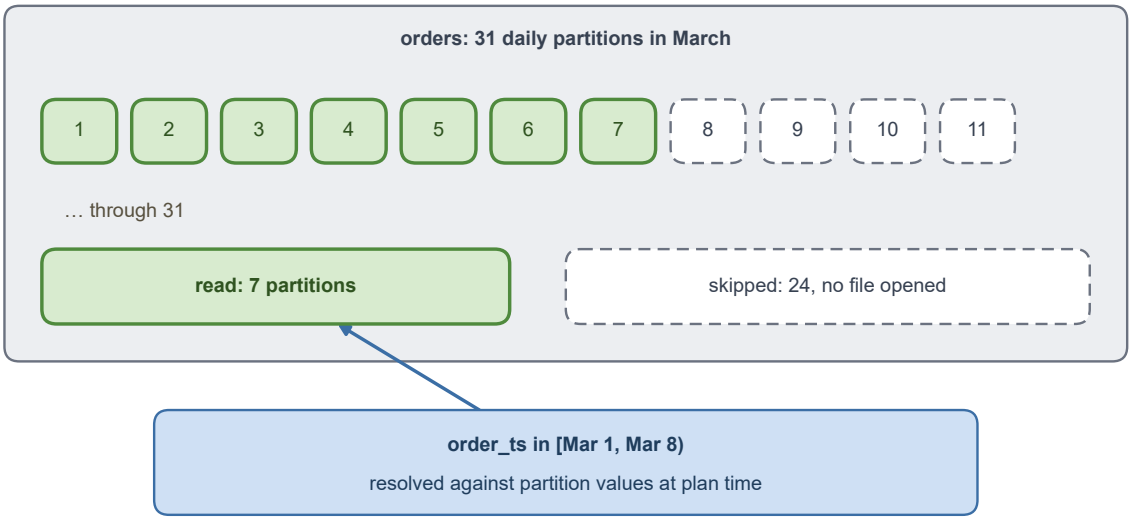


Figure 7.1: A filter that resolves against the partition value keeps 7 of 31 daily partitions of `orders` and skips the other 24 before any file is opened. Green marks what is read; dashed gray marks what is skipped.

Put numbers on it, from the lab. For the columns the specimen needs, one day of `orders` is 6.7 MB of Parquet and the month is 207.3 MB, so the week reads 46.8 MB in 7 tasks and an unpruned scan reads the month in 31. That is 4.4 times the bytes to answer the same question, and every extra byte becomes extra tasks and extra shuffle input downstream.

The most common reason pruning does not happen is the filter itself. A filter resolves against a partition value only when Spark can reduce it to a comparison of the bare column with a literal. The first draft of this chapter listed `date(order_ts)` and

`CAST(order_ts AS DATE)` among the filters that break this. The lab says otherwise. Table 7.2 gives five ways to ask for one day, and what each scan did; Figure 7.2 plots the bytes.

Filter as written	What the scan received	Input	Tasks
`order_ts >= … AND order_ts < …`	the range, as written	6.7 MB	1
`date(order_ts) = DATE '2026-03-03'`	rewritten to the same range	6.7 MB	1
`CAST(order_ts AS DATE) = DATE '…'`	rewritten to the same range	6.7 MB	1
`CAST(order_ts AS STRING) LIKE '2026-03-03%'`	no pushed filter	207.3 MB	31
`order_ts + INTERVAL 1 DAY > …`	no pushed filter	207.3 MB	31

Table 7.2: One day of `orders`, five ways. Spark rewrites a date comparison on a timestamp column into a range on the bare column, and the range prunes. A string cast or arithmetic on the column leaves nothing to push, and the scan reads the whole month.

The pattern is that Spark does the rewriting for comparisons it recognizes as equivalent to a range on the column, and gives up on anything it does not. So the rule is not "no function on the column"; it is "read the scan node and see what arrived." A filter that arrives as a range prunes. A filter that does not appear in `filters=` at all is being evaluated after the data has been read. The full catalogue of pushdown killers, with the fix for each, is Section C.11. The rewriting happens in Spark's optimizer before the scan, so it does not depend on the catalog, but check your own table's scan node once rather than assuming.

The specimen's own filter works for the simplest reason: it already is a range on the bare column, resolved once at plan time, with nothing standing between the column and the value.

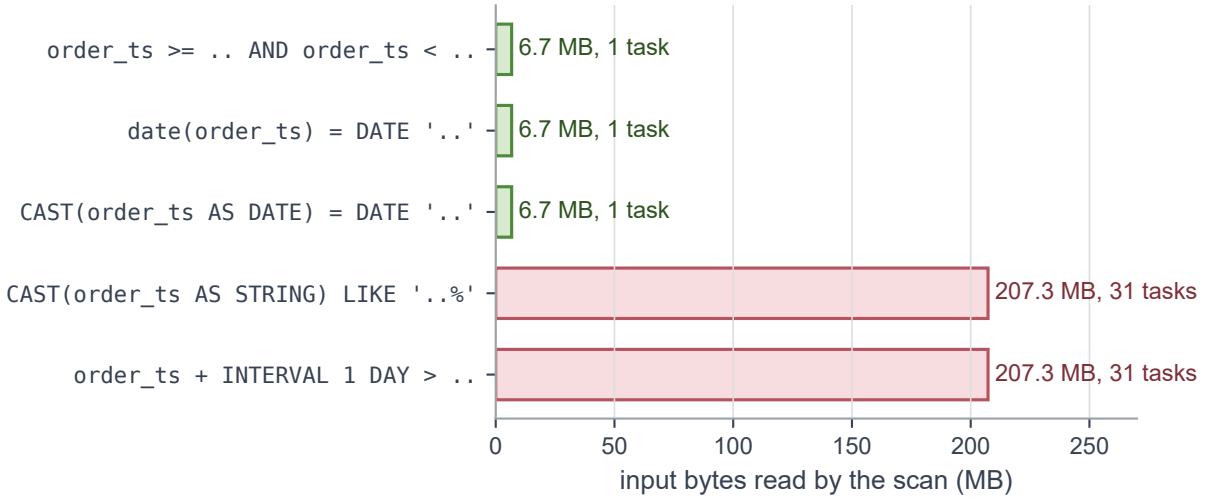


Figure 7.2: The same one day of `orders`, five filters, measured. Three forms reach the scan as a range and read 6.7 MB in one task; the two that do not read the whole month, 31 times as many tasks and 31 times the bytes.

7.3 Dynamic partition pruning

Static pruning needs a literal known before the query runs. A star-schema join — fact table filtered indirectly, through a small dimension — has no literal for the planner to use:

```
SELECT COUNT(*) AS n, SUM(o.order_total) AS total
FROM orders_d o
JOIN dim_date d ON o.order_date = d.date
WHERE d.day_of_week = 'Sat' AND d.month = 3;
```

Listing 7.2: A filter on the dimension, not the fact table

Here `orders_d` is a copy of `orders` partitioned by an identity transform on a `DATE` column, `order_date`; the lab builds it with a `CREATE TABLE … PARTITIONED BY (order_date) AS SELECT *, CAST(order_ts AS DATE) AS order_date`. The next subsection explains why the specimen's own `day(order_ts)` layout is not used.

Without help, Spark scans every partition of `orders_d`, joins, and discards everything that is not a Saturday — all the pruning happens *after* the expensive read. *Dynamic partition pruning* (DPP, on by default via `spark.sql.optimizer.dynamicPartitionPruning.enabled`) rewrites this (Figure 7.3). It runs the dimension side first, col-

lects the matching `date` values, and injects them into the fact scan as if the query had been written with `WHERE o.order_date IN (SELECT date FROM dim_date WHERE …)`, so the same partition-level skipping from Section 7.2 applies to a filter that, in the query as written, lived on the other table entirely.

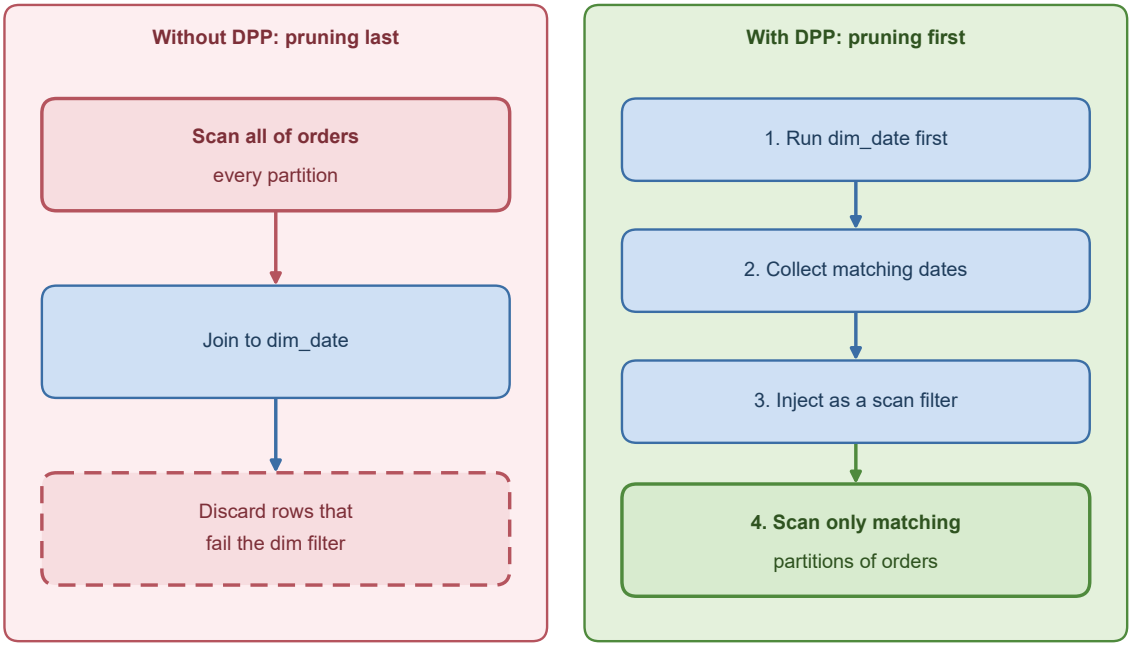


Figure 7.3: Without DPP, pruning happens last: the whole fact table is scanned before the join discards most of it. With DPP, the small dimension side runs first and its matching keys are injected as a filter *before* the fact scan, so pruning happens first.

The lab measures the saving. March 2026 has four Saturdays, so DPP should keep 4 of 31 partitions. Table 7.3 shows what the scan did across the cases that matter.

7.3.1 When DPP applies

The table is the chapter's best argument for reading `RuntimeFilters` instead of trusting the plan's shape. The rows with a filter that prunes nothing are the interesting ones, and the conditions in Table 7.4 explain each of them.

The broadcast condition deserves its reason. Spark runs the dimension side once, builds the broadcast hash table for the join out of it as it would anyway, and reuses that *same* in-memory result a second time as the list of matching keys for the fact-table filter: one execution, two consumers. In the lab's final plan the scan's subquery node reads

Case	RuntimeFilters on the fact scan	Input	Tasks
orders_d, identity partition, DPP off	empty	60.6 MB	31
orders_d, identity partition, DPP on	dynamicpruningexpression(order_date IN …)	7.8 MB	4
orders_d, DPP on, broadcast disabled	dynamicpruningexpression(true)	60.6 MB	31
orders, day(order_ts), join on CAST(order_ts AS DATE), DPP on	dynamicpruningexpression(cast(order_ts as date) IN …)	207.3 MB	31

Table 7.3: Dynamic partition pruning on the lab. It cuts the scan from 31 tasks to 4, an 87% reduction, only when the fact table is partitioned by the join column itself and the dimension side is a broadcast. The last two rows show filters that exist and prune nothing.

ReusedExchange over the join's own BroadcastExchange. If the dimension side is not broadcastable there is no cheap, already-materialized copy to reuse, and with the default reuseBroadcastOnly the filter collapses to dynamicpruningexpression(true), a sentinel that means "no pruning." With reuseBroadcastOnly set to false Spark may run the dimension side as its own query, but only when its estimate says the pruning saves more than it costs (spark.sql.optimizer.dynamicPartitionPruning.fallbackFilterRatio, default 0.5). Leave the default alone unless you have measured.

The first condition is the one a hidden-partitioned Iceberg table trips. The specimen's orders is partitioned by day(order_ts), so no column called order_date exists to join on, and a join on CAST(order_ts AS DATE) got a runtime filter that the Iceberg scan could not turn into a partition skip: 31 tasks, 207.3 MB, identical to the run with DPP off. Whether a different Iceberg version or a different join expression closes that gap is worth testing on your table; until you have, treat a DPP filter on a transformed partition column as unproven.

Condition	Why it is required	What fails without it
The fact table is partitioned by the join column itself	the injected filter is an `IN` on that column, and the scan must be able to map it onto partitions	the `day(order_ts)` row: a filter is injected, 31 tasks still run
The dimension side can be reused as a broadcast	the keys come free from the broadcast that the join builds anyway (`spark.sql.optimizer.dynamicPartitionPruning.reuseBroadcastOnly`, default `true`)	the broadcast-disabled row: the filter degrades to `true`
The dimension side has a selective filter	without one, every key matches and nothing is pruned	a `WHERE` on the dimension
The join is an equi-join on the partition column	the injected filter is an `IN` on that column	the join condition

Table 7.4: Conditions for dynamic partition pruning, each with the lab row that shows it failing.

> **Tip** `RuntimeFilters: [dynamicpruningexpression(true)]` is not a pruning filter. It is Spark telling you DPP considered the join and decided not to act. When a star join scans too much, look for that line before anything else.

7.4 Bucketing to avoid a shuffle

Partitioning divides a table by directory; *bucketing* divides it further, within a partition, by hashing a column into a fixed number of files. Writing with `bucket(16, customer_id)` computes `hash(customer_id) mod 16` for every row and sends it to the file for that remainder. This is the same `hash(key) mod N` idea Section 4.2 used to assign shuffle output partitions, applied at write time to decide which *file* a row lands in instead of which in-flight partition. Two tables bucketed on the same column with the same bucket count assign every row the same way from the same `customer_id`, so a row and its join partner always land in the file pair with the same bucket number.

Figure 7.4 shows what that buys. An ordinary join pays two shuffles every time it runs; a bucketed join pays them once, at write time. Each task reads one bucket from each table and joins them directly, which is precisely the precondition Section 5.8 used for a storage-partitioned join.

The lab bucketed `orders` and `dim_customer` on `bucket(16, customer_id)` and joined them with broadcasting disabled. Without Spark's storage-partitioned join enabled, the plan has two `Exchange hashpartitioning(customer_id, 200)` nodes under the join, and the reduce stage read 226.9 MB of shuffle data across 45 tasks. With it enabled, the scans carry `groupedBy=customer_id_bucket`, the two exchanges are gone, and the join runs in the 16 tasks of the scan stage itself with no shuffle read. The settings the lab turned on were `spark.sql.sources.v2.bucketing.enabled`, `spark.sql.iceberg.planning.preserve-data-grouping`, and `spark.sql.requireAllClusterKeysForCoPartition` set to `false`; the lab did not test which of the three can be left at its default.

```
+- SortMergeJoin [customer_id], [customer_id], Inner
   :- Sort [customer_id ASC NULLS FIRST]
   :  +- BatchScan lab.db.orders_b[customer_id]
   :       [filters=customer_id IS NOT NULL, groupedBy=customer_id_bucket]
```

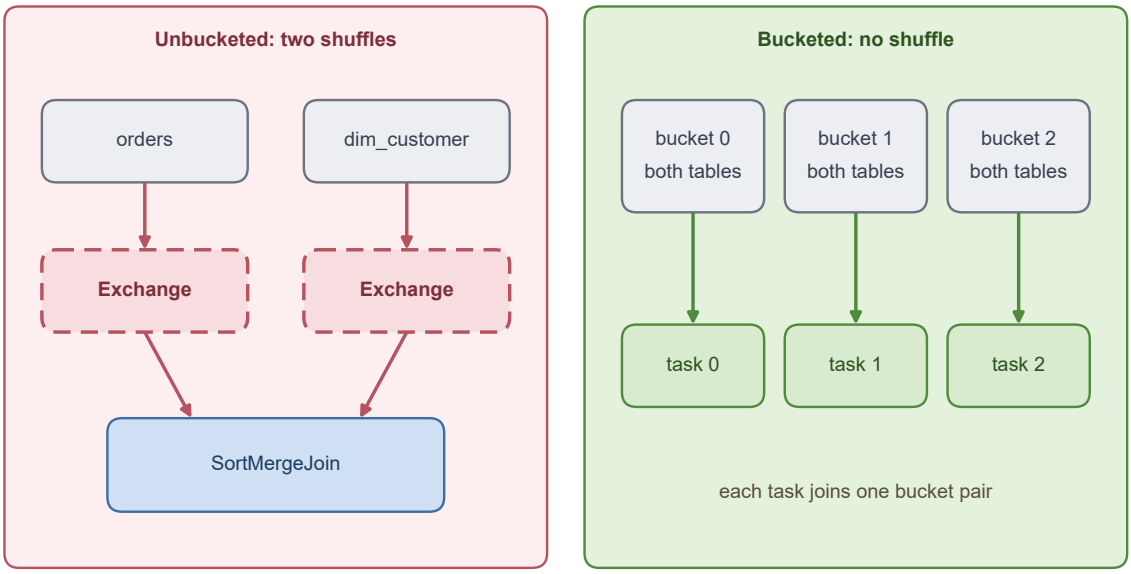


Figure 7.4: Without bucketing, a join shuffles both sides to line up the keys. With both tables bucketed on the join key, the matching rows are already in the same-numbered bucket on each side, and each task joins one bucket pair with no `Exchange`.

```
+- Sort [customer_id ASC NULLS FIRST]
   +- BatchScan lab.db.dim_customer_b[customer_id, segment]
        [filters=customer_id IS NOT NULL, groupedBy=customer_id_bucket]
```

Listing 7.3: A join over two bucketed Iceberg tables: no `Exchange` under the join (`EXPLAIN`, trimmed)

The `Exchange hashpartitioning(segment, 200)` above the join is the final aggregation's, and it appears in both plans; bucketing removes only the shuffle that lined up the join keys.

7.4.1 Sizing the bucket count

The bucket count is a design decision with a hard constraint: it fixes the join's parallelism and the size of each task's input, and you cannot change it without rewriting the table. Size it the way Section 4.4 sizes shuffle partitions. Suppose `orders` grows to 64 GB for the period you join on. Sixteen buckets would put 4 GB in every task, which spills; at a 128 MB target you need $64\,\text{GB}/128\,\text{MB} = 512$ buckets. On 40 slots that is about 13 waves, a comfortable number. Too many buckets is the opposite failure: thousands of

tiny files, and a task per file.

Bucketing has three costs you accept in exchange:

- **Rigidity.** Both tables need the same transform and the same bucket count on the join column. A join on any other column gets no benefit.
- **No skew rescue.** A hot key still lands in one bucket. AQE's skew split (Section 6.5) is a rewrite of a shuffle's output, and a bucketed join has no shuffle for it to act on, so a hot key becomes a slow task and stays one. The lab bucketed `orders` and `dim_product` into 16 buckets by `product_id`, joined them with storage-partitioned join on (no `Exchange` before the join), and read the join stage's 16 tasks: the median took 3.3 s, the task holding the hot product's bucket 8.7 s — 2.6 times the median, and nothing in the plan could split it.
- **Write cost.** Every write into the table pays the hash and the fan-out to every bucket.

Classic Spark bucketing (`df.write.bucketBy(n, col)`, for a Hive-catalogued table) established the idea but is brittle: the bucketing is a Spark-only convention, recorded in table properties, and invisible to any other engine reading the same files. Iceberg's `bucket(n, col)` partition transform (Section 11.1) is the modern equivalent on this stack, recorded in the table's own metadata so any engine reading the table sees and can exploit the same layout.

7.5 `repartition` versus `coalesce`

Both change how many partitions a DataFrame has; they get there by opposite mechanisms, and Table 1.2 already told you which is which. `repartition(n)` is wide: it shuffles, so it can raise *or* lower the partition count and redistributes rows evenly across the new count regardless of how they were distributed before. `coalesce(n)` is narrow: it only lowers the count, by merging existing partitions with no shuffle, which also means it cannot fix uneven input. Coalescing four huge partitions and four empty ones down to four still leaves four huge partitions; only a shuffle can redistribute the rows themselves.

Table 7.5 lists every way to ask for a partition count, because the DataFrame methods and the SQL hints do not all behave alike. The last column was measured on the lab for

the first three rows.

Call or hint	Shuffles?	Distributes by	Does AQE change the count?
`repartition(n)`	yes	round-robin	no: 200 requested, 200 tasks ran
`repartition(n, cols)`	yes	hash of columns	no: 200 requested, 200 tasks ran
`repartition(cols)`	yes	hash of columns	yes: 200 default, coalesced to 47
`repartitionByRange(cols)`	yes, after a sampling job	range of columns	no: 200 requested, 200 tasks ran
`coalesce(n)`	no	merges neighbors	not applicable
`/*+ REBALANCE */`	yes	balances sizes	yes: coalesced to 40 tasks (44 with a column)

Table 7.5: The ways to set a partition count. Only the forms that name a count fix it against AQE's coalescing.

The `REBALANCE` hint (Spark 3.2 and later) is the one to reach for before a write: AQE measures the shuffle output, splits oversized partitions, and merges tiny ones, so the files that come out are close to the advisory size without you guessing a count. It is controlled by `spark.sql.adaptive.optimizeSkewsInRebalancePartitions.enabled` (default `true`).

`coalesce` has a second cost beyond skew that surprises people. Because it adds no shuffle, it does not start a new stage: the narrow chain before it is folded into the same tasks, and *those* tasks are the only ones that exist. Table 7.6 and Figure 7.5 show it on two weeks of `orders`, read and written to a no-op sink.

The numbers are small because the lab is small, but the direction scales. `coalesce(10)` on a 560-split scan would run the scan and filter on 10 tasks of 56 splits each instead of 560 tasks spread over 40 slots: about 14 waves become 56 split-times in series on 10 of 40 slots. `repartition(10)` adds a shuffle but keeps the scan wide. Neither is free; the choice is whether you would rather pay in a shuffle or in a slow narrow stage.

Plan	Scan tasks	Median scan task	Reduce tasks	Shuffle read
no change	14	1688 ms	—	—
`coalesce(10)`	10	1660 ms	—	—
`repartition(10)`	14	3334 ms	10	1001.2 MB

Table 7.6: Fourteen days of `orders` (490.6 MB read), then 10 output partitions. `coalesce` merged the scan itself down to 10 tasks and shuffled nothing. `repartition` kept the scan at 14 tasks, wrote and then read 1001.2 MB of shuffle data (rows are larger once deserialized), and made each scan task about twice as slow because it now also writes shuffle files.

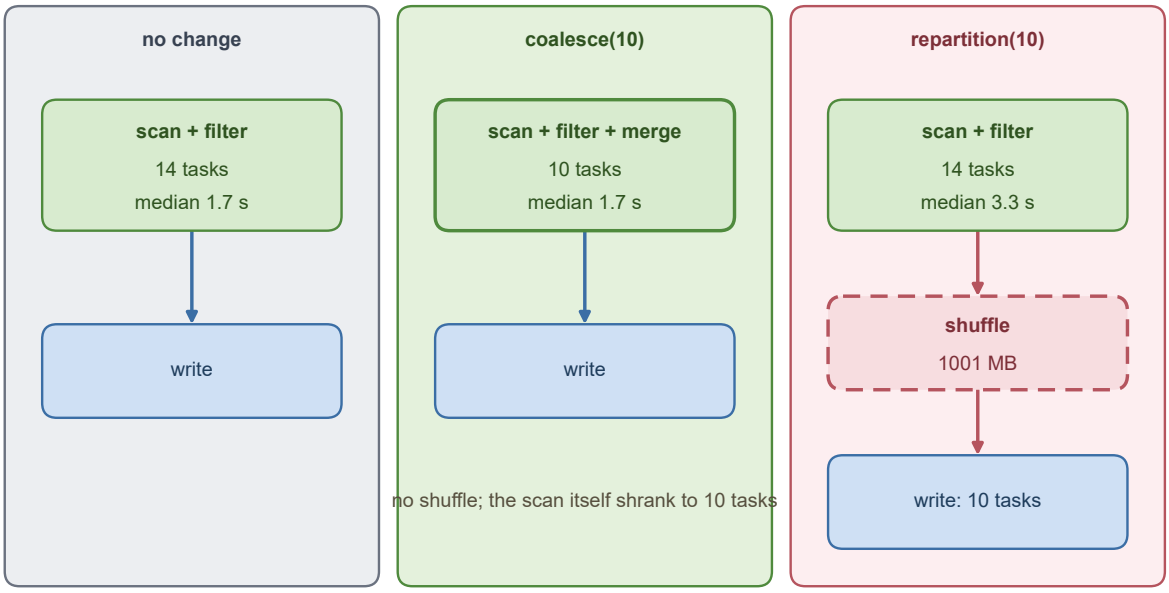


Figure 7.5: What each choice does to the stage graph for ten output partitions of two weeks of `orders`. `coalesce` folds the merge into the scan and adds no shuffle; `repartition` keeps the scan wide and pays for a 1001 MB shuffle.

> **Note** The `shuffle` flag in `coalesce(n, shuffle=true)` belongs to the RDD API (`rdd.coalesce`). The DataFrame method `df.coalesce(n)` has no such flag; the DataFrame equivalent of "coalesce with a shuffle" is `df.repartition(n)`.

7.6 Column pruning and `ReadSchema`

Column pruning — reading only the columns a query touches — is one of the optimizer's heuristic rules (Section 2.3) and shows up as the scan's column list (Listing 7.1). A list wider than the query needs almost always traces back to a `SELECT *` in a view, a CTE, or a UDF's input that the optimizer cannot see through. It pays most on a columnar format, where each column is stored contiguously and an unread column costs no I/O at all (Section A.5): the specimen reads 4 of the 7 `orders` columns, and `order_id` and `customer_id`, high-cardinality strings that compress poorly, are among the three it skips.

It goes one level deeper for a nested (struct) column. Parquet and Iceberg store nested fields independently enough that a query touching only `event.location.city` out of a wide `event STRUCT<…>` column can prune to that one leaf field, without reading its siblings. This is *nested schema pruning* (`spark.sql.optimizer.nestedSchemaPruning.enabled`, default `true`). It is visible in the scan's schema as the struct type narrowed to the accessed fields, not dropped whole:

```
-- SELECT event.location.city FROM events
ReadSchema: struct<event:struct<location:struct<city:string>>>   -- pruned

-- SELECT my_udf(event) FROM events    (whole struct passed to a UDF)
ReadSchema: struct<event:struct<location:struct<city:string,
                    lat:double,lon:double>,user:struct<...>,...>>
```

Listing 7.4: The read schema before and after nested pruning (shape only)

Whether this fires depends on the query projecting the leaf field explicitly; passing the whole struct into a UDF or a later transformation reintroduces the full column, the same way a `SELECT *` does. (The lab does not yet have a nested table, so Listing 7.4 is the expected shape, not a capture.)

> **Tip** To find which pruning layer failed on a slow scan, read the scan node in this order: the filters first, because they are the cheapest to fix and the largest saving; then the column list; then `RuntimeFilters`. A scan that reads too much usually fails the first of the three that you check.

7.7 Summary

Static partition pruning skips whole directories when the filter reaches the scan as a range on the bare column — Spark rewrites date comparisons on a timestamp into one, but not string casts or arithmetic — and the scan node's `filters=` list (`PartitionFilters` on a Hive table) is where you see it. Dynamic partition pruning gets the same skipping for a filter that lives on the other side of a join, by running that side first and reusing its broadcast; on the lab it cut a 31-task scan to 4, but only when the fact table was partitioned by the join column itself, and the filters that did not prune still printed as filters.

Bucketing extends pruning-by-layout into join cost: two tables bucketed identically on the join key need no shuffle to join, at the price of a fixed bucket count and no skew rescue.

`coalesce` merges partitions without a shuffle and therefore cannot fix skew, and it narrows the whole stage before it; `repartition` can fix both, at the cost of a shuffle. Column pruning reaches inside a struct, as long as the query projects the leaf field it needs. Table 7.7 turns each symptom into the evidence and the fix.

The rule the chapter repeats is that **the cheapest byte is the one the plan proves it never reads**: every mechanism here is verified in the scan node, not assumed from the query. Chapter 8 turns to the data you must read, and to keeping what is expensive to recompute.

Exercises

1. **Run and observe.** Write the same one-day filter on `orders` the five ways in Table 7.2, and `EXPLAIN` each. Which keep the scan's `filters=` populated, and how many tasks does each scan run?

2. **Read a plan.** Run Listing 7.2 with `spark.sql.optimizer.dynamicPartitionPr`

Symptom	Evidence	Fix	Cost
a scan reads every partition	no range on the partition column in `filters=`; month-sized input in the **Stages** tab	rewrite the filter as a bare-column range, not a string cast or arithmetic	none
a star join scans the whole fact table	empty or `dynamicpruningexpression(true)` `RuntimeFilters`	partition the fact table by the join column itself; keep the dimension broadcastable	a layout decision
a repeated join shuffles both sides every run	two `Exchange` nodes under the join	bucket both tables on the key, same count	write cost; fixed bucket count
a write produces thousands of tiny files or a few huge ones	file sizes in the output	`REBALANCE` before the write	a shuffle
a stage got slower after `coalesce`	few tasks in the scan stage	`repartition(n)` instead	a shuffle
the scan's column list includes unneeded columns	wide column list on the scan	project columns explicitly; avoid `SELECT *` in views	none

Table 7.7: The pruning and layout levers: each symptom, the evidence in the plan, the fix, and what it costs.

uning.enabled set to false, then true, then true with spark.sql.autoBroadcastJoinThreshold set to -1. Find the RuntimeFilters in each plan and compare the fact scan's task count.

3. **Predict, then check.** orders scans as 560 splits. Predict the number of tasks and the number of waves on 40 slots for the scan stage with coalesce(10) before a write, and with repartition(10). Then run both and compare the stage's task count in the **Stages** tab.

Chapter 8

Caching and Reuse

Caching keeps a result Spark would otherwise recompute — but the wrong cache wastes storage memory and starves execution, and a cache used once is pure overhead. On the lab, caching one week of `orders` *for two aggregations was slower than not caching at all; it needed about ten reads to break even. Spark also reuses work you did not ask it to: an identical exchange computed once and read twice, which cut a join from 10.0 to 7.5 seconds. This chapter covers when caching pays, which storage level to pick against the GC budget from Chapter 3, how to see reuse in a plan, and what happens when execution and storage memory finally meet at runtime. Every number is a capture from the lab of Section A.3.*

8.1 When caching pays

A cached DataFrame pays for itself only when two things are both true: it is read enough times to recover the cost of building the cache, and it fits comfortably inside the executor's storage memory (Section 3.3) without pushing out other useful cached data. Caching something read exactly once adds the cost of writing it into storage memory (and, if it does not fit, spilling it to disk) for a saving that never arrives. The classic mistake is caching immediately before a single `write()`.

How many reads is enough is a number you can measure rather than guess. The lab cached one week of `orders` (6.8 million rows, 563 MB cached) and ran the same

aggregation N times, with and without the cache. Table 8.1 and Figure 8.1 show the result.

Reads N	No cache	Build cache	N cached reads	Cache total
1	2.77 s	4.94 s	1.66 s	6.60 s
2	3.15 s	4.13 s	2.56 s	6.69 s
5	7.54 s	4.17 s	5.40 s	9.57 s
10	14.37 s	4.30 s	10.40 s	14.70 s

Table 8.1: Wall-clock seconds for N aggregations over one week of `orders`, single runs on a warm lab JVM. The cache costs about 4.2 s to build and saves about 0.4 s per read, so it only catches up near the tenth read.

The arithmetic generalizes. If building the cache costs M seconds, an uncached read costs R, and a cached read costs C, caching pays after $M/(R - C)$ reads: here $4.2/(1.44 - 1.04) \approx 10$. The break-even moves with the source. A Parquet scan from local disk is cheap, so the cache needs many reads; a scan of object storage, or an expensive upstream join, makes R large and the break-even small. Measure R for your source before deciding.

```
-- filtered_orders is read by two different aggregations
filtered_orders.cache()
filtered_orders.count()                    -- materializes the
   cache
result1 = filtered_orders.join(dim_product, "product_id")
result2 = filtered_orders.groupBy("product_id").sum("
   order_total")
```

Listing 8.1: Reused twice: worth caching only if the reads are costly

8.2 Lazy `cache()` versus eager `CACHE TABLE`

`df.cache()` does not cache anything by itself — it only marks the DataFrame for caching the next time an action runs it, consistent with transformations being lazy everywhere else

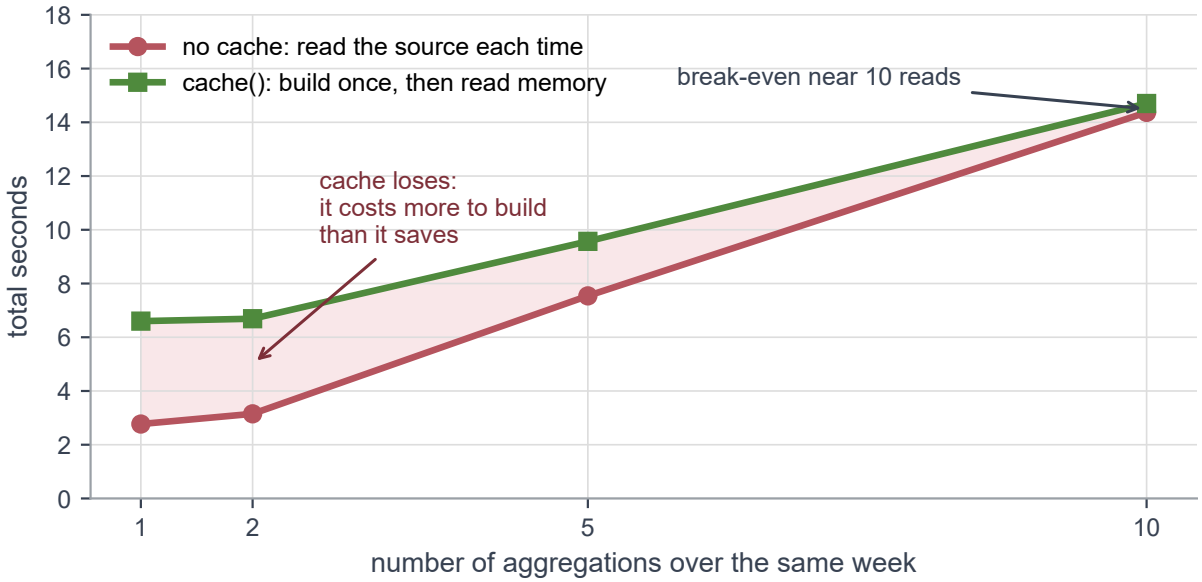


Figure 8.1: Total time for N aggregations over the same week, with and without `cache()`. The shaded gap is time the cache costs; the lines converge near ten reads because each cached read still scans 563 MB of cached batches against 58 MB of Parquet columns.

(Section 1.4). The lab's Storage tab shows it: Table 8.2 lists what was resident after each step.

Without a materializing action, the first real use of the DataFrame both computes it *and* populates the cache as a side effect, which works but hands you no control over when the cost is paid; a cheap action such as `count()` right after `cache()` pays it on your own terms. SQL's `CACHE TABLE` is the one place this default flips: it is *eager*, executing immediately. Use `CACHE LAZY TABLE` for the lazy behavior `df.cache()` gives you.

8.3 Storage levels and the GC trade-off

The default storage level for a DataFrame, `MEMORY_AND_DISK`, keeps cached blocks in storage memory and spills to disk only when they do not fit. What the blocks *are* matters for the arithmetic. Spark SQL caches a DataFrame as compressed columnar batches (`spark.sql.inMemoryColumnarStorage.compressed`, default `true`), not as millions of `Row` objects, so the per-object overhead of Section 3.2 applies to the RDD API, not here. The lab measured the same 6.8 million rows at three levels (Table 8.3).

Step	Cached partitions	In memory
`df.cache()`, no action yet	none	0
`df.count()`	7 of 7	563.3 MB
`CACHE LAZY TABLE wk_lazy AS …`	none	0
first query on `wk_lazy`	7 of 7	563.3 MB
`CACHE TABLE wk_eager AS …`	7 of 7	563.3 MB

Table 8.2: What the Storage tab holds after each step. Only `CACHE TABLE` caches before any query reads it.

How it was cached	In memory	On disk
`df.cache()` (Scala default, deserialized)	563.3 MB	0
`MEMORY_AND_DISK_SER` (serialized)	410.5 MB	0
`DISK_ONLY`	0	410.5 MB

Table 8.3: One week of `orders` at three storage levels. Serializing saved 27%, not the 2–3× that holds for caches of JVM objects.

So the deserialized default costs about a quarter more memory than serialized, and the serialized form must be decoded again on every read. Whether that trade is worth it depends on GC: a large, long-lived cache is the workload that pushes a collector toward its expensive full-heap path (Section 3.4). Two levels exist to trade that away:

- `MEMORY_AND_DISK_SER` keeps the cache serialized, one contiguous block per partition, at the cost of decoding rows back out on every read.
- `OFF_HEAP` (with `spark.memory.offHeap.enabled`) puts the cache outside the JVM heap, the pool from Section 3.3: zero GC overhead for that data, at the cost of the off-heap capacity it consumes. The lab cached 30 million rows (three columns) at `OFF_HEAP` with a 1 GB off-heap pool: Spark's memory manager showed 1,063 MB of storage in the off-heap pool and 0.1 MB on-heap, and the pool being full, 310 MB of the cache went to disk (the level reported was *disk, memory, offheap*). Off-heap caching therefore keeps the bytes out of the heap exactly as claimed, and a pool smaller than the data silently spills to disk rather than failing.

`DISK_ONLY` is rarely the right answer: reading a cache from disk is close enough to re-reading the source that the cache buys little.

> **Warning** PySpark's `StorageLevel.MEMORY_AND_DISK` is the *serialized* level (its `deserialized` flag is false), while Scala's `df.cache()` uses the deserialized one. On the lab the two differ by 153 MB for the same rows, which is the 27% of Table 8.3. Check the Storage tab's level column rather than trusting the constant's name.

Switch away from the deserialized default when GC time is 5–10% or more of task time for a job that caches heavily (Section 15.3). Below that, the decoding cost of a serialized level is not worth paying.

8.4 How execution and storage memory borrow at runtime

Section 3.3 described unified memory's split between execution and storage as a soft boundary set by `spark.memory.storageFraction`; here is what that means while a job actually runs. Either side may use the other's *idle* space: a job with nothing cached lets execution use the whole unified pool for shuffles and joins, and a job running no wide operators lets storage use all of it for cached blocks.

The boundary only matters once both sides want space at the same time. Suppose storage's own half of a 4 GB unified pool is 2 GB. A pipeline caches `filtered_orders` and the cache grows to 3 GB: 2 GB in storage's own half plus 1 GB borrowed from execution's idle half. A join then starts and its sort buffers need 1.5 GB. Execution is entitled to reclaim its space, so Spark evicts blocks from the borrowed 1 GB (LRU order); if the join later needs more than its own half, it spills to disk rather than blocking. Run the scenario in reverse — the join first, then the cache — and storage cannot evict a word of the join's live hash table; it is capped at whatever its guaranteed half has free until the join releases the borrowed space. The two directions are not symmetric:

- if execution needs memory that storage is borrowing, Spark evicts cached blocks (LRU) to reclaim it, down to storage's own guaranteed half;
- if storage needs memory that execution is borrowing, Spark cannot evict a running task's hash table or sort buffer; storage waits until the task finishes.

Execution always wins a live contest, because failing to complete a running operator is worse than evicting a cache entry and recomputing it later (Figure 8.2).

The lab reproduced the eviction. With a 4 GB heap (about 2.3 GB of unified memory) the whole month of `orders` was cached at `MEMORY_AND_DISK`: 2253 MB in memory and 176 MB on disk, all 31 of 31 partitions resident. A join to `dim_customer` then ran in the same application and needed execution memory. Afterwards the cache still held 31 of 31 partitions, but 1931 MB in memory and 410 MB on disk: 322 MB of blocks had moved from memory to disk to make room, arriving as 235 MB because the disk

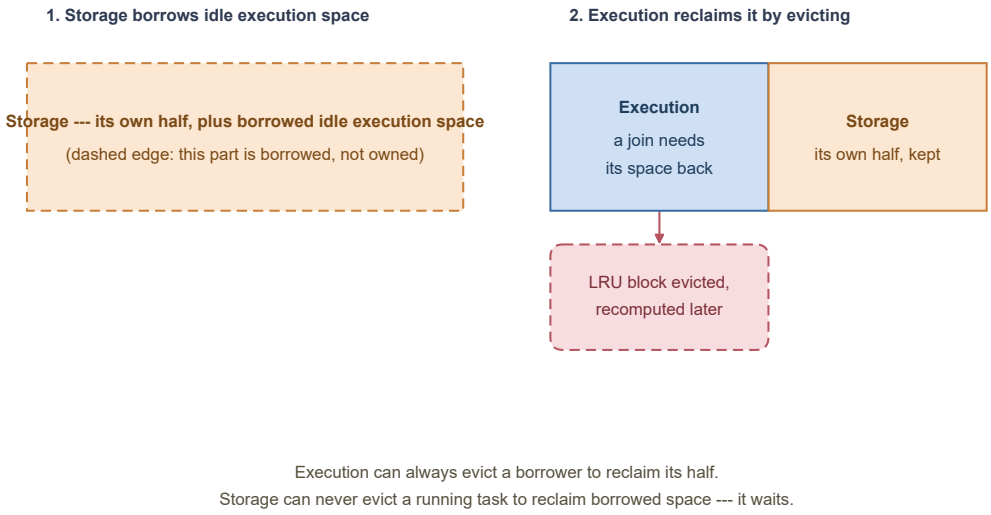


Figure 8.2: Borrowing is symmetric; reclaiming is not. Storage can spread into idle execution space, but the moment a join or sort needs that space back, execution evicts the borrower's cached blocks to reclaim it — storage has no equivalent power over a running task.

copy is serialized (Figure 8.3). Because the level includes disk, nothing was lost; with `MEMORY_ONLY` those blocks would have been dropped and recomputed.

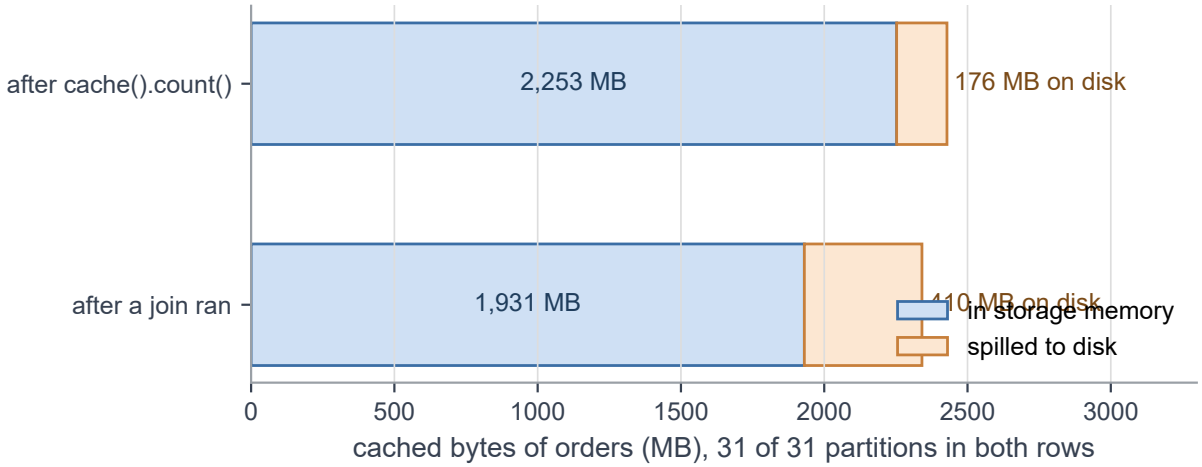


Figure 8.3: A cached month of `orders` before and after a join ran. All partitions stay cached, but 235 MB more of them sit on disk once execution claimed the memory it needed.

8.5 Exchange and subquery reuse

Caching is a decision you make; Spark also reuses identical work on its own when the physical plan contains the same subtree twice. If two operators need data shuffled the same way — same partitioning, same partition count, same aggregate — the planner reuses one exchange for both (`ReuseExchange`, Section C.4); an identical scalar subquery appearing more than once is deduplicated the same way (`ReuseSubquery`). Both are visible in the physical plan and need no caching from you. Figure 8.4 and Table 8.4 measure the effect on a self-join of a per-product aggregate.

Reuse is fragile in exactly the way Section 4.7 warned about: a partition count, column set, or aggregate function that differs by even a little between the two occurrences defeats the match. When the optimizer can drop an unused aggregate it will, and two sides that look different can collapse into one that is reused, so read the plan rather than the query: look for a `ReusedExchange` node, and count the scan stages.

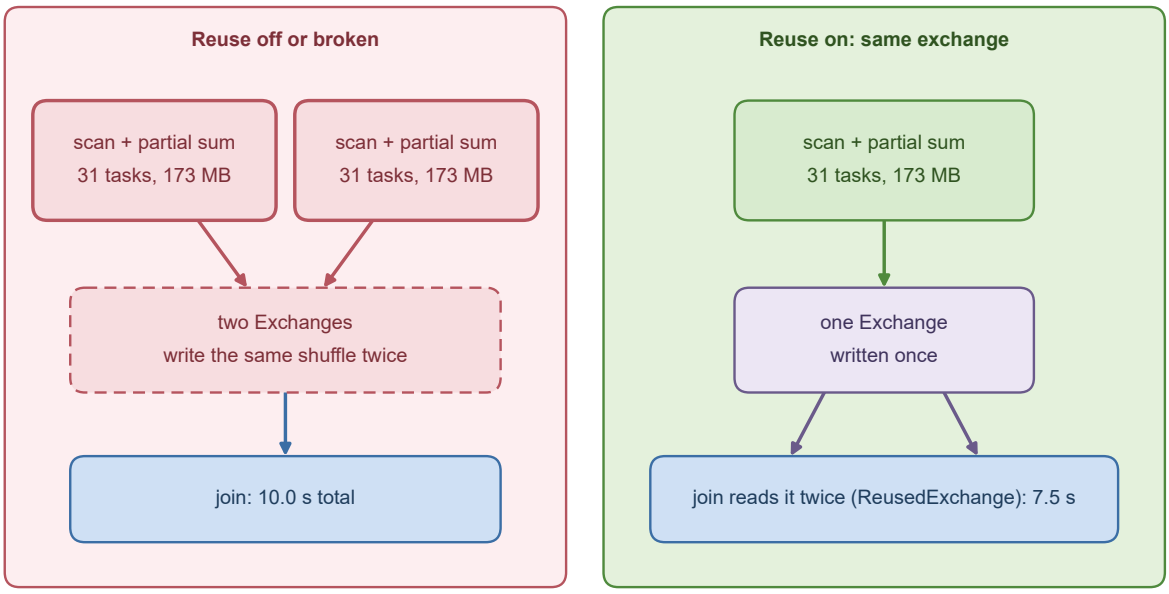


Figure 8.4: The same self-join of an aggregate planned two ways. With reuse, the scan and exchange run once and the join reads the result twice (`ReusedExchange`); without it, the scan and the shuffle write run twice.

Plan	Scan stages	`ReusedExchange`	Wall clock
same aggregate twice, reuse on	1 (173 MB)	1	7.5 s
same aggregate twice, `spark.sql.exchange.reuse`=false	2 (346 MB)	0	10.0 s
`sum` on one side, `max` on the other	2 (346 MB)	0	10.2 s

Table 8.4: Reuse on the lab. Disabling it doubles the scan and costs about a third more time. The last row is the fragile case: two aggregates over the same table and key differ in their aggregate function, so the exchanges differ and nothing is reused.

8.6 Unpersist and cache thrashing

A cached DataFrame you no longer need should be explicitly released with `unpersist()`. Left alone, storage memory fills with DataFrames nobody reads, and Spark's LRU eviction discards whichever cached *block* was used least recently to make room for new ones. Eviction happens per block, one per partition, not per DataFrame, so a large cache is evicted piecemeal.

The lab shows it. With the unified pool shrunk to about 1.1 GB (`spark.memory.fraction` = 0.3 on a 4 GB heap), three 8-day slices of `orders`, 469 MB each at `MEMORY_ONLY`, cannot all stay resident (Figure 8.5). After caching `ca`, `cb`, then `cc`, `ca` had lost 4 of its 8 partitions, the least recently used blocks, while `cb` and `cc` were whole. Re-reading `ca` then took 8.5 s, because the missing half was recomputed from the source, and re-caching it evicted half of `cb` in turn. Re-reading `cc`, still fully cached, took 0.4 s.

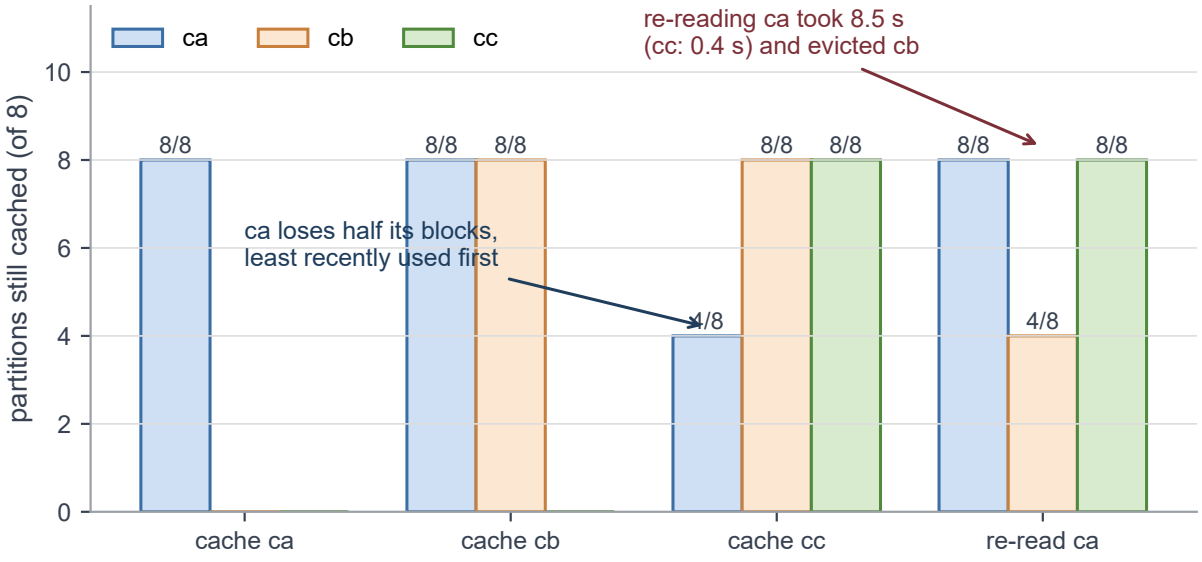


Figure 8.5: Three caches that do not fit together. Each new cache evicts the least recently used blocks of an older one, and re-reading the damaged cache evicts its neighbour: LRU has no notion of which DataFrame you still need.

LRU picks purely on recency of read, so it can evict the cache you still need before its next read, forcing a recompute you cached specifically to avoid. This *cache thrashing* is a caching bug that looks like a memory bug: the fix is fewer, more deliberate caches (and prompt `unpersist()` on ones you are done with), not more memory.

> **Note** Storage memory protects a cache from being evicted, not the executor from running out of heap. On the lab, caching a third 586 MB slice with a 4 GB heap and eight concurrent scan tasks ended in `java.lang.OutOfMemoryError: Java heap space`; the unified pool is soft, but the JVM heap is not.

8.7 Summary

Cache what is read enough times to repay the build and fits the storage budget; on the lab that took about ten reads of a cheap source. `cache()` only marks a DataFrame lazily, so materialize it on your own schedule with a cheap action; `CACHE TABLE` in SQL is eager by contrast. A DataFrame cache is columnar batches, so the serialized level saves about a quarter, not a multiple. Execution and storage share unified memory and may borrow each other's idle space, but only execution can evict a borrower to reclaim it: the lab saw 322 MB of a cache move to disk when a join started. Spark reuses an identical exchange on its own, fragile to any difference between the two occurrences; and a cache you forget to release does not just waste memory, it can evict the cache you still need. Table 8.5 turns each symptom into the evidence and the lever.

The rule the chapter repeats: **a cache is a bet on the number of reads, so measure the break-even before placing it**. Chapter 9 turns from keeping data in memory to the table format that organizes it on storage.

Exercises

1. **Run and observe.** Aggregate one week of `orders` 1, 2, 5, and 10 times with and without `cache()`, and fill in a row of Table 8.1. Where do your lines cross, and what does the crossing depend on?

2. **Read a plan.** Self-join a per-product `sum` aggregate and find the `ReusedExchange`. Change one side to `max`, and count the scan stages before and after.

3. **Predict, then check.** With a unified pool that holds two of three equal caches, predict how many partitions of the first cache survive after caching the third, and what a re-read of it costs. Then confirm in the Storage tab.

Symptom	Evidence	Lever	Cost
a cache made the job slower	build time above the saving; few reads	remove the cache; compute the break-even	none
a cached DataFrame is not resident	empty **Storage** tab after `cache()`	run a materializing action; `CACHE TABLE` is eager	the build cost, paid up front
heavy GC with a large cache	GC time at 5–10% of task time	serialized or off-heap level	decoding, off-heap capacity
a re-read is slow though cached	partial `cached/total` in **Storage**	fewer caches; `unpersist()` old ones	recompute on a miss
a plan scans the same table twice	two scan stages, no `ReusedExchange`	make the two sides identical	none

Table 8.5: The caching and reuse levers: symptom, evidence, lever, and cost.

Part III

The Iceberg Lakehouse

Chapter 9

Table Formats and the Iceberg Spec

An Iceberg table is not a directory of Parquet files with a manifest bolted on. It is a tree: a catalog pointer to a metadata file, to a manifest list, to manifests, to data and delete files — every level immutable, every commit a single atomic pointer swap. Reads consult this tree before touching any data. This chapter walks the spec (format version 2) against the metadata tables of a table the lab of Section A.3 built with three commits and two deletes, so you can draw the tree from memory.

9.1 From Hive tables to table formats

A classic Hive table's truth is the filesystem: a metastore records a table's schema and its partition-to-directory mapping, but knowing which *files* exist inside a partition means listing the directory at query time. That listing is slow at scale (Section 1.4's driver work before any task runs), and it is not atomic: a writer that has written three of five new files and a reader that lists the directory in between sees a table that never existed, with no snapshot to fall back to and no way to tell that a write is in progress.

A *table format* — Iceberg, Delta Lake, Hudi are the three in wide use — moves that truth out of the filesystem and into an explicit metadata layer the engine reads instead of listing directories. Every file that belongs to the table is named individually, in a structure written once and never mutated in place; a commit is complete the instant one pointer is swapped to reference a new version of that structure, and a reader either sees the whole

new version or the whole old one, never a partial write. Everything else in this chapter is what that metadata layer is actually made of.

9.2 The Iceberg spec, format version 2

Iceberg's metadata is a tree of four kinds of file, each immutable once written, each read by the layer above it, top to bottom (Figure 9.1):

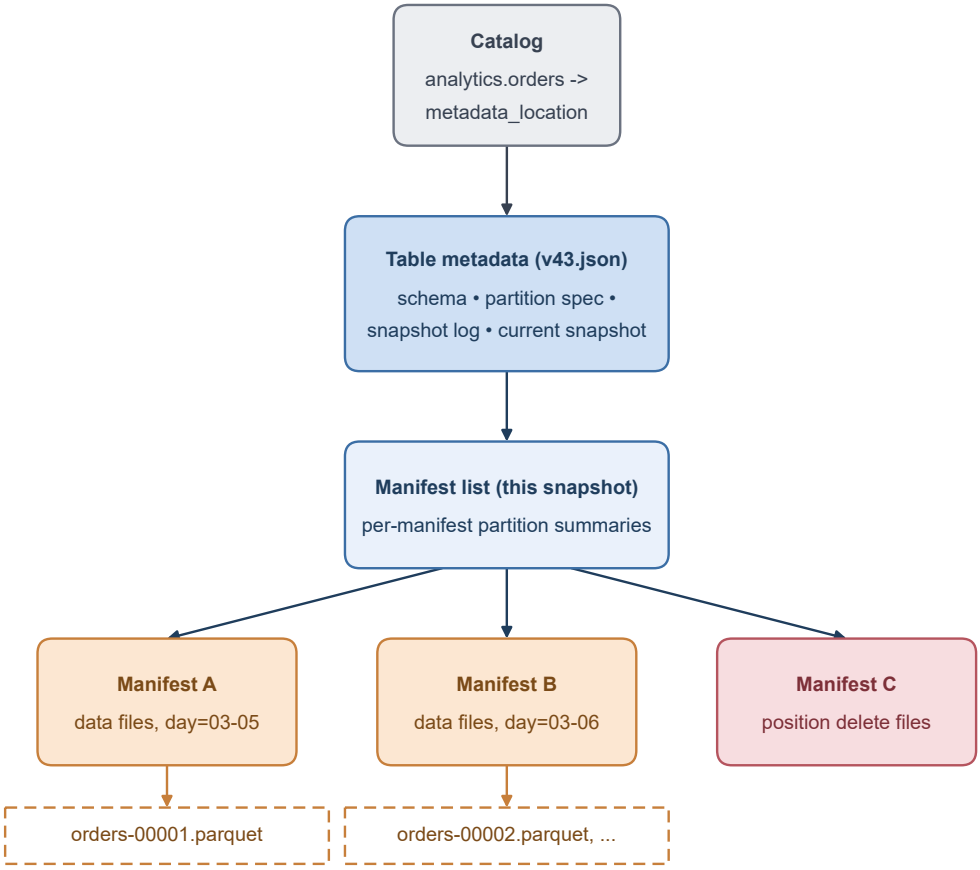


Figure 9.1: The metadata tree for one snapshot of `orders`. The catalog resolves to one table metadata file, which points to one manifest list, which points to the manifests that make up this snapshot, which list the actual data and delete files. Every level is immutable; a new commit writes new files at whichever levels changed and leaves the rest untouched.

Table metadata file (JSON) is the object a catalog entry points to. It holds the current schema, the partition spec, the full list of snapshots and a *snapshot log*, and the current snapshot's pointer to its manifest list. This is the one file a reader must lo-

cate through the catalog; everything below it is found by following pointers already inside it.

Manifest list (Avro), one per snapshot, is the list of manifest files that make up that snapshot, each entry carrying the manifest's path, its partition-spec ID, and summary statistics (row counts, per-column value ranges) for the partitions it covers — enough for a reader to skip an entire manifest without opening it, the first pruning layer Chapter 12 builds on.

Manifests (Avro) each list a set of data files *or* delete files (never both) as `manifest_entry` records, one row per file, carrying the file's path, format, partition values, per-column min/max and null counts, and the sequence number it was added at. Manifests are grouped by partition spec and reused across snapshots when their files have not changed, which is why an append does not rewrite every manifest in the table — only the ones touched by that commit. Concretely: appending one new day's worth of `orders` writes exactly one new manifest, listing only that new day's data files; the commit's new manifest list then references that one new manifest plus every manifest from the previous snapshot, unchanged and unrewritten, since none of those older files were touched by this append.

Data and delete files (Parquet on this stack) are the files themselves; delete files are the subject of Section 9.4.

9.2.1 The tree, measured

The lab created `t_meta` partitioned by day with one `CREATE TABLE AS SELECT` of three days, then two appends of one day each. Figure 9.2 is the tree after the third commit, and Listing 9.1 shows the three metadata tables that describe it.

```
snapshot_id          parent_id            operation  manifest_list
 3556728392267893512  NULL                 append     snap-3556...-ff6b38d0
 -3...
 4288545161950762915  3556728392267893512  append     snap-4288...-6561136d
 -3...
 6626548609302216676  4288545161950762915  append     snap-6626...-2c4a314e
 -5...
```

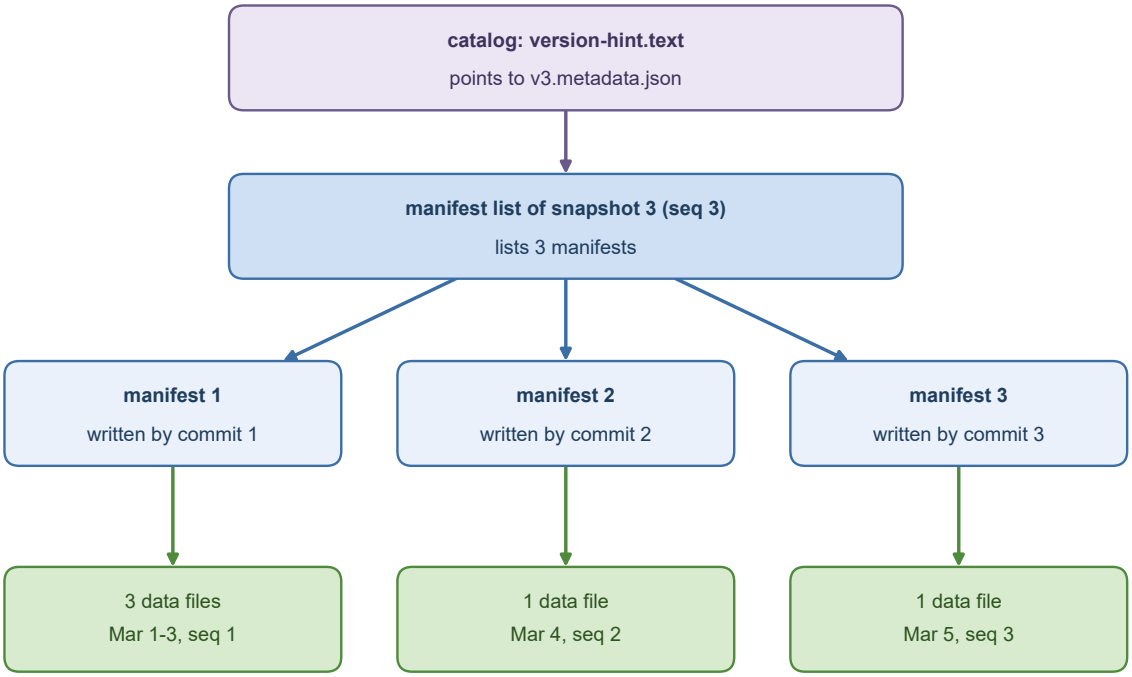


Figure 9.2: The metadata tree of `t_meta` after three commits, as the lab's metadata tables report it. The catalog points to `v3.metadata.json`; the snapshot-3 manifest list names three manifests, one written by each commit, which list five data files.

```
all_manifests  (snapshot -> manifest, files added by it)
 snapshot 1 -> m0 (3 files)
 snapshot 2 -> m0 (3 files, reused),  m0' (1 file)
 snapshot 3 -> m0 (3 files, reused),  m0' (1, reused),  m0'' (1 file)

metadata_log_entries
 v1.metadata.json  seq 1
 v2.metadata.json  seq 2
 v3.metadata.json  seq 3
```

Listing 9.1: The snapshots, manifests, and metadata log of `t_meta` after three commits (trimmed)

The tables confirm what the spec says. Each commit wrote exactly one new manifest and one new manifest list, and Figure 9.3 draws the reuse: the snapshot-3 manifest list names the manifests earlier commits wrote, byte for byte unchanged, plus the one new manifest. An append never rewrites an existing manifest. Each snapshot's manifest list is a complete description of the table at that moment, which is why a reader needs only the one it resolved.

On disk the same table, after two further deletes and one property change, holds exactly six `metadata.json` files (`v1` to `v6`), five manifest lists, and six manifests, plus a `version-hint.text` that the file-based catalog uses as its pointer. The sixth metadata file is worth a sentence: `ALTER TABLE … SET TBLPROPERTIES` wrote a new `metadata.json` and no new snapshot, because properties live in the metadata file and not in the data tree. The lab's catalog is file-based, so the pointer swap is a rename of that hint; a service-backed catalog swaps a different pointer, the subject of Chapter 10.

> **Note** Nothing in this tree is a directory listing. A reader that wants to know which files exist reads exactly the metadata file, manifest list, and manifests that the current snapshot's pruning has not already excluded — never the filesystem. Chapter 12 is entirely about how much of that tree a well-formed query gets to skip.

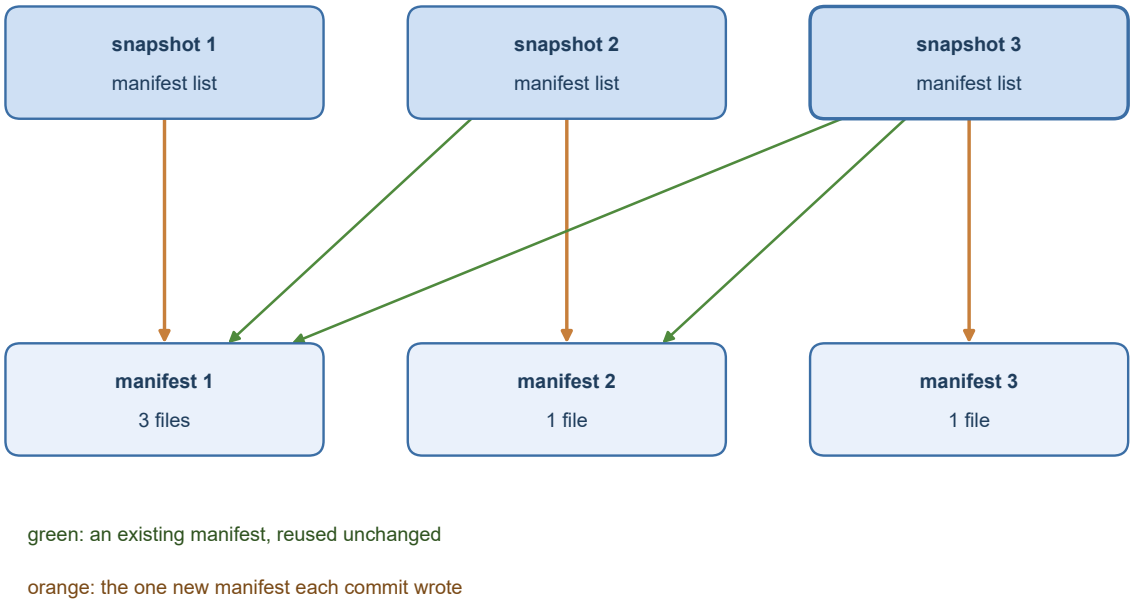


Figure 9.3: Three snapshots of one table. Each commit writes one new manifest (orange) and a new manifest list, and the list also names every earlier manifest unchanged (green). Older snapshots keep their own lists, so time travel reads them untouched.

9.3 Snapshots and the metadata tree

A *snapshot* is one committed version of the table: a `snapshot-id`, a pointer to its manifest list, an optional `parent-snapshot-id` linking it to the version it replaced, a `timestamp-ms`, and — the detail that makes format v2 usable for row-level deletes — a monotonically increasing `sequence-number`. Every file recorded anywhere in the tree inherits the sequence number of the snapshot that added it, and that number is what lets a reader decide, cheaply, whether a given delete file is even relevant to a given data file: the sequence numbers decide which delete files can apply to which data files, and the rule differs by delete type. A *position* delete file applies to a data file whose sequence number is less than *or equal to* its own, and an *equality* delete applies only to data files with a strictly smaller number.

A data file written after a delete was committed has a larger number and is never touched by it, so a reader skips the delete file without opening it. Figure 9.4 shows the numbers the lab recorded: data files were added at sequence numbers 1, 1, 1, 2, and 3; a copy-on-write delete replaced one of them with a rewritten file at 4; then a merge-on-

read delete committed a position delete file at 5. That delete applies to every data file numbered 5 or lower, and the next append, at 6, will not be checked against it.

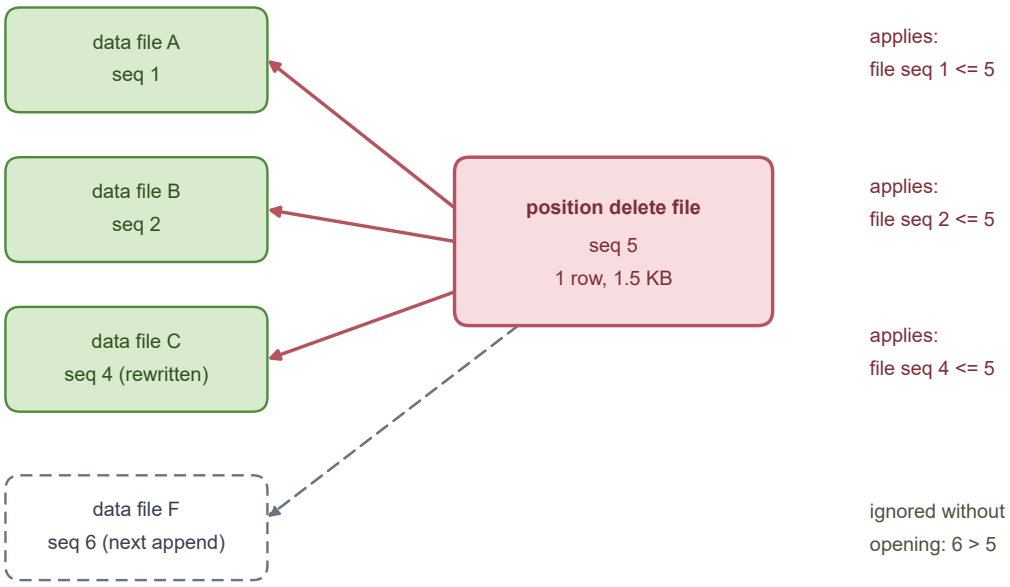


Figure 9.4: A position delete at sequence number 5 applies to the data files at 1, 2, and 4 and not to a later append at 6. File `C` carries number 4 because a copy-on-write delete rewrote it; sequence numbers record when a file was *added*, not when its rows were first written.

The table metadata file's *snapshot log* is the ordered history of every snapshot the table has ever had, and it is what powers time travel — `VERSION AS OF` / `TIMESTAMP AS OF` in Spark SQL resolve to a past `snapshot-id` by walking this log, then read that snapshot's manifest list exactly as if it were current. The lab confirms it: counting rows `VERSION AS OF` each of the three snapshots returned 2,904,029, 3,873,680, and 4,841,619, matching the three days, four days, and five days committed. A commit does not delete an old snapshot's entry from this log; that is a separate, deliberate operation (`expire_snapshots`, Chapter 13) — until it runs, the old files a superseded snapshot referenced are still reachable, which is also why an append does not immediately reclaim the disk space of files it replaced.

9.3.1 The commit as an atomic pointer swap

Writing new data and delete files, and new manifests and a new manifest list to describe them, does not change the table at all — none of it is visible until the table metadata file's pointer is swapped to reference the new snapshot, and that swap is the one atomic operation the catalog performs (Chapter 10 covers exactly how). A reader holding an open scan against the old snapshot is unaffected by a commit happening concurrently: it keeps reading the manifest list it already resolved, which still points at files that still exist, because nothing in this tree is ever mutated in place.

9.4 Position deletes versus equality deletes

Format v1 could only append or replace whole files; format v2's core addition is deleting or replacing individual *rows* inside an existing, untouched data file by recording the deletion in a separate delete file instead of rewriting the data file itself. Two encodings exist, and they trade off *write* cost against *read* cost in opposite directions.

Position deletes identify a deleted row by `(file_path, pos)` — the exact data file and its 0-indexed row ordinal. Applying one at read time is cheap: merge the sorted list of deleted positions against the data file's rows as they stream past, an operation with no lookup and no hashing. Writing one requires knowing the row's exact position, which means the writer has usually already read the data file — the natural output of a `MERGE` or `DELETE` that identifies matching rows during a scan (Section 13.2).

Equality deletes identify a deleted row by column value — `id = 5` — recorded once, independent of which data files happen to contain a matching row. Writing one is cheap, because the writer needs to know only the key, not any row's position; it is the only option available to a writer that never reads the data it is deleting, which is exactly the position a streaming or CDC job is in (Chapter 16). Applying one at read time costs a value comparison against every row of every data file it could apply to, which is why a table accumulating equality deletes without compaction gets measurably slower to read from over time.

Whether a Spark `DELETE` writes a delete file at all is a table setting, not a property of the statement. The lab deleted one row of a 969,000-row day twice: once with the default, and once after setting `write.delete.mode` to `merge-on-read` (Table 9.1 and Figure 9.5).

Mode	Snapshot operation	Data files	Bytes written
copy-on-write (default)	`overwrite`	1 added, 1 deleted	34,781,105
merge-on-read	`delete`	unchanged	1,492, in one position delete file

Table 9.1: Deleting one row, measured. Copy-on-write rewrote the whole 34.8 MB data file; merge-on-read wrote a 1.5 KB file that records the file path and the row position.

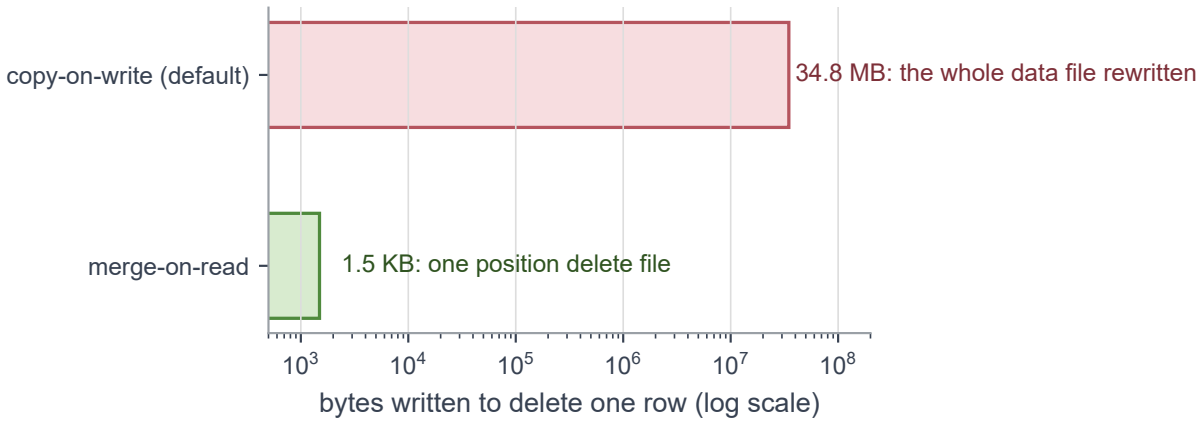


Figure 9.5: Bytes written to delete a single row: the whole data file under copy-on-write, one tiny delete file under merge-on-read. The log scale spans more than four orders of magnitude.

The price of the small write comes later, at read time: until compaction, every scan of that data file must apply the delete. The `files` table shows the shape directly: after the merge-on-read delete the table held five data files (content 0) and one delete file (content 1), and the delete file was the only newly written file. Chapter 13 turns this into a maintenance schedule.

> **Tip** A batch `MERGE INTO`, `UPDATE`, or `DELETE` on this stack writes position deletes *only if* the table's corresponding `write.*.mode` property is `merge-on-read`; by default it rewrites the affected data files, and the table never accumulates delete files at all. A table only accumulates equality deletes if something writes them deliberately, typically a streaming upsert; Spark SQL's own row-level statements write position deletes. If read performance is degrading on a table you know has no delete files, the cause is file count and pruning, not deletes (Chapter 12).

9.5 Iceberg, Delta Lake, and Hudi

All three formats solve the same problem this chapter opened with — metadata as truth instead of directory listing — with different defaults. Table 9.2 is not a ranking; it is what to expect walking into a table built on one of the others.

Table 9.2: Where the three table formats put their defaults

	Iceberg	**Delta Lake**	**Hudi**
Metadata	Manifest list + manifests (Avro)	JSON + Parquet transaction log	Timeline + per-partition metadata
Default write mode	Copy-on-write; merge-on-read opt-in per table	Copy-on-write; deletion vectors opt-in	Merge-on-read is the common case
Partitioning	Hidden, engine-computed (Chapter 11)	Explicit partition columns	Explicit partition columns
Primary engine origin	Netflix / multi-engine from the start	Databricks / Spark-first	Uber / Spark-first, CDC-oriented

The mechanism-level questions this book answers — what makes a commit atomic, why a delete file changes read cost, why hidden partitioning survives a scheme change — transfer directly to Delta and Hudi tables even though the concrete file layout differs;

only the exact metadata format is Iceberg-specific.

9.6 Summary

A table format replaces filesystem listing with an explicit, immutable metadata tree: a catalog-resolved metadata file points to a manifest list per snapshot, which points to manifests, which list data and delete files. A commit is one atomic swap of that top pointer, which is why a reader never sees a partial write. Format v2's sequence numbers let a reader tell which delete files apply to which data files without comparing every pair; position deletes are cheap to read and require the writer to have already scanned the row, while equality deletes are cheap to write and progressively more expensive to read as they accumulate. The next chapter covers exactly how that top-level pointer swap becomes atomic when two writers race for it.

Exercises

1. Commit three appends to a table, then inspect `metadata_log_entries`, `snapshots`, and `manifests`. Explain how many metadata files and manifests exist and why.

2. Delete one row from a copy-on-write table and then from the same table set to merge-on-read. Compare the snapshot summaries and the `files` metadata table, and explain the difference in bytes written.

3. Query a table with `VERSION AS OF` an old snapshot after a subsequent commit. Confirm from `metadata_log_entries` which manifest list that snapshot still points to.

Chapter 10

Catalogs and Commits

The catalog is the one component that turns a table name into the location of its current metadata — and the point at which a commit becomes atomic. Writing new metadata is not a commit until the catalog's pointer is swapped to it. When two writers race, one wins the swap and the other retries. This chapter covers what a catalog does, how the atomic swap works, and how to tune retries so a busy table does not fail commits under contention. The lab raced twelve writers for one table: 34 of 48 commits lost a swap and won on the second attempt, one gave up after four, and not a single row was lost or duplicated.

10.1 What a catalog does

Everything in Chapter 9's metadata tree is reachable once you have the path to the current table metadata file — the catalog's only job is to be the durable, atomically-updatable record of that one path per table name. Concretely, a catalog answers two questions: given `db.table`, what metadata file is current, and, on a commit, can I swap that pointer to a new file *only if* no one else has moved it since I last read it. Nothing about the manifest tree, delete files, or partitioning from Chapter 9 lives in the catalog at all — it is small and simple by design, because it is the one piece of the whole system every reader and writer has to agree on synchronously.

Catalogs differ only in *where* that one pointer lives and *how* the conditional swap is

enforced (Figure 10.1). The lab uses the simplest: a file-based catalog whose pointer is a `version-hint.text` file naming the latest `vN.metadata.json`, and whose swap is the creation of the next numbered file, which fails if that file already exists. That works on a filesystem with atomic create or rename; Iceberg's documentation advises against a file-based catalog on object stores whose rename is not atomic, which is why production deployments use a catalog backed by a service (Section 10.2).

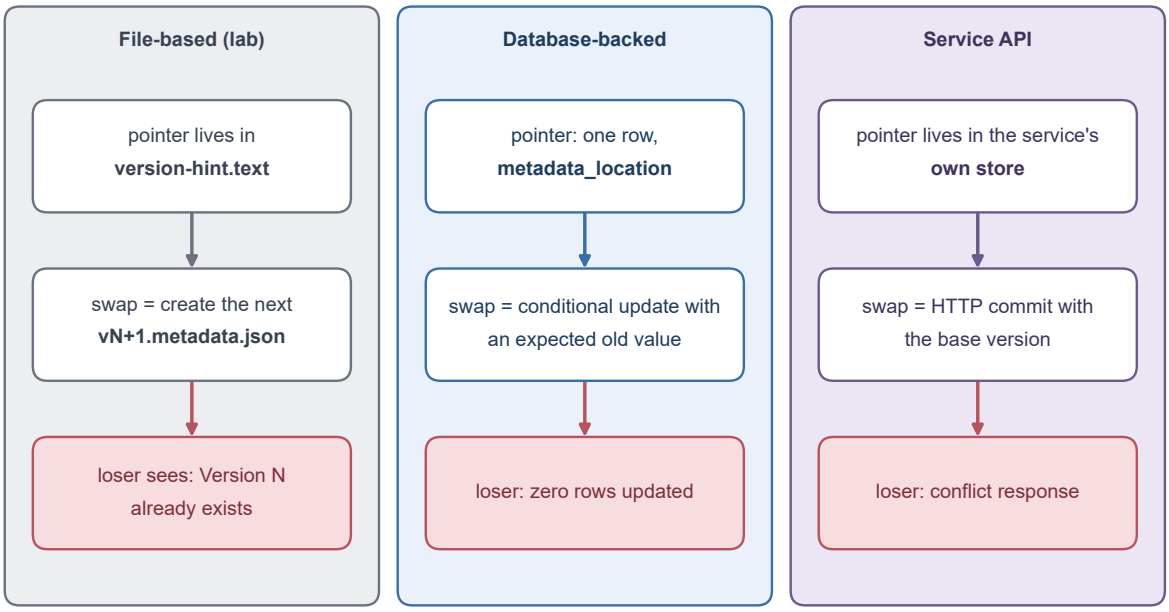


Figure 10.1: Three catalogs, one job. Each keeps a single pointer per table and swaps it conditionally; a writer that loses sees a rejection from whichever mechanism guards the pointer. Only the file-based catalog was run on the lab; the other two are described from the Iceberg documentation.

10.2 Service-backed catalogs

The lab ran one service-style catalog: Iceberg's JDBC catalog on a SQLite database. Its pointer is one row of a table `iceberg_tables` with columns `catalog_name`, `table_namespace`, `table_name`, `metadata_location` and `previous_metadata_location`; after a `CREATE` and one `INSERT` the row pointed at `00001-….metadata.json` and remembered `00000-…` as its predecessor. A commit is a conditional `UPDATE` that names the old location: replaying a stale writer's update (old location as the condition) changed **0 rows**, which

is the lost swap. REST and other service catalogs were not run; the description below is the mechanism. A service-backed catalog puts the pointer in a database or a service instead of a file. The commit is then one request carrying the base metadata file's identity alongside the new one, and the service rejects it with a conflict if its own record of the current metadata no longer matches that base — the same expected-old-value check the file-based catalog gets from "create the next file, fail if it exists," enforced by the service's own backing store (a database row, most commonly). Every client, whatever its language or engine, calls the same small API to read or commit a table, and the service can offer what a bare file cannot: multi-table transactions, or credential vending, in which the catalog hands each client short-lived storage credentials scoped to the one table it asked for. The cost is one more service to run. Iceberg ships several such implementations; which one a table uses changes where the pointer lives, not the metadata tree beneath it.

> **Note** Every catalog answers the same two questions from Section 10.1; nothing in Chapter 9 or the rest of this book depends on which one a table uses. Pick based on what already exists in the platform and what the ecosystem around the table needs, not on a claimed metadata-format difference — there is none.

10.3 The atomic pointer swap

A commit's actual sequence, regardless of catalog:

1. read the current metadata file to get the base snapshot;
2. build new manifests, a new manifest list, and a new snapshot on top of that base, writing all of it to storage — none of this is visible to any reader yet;
3. write a new table metadata file referencing the new snapshot;
4. ask the catalog to swap its pointer from the base metadata file to the new one, *conditioned on* the pointer still being the base file.

Step 4 is the only step that touches shared, contended state; steps 1–3 are pure writes to storage that never conflict with another writer's steps 1–3, because each writer is writing

its own new files. This is exactly why Iceberg calls its concurrency model *optimistic*: every writer does the expensive part assuming it will win, and only the cheap, final compare-and-swap can fail.

10.4 Optimistic concurrency and retries

When step 4 fails because another commit already moved the pointer, Iceberg does not discard the writer's work — it re-reads the new current metadata file, re-applies the same logical change on top of it (a new manifest referencing the same underlying data/delete files, at most a cheap manifest rewrite rather than a data rewrite), and retries the swap. That rewrite is cheap specifically because a manifest is a small listing of file paths and statistics (Section 9.2), not the data itself — its size tracks the number of files touched by *this* commit, entirely decoupled from how large the table as a whole has grown. Table 10.2 controls how persistent that retry is.

Watching it happen. The lab raced twelve writer threads, each appending four small batches of 20,000 rows to one table: 48 commits that all reach step 4 within milliseconds of each other. Most of them lose at least one swap, and the Iceberg client reports each commit's *attempts* counter. Two messages show up in the log when a swap loses:

```
CommitFailedException: Version 14 already exists: .../t_race_r4/metadata/v14.
metadata.json
CommitFailedException: Cannot commit changes based on stale table metadata
```

Listing 10.1: Why a commit retried (from the lab's log, trimmed)

The first is the file-based catalog's compare-and-swap: two writers both built `v14.metadata.js` from the same base, one created it first, and the second found it already there. The second message is the client noticing before it tries. Either way the writer re-reads the current metadata and goes again, without surfacing an error to the job (Figure 10.4 shows the sequence). Table 10.1 and Figure 10.2 tally the outcome.

A retry is cheap, as the next section explains, and the lab measured it: a commit that won on the first try took 0.60 s on average and one that needed a second attempt 0.92 s, about 0.3 s more. The tail is the cost to watch: the one commit that needed four

`commit.retry.num-retries`	Commits	First try	Two attempts	Four attempts	Failed	Rows vs. expected
4 (default)	48	12	34	1	1	940,000 = 940,000
0	48	41	0	0	7	820,000 = 820,000

Table 10.1: Twelve writers, one table, 48 commits, measured. With the default, 34 commits needed a second attempt and one needed four; with no retries, 7 failed outright. In both runs the row count equals 20,000 times the commits that succeeded: a lost swap loses no data, only the attempt.

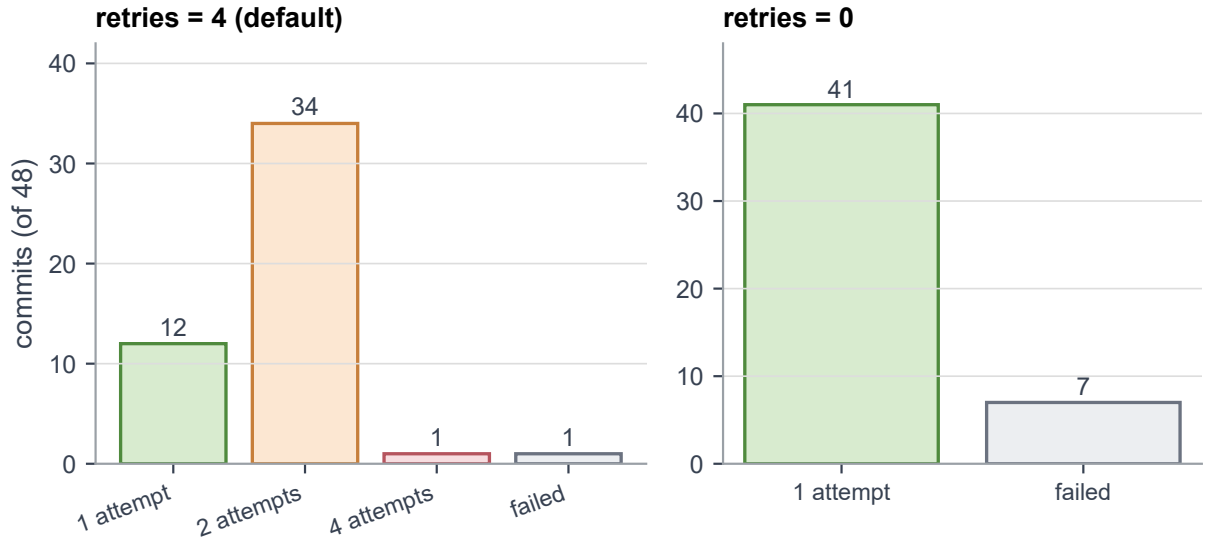


Figure 10.2: How many attempts each of the 48 commits needed. The default retries absorbed almost every lost race; with retries off, a lost race is a failed job.

attempts took 11.8 s, because each failed attempt backs off before it goes again. Retries also do not raise the table's ceiling. The 47 successful commits landed over 18 s, about 2.6 per second, and they form one line in the snapshot log: every snapshot's parent is the previous snapshot, however many writers there are (Figure 10.3).

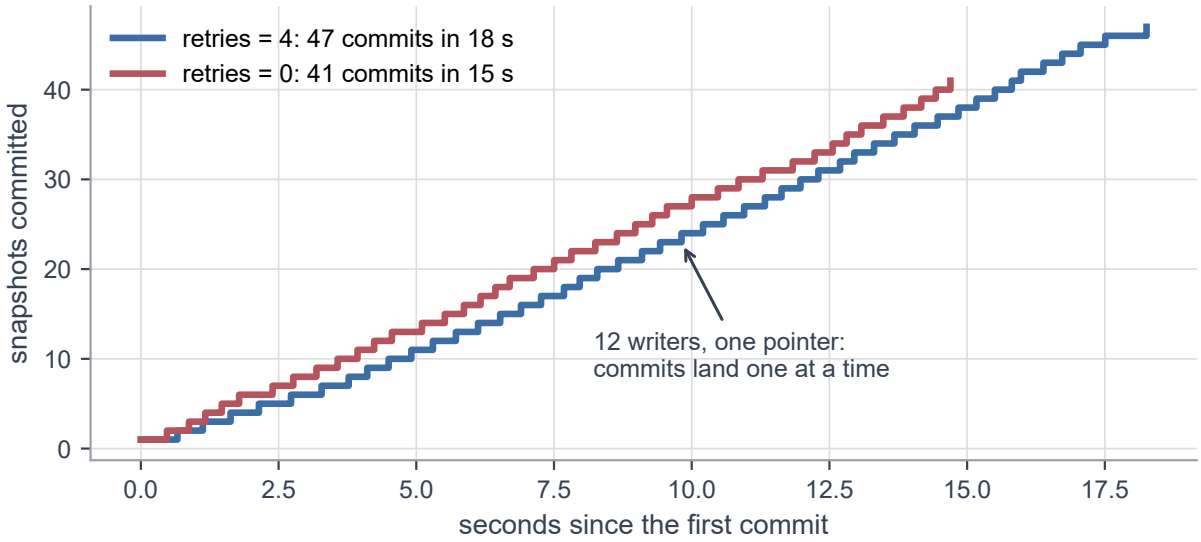


Figure 10.3: Snapshots committed over time for the two runs. Twelve writers still produce a single chain of commits: the catalog pointer admits one writer at a time, so the table's write rate is bounded by commit latency, not by writer count.

Table 10.2: Iceberg commit-retry table properties

Property	Default	Meaning
`commit.retry.num-retries`	4	Max retry attempts before failing the commit
`commit.retry.min-wait-ms`	100	Initial backoff between attempts
`commit.retry.max-wait-ms`	60,000	Backoff ceiling
`commit.retry.total-timeout-ms`	1,800,000	Overall time budget across all retries

A table with many concurrent writers hitting the same partitions — several streaming jobs upserting into one table, say — can exhaust the default 4 retries under sustained contention, as one commit of 48 did on the lab; raising `commit.retry.num-retries` (a table property, set via `ALTER TABLE ... SET TBLPROPERTIES`) trades a slower worst-case commit for a much lower failure rate. It does not help a table where every writer touches the *same* partition on every commit — retries only help when the underlying

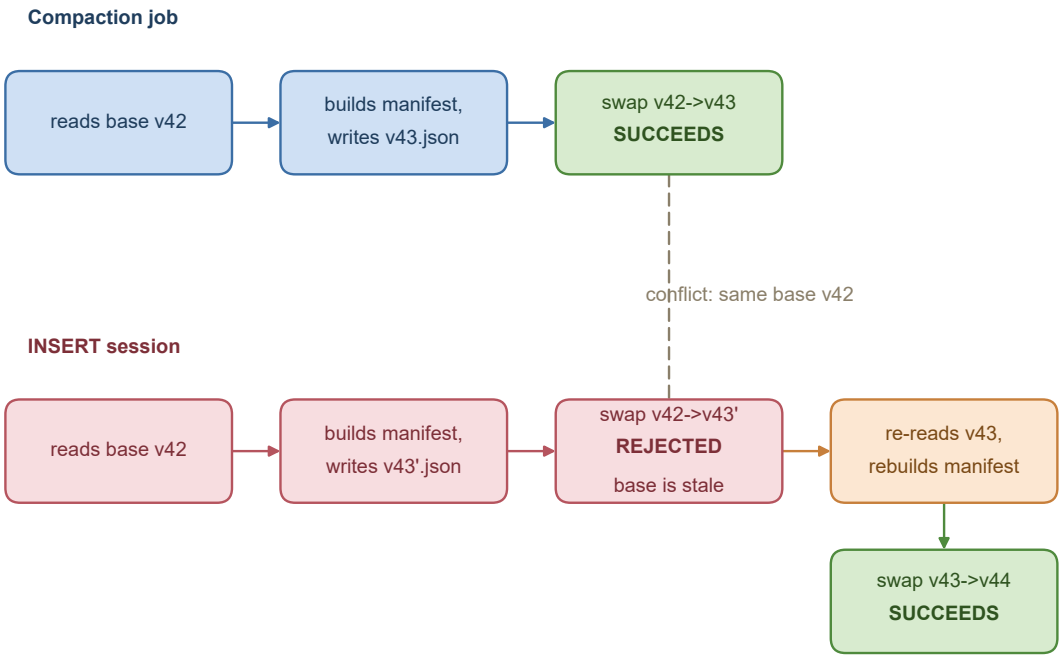


Figure 10.4: Schematic: two writers racing for the same base version. The first writer's swap lands first and succeeds; the INSERT session's swap against the same stale base is rejected, so it re-reads the new current version, rebuilds its manifest on top of it, and retries — succeeding one version later, with no error surfaced to either caller.

data actually merges cleanly with the new base, which Section 11.1's partition design controls more than any retry setting does.

> **Warning** Retrying a failed swap is safe because retries re-read the real current state and reapply on top of it — it is not blind resubmission of the same metadata file. A writer that instead cached the base snapshot and kept resubmitting it would silently lose every commit that happened in between.

10.5 Catalog configuration in Spark

A catalog is registered per Spark session under a name of your choosing:

```
spark.sql.catalog.lab=org.apache.iceberg.spark.SparkCatalog
spark.sql.catalog.lab.type=hadoop
spark.sql.catalog.lab.warehouse=/lab/warehouse
```

Listing 10.2: Registering the lab's file-based Iceberg catalog

Every table reference is then three-part, `lab.db.orders`, resolved through that specific catalog — which matters the moment more than one catalog is registered (say, a file-based one for experiments and a service-backed one for shared tables): the catalog name in the identifier, not any global default, decides which metadata store and which commit protocol from this chapter actually apply.

> **Note** `org.apache.iceberg.spark.SparkCatalog` has to actually be on the JVM's classpath for this registration to resolve to anything — the lab passes the Iceberg Spark runtime jar with `--jars`; it is not something set by this config block alone. A catalog name that fails to resolve with a `ClassNotFoundException` rather than a clear catalog-not-found error is almost always this: the config is correct, the jar is missing from the engine that tried to honor it.

10.6 Summary

The catalog's whole job is a name-to-current-metadata-file mapping that can be swapped atomically; the file-based catalog does this with a create-if-absent of the next meta-

data file, a service-backed catalog with a conditional update behind its API, and neither changes anything about the metadata tree itself. A commit writes all its new files optimistically, then attempts one conditional swap; on conflict it re-reads the real current state, reapplies the same change on top of it, and retries within the bounds of `commit.retry.*`, which is the lever to raise for a table under genuine concurrent-writer contention.

Exercises

1. Run twelve writers into the same table concurrently, four commits each. Count the commits that needed a second attempt and check that the row count equals the rows of the commits that succeeded.

2. Register the same physical warehouse location under two differently named catalogs in one Spark session. Confirm both resolve the same table to the same current snapshot.

3. Lower `commit.retry.num-retries` to 1 on a table and reproduce a commit failure under two concurrent writers; raise it back and confirm both succeed.

Chapter 11

Iceberg Partitioning and Layout

Iceberg partitioning is hidden: you filter on `order_ts` *and Iceberg derives the predicate on* `day(order_ts)` *for you, so a query never references a partition column directly and pruning does not break when the scheme changes. This chapter covers the partition transforms, evolving a spec without rewriting history, sort orders and Z-ordering, target file size, and the write-distribution mode — the choices that decide how much a reader and a writer have to move. On the lab, hour partitions turned 31 files into 744 for no pruning gain, Z-ordering read 3 files for a lookup that a customer sort needed 43 for, and one viral product made a bucket seven times its neighbours. Every number is a capture from the lab of Section A.3.*

11.1 Hidden partitioning and partition transforms

A classic Hive table's partition column is a real column duplicated into the directory path — a query has to filter on that exact derived value (`WHERE order_date = '2026-09-20'`) for pruning to fire, and a filter on the underlying timestamp (`WHERE order_ts >= …`) does not automatically resolve to it. Iceberg's partition spec instead names a *transform* of a source column — `day(order_ts)`, `bucket(16, customer_id)`, `truncate(4, category)` — and stores the computed value in the manifest, not as a queryable column at all. The planner derives the partition predicate from a filter on the source column itself, so the specimen query's plain `order_ts >= TIMESTAMP '…'` filter from Section 7.2

prunes correctly with no partition column ever appearing in the query text.

Table 11.1 covers the transforms this stack uses.

Table 11.1: Common Iceberg partition transforms

Transform	Use case	Notes
`day` / `hour`	Time-series data filtered by a timestamp range	Coarser (`day`) means larger partitions; `hou` high-volume streams ter 16)
`bucket(n, col)`	High-cardinality key with equality or join predicates	Even file distribut hash; enables s partitioned joins (jo skip the shuffle wh sides share the same spec, Section 5.8)
`truncate(n, col)`	String or numeric prefix filters	Groups by a value without one partition tinct value
`identity`	Low-cardinality column filtered directly	Same as classic parti just recorded the hidd

> **Tip** Prefer `day` over `identity` on a raw timestamp: an `identity` partition on `order_ts` would create one partition *per distinct timestamp value*, which is not partitioning at all — it is one file per row's worth of metadata. Always partition on a transform coarse enough to group many rows per partition. The lab tried it on one hour of orders (40,582 rows): an `identity` partition on `order_ts` wrote 3,600 files of 2.8 KB each, about eleven rows apiece, and took 32.4 s; the same hour under `hour(order_ts)` wrote one 1.4 MB file in 1.8 s.

Choosing the specimen's actual spec. The right transform is the coarsest one that keeps a typical partition's files near the target size, and the lab measures what finer costs.

It wrote the same 30 million rows twice, once partitioned by `day(order_ts)` and once by `hour(order_ts)` (Figure 11.1).

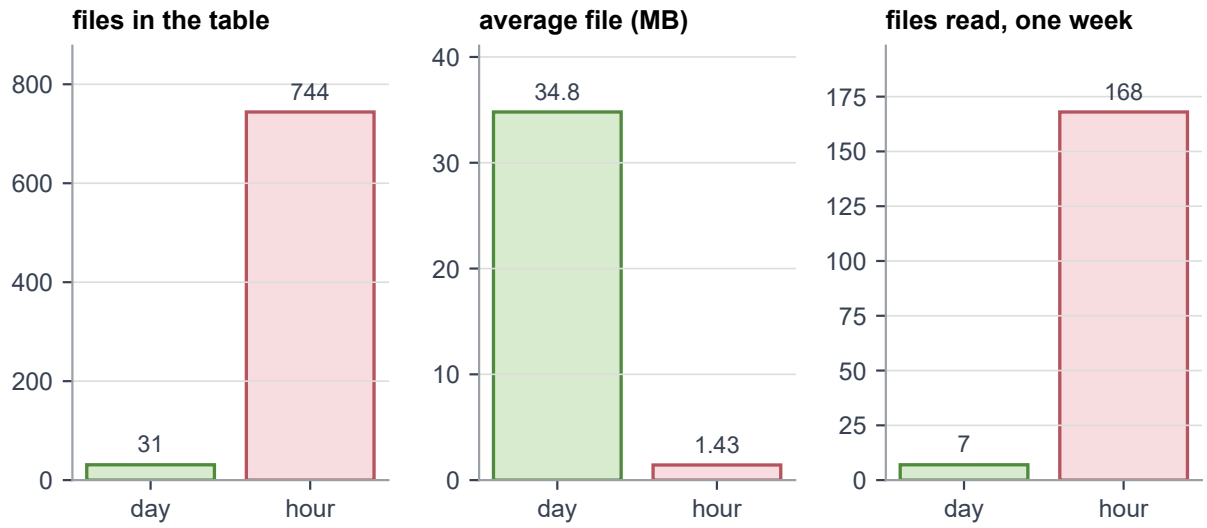


Figure 11.1: The same month of `orders` at two partition granularities. The hour spec has 24 times the files at one twenty-fourth the size, and a week-long query opens 168 files instead of 7 for the same bytes.

Spec	Files	Avg file	Files read (week)	Scan tasks	Planning
`day`	31	34.8 MB	7	7	48 ms
`hour`	744	1.43 MB	168	42	73 ms

Table 11.2: Day against hour for a one-week filter. Both read 6.8 million rows and about 47 MB; the hour layout reads 24 times the files and plans 50% slower, and its 1.4 MB files were packed four to a task only because the split planner combined them (Section 12.6).

A finer transform buys pruning only for queries narrower than the coarser unit. A one-week query gains nothing from hours and pays for them in file opens, task overhead, and manifest entries. Too coarse costs on the other side: a `month` partition of the first half of March held a single 517 MB file, and a one-day query against it decoded 11.3 million rows (Section 11.2). The lab's 35 MB-a-day partitions are already below the 128 MB floor this chapter's exercises set, which is the signal to stay at `day`; `hour` earns its keep only when a day's volume passes a few gigabytes and queries routinely ask for a few hours.

11.2 Partition evolution

A classic Hive table's partitioning is fixed at creation; changing it means rewriting every file under the new scheme. Iceberg's partition spec can change (`ALTER TABLE … ADD PARTITION FIELD`) without touching a single existing file, because the transform that produced a file's partition value is recorded per-manifest, not assumed globally — old manifests keep describing old files under the old spec, new manifests describe new files under the new spec, and a query spans both by evaluating each manifest against its own recorded spec.

This is why moving from `month(order_ts)` to `day(order_ts)` as a table grows is a metadata-only operation, not a maintenance project: new data lands under the finer scheme immediately, old data keeps the coarser scheme it was written with, and a query filtering by date prunes correctly against both — coarser pruning on the old partitions, finer pruning on the new ones, with no explicit migration step and no downtime (Figure 11.2).

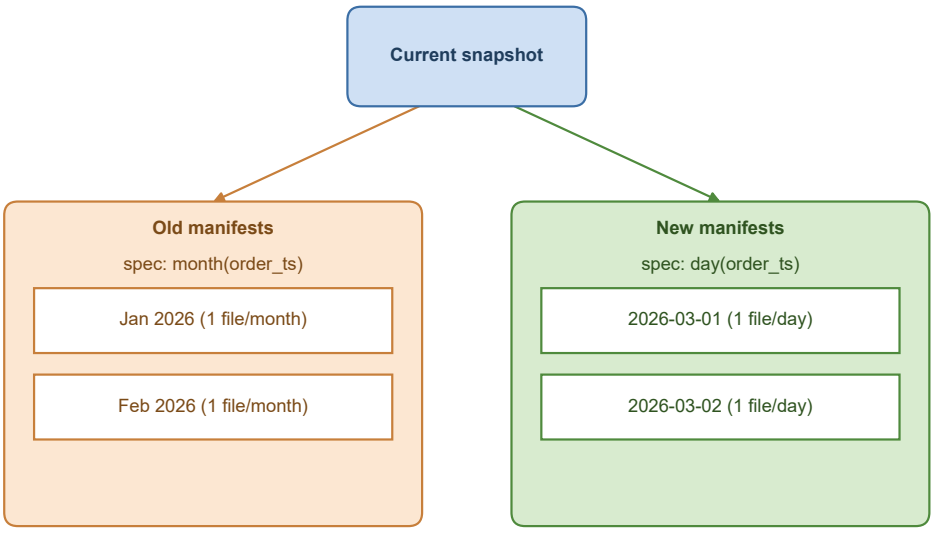


Figure 11.2: After evolving `orders` from `month(order_ts)` to `day(order_ts)`, both specs coexist under the same current snapshot. Old manifests keep describing month-partitioned files; new manifests describe day-partitioned files. One scan reads both, pruning each manifest against its own recorded spec.

The lab built exactly this. A table was partitioned by `months(order_ts)` and loaded with the first 15 days of March, then evolved with `ALTER TABLE … REPLACE PARTITION FIELD months(order_ts) WITH days(order_ts)`, and loaded with the remaining 16 days. No file was rewritten, and the `partitions` metadata table reports both specs side by side:

```
spec_id  partition            record_count  file_count
0        {674, NULL}         14,514,604    1      -- month 674 since 1970:
Mar 1-15
1        {NULL, 2026-03-16}      968,819    1      -- day partitions from
here on
1        {NULL, 2026-03-17}      969,559    1
...                                                 -- 16 day partitions,
34.8 MB each
```

Listing 11.1: The `partitions` table of an evolved table (trimmed)

A query spans both, but it prunes each at its own granularity. Figure 11.3 measures one-day queries against the old and new data.

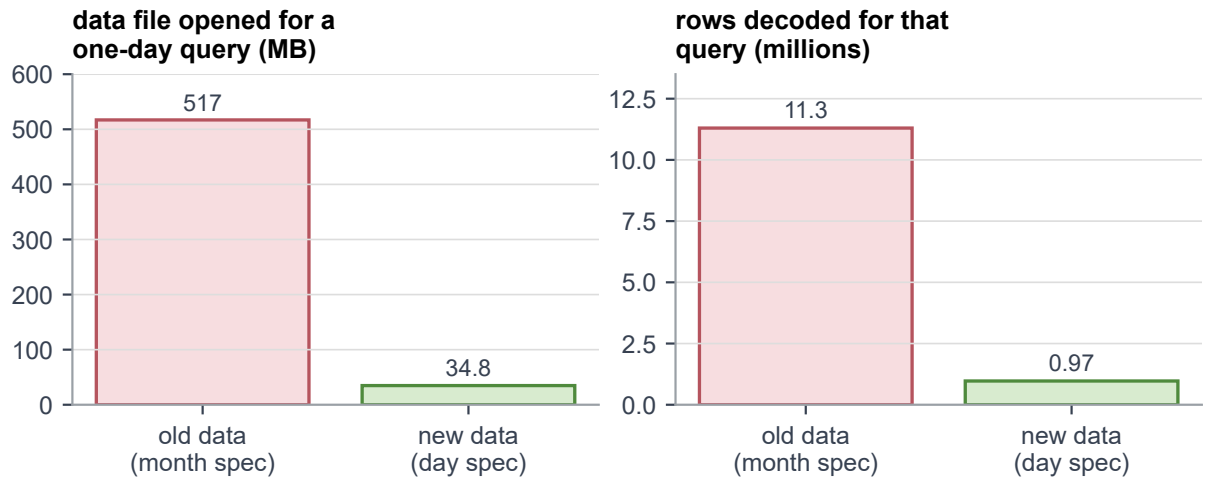


Figure 11.3: One day of old data lives in the 517 MB month-partition file and decodes 11.3 million rows; one day of new data lives in its own 34.8 MB file and decodes 0.97 million. Evolution kept the old data readable, not as fast.

Evolution is free of rewrites, and equally free of improvement for history: the old data keeps its coarse layout until a compaction rewrites it under the new spec (Chapter 13).

11.3 Sort orders and Z-ordering

A partition spec controls which *files* a filter can skip; a *sort order* controls how much of the *content* within a kept file can still be skipped, via the column min/max statistics Chapter 9's manifests already carry per file. Files written sorted by a column have tight, non-overlapping min/max ranges across files — a filter on that column can then skip whole files even within one partition, the same statistics-based pruning Section C.11 described for row groups, just at the file level.

A single sort order only helps predicates on the columns it is sorted by, front to back — a table sorted by `(category, customer_id)` gives `category` clean file-skipping and gives `customer_id` almost none, because rows are grouped by category first and only ordered by customer *within* that grouping.

Z-ordering (`rewrite_data_files` with `sort_order` type `zorder`) fixes exactly that asymmetry by building one sort key out of *interleaved bits* from several columns, so no single column gets front-of-key priority. Interleaving means taking each column's bits one at a time in round-robin order: with two columns A and B whose binary values are $a_2a_1a_0$ and $b_2b_1b_0$, the Z-order key is $a_2b_2a_1b_1a_0b_0$. Sorting by that combined key groups rows that are close in *either* column, because a small change in A or B alone only flips a few bits near the end of the interleaved key — rows near each other in the two-dimensional (A, B) space end up near each other in the one-dimensional sort order, in both directions at once, at the cost of neither column getting the clean front-of-key locality Section C.11 described for a plain sort. This is the right trade when queries filter on `category` sometimes and `customer_id` other times, rather than one column consistently.

The lab measured both on 9.7 million rows in about 45 files, with lookups on `customer_id`, on `product_id`, and on both (Figure 11.4).

The sorted table behaves exactly as described: the leading column prunes completely, the second column not at all (43 of 43 files read). Z-order spreads the pruning: no lookup is as cheap as the sorted layout's best case, but none is as bad as its worst. Use the one that matches how you query; if one column dominates, sort by it.

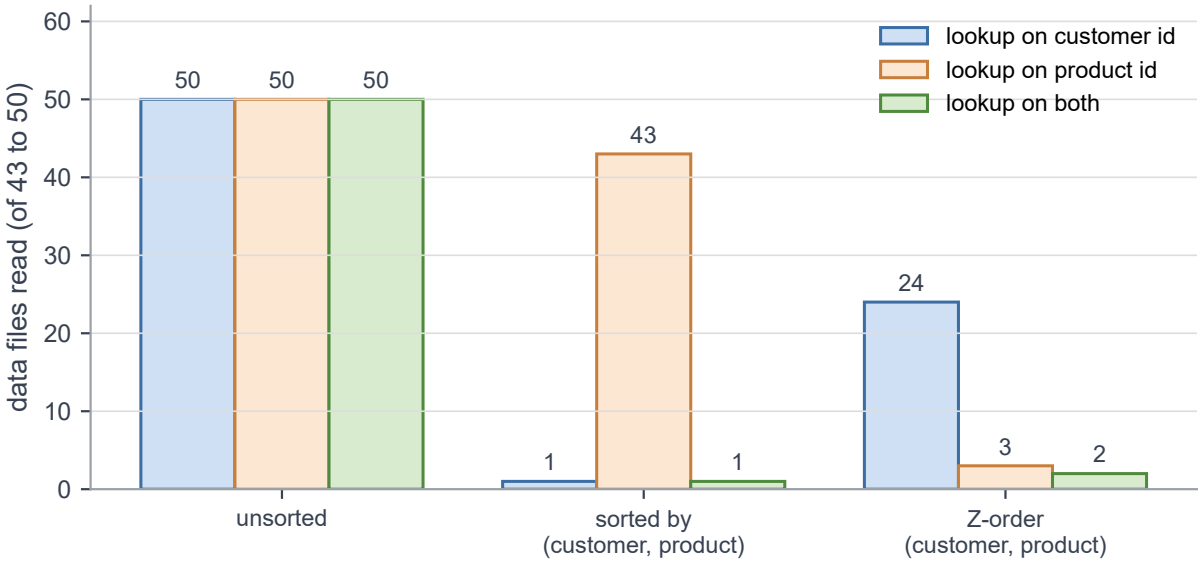


Figure 11.4: Data files opened by three lookups on three layouts of the same rows. Sorting by `(customer_id, product_id)` makes customer lookups nearly free (1 file) and leaves product lookups reading 43 of 43. Z-ordering gives up the best case for the other lookup: 24 files for a customer, 3 for a product, 2 for both.

> **Warning** `rewrite_data_files` only rewrites files it considers badly sized. The lab's first sort attempt left most files alone, because they were already near the 8 MB target, and the customer lookup still skipped just 3 of 44 files. Adding `rewrite-all=true` rewrote every file and the same lookup then read one. Check `skippedDataFiles` after a sort, not just the procedure's success message.

11.4 Target file size and the small-files problem

Every file a reader opens costs a fixed overhead — opening the file, reading its footer, resolving its schema — independent of how many rows it holds (Section 1.4's input-partition sizing runs into the same fixed cost from the other direction). A table accumulating many small files, typically from frequent low-volume commits (a streaming job flushing every micro-batch, Chapter 16), pays that overhead over and over for little data each time. `write.target-file-size-bytes` (512 MB by default) sets what a write aims for; Chapter 13's compaction procedures are the after-the-fact fix for files that already landed too small.

11.5 Write distribution modes

Before a write's rows are grouped into per-partition files, Spark may need to shuffle them so that all rows for a given partition value land together rather than scattered across every task's output — exactly the shuffle mechanics from Chapter 4, applied to a write instead of a read. `write.distribution-mode` chooses how:

`none` writes rows in whatever order the upstream plan already produced them, with no shuffle. Cheapest, but every task can touch every partition, so a table with many partitions ends up with many small files per commit — one per (task, partition) pair touched.

`hash` (the default for unpartitioned or partitioned tables with a defined spec) shuffles by partition value first, so each task after the shuffle owns one partition's rows and writes one file for it. This is the direct fix for the `none` mode's small-files problem, at the cost of the shuffle itself.

`range` shuffles by a sampled range of the sort order instead of a hash, producing files with tighter, non-overlapping value ranges — the write-time complement to Section 11.3's pruning benefit. A hash sends a given key to the same partition every time, but two different keys can land in the same partition or two adjacent keys can land in different ones — a hash has no notion of order. Range partitioning needs to know *where* the value boundaries between partitions should fall to keep them evenly sized, so a lightweight sampling pass reads a small fraction of the rows first, estimates the value distribution, and picks boundaries from that sample before the real, boundary-aware shuffle runs — the extra planning cost this mode carries.

The modes only differ when the rows reaching the writer are not already grouped by partition. The lab loaded ten days of `orders` through a round-robin repartition into ten tasks, so every task held rows from every day, into a day-partitioned table under each mode (Figure 11.5).

The shuffle bought a tenfold reduction in files and cost nothing measurable in this run, because the write itself dominated. `none` also needs the fanout writer (`write.spark.fanout.enabl`

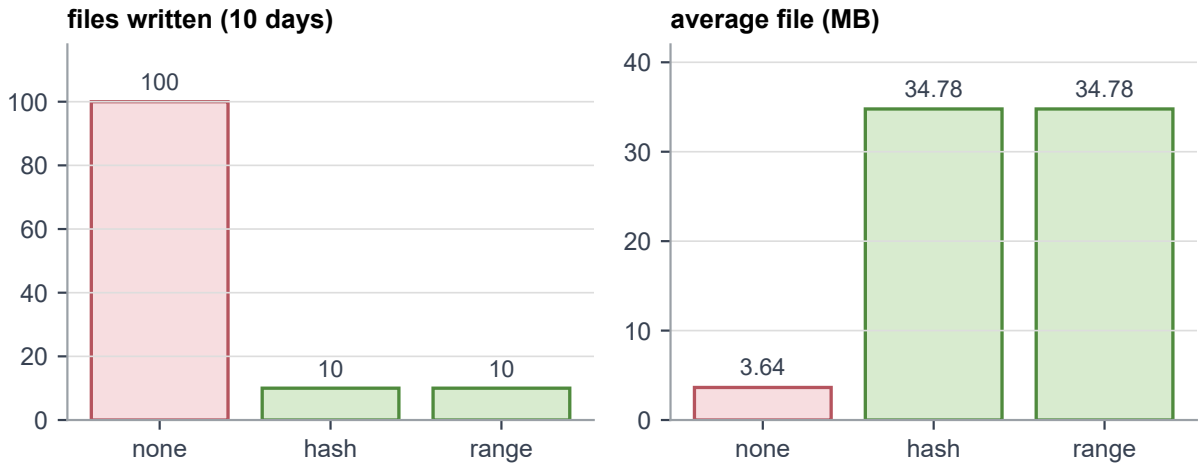


Figure 11.5: The same ten days written under each distribution mode. With no shuffle, each of the ten tasks wrote one file per day it held, 100 files of 3.6 MB; with `hash` or `range` each day's rows were gathered first, giving 10 files of 34.8 MB. The inserts took 25.1, 24.3, and 23.8 s.

since its input is not clustered by partition; without it the write fails. If you control the upstream ordering, as a daily partitioned scan does, `none` is free; otherwise `hash` is the safe default.

For the specimen table, this is the mode that determines what a write does with the viral product from Section 1.6's skew: under `hash` distribution, every row for that one `product_id` lands in the same write task regardless of how many there are, so a write task for that key can end up far larger than the others — the write-side version of the same skew, covered in full in Section 13.3. The lab shows the file-size version on `bucket(16, product_id)` with `hash` distribution (Figure 11.6): the viral product's bucket held 3.33 million rows in a 110 MB file, while the other fifteen held about 425,000 rows and 15.4 MB each. The 7.1-times-larger task that wrote it is the straggler of Chapter 6 appearing on the write side.

11.6 Summary

Iceberg partitioning is hidden behind a transform of a source column, so pruning works from a plain filter on that column with no partition column in the query; `day`/`hour` suit time-series filters, `bucket` suits high-cardinality keys and storage-partitioned joins, and

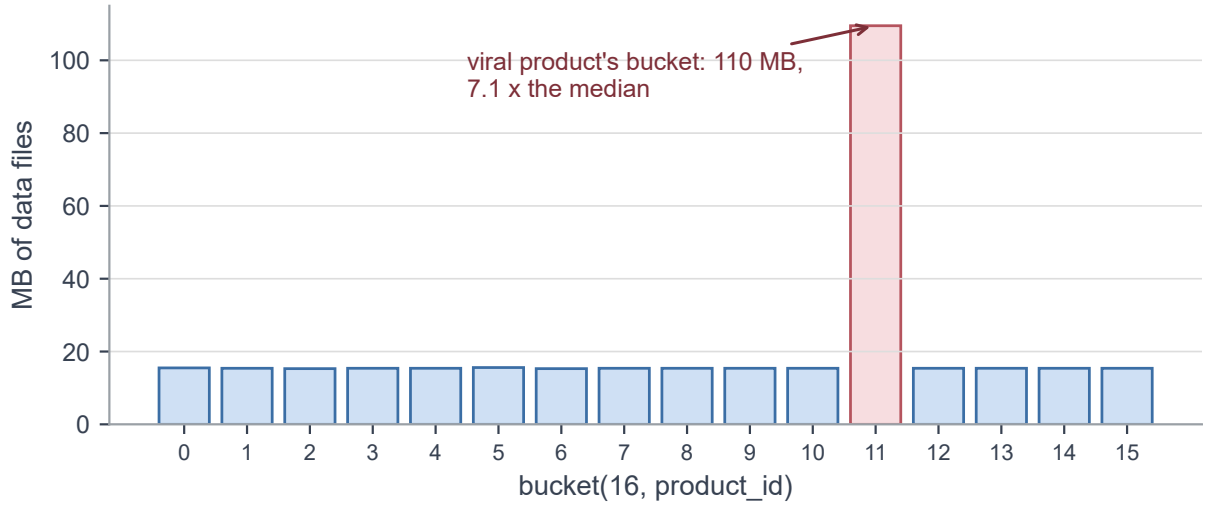


Figure 11.6: Bytes per bucket for `bucket(16, product_id)` on 9.7 million orders. Hashing sends every row of the viral product to the same bucket, so one file and one write task carry 7.1 times the median.

an `identity` transform on a high-cardinality column is a modeling mistake, not a partitioning choice. Evolving the spec is metadata-only because old files keep the spec they were written under. A sort order or Z-order tightens the per-file statistics that let a reader skip whole files inside a kept partition; target file size and the write distribution mode are what decide whether a write produces a few well-sized files or many small ones in the first place.

Table 11.3 turns each symptom into the evidence and the lever.

Exercises

1. Given a daily data volume and a query pattern, choose a partition spec. Justify it against the target of 128 MB–1 GB per partition per write.

2. Evolve a table's partition spec from `month(order_ts)` to `day(order_ts)`. Confirm from the `partitions` metadata table that old and new data report different specs.

3. Write the same DataFrame with `write.distribution-mode` set to `none` and then `hash`. Compare the file count per partition in each case.

Symptom	Evidence	Lever	Cost
thousands of tiny files	`files` table: small average size, many rows of it	coarser partition transform; `hash` distribution	coarser pruning; a shuffle
a lookup opens every file	`skippedDataFiles` is 0	sort or Z-order by the filter columns	compaction job; small files
history is slower than new data	old spec partitions in `partitions`	rewrite old files under the new spec	a rewrite
one file or task much larger	skewed row count per bucket	a different bucket column; salt the key	less clean joins
writes produce one file per task per partition	`none` mode on unclustered input	`hash` or `range` distribution	a shuffle

Table 11.3: The layout levers: symptom, evidence, lever, and cost.

Chapter 12

Reading Iceberg Efficiently

Before the Iceberg reader opens a single data file, it has already skipped most of them — by partition, by manifest, by column statistics, and optionally by bloom filter. Each layer needs the query's filter in a form it understands, and each reports what it skipped. A week of `orders` *opens 7 of 31 files; a lookup of one* `customer_id` *opens all 10 files of an unsorted table and exactly one of the same rows sorted. This chapter covers the pruning layers, the statistics that feed them, the counters that prove they fired, and the read-time settings that decide how many tasks a scan becomes. Every number is a capture from the lab of Section A.3.*

12.1 Manifest pruning and metadata filtering

A scan resolves in layers, each cheaper than the last and each capable of excluding the whole layer below it (Table 12.1 lists them top to bottom): the manifest list's per-manifest partition summaries (Section 9.2) let the planner skip an entire manifest without opening it; the surviving manifests' per-file partition values and column statistics let it skip individual files without opening them; only files that survive both layers are opened at all, and even then column pruning (Section 7.6) and row-group statistics (Section C.11) keep working exactly as they do for any Parquet scan. Static and dynamic partition pruning (Chapter 7) produce the values these layers filter against; this chapter is about what Iceberg does with a value once Spark has produced one.

Table 12.1: The pruning layers a scan passes through, in order

Layer	Skips
Manifest list	Whole manifests, via per-manifest partition summaries
Manifest entries	Whole data/delete files, via partition value and column min/max/null statistics
Bloom filter (optional)	Row groups, for an equality lookup on a high-cardinality column
Row-group statistics	Row groups within an opened file, via Parquet's own min/max (Section C.11)

Figure 12.1 draws the same four layers as what they are: a funnel, narrowing from every file in the table down to the rows a query actually reads, each layer paying only for what survived the one above it.

12.1.1 Counting what each layer skipped

Each layer reports its work as counters on the scan node: in the Spark UI's **SQL** tab, the `BatchScan` box lists them as metrics, and they are *not* part of `EXPLAIN`. Figure 12.2 reads them for the specimen's week on a table built by 31 daily commits.

How a table was written decides which layer does the work. Table 12.2 runs the same week query on two physical layouts of the same 30 million rows.

The result is the same, and the cost is not: opening one manifest to inspect 31 entries is cheap, but a table with thousands of manifests turns the first layer from a convenience into the thing that keeps planning fast. It works only when manifests are clustered by partition, as they are for a daily load. After streaming commits or many small appends each manifest spans many days, its partition summary covers the whole range, and nothing is skipped; rewriting manifests restores the clustering (Chapter 13).

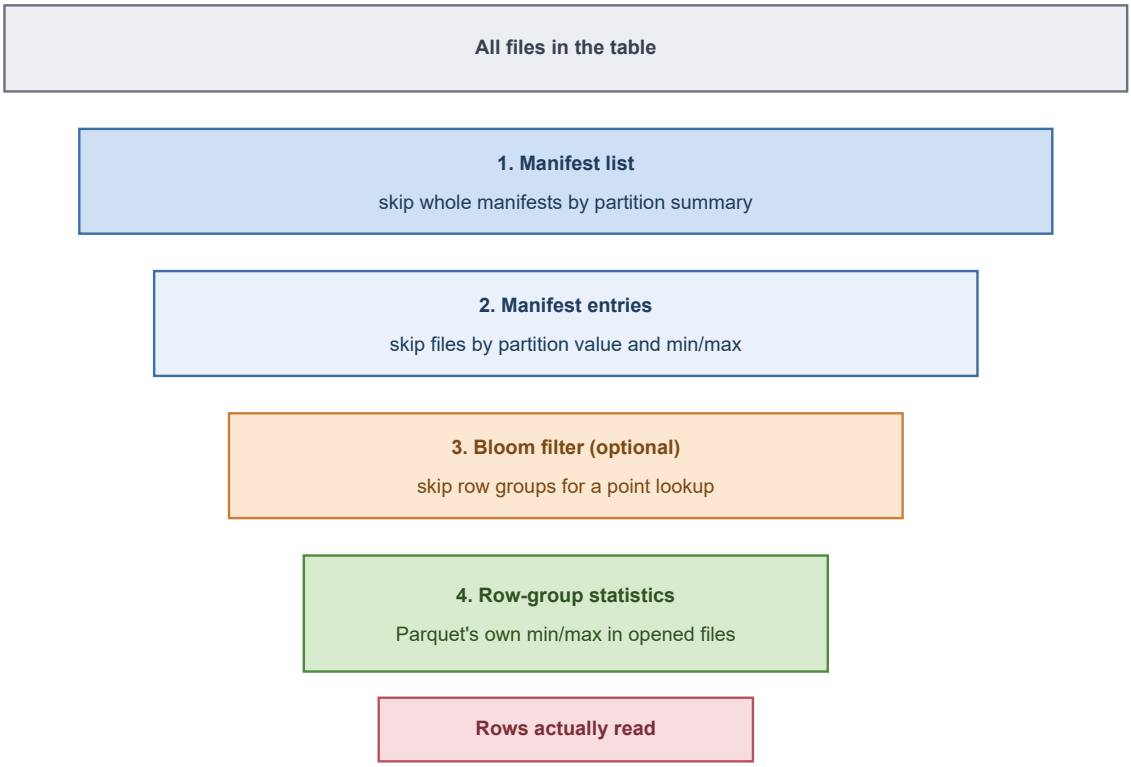


Figure 12.1: The four pruning layers, narrowing top to bottom. Each layer only ever has to consider what the layer above it did not already exclude — a file skipped by the manifest list is never even checked against a bloom filter or row-group statistics.

Table (same rows)	Manifests	Skipped manifests	Result files	Skipped files	Tasks
one commit, one manifest	1	0	7	24	7
31 daily commits	31	24	7	0	7
31 daily commits, whole month	31	0	31	0	31

Table 12.2: The week query on two layouts. Both read the same 7 files and 46.8 MB; the layer that did the skipping differs. With a single manifest the 24 files were excluded by their per-file partition values, one layer down. With one manifest per commit, 24 whole manifests were never opened.

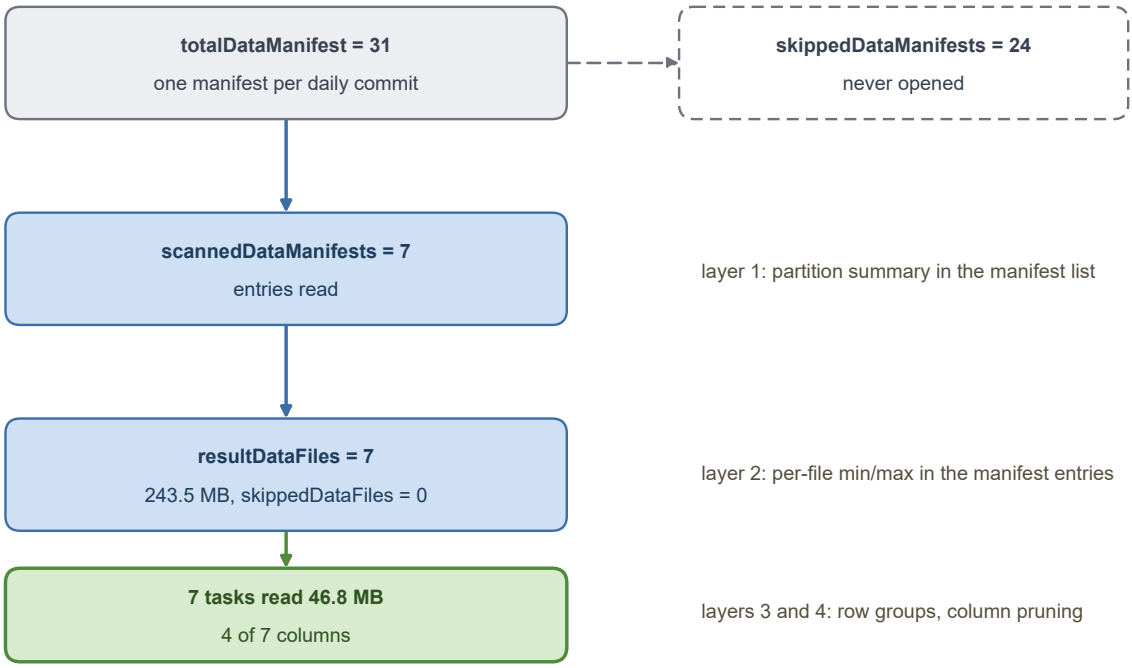


Figure 12.2: The scan counters for one week of `orders` on a table with one manifest per daily commit. The partition summaries in the manifest list excluded 24 of 31 manifests before any was opened; the 7 survivors held 7 files, all needed; the tasks then read 46.8 MB, the four columns the query names.

12.2 Column statistics and predicate pushdown into Iceberg

Partition values are the easy case, because the partition column is the layout. The second layer works for any column, but only if the statistics can tell files apart. The per-file minimum and maximum of `product_id` on the lab's `orders` are a good example of statistics that cannot:

```
file (day)   order_id (lo .. hi)                product_id (lo .. hi)
03-01        000004a2-29b7-4d .. ffffcf0b-ec3e-4e   p0000005 .. p1999998
03-02        00000221-2318-41 .. ffffa331-4b9b-4f   p0000001 .. p1999999
03-03        000003c6-0c3e-41 .. ffffed77-6266-4g   p0000004 .. p1999998
```

Listing 12.1: Per-file bounds for three `orders` files (`readable_metrics` of the `files` table, trimmed)

Every file's range covers the whole key space, because rows were written in arrival order and every day contains every product. A predicate `product_id = 'p0000042'` is not outside any file's range, so no file is skipped. Pruning by statistics is a property of the *clustering* of the data, not of the predicate. Figure 12.3 measures it with a lookup of one `customer_id` on a 10-day copy of `orders` written two ways.

Table 12.3 and Figure 12.4 show what that does to the scan.

Sorting is the strongest lever by a wide margin: 0.2 MB read against 60.5 MB. It has a price the table also shows. The sorted copy is 200 files of 1.7 MB each instead of 10 files of 35 MB, a small-files layout that costs planning time and file opens on every other query (Section 11.4). Choosing a sort order is choosing which lookups you are willing to make cheap (Chapter 11).

A filter that cannot be pushed to this layer still runs correctly — Spark applies it after the read — so what is lost is pruning, never correctness. The rules for what pushes down are the same as for plain Parquet, catalogued in Section C.11.

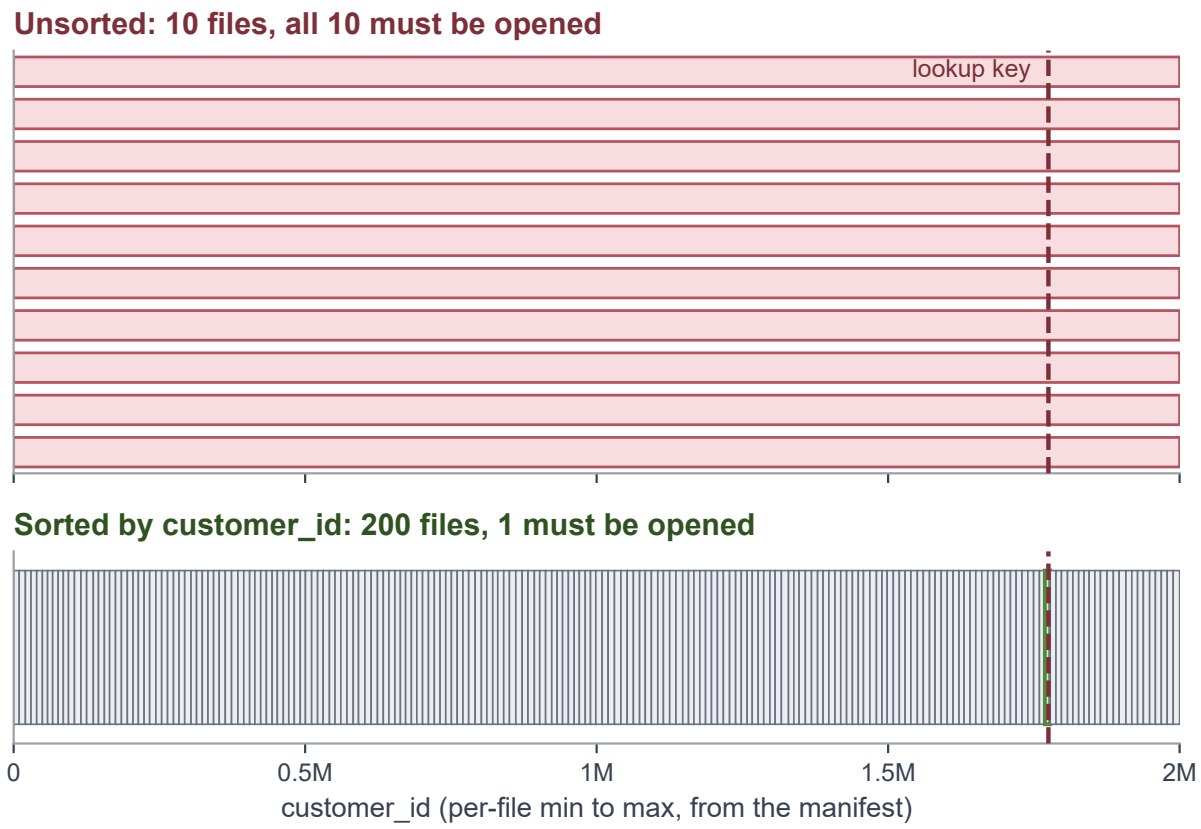


Figure 12.3: Per-file `customer_id` ranges, from the manifest, for the same 9.7 million rows. Unsorted, each of 10 files spans the whole key range and the lookup key falls inside all of them. Sorted by `customer_id`, the 200 files have disjoint ranges and the key falls inside exactly one.

Copy of 10 days	Files opened	Files skipped	Rows decoded	Bytes read
unsorted	10	0	9 678 114	60.5 MB
unsorted, bloom filter on the column	10	0	2 905 063	28.9 MB
sorted by the column	1	199	44 405	0.2 MB
sorted, `metrics.default = counts`	200	0	48 742	1.5 MB

Table 12.3: One `customer_id` equality lookup, measured on four copies of the same rows. Sorting lets the manifest skip 199 of 200 files before any is opened. The bloom filter helps inside each file, but all 10 are still opened. Storing only counts removes the min/max the manifest needs, so all 200 files are opened, and Parquet's own row-group statistics rescue most of the bytes.

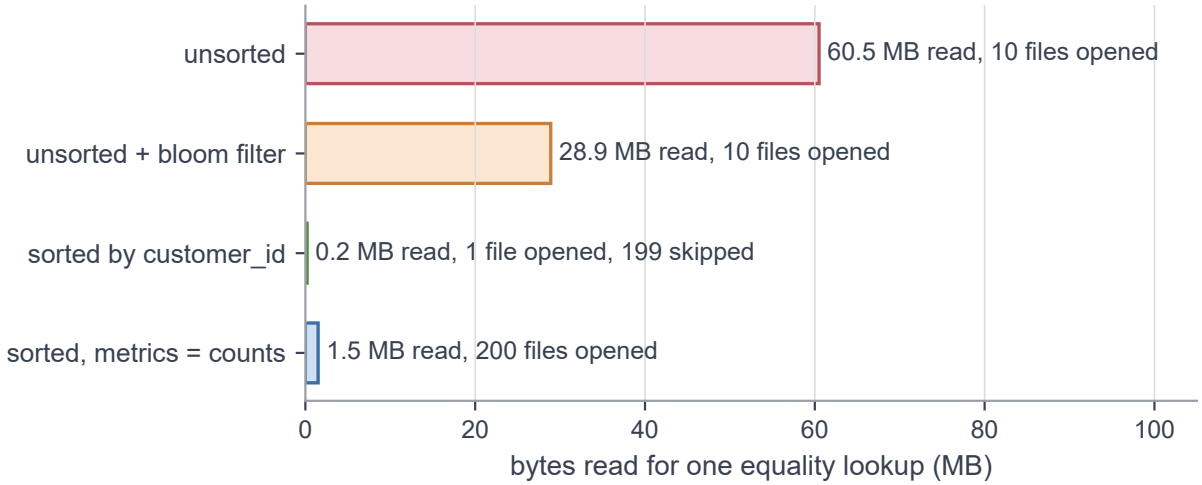


Figure 12.4: Bytes read for one `customer_id` lookup, by table layout. Sorting wins by two orders of magnitude; the bloom filter halves the read; storing only counts throws the sorted layoutś advantage away at the manifest and leaves Parquet footers to recover most of it.

12.3 Metrics modes: `full`, `truncate`, `counts`, `none`

`write.metadata.metrics.default` (default `truncate(16)`) controls how much of each column's statistics get written into the manifest in the first place. It is a write-time property with a direct read-time consequence:

`full` stores the exact min and max value, uncapped. Best pruning, largest manifest entries — expensive for a wide string column with long values.

`truncate(16)` (the default) stores only the first 16 bytes of a string's min and max. Pruning still works for any predicate that differs within the first 16 bytes; two values agreeing on their first 16 bytes look identical to the pruning layer.

`counts` keeps null and value counts only, no min/max at all.

`none` keeps nothing; the file is never skipped by this layer regardless of the filter.

The lab's `order_id` is a 36-character UUID, and Listing 12.1 shows the truncation at work: every bound is exactly 16 characters. The upper bound of the third file ends in `4g`, not `4f`: Iceberg cannot store a truncated maximum that is *below* some real value, so it increments the last kept character to produce a bound that is still an upper bound. A

truncated bound is therefore always safe and sometimes loose, and a UUID lookup gains nothing from it, since the first 16 characters of random UUIDs never cluster.

The last row of Table 12.3 is the `counts` mode seen from the read side. With no min/max in the manifest, the planner cannot exclude any of the 200 sorted files, and plans 200 splits (50 tasks, after grouping). The Parquet footers of those files still carry row-group statistics, so each task skips its row groups and the lookup reads only 1.5 MB; but it has opened 200 files to find that out, where the `truncate` copy opened one.

`write.metadata.metrics.column.<col>` overrides the default per column, which is the practical lever: set `full` on the column your biggest queries filter by most (the specimen's `order_ts`, say) and leave wide free-text columns at `counts` so their manifests do not balloon for no pruning benefit.

12.4 Bloom filters

Min/max statistics prune well for a range predicate, or for a point lookup on data clustered by that column. They do not prune a point lookup on an unsorted high-cardinality column — Figure 12.3's top panel. A Parquet bloom filter, enabled per column with `write.parquet.bloom-filter-enabled.column.<col>=true`, answers a narrower question directly — "can this row group possibly contain this exact value" — with no false negatives and a tunable false-positive rate.

On the lab it helped, but less than sorting did. The 10-day unsorted copy decoded 9.7 million rows without a filter and 2.9 million with one, reading 28.9 MB instead of 60.5 MB: about two thirds of the row groups were skipped. All 10 files were still opened, because the filter lives inside the file, below the manifest layer. And not every row group was skipped, since a bloom filter has false positives and the column holds about a million distinct values per row group, more than the default size cap (`write.parquet.bloom-filter-max-bytes`, 1 MB, per row group) can represent at a tight false-positive rate. It costs write-time CPU and file size (the bloom copy is 35.8 MB per file against 34.8 MB), and is worth enabling only on columns queried by equality that you cannot sort by.

12.5 Vectorized reads

`read.parquet.vectorization.enabled` (default `true`) reads a *batch* of rows into a columnar, Tungsten-friendly in-memory format (Chapter 17) per call instead of materializing one Java object per row. `read.parquet.vectorization.batch-size` (default 5,000) sets how many rows make up one batch; larger batches amortize per-batch overhead further but hold more decoded data in memory at once.

You can see whether it is on from the plan: with it enabled, the scan is followed by a `ColumnarToRow` node, which converts the batches the scan produces; with it disabled, the node is absent. The lab ran one aggregate over 9.7 million rows both ways:

Setting	`ColumnarToRow` in plan	Median task	Bytes read
vectorization on	yes	101 ms	19.5 MB
vectorization off	no	721 ms	19.5 MB

Table 12.4: The same scan with vectorized reads on and off. The bytes read are identical; decoding row by row made each task about seven times slower.

The I/O is the same and only the decoding differs, which is why the setting belongs to the read path and not to pruning. Leave it on; turn it off only to isolate a suspected reader bug.

12.6 Split planning

Section 1.4 described input partitions as roughly `read.split.target-size` (128 MB default) worth of data each. For an Iceberg scan that is the configured *ceiling*, and not always the number you get. The planner bins manifest entries toward the target, using `read.split.open-file-cost` (4 MB default) as a floor weight per file so that many tiny files are not treated as free to combine, and `read.split.planning-lookback` (10) to bound how many candidate bins it considers at once.

On the lab the ceiling did nothing, and the cause is a feature. The 10 unsorted files total 348 MB. At the 128 MB default the arithmetic says about 4 splits, and the scan

ran 10 tasks; setting `read.split.target-size` to 512 MB changed nothing. Iceberg 1.6 shrinks the target when the scan is small relative to the cluster's default parallelism (`read.split.adaptive-size.enabled`, default `true`), so that a small scan still produces enough tasks to keep the slots busy; here the lab's 40-slot parallelism pulled the target well below a file. Table 12.5 turns the adaptive size off to expose the setting underneath.

Settings	Files	Tasks
adaptive on, target 128 MB (default)	10 files, 348 MB	10
adaptive on, target 512 MB	10 files, 348 MB	10
adaptive off, target 128 MB	10 files, 348 MB	4
adaptive off, target 512 MB	10 files, 348 MB	1
adaptive off, target 8 MB	10 files, 348 MB	10

Table 12.5: Scan tasks for ten 35 MB files. Adaptive sizing hides `read.split.target-size` for a small scan. With it off, the 128 MB target packs three files per task and a 512 MB target packs all ten into one. An 8 MB target cannot split a file that has no split offsets, so it leaves 10 tasks.

The 128 MB row is the packing the earlier arithmetic predicted: 35 MB files, three per bin, ten files in four bins. The last row is the other limit: a split can only fall on a row-group boundary, and these files report no split offsets, so a smaller target does not create more tasks. For a scan to feed a large cluster, the data must be in more files than slots, or in files whose row groups can be split.

12.7 The metadata tables as a diagnostic

Every layer above is inspectable directly, without guessing from query latency. Listing 12.2 runs the three most useful metadata tables against the lab's `orders`.

```
SELECT count(*) AS files, round(avg(file_size_in_bytes)/1e6,1) AS avg_mb,
       sum(record_count) AS rows
```

```
FROM lab.db.orders.files;
-- files = 31, avg_mb = 34.8, rows = 30000000

SELECT partition, record_count, file_count
FROM lab.db.orders.partitions ORDER BY partition LIMIT 2;
-- {2026-03-01}  967542  1
-- {2026-03-02}  968774  1

SELECT added_data_files_count, existing_data_files_count
FROM lab.db.orders.manifests;
-- 31, 0     (one manifest holding all 31 files)
```

Listing 12.2: Three metadata tables on `orders`, with the lab's output

`table.files` lists every live data and delete file with its size, record count, and per-column bounds; `table.manifests` shows manifest-level summaries; `table.partitions` aggregates file counts and sizes per partition, the fastest way to spot a partition accumulating too many small files (Section 11.4); `table.snapshots` and `table.history` show the commit sequence a scan resolves against. Reading these tables directly is usually the faster way to confirm a suspicion from the scan counters.

Inside Spark, name a metadata table with its full catalog path (`lab.db.orders.files`): after `USE lab.db`, the short form `orders.files` is parsed as table `files` in namespace `orders` and fails.

> **Tip** The pruning ratio at each layer comes straight from the scan node's counters: `skippedDataManifests / totalDataManifest` for the first layer and `skippedDataFiles / (skippedDataFiles + resultDataFiles)` for the second. Compare `resultDataFiles` with the table's file count from `table.files` to see how selective the whole scan was.

12.8 Summary

A scan prunes in layers of increasing cost — manifest list, manifest entries, optional bloom filter, row-group statistics — and reports each layer's work as counters on the scan node. The first layer works when manifests are clustered by partition; the second

works for any column whose per-file ranges are disjoint, which is a property of how the table was sorted and written, not of the query. Metrics modes decide whether the ranges exist at all, and a truncated bound is always safe and sometimes loose. A bloom filter rescues an unsorted equality lookup partly, below the file layer. Vectorized reads change decoding speed and nothing else, and split planning's 128 MB target is a ceiling that Iceberg's adaptive sizing lowers for a small scan. Table 12.6 turns each symptom into the evidence and the lever.

Symptom	Evidence	Lever	Cost
a selective query opens every file	`skippedDataFiles` is 0; `files` ranges overlap	sort or cluster by the filter column	small files; write cost
planning is slow, many manifests	large `totalDataManifest`; few skipped	rewrite manifests so each spans few partitions	a maintenance job
min/max is missing for a column	`lower_bound` is `NULL` in `files`	set the column's metrics mode to `full` or `truncate`	larger manifests
an equality lookup on an unsorted column	overlapping ranges, high `rows decoded`	bloom filter on that column	write CPU, file size
scan feeds too few tasks	tasks fewer than slots for a large scan	more or smaller files; check adaptive split size	more files to plan
decoding is the bottleneck	no `ColumnarToRow` node	`read.parquet.vectorization.enabled=true`	

Table 12.6: The read-path levers: symptom, evidence, lever, and cost.

The rule the chapter repeats: **statistics skip files only when the data was laid out so that they can**. Chapter 13 turns from reading the layout to maintaining it.

Exercises

1. **Run and observe.** Run a one-week query on a table written by one commit and on a copy written by daily commits. From the scan node's counters, fill in a row of Table 12.2 for each, and say which layer skipped the other 24 partitions' worth of

data.

2. **Read the metadata.** Query `table.files` for a column you filter on and print its lower and upper bounds with `readable_metrics`. Do the per-file ranges overlap? Predict how many files an equality lookup on that column opens, then check with the scan's `resultDataFiles`.

3. **Predict, then check.** For ten 35 MB files, predict the task count for a target of 128 MB and for 512 MB with adaptive sizing off, then run both. What happens with adaptive sizing on?

Chapter 13

Writing and Maintaining Iceberg

Copy-on-write rewrites whole files on every update: slow writes, fast reads. Merge-on-read writes small delete files instead: fast writes, slower reads until compaction catches up. The choice is a function of your update rate and your read latency budget, and it commits you to a maintenance schedule — compaction, snapshot expiration, orphan-file cleanup — in a specific order. This chapter covers the write modes, optimizing `MERGE`*, and running maintenance without deleting data a reader still needs. On the lab, a one-row update cost 7.9 s and 34.8 MB under copy-on-write and 2.9 s and 4 KB under merge-on-read, sixty tiny commits became one file and a scan four times faster, and expiring snapshots reclaimed 35 MB, all measured on the lab of Section A.3.*

13.1 Copy-on-write versus merge-on-read

Every `UPDATE`, `DELETE`, or `MERGE` against a matched row is, underneath, one of two things: rewrite the data file that row lives in with that row changed or removed (*copy-on-write*, CoW), or leave the data file untouched and record the change as a delete file plus, for an update, a new data file holding only the changed rows (*merge-on-read*, MoR, built on Section 9.4's delete files). `write.delete.mode`, `write.update.mode`, and `write.merge.mode` set this independently per operation type; check a table's current properties rather than assuming a default, since Iceberg has changed which mode ships as the default across releases.

The trade is exactly where each mode's name says it is: CoW pays the full cost of rewriting a file at write time so every subsequent read sees plain data files with no delete files to merge; MoR pays almost nothing at write time — a small delete file, no data rewrite — and pushes the cost onto every read until compaction (Section 13.4) catches up. Table 13.1 is the concrete rule: a table with a low update rate and latency-sensitive reads wants CoW; a table absorbing frequent small updates (a streaming upsert, Chapter 16) wants MoR, because CoW there would mean rewriting the same large file on nearly every micro-batch.

Table 13.1: Choosing copy-on-write versus merge-on-read

	Copy-on-write	**Merge-on-read**
Write cost	Rewrites whole matched files	Small delete file, no rewrite
Read cost	Plain data files, no merge	Merges delete files at read time
Best fit	Infrequent updates, latency-sensitive reads	Frequent small updates, write-latency-sensitive
Needs	—	Regular compaction to bound read cost

The lab measured the trade on ten days of `orders`, partitioned by day, with ten small `UPDATE`s of five rows each (Figure 13.1 and Table 13.2). On Iceberg 1.6.1 the default for `UPDATE`, `DELETE`, and `MERGE` was copy-on-write.

Two details are easy to miss. A merge-on-read `UPDATE` is recorded as an `overwrite` snapshot with one added data file (the new row versions) and one added delete file (the old ones), not as a `delete`. And after `rewrite_data_files` rewrote the affected data file, the `files` table still listed the ten delete files: they no longer point at live rows but were not removed. A follow-up run confirmed it on six deletes: `rewrite_data_files` with `rewrite-all` left one delete file standing, `rewrite_position_delete_files`

Mode	Per update	Bytes written	Files after ten	Full scan
copy-on-write	7.9 s	34.8 MB	10 data	0.28 s
merge-on-read	2.9 s	4 KB	20 data, 10 delete	0.35 s
merge-on-read, compacted	—	—	10 data, 10 delete	0.31 s

Table 13.2: Ten five-row updates, measured. Copy-on-write rewrote a 34.8 MB data file each time; merge-on-read wrote one tiny data file and one tiny delete file. The price shows up in the read: 25% slower with ten delete files outstanding, and only partly recovered by compaction.

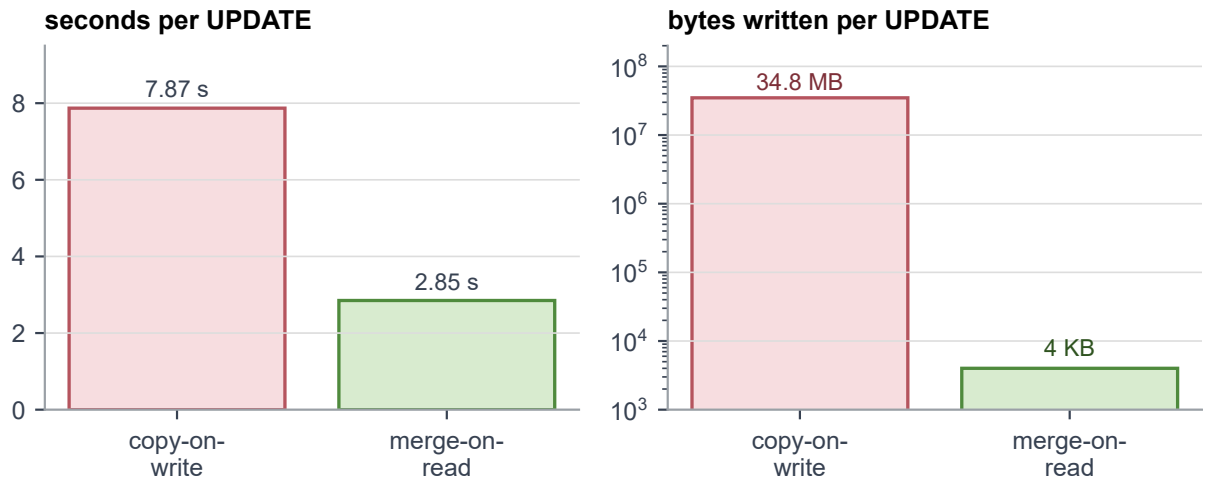


Figure 13.1: One small `UPDATE`, two paths. Merge-on-read is nearly three times faster and writes about nine thousand times fewer bytes.

did not remove it, and the `remove-dangling-deletes` option was rejected by Iceberg 1.6.1 ("Cannot use options [remove-dangling-deletes], they are not supported by the action or the rewriter BIN-PACK") — the option belongs to a later release. That is why read cost recovered only from 0.35 to 0.31 s.

Figure 13.2 follows the same `UPDATE` down both paths to make the trade concrete: same three matched rows, opposite placement of the cost.

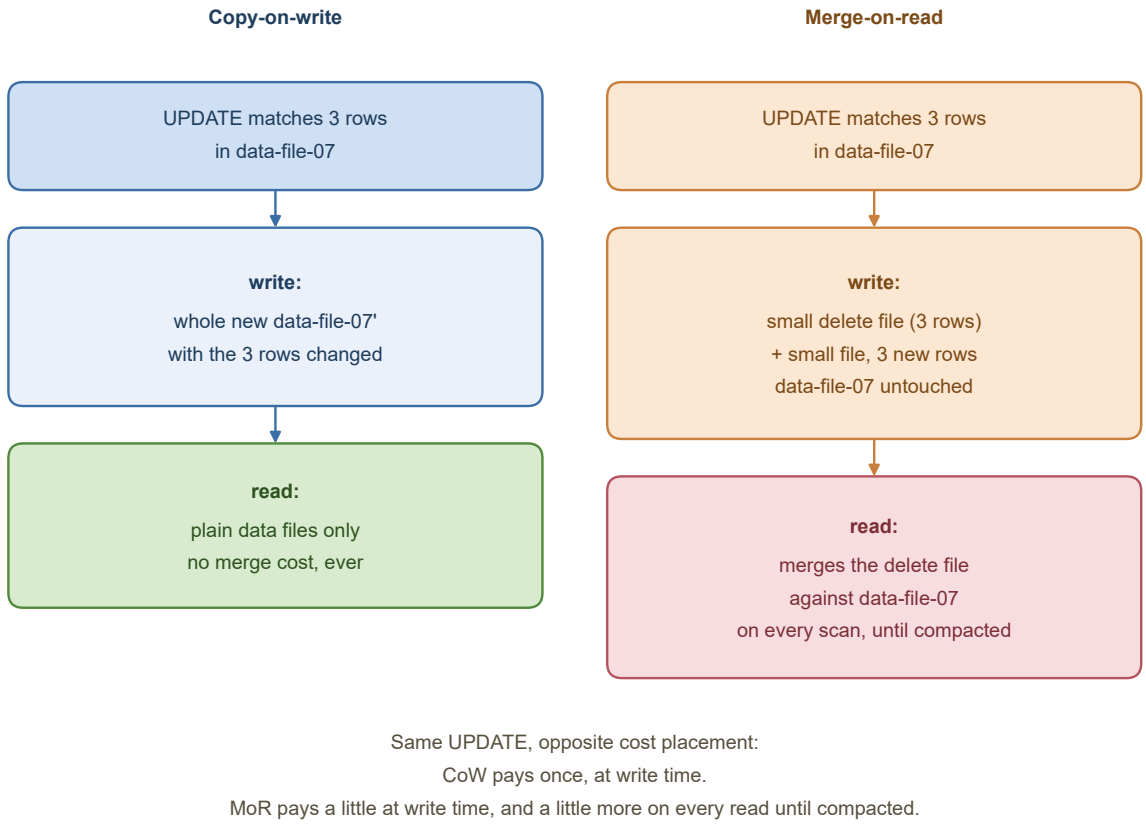


Figure 13.2: The same `UPDATE` under copy-on-write and merge-on-read. CoW pays once, at write time, and every later read is plain data with no merge; MoR pays a small write-time cost and pushes the rest onto every read until the next compaction.

13.2 `MERGE`, `UPDATE`, and `DELETE`

A row-level operation's `ON` or `WHERE` predicate does two different jobs, and conflating them is the single most common performance mistake against these tables. If the pred-

icate matches whole partitions — resolvable the same way Section 7.2 resolves a scan filter — Iceberg performs a *metadata-only delete*: it drops the file references from the manifest without reading or rewriting a single row, because no row inside an untouched partition needed to change. If the predicate only narrows which *rows* inside a file match, Iceberg has to open those files, find the matching rows, and rewrite (CoW) or emit deletes for (MoR) exactly the files that contained a match — every other file in the table is untouched either way, provided the predicate lets partition pruning rule them out first.

```
MERGE INTO orders o
USING staged_orders s
ON o.order_id = s.order_id
   AND o.order_ts >= TIMESTAMP '2026-03-03 00:00:00'
   AND o.order_ts <  TIMESTAMP '2026-03-04 00:00:00'
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;
```

Listing 13.1: A predicate on the target's partition column keeps the scan narrow

Dropping the two `order_ts` lines does not change which rows match — `order_id` alone is still sufficient — but it removes the one predicate the planner can resolve against partition values before scanning. The lab merged the same 1,000 staged rows both ways (Figure 13.3): without the predicate the scan read 200.7 MB across 10 tasks, every day of the table; with it, 35.1 MB in one task. Both rewrote the same single 34.8 MB data file, because the matching rows lived in one partition either way, so the saving is entirely in the scan, and the merge ran 16% faster. A `MERGE` without a partition predicate is not wrong; it is a full scan wearing a row-level operation's syntax.

The predicate has to be a filter on the target's own column. A join key alone, such as equality on the staged table's timestamp, is not a static range and should not be assumed to prune; check the scan's bytes, as above.

A predicate that matches whole partitions takes the other path entirely. The lab deleted one day of a ten-day table with a range on `order_ts`: the snapshot was an operation of type `delete` with one deleted data file and no added files, the statement took 0.54 s, and no data file was read. That is a metadata-only delete, the cheapest row-level operation there is.

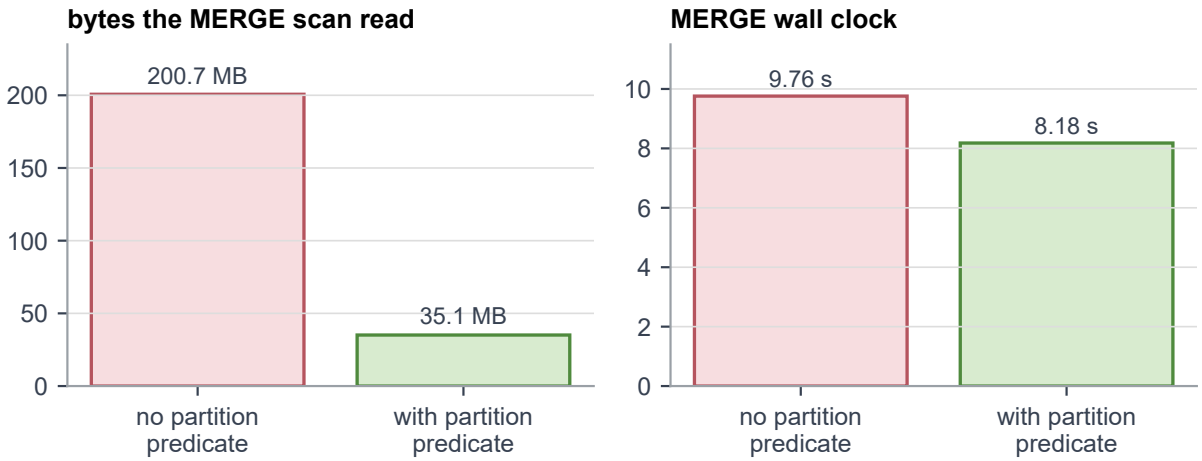


Figure 13.3: The same `MERGE`, with and without a static range on the partition column in the `ON` clause. The predicate cut the bytes the scan read by 5.7 times.

13.3 Write distribution and skew on write

Section 11.5 covered `write.distribution-mode` for a plain append; a row-level operation inherits the same choice, with one extra failure mode. A `MERGE` whose matched rows are concentrated in a handful of partitions — a backfill touching one day's data, say — shuffles unevenly under `hash` mode exactly the way a skewed join does (Section 6.5), except AQE's skew-split machinery is a read-side mechanism and does not rebalance a write.

The specimen's own viral, heavily-discounted product (Section A.4) makes this concrete. Say a `MERGE` runs against a month of `orders` to correct a pricing error, matching on `product_id`. Under `hash` distribution, matched rows shuffle by `product_id` before the rewrite, so every row for the viral product lands in a single shuffle partition. The lab's join shuffle on the same key showed the shape: one partition of 72.8 MB against a median of 1.6 MB (Figure 6.6), and the write-side bucket of Figure 11.6 was 7.1 times its neighbours. The task that draws the viral product's partition does the work of many while the rest of the stage finishes early and sits idle: the straggler shape of Section 6.5, except that here it is a rewrite task, and AQE's join-side skew split does not apply to it. Nothing currently detects or splits an oversized write task the way AQE does for a read. (Claim — the write-side straggler itself was not separately timed on the lab.)

13.4 Compacting data files

`rewrite_data_files` is the direct fix for both small files (Section 11.4) and MoR read cost: it rewrites a set of files into fewer, better-organized ones, in one of three strategies. `binpack` (the default) merges small files toward the target size with no reordering — cheapest, and sufficient when the only problem is file count. `sort` rewrites according to a defined sort order (Section 11.3), restoring tight min/max ranges that ordinary appends erode over time. `zorder` does the same for several columns interleaved together, at a higher compute cost, when queries filter on different columns unpredictably. All three also apply outstanding delete files to the data they rewrite, which is why running compaction is the concrete answer to a MoR table whose reads have been getting slower.

The lab measured the small-files case with sixty one-minute appends to a table, the shape a streaming job leaves behind (Figure 13.4). The table held 60 data files averaging 25 KB; one `binpack` call rewrote all 60 into a single 1.43 MB file, and a full scan fell from 0.45 to 0.12 s, nearly four times faster, with identical results.

13.5 Rewriting manifests



Figure 13.4: Sixty tiny commits, then compaction, then a manifest rewrite. Compaction fixed the files and the scan time but *added* a manifest; the manifest rewrite then merged all 61 into one, with no further change to this small scan.

Manifests accumulate their own version of the small-files problem: many small com-

mits each add a small manifest, and a scan's manifest-list pruning (Section 12.1) works less well the more, smaller manifests it has to consult individually. `rewrite_manifests` consolidates them into fewer, larger manifests with the same entries, improving planning time without touching a single data or delete file — a metadata-only compaction, distinct from and cheaper than `rewrite_data_files`. On the lab it merged 61 manifests into one, but the scan time did not move (0.12 against 0.13 s): with sixty small manifests on a local disk, planning was never the bottleneck. The benefit appears when thousands of manifests make planning itself slow (Section 12.1), which this lab does not reach.

13.6 Snapshot expiration and orphan files

Section 9.3 noted that superseded files stay reachable until something removes their snapshot from the log — that something is `expire_snapshots`: past a retention window, it drops old snapshot entries and, for any file no longer referenced by *any* remaining snapshot, deletes the physical file. This is the mechanism that actually reclaims the disk space compaction's rewrites and a row-level operation's rewritten files leave behind; skipping it does not corrupt anything, it just means old files accumulate indefinitely.

`remove_orphan_files` is a different, independent safety net: it scans the table's storage location directly for files that are not referenced by *any* manifest at all — typically the debris of a failed write that never committed, since a successful commit's files are always referenced. Because it does its own storage listing rather than reasoning from snapshots, it needs a generous `older_than` window (the default is three days) to avoid racing an in-flight write that has written files but not yet committed the metadata referencing them.

The lab ran both on the merge-on-read table left by the update experiment, which had 12 snapshots and 20 data files in a 386.1 MB directory (Figure 13.5). `expire_snapshots` with only the latest snapshot retained dropped 11 snapshots, deleted 11 data files, 11 manifests, and 11 manifest lists, and left a 350.8 MB directory: 35.3 MB reclaimed, the one large data file each superseded snapshot had kept alive. A 5 MB stray file, the kind a failed write leaves, then added 5 MB that no manifest named. `remove_orphan_files` first *refused* to run with a one-hour cutoff ("Cannot remove orphan files with an interval

less than 24 hours"); with the stray file aged three days and a 48-hour cutoff, a dry run listed exactly that file and the real run removed it, restoring 350.8 MB. The table still counted 9,678,114 rows.



Figure 13.5: Table directory size through the two cleanup procedures. Expiring snapshots frees what superseded history held; a stray file from a failed write adds 5 MB until `remove_orphan_files` deletes it.

Warning Guides disagree on a strict global order for these two procedures, and the official documentation does not mandate one. The mechanism explains why: `expire_snapshots` is what makes old files eligible for deletion by un-referencing them, and it is reasonable to run it before or interleaved with `remove_orphan_files`, which only ever touches files no snapshot ever referenced in the first place — the two operate on disjoint sets of files by construction, as long as `remove_orphan_files`'s retention window is wide enough to never touch a write still in flight. Treat the "run compaction, then expire, then remove orphans" sequencing some operational guides recommend as a safe default cadence, not a correctness requirement. Figure 13.6 shows it, and which step pays for which.

13.7 Metadata management for streaming tables

A streaming job committing every micro-batch (Chapter 16) is the worst case for every mechanism in this chapter at once, and each consequence compounds a different

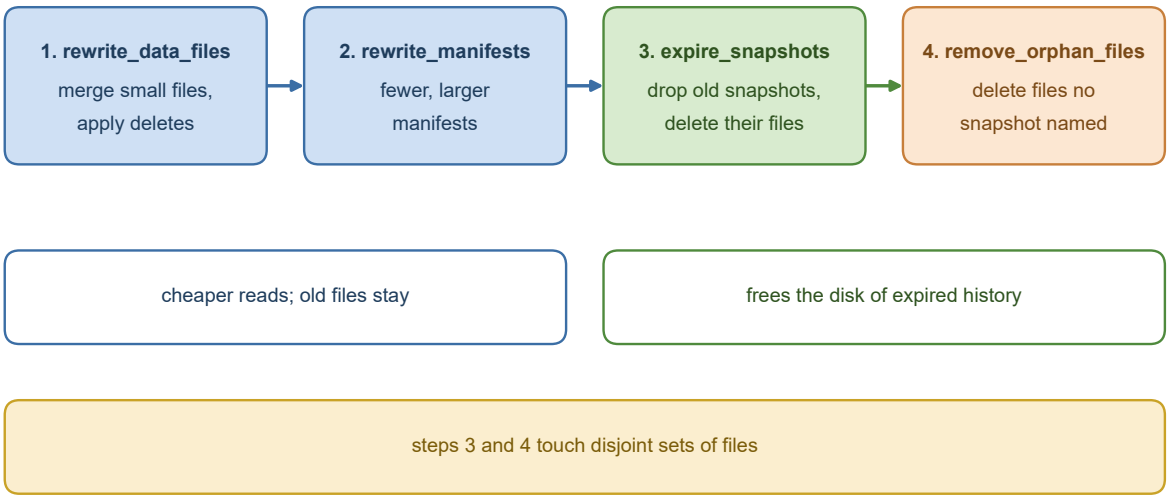


Figure 13.6: The four maintenance procedures in the usual order. The first two make reads cheaper and leave old files behind; the third frees the disk of expired history; the fourth removes files no snapshot named, which is a disjoint set.

cost already covered above. Many small commits mean many small files, which is Section 13.4's small-files problem, but arriving continuously rather than from one bad batch job. They also mean many small manifests, which slows manifest-list pruning (Section 12.1) on every subsequent read, since more, smaller manifests must be opened and checked individually to plan the same scan. And every commit adds one more entry to the table metadata file's snapshot log (Section 9.3), so that log — and the metadata JSON a client must parse just to resolve the current snapshot, and the search space time-travel queries walk — keeps growing for as long as the stream runs uncompacted, all from normal operation rather than a mistake. The practical answer is scheduling all three maintenance operations to run continuously and automatically against a streaming table — compaction and manifest rewriting on a short cadence matched to the commit interval, snapshot expiration on a looser one — rather than treating maintenance as an occasional manual task the way it can be on a table that only receives periodic batch writes.

13.8 Summary

Copy-on-write and merge-on-read trade write cost against read cost in opposite directions, set independently per operation type; pick from the update rate and the read-latency budget, not by default. A row-level operation's predicate does double duty — matching rows and, if it also resolves against partition values, keeping the scan narrow the same way any other filter does. Compaction and manifest rewriting are metadata- and data-level cleanups that keep reads fast; snapshot expiration and orphan-file removal are what actually reclaim disk space, operating on disjoint sets of files with no single mandated order between them. A streaming table needs all of this running continuously rather than occasionally.

Table 13.3 turns each symptom into the evidence and the lever.

Symptom	**Evidence**	**Lever**	**Cost**
frequent small updates are slow and heavy	snapshot summary: tens of MB added per update	`write.*.mode` = merge-on-read	slower reads; compaction
reads slower on a merge-on-read table	delete files in `files` (content 1)	`rewrite_data_files` with a delete threshold	a rewrite job
a `MERGE` scans the whole table	scan bytes of every partition	a range on the partition column in `ON`	none
many small files after streaming	tiny average size in `files`	`rewrite_data_files` (`binpack`)	a rewrite job
disk keeps growing after compaction	large directory, many snapshots	`expire_snapshots`	loses time travel past the retention
files in storage no snapshot names	dry-run `remove_orphan_files` lists them	`remove_orphan_files` with a wide cutoff	a listing of the location

Table 13.3: The write and maintenance levers: symptom, evidence, lever, and cost.

Exercises

1. Run a `MERGE` with and without the partition column in the `ON` clause. Explain the difference in files scanned from the plan.

2. Set a table to merge-on-read, run ten small updates, then run `rewrite_data_files`. Compare scan latency before and after.

3. Delete an entire partition's worth of rows in one statement and confirm from the plan that it resolved as a metadata-only delete.

Part IV

Operating Spark

Chapter 14

Tuning Resources and Configuration

This chapter does not hand you a table of recommended settings. It derives an executor shape from the memory map in Chapter 3 and the shuffle in Chapter 4, and then checks each derivation against a measurement: twelve executor shapes running the same stage, eight shuffle-partition counts, four dynamic-allocation setups, and four layers of configuration. The numbers come from the lab in Section A.3 (one JVM on one machine), so read them as shapes of curves and thresholds, not as production sizes. The chapter closes with a symptom-to-lever table; the one cluster-manager topic the lab cannot exercise, the Kubernetes shuffle story, is marked as such.

14.1 Executor sizing from the mechanism

Sizing an executor is choosing three numbers — `spark.executor.cores`, `spark.executor.memory`, `spark.executor.memoryOverhead` — and each one is bounded by a mechanism from an earlier chapter. The cleanest way to see the bounds is to hold the work constant and change only the shape.

The experiment. One stage, twelve shapes. The stage hash-partitions the 30-million-row `orders` table by `customer_id` into 8 shuffle partitions and sorts each partition by `order_ts` for a window function; with AQE off, every partition holds about 3.75 million rows. The lab runs Spark in local mode, so the single JVM *is* the executor: the heap is the executor heap and the core count is the slot count. The grid is heap $\in \{1, 2, 4\}$ GB by cores $\in \{1, 2, 4, 8\}$. Figure 14.1 shows what happened.

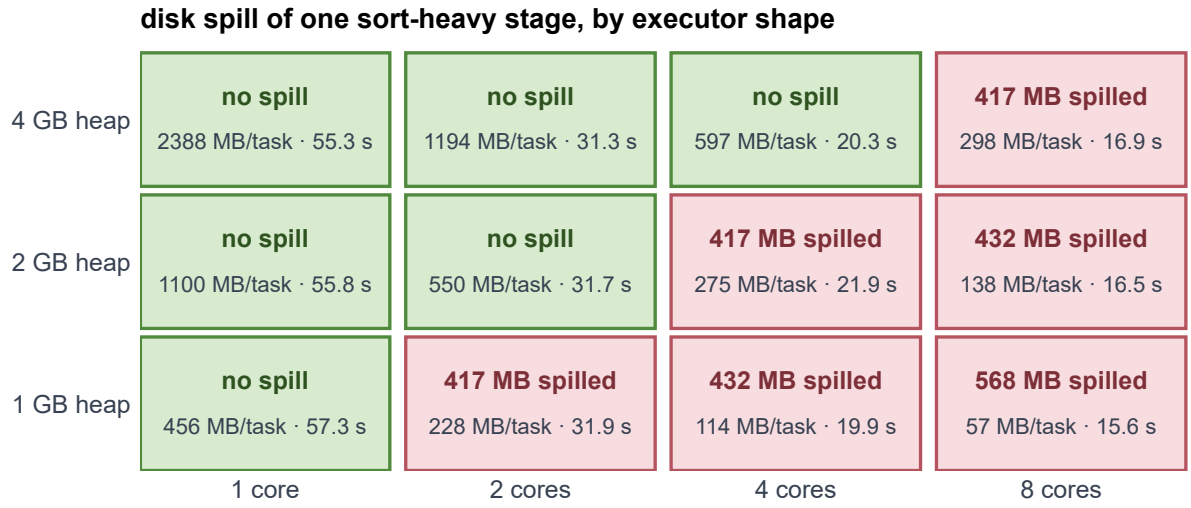


Figure 14.1: The same stage under twelve executor shapes. The spill pattern follows a diagonal: it appears whenever the execution memory per concurrent task falls below a threshold, whichever of heap or cores moved it there. Each cell is one run.

The diagonal is the memory model from Section 3.3 made visible. The unified pool is $(\text{heap} - 300\ \text{MiB}) \times 0.6$ (`spark.memory.fraction`), and the lab confirms the formula on all three heaps (Table 14.1). That pool is shared by the tasks running at the same moment, so each task's fair share is the pool divided by the core count — and that, not the heap on its own, is what a sort or a hash table runs into.

Figure 14.2 replots the twelve runs against that share. Every shape with 298 MB per task or less spilled; every shape with 456 MB or more did not. The stage's largest tasks therefore need somewhere between 300 and 450 MB of execution memory — the figure that a sizing exercise has to find, and the reason the heap alone is the wrong thing to tune.

What did the spilling cost? Less than the cores bought. Figure 14.3 shows wall-clock time falling from 55–57 seconds on one core to 15.6–16.9 seconds on eight, almost independent of the heap, even though the small heaps spilled 0.4–0.6 GB to disk along

Heap	Unified pool	Pool ÷ cores (MB per task)			
	measured	1 core	2 cores	4 cores	8 cores
1 GB	456 MB	456	228	114	57
2 GB	1100 MB	1100	550	275	138
4 GB	2388 MB	2388	1194	597	298

Table 14.1: The unified pool read from Spark's memory manager at start-up (it equals $(\text{heap} - 300\ \text{MiB}) \times 0.6$), and the share each concurrent task gets. Spilling cells in Figure 14.1 are exactly the ones below roughly 300–450 MB.

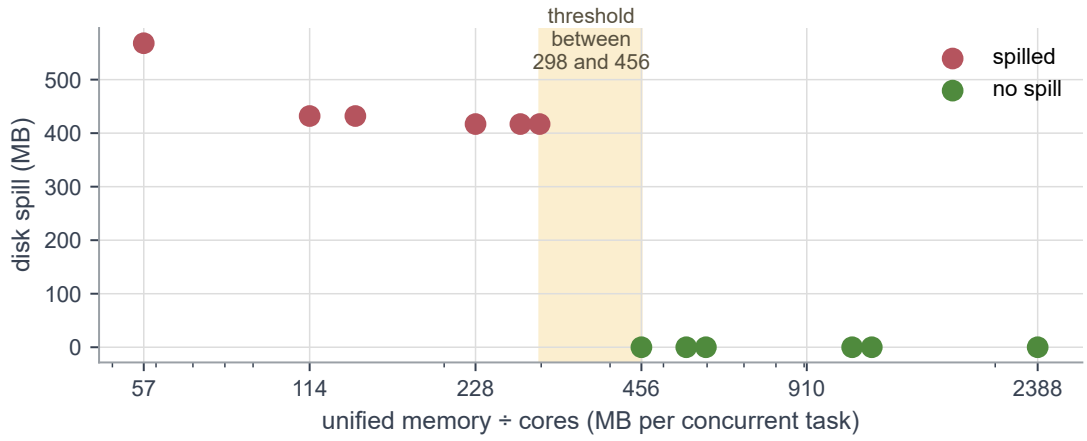


Figure 14.2: Spill against execution memory per task. The twelve runs separate cleanly at a single threshold, not by heap size and not by core count.

the way. The 2 GB heap at four cores spilled and took 21.9 s; the 4 GB heap at four cores did not and took 20.3 s, an 8% difference from one run each. On a laptop SSD with the operating system's page cache absorbing the spill files, a spill is cheap; on a cluster with network-attached local disks it may not be, and this lab cannot say. What does grow steadily with cores is garbage-collection time (right panel: 0.16 s to 1.0–1.8 s summed over all tasks), because more concurrent tasks hold more live rows in the same heap.

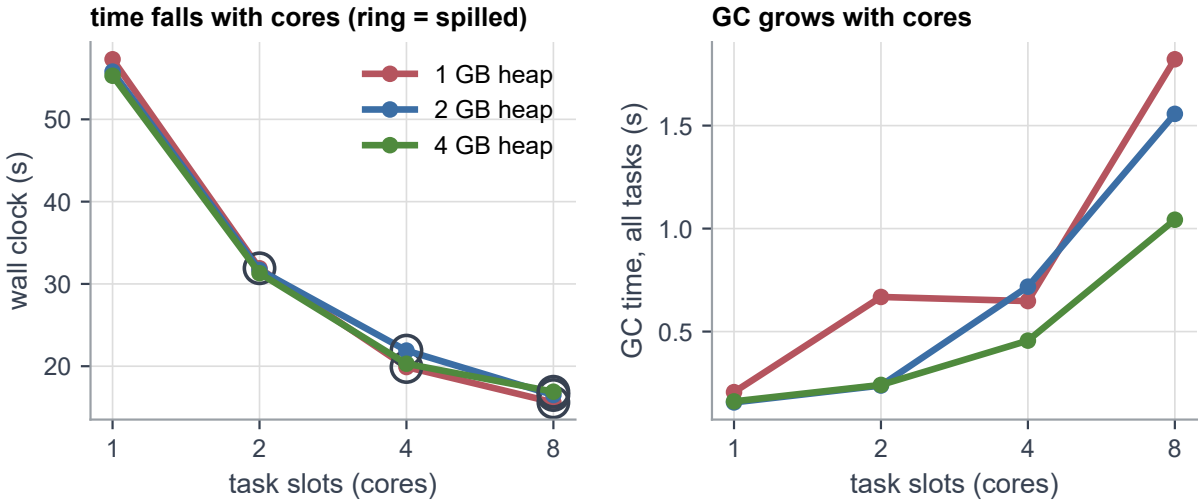


Figure 14.3: More slots buy time; more slots in the same heap cost GC. The rings mark runs that spilled. The three heaps lie almost on top of each other in the left panel: for this stage, spilling to a fast disk cost less than the run-to-run noise one would expect between single runs.

Cores per executor therefore trades two things: tasks run concurrently (time falls) against the share each gets of the pool (spill appears) and the heap they pollute together (GC grows). Folklore sets a ceiling of about five cores per executor for reasons tied to the storage client; the lab has no remote storage, so that is untested here — verify on your cluster by comparing scan-stage time at five and eight cores on the same data. The memory-side reason to stop early is the one measured above.

Executor memory is sized backward from the largest per-task need. For the specimen's reference cluster (4 cores per executor, Appendix A) the lab stage's 300–450 MB per task means a unified pool of $4 \times (300 \text{ to } 450) = 1.2$ to 1.8 GB, hence a heap of $1.2/0.6 + 0.3 \approx 2.3$ to 3.3 GB. `spark.executor.memory=4g` sits above that range with headroom for the storage side of the unified pool (Chapter 8) and is the defensible

first choice; it is a prediction for the *reference* data, and the same arithmetic with your own largest-task need replaces the 300–450 figure.

Memory overhead (`spark.executor.memoryOverhead`, the greater of 10% of executor memory or 384 MB) covers everything outside the heap: Tungsten off-heap pages if enabled, thread stacks, metaspace, and PySpark's worker processes. The formula is a floor, not a measurement of *your* workload. When a container is killed with no Java exception anywhere, the overhead was the number that was too small; Section 15.4 reproduces that failure on purpose.

> **Tip** On Kubernetes the pod's memory limit is executor memory plus overhead plus any off-heap size. Exceeding it kills the pod from outside the JVM, so the Spark log shows nothing — the evidence is in the pod's status, not in the executor's.

14.2 Parallelism: partitions against slots

`spark.default.parallelism` governs RDD operations with no other hint (defaulting to the total core count); for DataFrame and SQL workloads `spark.sql.shuffle.partitions` (Section 4.4, Section 6.3) is the number that matters. It is also a resource-sizing lever, because it decides how many of the executors' task slots a reduce stage can fill. Figure 14.4 sweeps it from 2 to 5000 on the lab's 8 slots, with AQE off, for two stages with opposite bottlenecks.

On the right, 2 partitions leave six of eight slots idle and take 35.9 s; 8 partitions (one wave) take 10.3 s; the bottom of the curve is 8.8 s at 40, and the stage slows again to 15.7 s at 5000 — the same time as 4 partitions, for the opposite reason. On the left the scan stage (about 32 input tasks) does most of the work, so the shuffle partition count hardly matters until 1000 and 5000 partitions add scheduling cost. The practical rule is the one Chapter 4 derived from bytes, now with its other boundary: a partition count at or above the slot count, and small enough that per-task overhead stays negligible. AQE's coalescing (Chapter 6) automates the upper boundary; nothing automates the lower one if the input to the shuffle is genuinely small.

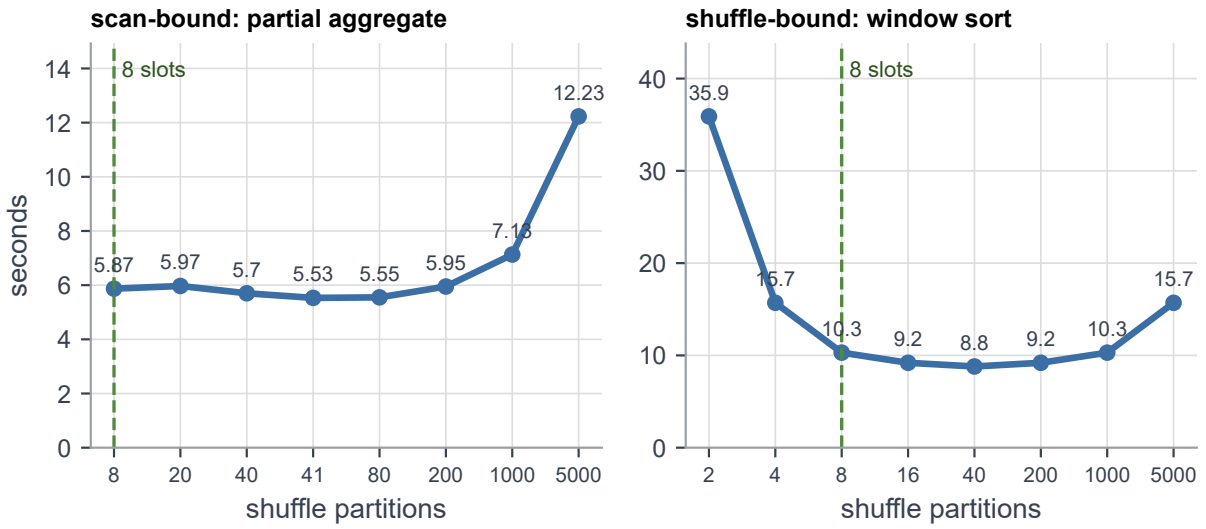


Figure 14.4: The same sweep on two stages. When the scan dominates (left) the shuffle partition count is nearly invisible until it reaches the thousands. When the shuffle stage does the work (right) too few partitions leave slots idle and too many pay a per-task cost: a hyperbola with a flat bottom.

14.3 Dynamic allocation

A fixed executor count is either wasted (idle executors between jobs in a long-lived session) or insufficient (a burst of work with no room to grow). `spark.dynamicAllocation.enabled` lets Spark request and release executors against the cluster manager as work arrives and drains: it adds executors when the scheduler's pending-task backlog persists past `spark.dynamicAllocation.schedulerBacklogTimeout` (1 s default), and removes an executor once it has been idle past `spark.dynamicAllocation.executorIdleTimeout` (60 s default), bounded by `spark.dynamicAllocation.minExecutors` and `.maxExecutors`.

The one condition that makes this safe is the failure mode of Section 15.4: an executor holding shuffle files that a later stage still needs cannot be released. Spark refuses to start otherwise. With dynamic allocation on and nothing protecting shuffle data, the lab's application died at start-up:

```
org.apache.spark.SparkException: Dynamic allocation of executors requires
one of the following conditions: 1) enabling external shuffle service through
spark.shuffle.service.enabled. 2) enabling shuffle tracking through
spark.dynamicAllocation.shuffleTracking.enabled. 3) enabling shuffle blocks
```

```
decommission through spark.decommission.enabled and
spark.storage.decommission.shuffleBlocks.enabled. 4) (Experimental)
configuring spark.shuffle.sort.io.plugin.class ...
```

Listing 14.1: Dynamic allocation without a plan for shuffle data (Spark 3.5.2, standalone cluster)

Figure 14.5 shows what shuffle tracking (`spark.dynamicAllocation.shuffleTracking.enabled`, on the lab cluster set explicitly) does in practice. The lab ran a standalone master and one four-core worker, executors of one core each, a maximum of four, and a 5-second idle timeout. Each run executes a job, idles for 25 s, and then runs the *same* DataFrame again, so that a surviving shuffle can be reused. (A first version of this experiment built a new DataFrame for the second job; Spark then recomputed the map stage whatever happened to the executors, and the comparison measured nothing.)

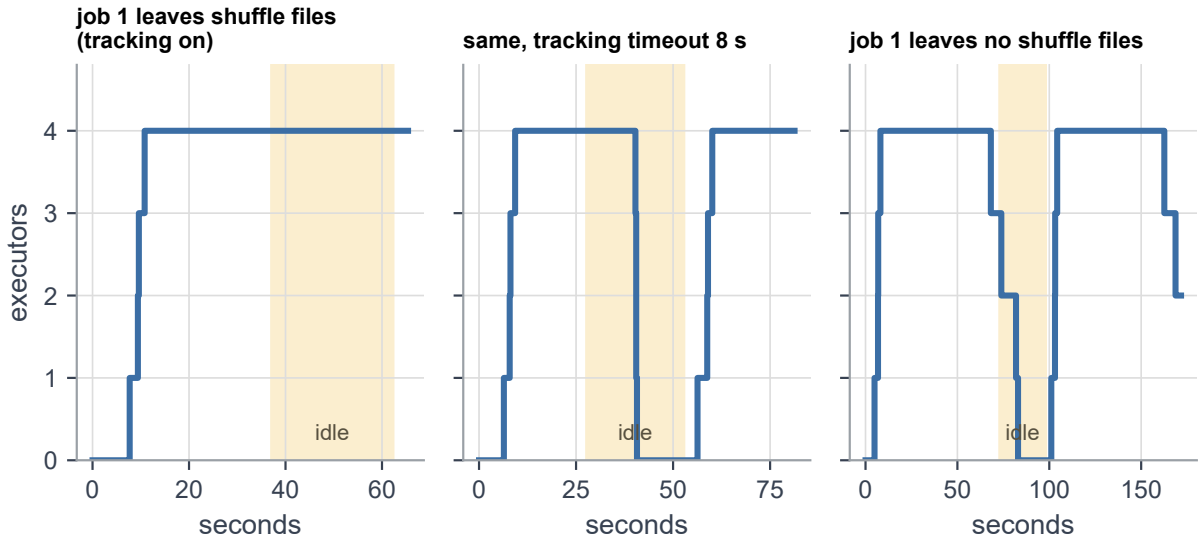


Figure 14.5: Executors over time, three configurations. Left: job 1 leaves shuffle files, and tracking keeps all four executors through the idle period. Middle: the same, with a tracking timeout of 8 s; the executors holding the files are released and the second job has to ask for them again. Right: a job that leaves no shuffle files, so every executor is released after its idle timeout.

Tracking makes scale-down conservative: Spark keeps an executor while it holds shuffle output that a live job or a re-run could use, and drops it only after `spark.dynamicAllocation.shuffleTracking.timeout` (infinite by default). The price is visible in the left panel: four executors sit idle for 25 seconds because their files might be needed

Setup	Executors in the idle period	Job 2 duration	Why
tracking on, shuffle files exist	4 (held)	0.2 s	the map stage is skipped; its output is still on the executors
tracking on, timeout 8 s	0 after about 13 s	25.6 s	executors re-requested; map stage recomputed
no shuffle files left	0	(no shuffle stage)	nothing to protect; released at the idle timeout
tracking off, no external service	—	—	application refuses to start (Listing 14.1)

Table 14.2: What dynamic allocation did in each lab setup. The first two rows run the same job; the difference, 0.2 s against 25.6 s, is the price of letting shuffle-holding executors go.

again — and the benefit is a second job that finishes in 0.2 s because its map stage is skipped, compared to 25.6 s when the executors are let go and the stage recomputed on freshly requested ones. An external shuffle service removes the price by keeping the files servable without the executor; the lab ran no external service, so that statement rests on the Spark documentation and the mechanism in Section 14.4, not on a measurement.

14.4 The external and push-based shuffle service

Without an external shuffle service, a reduce task fetches shuffle blocks directly from the executor that wrote them (Section 4.2) — the dependency that dynamic allocation has to work around, and the one that turns a premature executor removal into a `FetchFailedException`. `spark.shuffle.service.enabled` moves block serving out of the executor JVM into a separate long-lived process, so shuffle data stays servable after the executor that wrote it is gone and scale-down carries no data-loss risk. Push-based shuffle (Section 4.6) builds on the same external service to merge blocks on the reduce side.

> **Warning** Claim — from the Spark documentation, not tested in the lab: on YARN the external shuffle service is a NodeManager auxiliary service; on Kubernetes there is no such service, and dynamic allocation there relies on shuffle tracking (above) or on decommissioning. Before relying on `spark.shuffle.service.enabled` on a Kubernetes cluster, confirm from the executor logs that a shuffle service endpoint exists. Setting the key with nothing listening produces fetch failures that look exactly like the problem it was meant to prevent.

14.5 Configuration precedence and profiles

A key can be set in more than one place, and Spark resolves a conflict by a fixed order. The lab set the same key — `spark.sql.shuffle.partitions` — in every layer and asked the live session which value it saw (Table 14.3).

The order is therefore: a runtime `SET` for SQL settings, then values set in application code, then `--conf` flags, then `spark-defaults.conf`, then the built-in default. The code layer in the experiment set only the SQL key, so the memory column carries the

Where the key was set	shuffle.partitions	executor.memory
properties file only (`spark-defaults.conf` equivalent)	111	3g
file and `--conf` on `spark-submit`	222	5g
file, `--conf` and `SparkSession.builder.config`	300	5g
the above, then `spark.conf.set` on the running session	400	—
`spark.conf.set("spark.executor.memory", ...)` on the running session	rejected	—

Table 14.3: The value a session reported after each layer was added. Each layer overrides the one above it in the table; static settings such as executor memory cannot be changed once the application has started.

`--conf` value forward unchanged. The rejected last row is the practical rule: only SQL configuration is changeable mid-session. A value that looks ignored is usually overridden from a higher layer than the one you edited, and `spark.conf.get(key)` in the running session reports the value actually in effect — faster than re-reading files.

14.6 From symptom to lever

Table 14.4 collects the chapter's levers with the evidence that tells you which one applies and what it costs.

14.7 Summary

An executor's three sizes each trace to a mechanism: the heap bounds the unified pool as $(\text{heap} - 300\ \text{MiB}) \times 0.6$; that pool divided by the cores is the share each concurrent task spills against, and in the lab the threshold sat between 298 and 456 MB per task; the overhead covers everything outside the heap. Cores bought time (55 s to 16 s from one

Symptom	Evidence	Lever	Cost
sort or hash spills	*Spill* columns in the stage page, only on some tasks	unified pool ÷ cores below the largest per-task need: raise heap or lower cores (Figure 14.2)	memory per executor, or fewer slots
many idle slots in a shuffle stage	stage has fewer tasks than slots (2 partitions on 8 slots took 3.5× longer than 8)	raise shuffle partitions to at least the slot count	scheduling cost beyond a few thousand
GC time grows with cores	GC column in the executors summary	fewer cores per executor, or more heap	slower per-executor parallelism
idle executors held for minutes	timeline of executors vs jobs (Figure 14.5)	lower `shuffleTracking.timeout`	recompute if the files are needed
application will not start with dynamic allocation	the exception in Listing 14.1	enable shuffle tracking, or an external service	held executors
a setting seems ignored	`spark.conf.get` differs from the file	find the higher layer (Table 14.3)	none

Table 14.4: The tuning levers of this chapter: symptom, evidence, lever, cost.

to eight) and cost GC; spilling to a fast local disk cost far less than the cores returned. The shuffle partition count has a lower boundary set by the slot count and an upper one set by task overhead, with a flat bottom in between. Dynamic allocation is safe only when something protects shuffle data; tracking does so by keeping executors, and the lab measured what keeping them saves (0.2 s against 25.6 s) and what it holds idle. Configuration resolves in a fixed order in which only SQL keys can still change at runtime. The Kubernetes shuffle story is a design point the lab could not measure.

Exercises

1. Pick a stage of your own that spills. From its task-level metrics (Section 15.3) read the largest task's memory use, compute the unified pool per concurrent task, and predict whether halving the cores or doubling the heap removes the spill. Then test the prediction.

2. Repeat the shuffle-partition sweep of Figure 14.4 on a stage of your own with AQE off. Where is the bottom of your curve relative to the slot count?

3. Run a job, idle for longer than `spark.dynamicAllocation.executorIdleTimeout`, and run a second job that reuses the first job's shuffle. Compare the second job's duration with shuffle tracking on, with a short tracking timeout, and (on a cluster that has one) with an external shuffle service.

Chapter 15

Debugging and Profiling

A slow job leaves evidence: in the SQL tab, the stages tab, the executor metrics, the event log, and — when you need it — a flame graph. This chapter teaches you to read that evidence in order, so that "the job is slow" becomes "stage 2 has one task nine times the median" in a few minutes. Every failure it describes was reproduced on purpose on the lab (Section A.3): a skewed join, three different out-of-memory failures, a straggler with and without speculation, and a stage that lost half its shuffle files. The numbers in each section are the real output of those runs.

15.1 The Spark UI as a diagnostic

The UI is organized as a funnel (Figure 15.1), and reading it in the wrong order is the most common way to waste time on a slow job. The **SQL** tab shows the physical plan with live metrics on every node, so you can see *which operator* is expensive before worrying about which stage or task. The **Stages** tab is next: each stage's task-duration distribution is where skew becomes visible as a gap between median and maximum. The **Executors** tab aggregates across the application and shows imbalance between executors rather than between tasks. Only once the operator and the stage are named does it pay to open individual task metrics (Section 15.3).

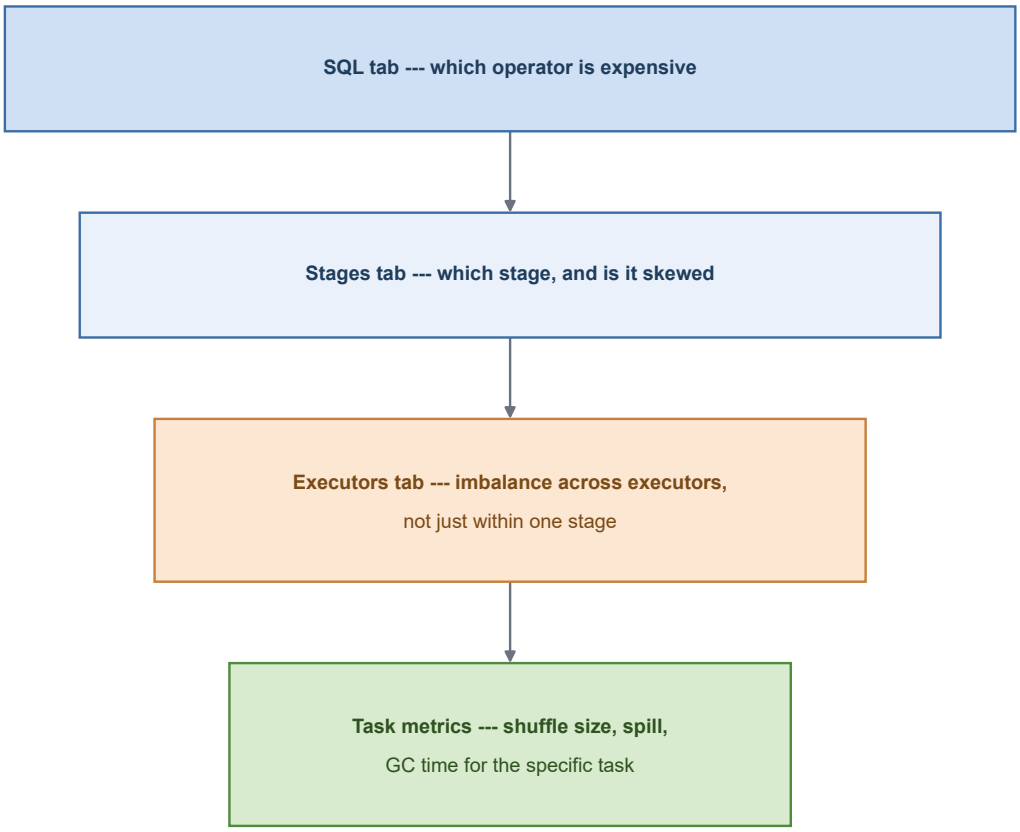


Figure 15.1: The UI read top to bottom: each tab narrows the search before the next is worth opening.

15.2 Event logs and the history server

Everything the live UI renders is first written to an *event log*: a JSON-lines file of every scheduler event, one line per event. The History Server replays that file through the same listener objects that build the live UI, so by the last line it holds the picture the live UI would have shown at exit — which matters for the jobs you most need to debug, those that failed and no longer have a UI. On Kubernetes it is the only diagnostic that outlives the executor pods.

The lab opened one. A single join query (125 tasks over four stages) produced an event log of 1.02 MB holding 276 events (Figure 15.2).

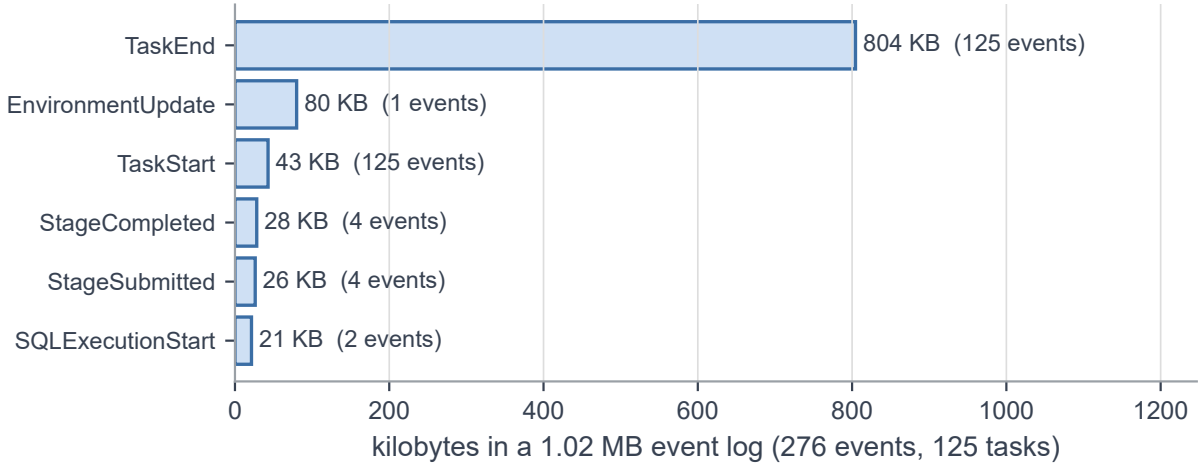


Figure 15.2: Bytes in the event log by event type. `TaskEnd` events, which carry every task's metrics, are 79% of the file at about 6.4 KB each; the environment snapshot is the second-largest entry at 80 KB.

The size rule follows: an event log grows with the number of *tasks*, about 6.4 KB per task here, so a job with 200,000 tasks writes about 1.3 GB, which is why very large jobs need a log compressed (`spark.eventLog.compress`) and rolled over (`spark.eventLog.rolling.enabled`). The lab ran the same job with compression on: the log shrank from 1,021,128 to 70,782 bytes, 14 times smaller. Rolling was not tested.

15.3 Reading task, shuffle, spill, and GC metrics

A single task's detail is the finest-grained evidence the UI offers, and the lab produced a real skewed stage to read. A sort-merge join of `orders` to `dim_product` on the hot product (30% of the rows share one `product_id`), 40 shuffle partitions, a 1 GB heap and four cores, AQE off. Figure 15.3 sorts the 40 tasks of the join stage by duration: 39 sit between 0.43 and 1.04 s and one takes 5.44 s.

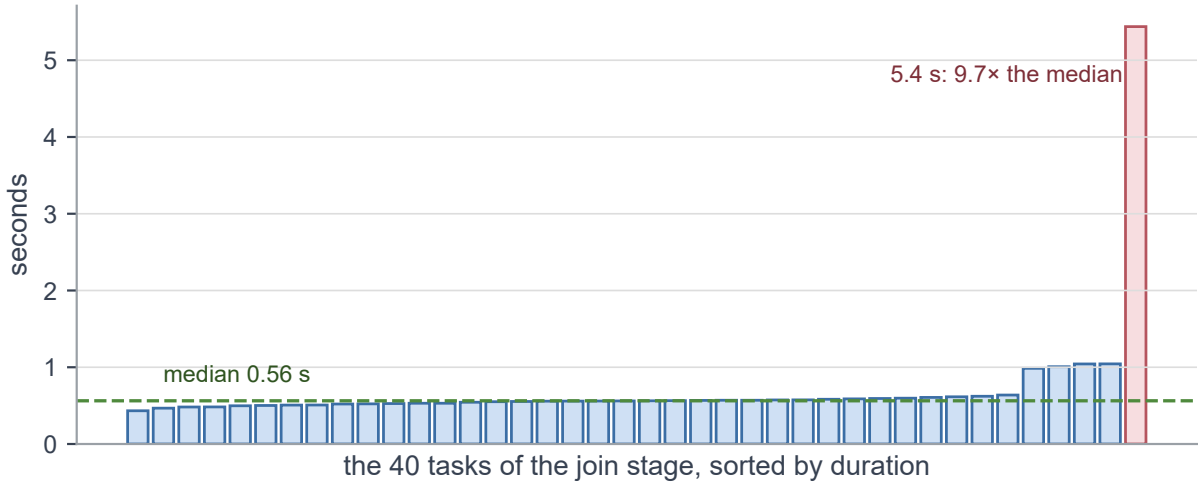


Figure 15.3: The 40 tasks of the join stage. One task is 9.7 times the median.

Figure 15.4 compares the median task with the slow one on the five numbers that do the diagnostic work.

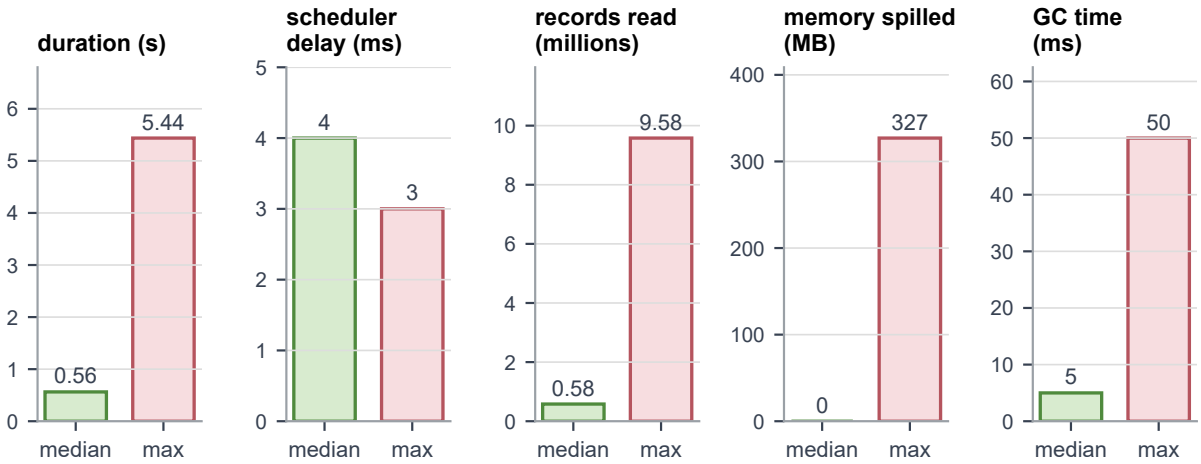


Figure 15.4: Median task against the slowest task of the same stage.

Task duration against scheduler delay separates real work from waiting for a slot. The

slow task ran 5.44 s with a scheduler delay of 3 ms: all of it was work, not queueing, which rules out "not enough slots" and points at "this task has too much to do."

Records or bytes read in the same stage is the most direct skew signal. The median task read 0.58 million records (8.2 MB); the slow one read 9.58 million (82.2 MB) — 16.5 times the records for 9.7 times the time.

Spill appears when a task's share of execution memory was too small for its sort. Only the slow task spilled: 327 MB of memory spilled, 72 MB written to disk, where every other task spilled nothing (the share was 456 MB divided by 4 cores, Chapter 14).

GC time was *not* the story here: 50 ms of a 5.4 s task, about 1%. A common narrative says skew drives GC; this measurement says check before assuming it, and that spill cost, not GC, was what the extra memory pressure bought.

Read together, the four numbers tell one story — a task with 16 times the input that spilled and ran ten times longer — and it is the same cause Section 6.5 detects and fixes automatically for a join, made visible here for any stage.

15.4 Common failure signatures

A job rarely fails as one incomprehensible error. It fails as one of a small number of recognizable signatures, each with its own evidence.

Out of memory: three failures, three traces. The lab triggered each on purpose (Table 15.1).

Two observations. First, the exception *class* separates the first two: `java.lang.OutOfMemoryEr` is the JVM refusing an allocation; `SparkOutOfMemoryError` is Spark's memory manager (Section 3.3) refusing to grant a page, with the byte count it could not get. Second, in local mode a driver `collect()` OOM was logged by an executor thread, because the driver and executor share one JVM: on a cluster the same failure would appear in the driver's log, and telling the two apart there is part of the diagnosis. The third row has

How triggered	Evidence	What overflowed
`collect()` of 30 M rows, 512 MB heap	`java.lang.OutOfMemoryError: Java heap space`, logged as `Executor: Exception in task`	the JVM heap, by an allocation Spark does not track
`collect_list` of one hot key, 512 MB heap	`SparkOutOfMemoryError: [UNABLE_TO_ACQUIRE_MEMORY] Unable to acquire 65536 bytes of memory, got 0`	Spark's own execution-memory accounting: the pool had no 64 KB page to give
off-heap sort in a 1 GB container	exit code 137, `OOMKilled=true`, **zero** `OutOfMemoryError` lines in the log	the container's cgroup limit: heap plus off-heap exceeded what the kernel allowed

Table 15.1: Three OOM signatures, reproduced on the lab. The distinguishing evidence is the exception class (or its absence), not the words "out of memory."

no exception at all, which is the signature — the JVM never got a chance to log, and the evidence is the container's exit status. Raising `spark.executor.memoryOverhead` is the fix for this one (Section 14.1), not the heap.

Stragglers and speculation. A straggler is one task running far longer than the rest with no data-size explanation, usually a node problem. The lab simulated one: eight tasks of 3 s, with the first attempt of task 3 sleeping 40 s, run on two workers registered under two different host names. Without speculation the job took 41.3 s. With `spark.speculation` on (multiplier 1.5, quantile 0.75) it took 11.2 s (Figure 15.5): once six of eight tasks had finished, Spark launched a copy of task 3 on a different executor at 8.1 s, which finished 3.1 s later, and the original was killed.

A detail that cost the lab one failed run: on a single worker with four executors all on one host, speculation had no effect (41.6 s both ways). Spark does not place a speculative copy on the host that already runs the original, so a cluster with one host per executor pool cannot speculate. Speculation cuts tail latency for node-related stragglers; for genuine skew it would just run the same oversized task twice.

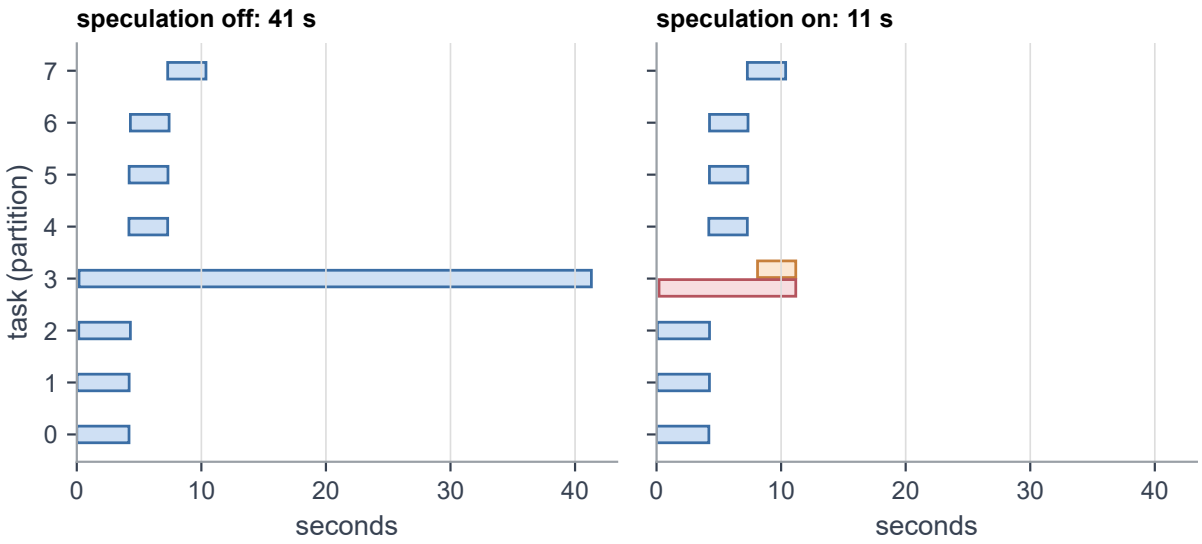


Figure 15.5: Task attempts without and with speculation. The speculative copy (orange) started at 8.1 s and won; the original (red) was killed when it did.

Fetch failures and stage retries. Shuffle files are local to the executor that wrote them, so losing the executor — an OOM kill, a node drain, dynamic allocation scaling down — loses them. The lab reproduced the recovery: after a job had run, it deleted 30 of the 57 shuffle files written by one of two executors, then ran the same query again (Figure 15.6).

A fetch failure is not a generic task failure. The scheduler recognizes it, invalidates that executor's map outputs in the map output tracker (Section 18.2), and resubmits only the missing map tasks before retrying the reduce stage. What the figure adds is that a stage chain can fail in *cascade*: the stage downstream of the lost output failed, and when that recomputation reached a second exchange it hit missing files again. You see this in the UI as stages with several attempts and a task count that varies by attempt. Two layers retry, and tuning the wrong one does nothing: the shuffle client retries a fetch from a live executor (`spark.shuffle.io.maxRetries`, `spark.shuffle.io.retryWait`); a genuinely lost executor is handled by stage resubmission, bounded by `spark.stage.maxConsecutiveAttempts` (4) and `spark.task.maxFailures` (4). Raise both on a cluster where executors are routinely reclaimed.

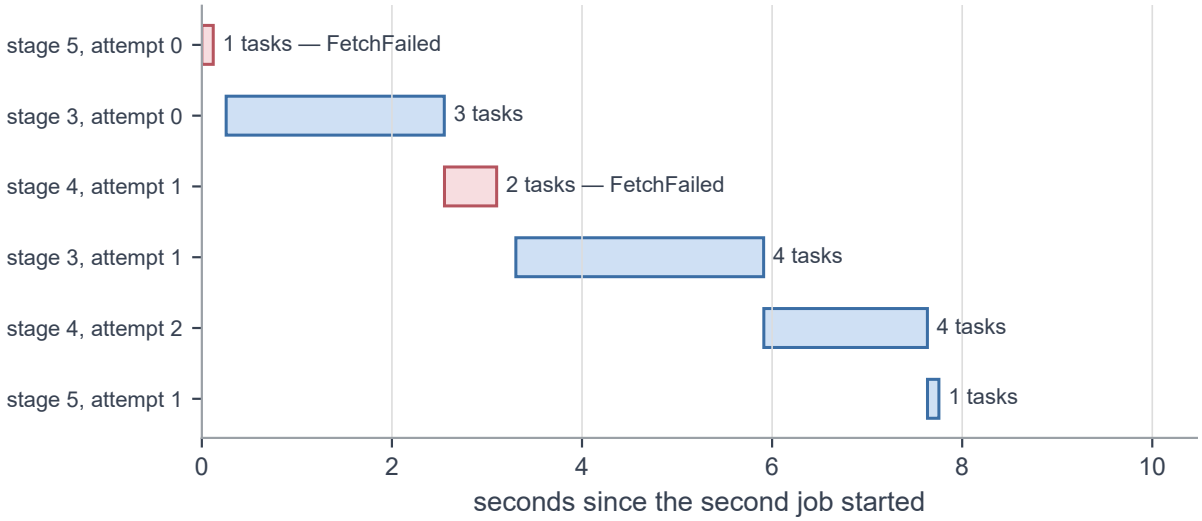


Figure 15.6: The stage attempts of the second run. The final stage failed at once with a `FetchFailedException`; the map stage re-ran for the missing output (3 tasks); a middle stage failed with a second `FetchFailedException`; the map stage ran again (4 tasks); then the middle and final stages succeeded. Three tasks failed with `FetchFailed`, 7.7 s in all.

15.5 async-profiler and flame graphs

The UI's metrics answer *which stage and which task*; they do not answer *which method is burning CPU* in a task that is neither spilling nor GC-bound. Instrumenting profilers rewrite every method to time it, so their overhead scales with call frequency. `async-profiler` samples instead: it interrupts the JVM at a fixed interval and reads the call stacks, so the cost is fixed per interrupt and independent of how hot the code is. The fraction of samples in a method converges on the fraction of time it consumes, and a flame graph draws exactly that: one bar per stack frame, width proportional to samples. A wide bar under a Spark-internal frame usually means the bottleneck is in a UDF or expression the generated code calls (Chapter 17, where a Python UDF cost 32 times a built-in), not in Spark's execution machinery. (Claim — not exercised in the lab, which has no profiler installed: the sampling mechanism and its low overhead are as documented by the project.)

15.6 Debugging query plans

A plan that does not do what you expect is diagnosed the way Chapter 2 taught you to read one, applied adversarially: compare the *analyzed* plan with what you intended (a wrong join condition or an implicit cast shows up here); check the *optimized* plan for whether pushdown and pruning fired (Figure 19.2 is the catalogue of what defeats them); and compare the *physical* plan's joins and exchanges with what Chapter 5 and Chapter 4 predict. `EXPLAIN FORMATTED`'s numbered node list plus per-node detail is almost always more useful than the tree alone, because you can cross-reference a node's exact filters and build side. A hint that appears ignored is checked the same way: read the operator (Section 5.6).

15.7 From symptom to lever

15.8 Summary

Diagnose top-down: the SQL tab names the expensive operator, the Stages tab names the stage and shows skew as a task-duration outlier, and only then do individual task metrics confirm the mechanism. In the lab's skewed stage one task read 16.5 times the records, spilled while its 39 neighbours did not, and took 9.7 times as long, with GC at 1% — a reminder to read the numbers before telling the textbook story. The event log makes this possible after the job has exited and costs about 6.4 KB per task. Three OOM failures leave three different traces, told apart by the exception class or its absence. A straggler on a separate host is cut from 41.3 to 11.2 s by speculation, which cannot act on a single host. A lost executor's shuffle files trigger a cascade of stage attempts, not a single retry.

Exercises

1. Take a stage summary of your own and compute the max-to-median ratio of duration and of records read. If the ratios differ by a large factor, what does that say about the cause?

2. Trigger each of the three OOM signatures on a test cluster and record the exception

Symptom	Evidence	Lever	Cost
one task far slower than the rest	max/median duration 9.7×; records read 16×	skew handling (Chapter 6) or repartition by a better key	shuffle cost
one slow task, ordinary input	records read near the median; scheduler delay low	`spark.speculation` (and only if node-related)	duplicate compute; needs more than one host
`java.lang.OutOfMemoryError`	exception class names the JVM heap	raise executor heap, or shrink what the task holds	memory per executor
`SparkOutOfMemoryError`	"Unable to acquire … got 0"	raise unified pool per task: more heap or fewer cores (Figure 14.2)	memory, or slots
executor vanishes, no exception	exit code 137, `OOMKilled`	raise `spark.executor.memoryOverhead`	pod size
stage runs several times	`FetchFailedException`, attempts > 1	shuffle service, tracking timeout, retries (Chapter 14)	held executors
event log is huge	about 6.4 KB per task	roll and compress the log	history-server CPU

Table 15.2: The debugging levers: symptom, evidence, lever, cost.

class (or the absence of one) and the exit status.

3. Reproduce Figure 15.6 by deleting one executor’s shuffle files between two runs of the same DataFrame, then count the stage attempts and `FetchFailed` tasks in the event log.

Chapter 16

Streaming into the Lakehouse

Structured Streaming is the same query engine over an unbounded table, processing one micro-batch at a time and ending each batch with a single Iceberg commit. The whole model from the earlier parts still applies — the shuffle, the executor budget, the commit — but with two new pressures: state that must be kept between batches, and a commit cadence that produces small files faster than a batch job. This chapter measures both on the lab (Section A.3): a rate source writing 20,000 rows per second into an Iceberg table at three trigger intervals, a windowed aggregation with and without a watermark, the three output modes, a query killed and restarted from its checkpoint, and `AvailableNow` *processing a growing directory.*

16.1 The micro-batch model and the unbounded table

Structured Streaming treats a stream as a table that keeps growing, and a streaming query as an ordinary DataFrame query re-run incrementally against the new rows since the last run. Each run is a *micro-batch*: the driver's `StreamExecution` asks the source for the range of new offsets available, plans and executes an ordinary batch job over exactly that range — the same Catalyst pipeline, the same job/stage/task machinery from Chapter 1 — and, on success, records the offsets it has processed to a *checkpoint* location before asking for the next range. Nothing about execution is special to streaming; what is special is that this loop runs forever and that recovery after a failure means replaying from the

last checkpointed offset rather than restarting the whole computation (Figure 16.1).

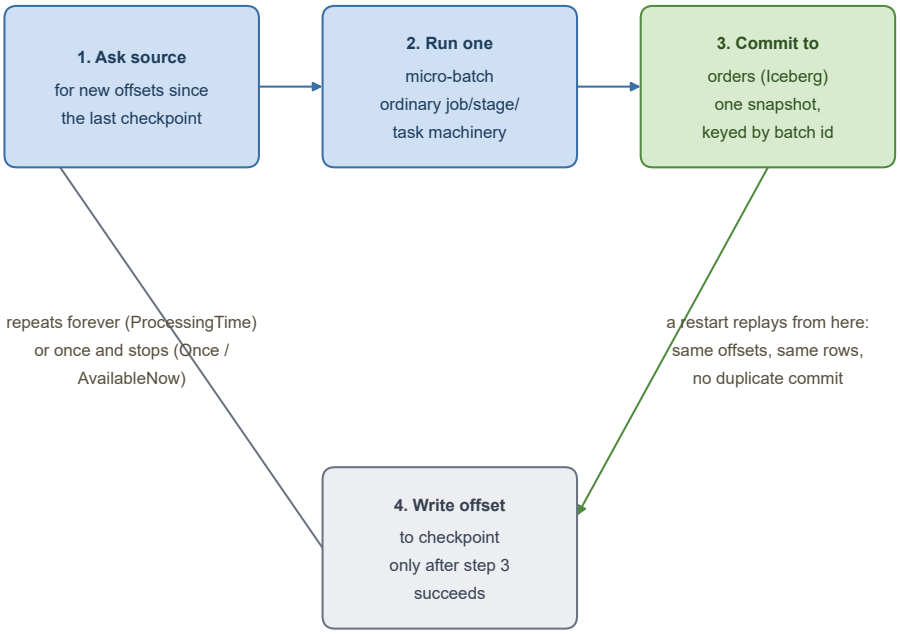


Figure 16.1: One iteration of the micro-batch loop. The offset log is written before the batch runs and the commit log after it, which is what makes a restart safe: a replay re-runs exactly the range that either never committed or already did, never anything in between.

The lab's checkpoint directory shows the loop's two logs. After the run described in Section 16.6, the checkpoint held `offsets/0` through `offsets/28` and `commits/0` through `commits/28`: one offset entry and one commit entry per micro-batch, 29 of each, matching the table's 29 snapshots.

16.2 Sources and sinks

A source needs one contract: given a starting offset, return the new records since it, and report the latest offset available. Kafka is the most common source, with a partition's offset serving directly as the streaming offset; the `file` source uses file names as offsets; the lab's `rate` source generates rows at a fixed speed and is used here so that the experiments are reproducible. A sink is symmetric: it must support exactly-once writes

idempotently keyed by batch id, or tolerate the batch being reprocessed. Writing to an Iceberg table satisfies this because a commit is already atomic (Section 9.3.1): a batch that failed after writing files but before committing never became visible (Figure 19.1 in Chapter 19 measured that for a batch write).

16.3 Event time and watermarking

Grouping by *event time* (when something happened, carried in the data — the specimen's `order_ts`) rather than *processing time* needs a rule for when a time-based group is complete, since late data can always arrive for a window already emitted. A *watermark* (`withWatermark("order_ts", "10 minutes")`) declares how late data is tolerated: Spark tracks the maximum `order_ts` seen so far and treats any window ending more than the watermark delay before that maximum as closed. The rule, applied by hand to one moment of the stream, looks like this (an arithmetic illustration of the rule, not captured output):

```
Max order_ts seen so far: 10:47   ->  watermark = 10:37
Window [10:30, 10:40) already closed (ends before 10:37)   ->
   emitted, forgotten
Window [10:40, 10:50) still open   (ends after 10:37)      ->
   still accepting rows
An order_ts=10:35 row arrives late -> closed window -> dropped,
    not counted
An order_ts=10:41 row arrives late -> open window   -> counted
   normally
```

Listing 16.1: Which orders close a 10-minute window, and which arrive too late (illustration of the rule)

The watermark is also what bounds state. The lab ran the same windowed count (5-second windows by 1,000 customer keys, 2,000 rows per second) twice for 100 seconds, once without a watermark and once with a 10-second watermark, and recorded the state store's row count after every micro-batch (Figure 16.2).

A batch aggregate never has this problem: it reads its input once and the state disappears when the job ends. A streaming aggregation's state keeps accumulating for as long

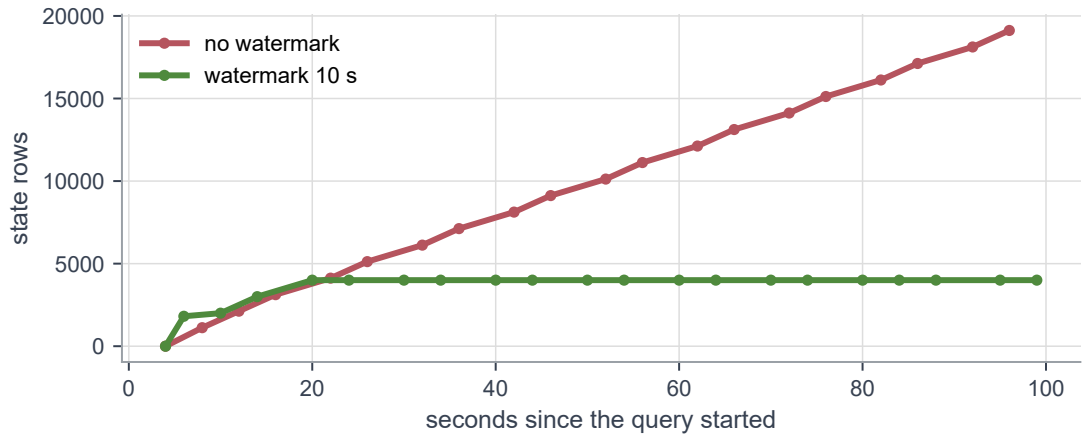


Figure 16.2: State rows against time. Without a watermark nothing tells Spark a window is complete, so state grows by about 1,000 rows every 5-second window and was still climbing (19,121 rows, 5.0 MB) when the run stopped. With a 10-second watermark it plateaus at 4,000 rows (1.4 MB) as closed windows are evicted — 16,000 rows removed over the run.

as the query runs, which is why state size, not input size, is what eventually threatens the heap in a way no batch operator ever has to plan around.

16.4 Stateful operations and the state store

The *state store* holds the partial results a windowed aggregation or a streaming join needs across micro-batches, checkpointed alongside the offsets so it survives a restart. The default provider keeps state as JVM objects in memory per task and snapshots it to the checkpoint; `RocksDBStateStoreProvider` (`spark.sql.streaming.stateStore.providerClass`) moves it into an embedded RocksDB instance per task — a log-structured merge tree that never updates a file in place, flushes a small in-memory buffer as an immutable sorted file, and merges files in the background. That structure lets it hold more state than fits in the heap, at the cost of occasional compaction pauses and a disk read for state outside its block cache.

The lab compared the two providers on a heavier state: 20,000 rows per second into 10-second windows by 40,000 customer keys, 120 seconds, 2 GB heap (Figure 16.3). At about 520,000 state rows the default provider reported 144 MB of state memory and the RocksDB provider 22 MB — 6.5 times less memory held by the store — while

the micro-batch took 1.2 s against 0.8 s, with the state-store commit at 1.05 s against 0.53 s. RocksDB trades per-batch latency for a bounded memory footprint; at this size the lab's heap was not under pressure, so the benefit shown is footprint, not avoided GC. The two memory figures are not perfectly comparable — they are each provider's own accounting.

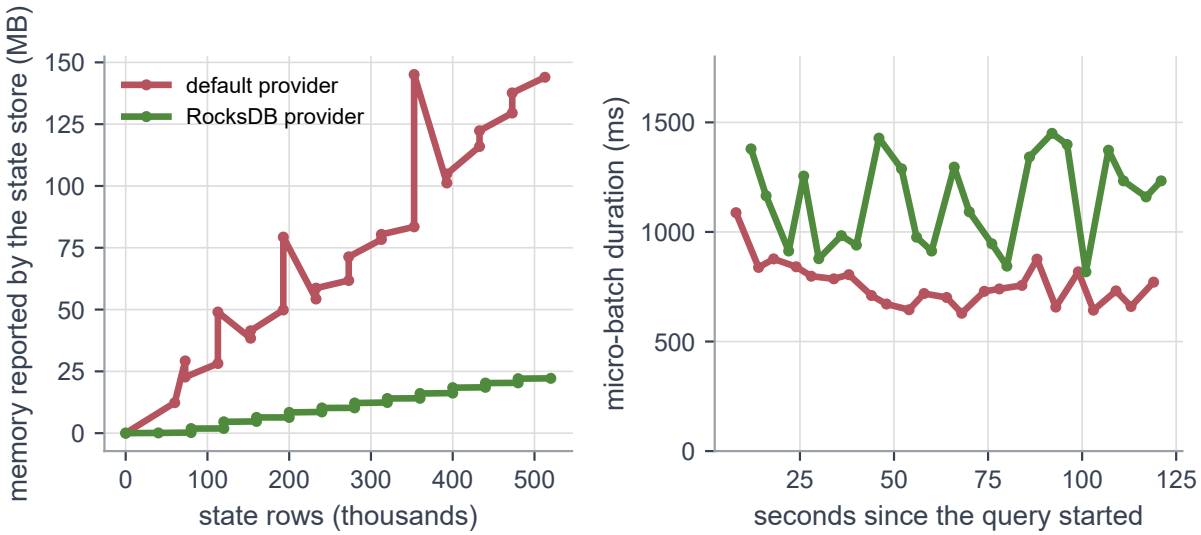


Figure 16.3: Left: reported state memory against state rows for the default and RocksDB providers. Right: micro-batch duration over time. RocksDB held the same rows in one sixth of the reported memory and paid with slower commits.

16.5 Output modes and triggers

Output mode controls what a micro-batch emits. The lab ran the same windowed count under all three modes with a 5-second watermark and recorded the rows each batch emitted (Figure 16.4).

`append` is final-only: a row leaves once its window is closed by the watermark, so output lags the data by the watermark delay. `update` emits changes, including rows already emitted before. `complete` is the one that does not scale: its output is every window ever seen, so its cost grows with the life of the query — viable only for a small, bounded aggregate.

Trigger controls when a micro-batch runs. `ProcessingTime` fires on a fixed interval; `AvailableNow` reuses the same loop for one iteration over everything available and exits,

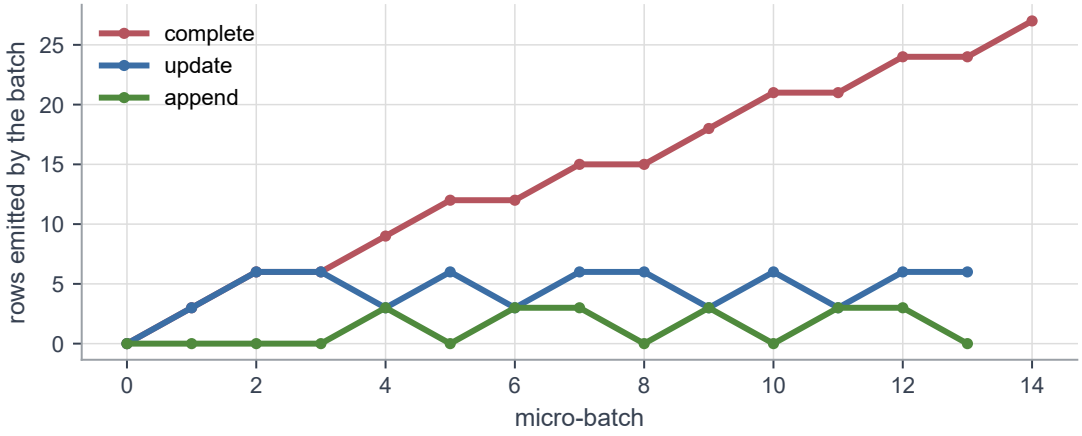


Figure 16.4: Rows emitted per micro-batch for the same query. `append` emits nothing until a window closes (0 or 3 rows); `update` emits the windows that changed (3 to 6 rows); `complete` re-emits the whole result table, which grew from 3 to 27 rows over 15 batches and keeps growing.

which turns a streaming query into a scheduled incremental job. The lab ran it three times against a directory of Parquet files (Table 16.1).

Run	New rows landed	Seconds	Table rows	Snapshots
1	100,000	6.6	100,000	1
2	0	1.6	100,000	1
3	50,000	4.2	150,000	2

Table 16.1: `Trigger.AvailableNow` over a directory that grows between runs. The first run processes everything, the second finds nothing and commits nothing (no empty snapshot), and the third processes only the new files.

16.6 Exactly-once and the commit

"Exactly-once" is two guarantees, neither sufficient alone: the checkpoint's offset log records which input range a batch covers before the batch runs, so a restart replays the same range rather than skipping or duplicating input; and the sink makes committing that range idempotent. For Iceberg the sink writes its identity into each snapshot's summary. The lab read it back from a streaming-written table:

```
spark.sql.streaming.queryId   = c3fa69d9-60d4-4f1b-b5a2-a55d422fe0f3
spark.sql.streaming.epochId   = 49
added-data-files = 1     added-records = 40000     added-files-size = 65127
```

Listing 16.2: Streaming identity recorded in a snapshot summary (Iceberg 1.6.1, selected keys)

The keys are `spark.sql.streaming.queryId` and `spark.sql.streaming.epochId`: the query that wrote the snapshot and the micro-batch number. The lab tested the replay directly: after a query had committed epochs 0 to 8, the last commit-log entry (and its checksum file) was deleted, which makes Spark believe epoch 8 never committed, and the query was restarted. It re-ran epoch 8, carried on to epoch 23, and the table held 79,000 rows, all distinct, with epoch 8 appearing in exactly one snapshot. A replayed batch that had already committed was not committed twice.

The lab then tested the behaviour end to end. A query writing 2,000 rows per second into a fresh table with a 3-second trigger was killed with `docker kill` about 60 seconds in, and restarted with the same checkpoint for another 30 seconds. The result (Figure 16.5):



Figure 16.5: Records per snapshot across a kill and restart. The table ended with 29 snapshots whose epoch ids run 0 to 28 without a gap under one query id; the first batch after the restart (epoch 19) carried 20,000 records, catching up on the downtime. The table held 176,000 rows, all distinct, covering sequence numbers 0 to 175,999 with no gap.

No row was duplicated or lost, and the 28 data files on disk equalled the 28 in the

table, so the kill left no orphan. The experiment does not prove that a kill in the narrow window between writing files and committing is safe — it cannot say where in the batch the kill landed — but it shows the replay behaviour on a real kill: the restart resumed from the checkpoint's last batch and the offset range it covered.

16.7 Writing to Iceberg: cadence, small files, maintenance

Every micro-batch that writes to Iceberg ends in exactly one commit — one new snapshot, one new manifest list, at least one new manifest — regardless of how little data it contained. The lab ran the same 20,000-rows-per-second stream into a day-partitioned table at three trigger intervals for 100 seconds each (Figure 16.6).

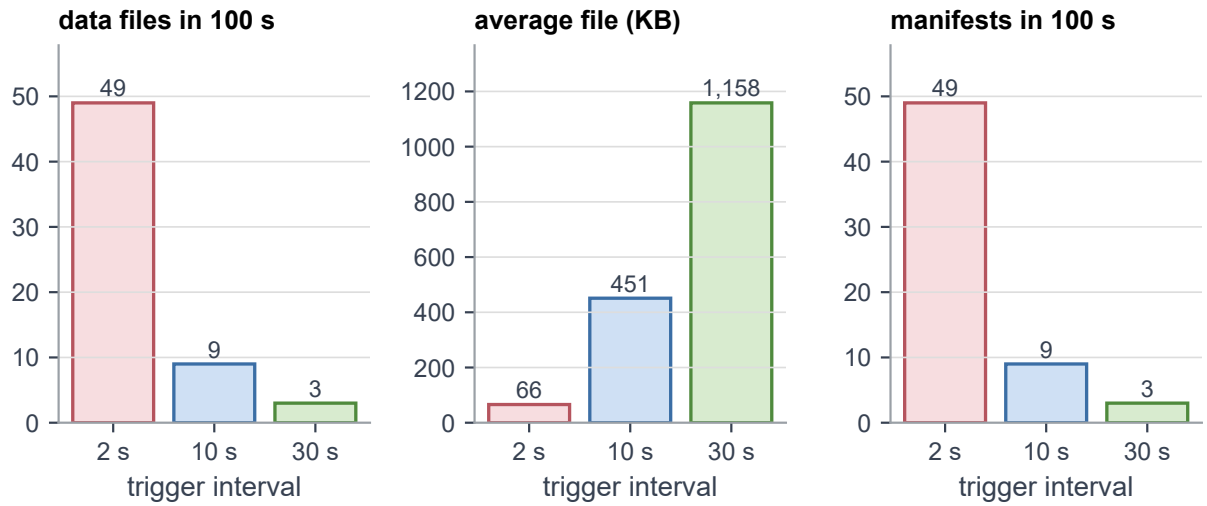


Figure 16.6: The trigger interval sets the commit count, and with it the file count, the file size and the manifest count. A 2-second trigger produced 49 files of 66 KB and 49 manifests in 100 seconds; a 30-second trigger produced 3 files of 1.16 MB and 3 manifests.

The relationship is exact and simple: files, snapshots and manifests grow as one over the trigger interval, and average file size grows in proportion to it. A stream with a short trigger is therefore the fastest way to produce the small-files and manifest-growth problems of Chapter 13. The lever is the trigger interval against the latency budget — decide the interval from how fresh a row must be, then treat continuous compaction

Trigger	Snapshots / 100 s	Files / 100 s	Average file	Files per day (extrapolated)
2 s	50	49	66 KB	about 42,000
10 s	10	9	451 KB	about 7,800
30 s	4	3	1,159 KB	about 2,600

Table 16.2: The lab's measurements and their extrapolation to a day ($\times 864$). Each trigger produced one file per commit because the table has one partition for the day; the lab's rate (about 1.7 bytes per row after compression) is far lower than a production stream, so the shape of the relationship is the transferable part, not the absolute sizes.

(Section 13.4) as the backstop for the file size that interval implies, not as a substitute for choosing it.

16.8 From symptom to lever

16.9 Summary

A streaming query is an ordinary batch query re-run incrementally over new offsets, recovering by replaying from a checkpoint; the lab's checkpoint held one offset and one commit entry per micro-batch. A watermark is what makes a windowed aggregation ever finish a window and what bounds state: without it state grew without limit (19,121 rows and counting), with it state plateaued at 4,000. Output mode decides what a batch emits, and `complete` re-emits a result that grows for the life of the query. Exactly-once combines the offset log with an idempotent sink; Iceberg records `queryId` and `epochId` in each snapshot, and a killed and restarted query produced 176,000 distinct rows with no gap and no orphan file. Every commit's fixed overhead multiplied by a short trigger is the cause of a streaming table's small-files and manifest pressure: 49 files in 100 seconds at a 2-second trigger against 3 at 30 seconds.

Exercises

1. Given a trigger interval and an input rate, estimate files committed per hour and the

Symptom	Evidence	Lever	Cost
thousands of tiny files per day	`files`: 66 KB average at a 2 s trigger	widen the trigger; compact continuously	latency; a rewrite job
manifests multiply with commits	`manifests`: one per commit (49 in 100 s)	`rewrite_manifests` on a schedule	a metadata rewrite
state grows without bound	state rows still rising at the end of the run	add a watermark (4,000-row plateau)	late rows dropped
output table grows every batch	`complete` mode: 3 to 27 rows in 15 batches	use `update` or `append`	none
need a scheduled job, not a service	`AvailableNow`: a no-data run commits nothing	`AvailableNow` from a cron job	start-up time per run
duplicate or missing rows after a crash	row count vs distinct count; epoch ids with a gap	same checkpoint, same query id	none

Table 16.3: The streaming levers: symptom, evidence, lever, cost.

compaction cadence needed to keep the table healthy.

2. Run a windowed aggregation with and without a watermark; read `stateOperators.numRowsTo` from `recentProgress` after each batch and plot it.

3. Kill a streaming query mid-batch and restart it with the same checkpoint. Confirm from the table's snapshot summaries that the epoch ids have no gap and that the row count equals the distinct count of a unique column.

Part V

Internals

Chapter 17

Tungsten and Code Generation

Below the plan, a stage does not interpret operators row by row. Tungsten stores rows as packed bytes, and whole-stage code generation collapses an operator subtree into a single generated Java method that the JIT compiles to tight machine code. This chapter opens that layer with the real thing: the binary row format, the `EXPLAIN CODEGEN` *output of a stage from the lab, and four measurements — what turning code generation off costs, what a Python UDF costs, where the 100-field limit bites, and what off-heap execution memory did and did not change. All numbers come from the lab in Section A.3 (Spark 3.5.2, local mode, 30 million rows); single runs unless stated.*

17.1 The Tungsten binary row format

A plain JVM object holding a row — one object per field, boxed primitives, object headers — is the per-object overhead Section 3.2 measured, multiplied by every row in every partition. `UnsafeRow` avoids it by laying a row out as one contiguous byte buffer: a null-tracking bitset first, then one 8-byte slot per field (a primitive stored inline, or an offset-and-length pair pointing into a variable-length region at the end of the same buffer for a string or binary value). Figure 17.1 shows this for one row of the specimen's `orders` table.

Reading a field is a direct byte-offset access: for a fixed field, read 8 bytes at a statically known offset; for a variable field, read the (offset, length) pair, then that many

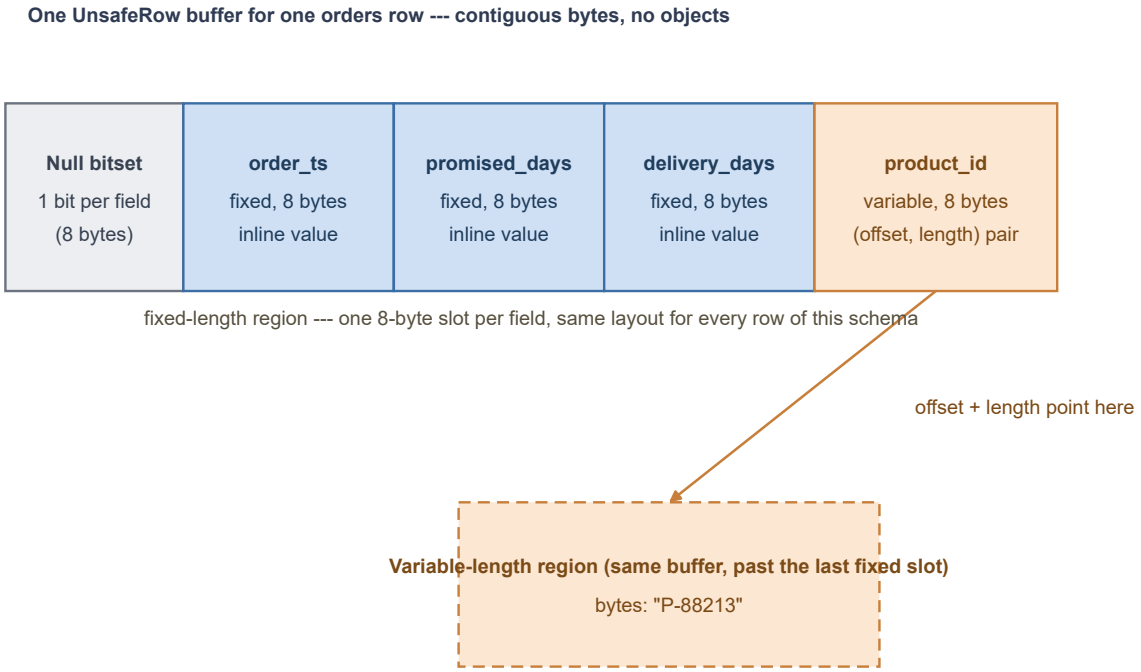


Figure 17.1: One `UnsafeRow`: a null bitset, one 8-byte slot per field, and a variable-length region past the last fixed slot. A fixed field's slot holds its value directly; a variable field's slot holds an (offset, length) pointer into the variable region of the same buffer.

bytes. There is no pointer chasing through an object graph and no per-field object to allocate or collect. The trade is that every value must fit this fixed encoding, which is why Catalyst carries a fully resolved schema (Chapter 2) all the way to execution.

By the layout rule an `orders` row has a 64-byte fixed part (an 8-byte bitset plus seven 8-byte slots) and a variable part padded to 8-byte words — the 36-character `order_id` takes 40 bytes, the 8-character `product_id` 8, the 9-character `customer_id` 16 — about 128 bytes in total (computed from the layout, not measured). That is more than the 84 bytes per row the source reports to the planner (Table 19.2) and more than the 36 bytes per row the same data occupies in compressed Parquet (1.08 GB of files for 30 million rows): three different sizes for the same row, one for each place the engine needs a number.

17.2 Off-heap memory management

`UnsafeRow` buffers do not have to live on the JVM heap. When `spark.memory.offHeap.enabled` is set, Tungsten's `TaskMemoryManager` allocates fixed-size pages directly from off-heap memory using `sun.misc.Unsafe`-style raw access. A row in an off-heap page is invisible to the garbage collector — not cheap to collect, never considered — and the price is manual lifecycle management: a task must free a page explicitly when it no longer needs it.

The mechanism suggests off-heap memory should cut garbage-collection time. The lab tested that on the sort-heavy stage of Chapter 14, adding a 1 GB off-heap pool to a 1 GB and a 2 GB heap at four cores (Figure 17.2).

On this stage the hypothesis did not hold: GC time was already small (0.65–0.72 s summed over 40 tasks and about 20 s of wall clock), the spill volume did not change, and the off-heap runs were slower. The lab does not isolate why — one plausible reading is that the stage's garbage was not the sorted rows but the short-lived objects around them, and that off-heap allocation adds its own cost — so the safe conclusion is narrower than the folklore: *measure GC time first* (Section 15.3); off-heap pays only when GC time is a large share of task time, and in this stage it was about 1%. When off-heap is enabled the pool is extra memory and must be added to the container limit (Section 14.1).

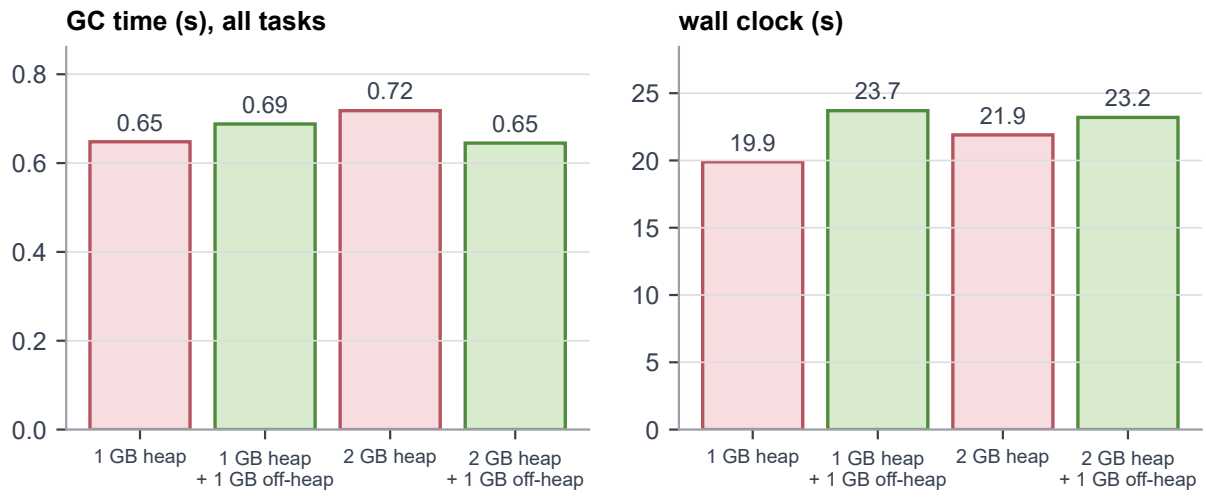


Figure 17.2: The same stage on-heap and with a 1 GB off-heap pool added. GC time did not fall (0.65 to 0.72 s in all four), spill was identical (417 MB to disk), and wall-clock time was 6–20% higher with off-heap enabled. One run each.

17.3 Cache-aware computation

A CPU cache miss costs orders of magnitude more than a hit, and sorting an array of object pointers — the classic JVM layout — follows a different, effectively random pointer on every comparison. Tungsten's sort packs a compact *(prefix, pointer)* pair per row into one flat array: the prefix is a fixed-size encoding of enough of the sort key to settle most comparisons, so the hot loop reads sequentially through one small array and chases the pointer to the full row only on a tie. For fixed-width numeric keys Spark goes further with a radix sort that orders by key bytes with no comparisons at all, controlled by `spark.sql.sort.enableRadixSort` (default true).

The lab sorted nine days of `orders` by `order_ts` (a timestamp, so radix-eligible) through a window function, with the setting on and off (Figure 17.3).

17.4 Whole-stage code generation internals

Interpreting a plan operator by operator means a virtual call and a row hand-off at every boundary, none of which the JIT can see through because the call target varies with the plan. *Whole-stage code generation* collapses a chain of adjacent operators that implement `CodegenSupport` into one generated Java method using a *produce/consume* proto-



Figure 17.3: The same sort with radix sort on (the default) and off. Radix sort made the stage 1.7× faster; the comparison sort the setting falls back to is the prefix-based one described above.

col: each operator's `produce()` emits the code that generates its rows and calls into the next operator's `consume()` *inline*. The `*(n)` markers this produces were introduced in Section 2.6. To see what the generated method looks like, the lab ran `EXPLAIN CODEGEN` on a scan, filter, projection and partial aggregate over `orders`.

```
*(1) HashAggregate(keys=[], functions=[partial_sum(order_total)])
+- *(1) Project [order_total]
   +- *(1) Filter (order_total > 100.0 AND promised_days < 5 ...)
      +- *(1) ColumnarToRow
         +- BatchScan lab.db.orders[promised_days, order_total]

/* 037 */ private void hashAgg_doAggregateWithoutKey_0() {
/* 045 */   while (columnartorow_mutableStateArray_1[0] != null) {
/* 048 */     for (int localIdx = 0; localIdx < localEnd; localIdx++) {
/* 051 */       boolean isNull_1 = vector[1].isNullAt(rowIdx);
/* 052 */       double value_1 = isNull_1 ? -1.0 : vector[1].getDouble(rowIdx
);
/* 058 */       filter_value_3 = SQLOrderingUtil.compareDoubles(value_1,
100.0D) > 0;
/* 059 */       if (!filter_value_3) continue;
/* 067 */       filter_value_9 = value_0 < 5;
/* 068 */       if (!filter_value_9) continue;
/* 074 */       hashAgg_doConsume_0(value_1, false);
```

Listing 17.1: The plan of the lab's stage 1 and an excerpt of its generated method (`EXPLAIN CODEGEN`, Spark 3.5.2; lines shortened)

Read the listing as the mechanism made concrete. The batch loop (`while`/`for`) sits *inside* the generated method: the scan hands over a whole `ColumnarBatch`, and the generated code walks it row by row reading each column through a typed accessor (`getDouble`) — no `InternalRow` object per row. The filter is two inline comparisons with `continue` for the rows that fail; the aggregation call (`hashAgg_doConsume_0`) is the next operator's `consume()` spliced in at that point. Scan adapter, filter, projection and partial aggregate are one method, which is what lets the JIT compile them as a unit. Counting the generated source is also a quick check on size: this one stage compiled to 283 bytes of bytecode in its largest method (`maxMethodCodeSize` in the output header), far from the 64 KB method limit.

Figure 17.4 measures what the fusion is worth, by running the same three queries with `spark.sql.codegen.wholeStage` on and off.

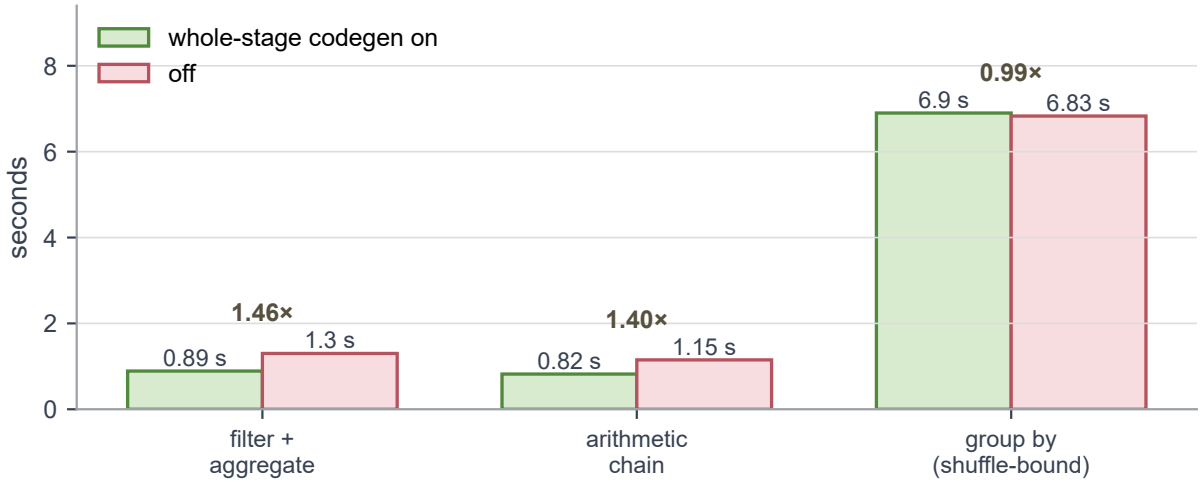


Figure 17.4: Whole-stage codegen on and off. A filter-and-aggregate and an arithmetic chain ran 1.4–1.5× faster with it; a shuffle-bound group-by did not change (0.99×), because its time is in the exchange, not in the fused loop.

The scan itself is already vectorized (Section 12.5 measured that at about 7× over the row-based reader), so the 1.4–1.5× here is what fusion adds on top; and fusion pays where the stage's time is CPU in the fused operators, not where it is data movement.

> **Note** Not everything fuses. An `Exchange` always breaks the chain — there is no straight-line loop across a network boundary — so codegen restarts on the far side of every shuffle; this is the stage boundary Section 2.5 described from the other direction. A sort-merge join implements `CodegenSupport` and fuses like any other operator (Listing C.1). What does not fuse is an operator Catalyst cannot generate code for: a row-at-a-time Python UDF, below.

17.5 Expression code generation and its limits

Beneath whole-stage codegen, an individual Catalyst `Expression` generates its own Java fragment rather than being evaluated by a generic interpreter tree. Two ceilings apply. A single JVM method is limited to 64 KB of bytecode, and Spark splits oversized generated code across methods first. And `spark.sql.codegen.maxFields` (default 100) makes Spark *give up* on whole-stage codegen for a stage that handles more than 100 fields, falling back to interpreted evaluation. The lab built a `SELECT` with 10 to 400 derived columns (each `order_total * i + promised_days`) and checked whether the projection stayed inside whole-stage codegen (Figure 17.5).

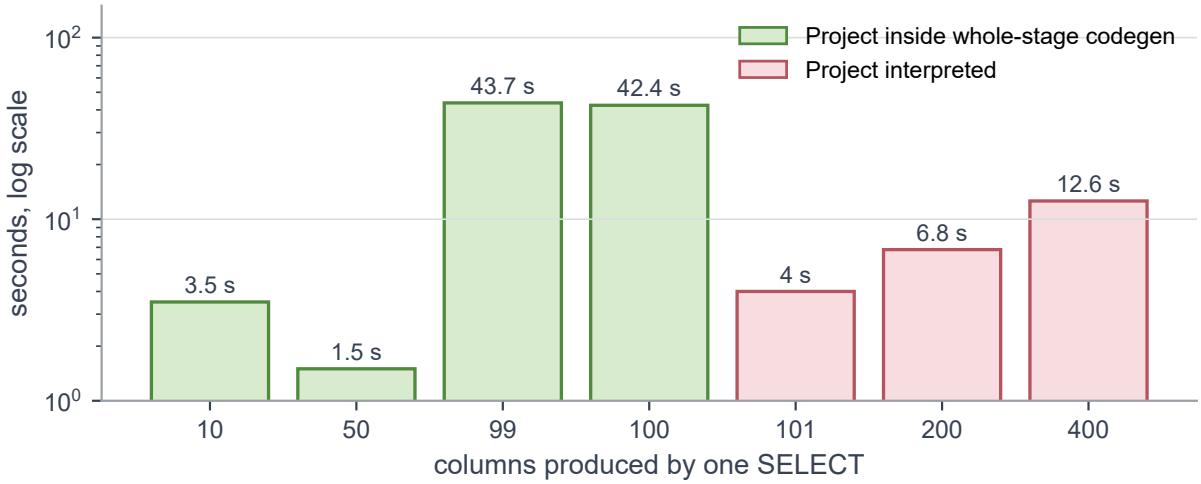


Figure 17.5: Whole-stage codegen holds up to exactly 100 columns and is abandoned at 101. The interesting part is the time: the 99- and 100-column stages took about 43 s, ten times the 4 s of the 101-column interpreted fallback. One run each, log scale.

The limit is where the documentation says: 100 columns fuse, 101 do not. What the documentation does not say is that fusing the last few columns before the limit was the

slowest configuration in the experiment. The measurement does not isolate the cause; the mechanism that fits is that the generated method for 99–100 wide expressions is huge, and a method above the JVM's `HugeMethodLimit` (8,000 bytes of bytecode) is not JIT-compiled by default and runs in the bytecode interpreter, so "generated code" stops being fast code. The lab tested that: with the JVM flag `-XX:-DontCompileHugeMethods` the 99-column stage ran in 4.5 s instead of 40.1 s, while the 50-column stage was unchanged at 4.4 s. The generated code was fine; the JIT was refusing to compile it. The practical reading is for tooling that emits hundreds of columns: wide projections are not where whole-stage codegen helps, and a few more columns can be cheaper than a few fewer.

17.6 Columnar processing and vectorization

`UnsafeRow` is row-oriented: one contiguous buffer per row. `ColumnarBatch` stores a batch the other way, one contiguous buffer per *column* across many rows — the layout Section 12.5's vectorized Parquet reader produces directly from the file's own columnar encoding. It is also the representation a pandas UDF communicates through, and the reason it is fast is specific: `ColumnarBatch` and Arrow's in-memory format are close enough that a batch can be handed to the Python process with no per-value (de)serialization loop. A row-at-a-time Python UDF has no such shortcut: every row crosses the JVM/Python boundary as its own converted object.

Figure 17.6 measures the three ways to compute `order_total * 1.07` and sum it over 30 million rows.

The row-at-a-time UDF is 32× slower than the built-in; the Arrow-based one recovers a factor of 3.7 of that and is still 8.6× slower, because the data must still leave the JVM. The fix that costs nothing is to express the logic in built-in functions so that Catalyst can generate code for it. (The pandas UDF needed `pyarrow`, which the lab image does not ship; the lab installed it into its throwaway container for this run.)

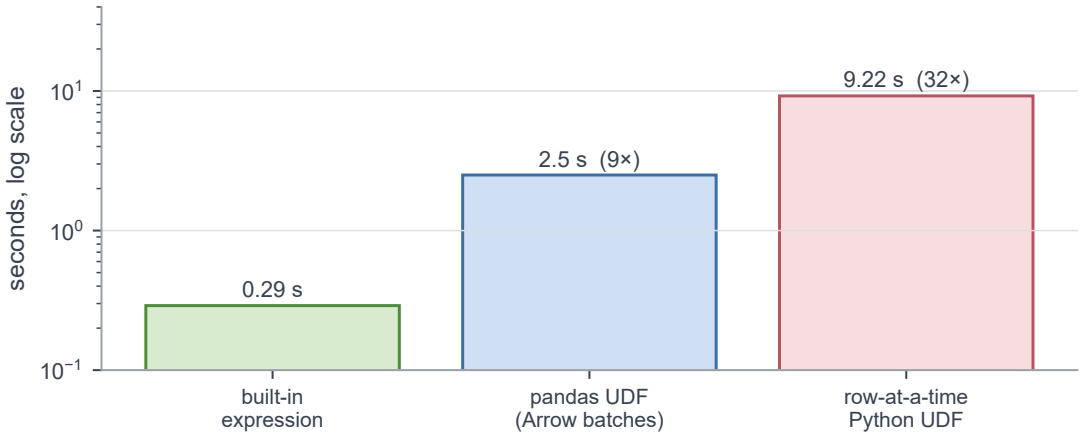


Figure 17.6: The same arithmetic as a built-in expression (0.29 s), a pandas UDF evaluated in Arrow batches (2.5 s) and a row-at-a-time Python UDF (9.2 s). The plan shows `ArrowEvalPython` for the second and `BatchEvalPython` for the third; neither is inside the whole-stage-codegen loop.

Symptom	**Evidence**	**Lever**	**Cost**
a CPU-bound stage is slow	no `*(n)` on its operators in `EXPLAIN`	find what blocks fusion (a UDF, more than 100 fields)	rewrite of the expression
a Python UDF dominates	`BatchEvalPython` in the plan; 30× slower than built-ins	built-in functions, or a pandas UDF (Arrow batches)	pyarrow dependency
a wide `SELECT` is slower than a narrower one	codegen time, not run time; 99–100 columns slowest in the lab	split the projection or stay above the 100-field limit	none
sort of a numeric or timestamp key is slow	`enableRadixSort` false or key not radix-eligible	leave radix sort on (default)	none
GC time is high	GC column in the UI	measure first; off-heap did not help when GC was 1% (Figure 17.2)	container memory

Table 17.1: The Tungsten levers: symptom, evidence, lever, cost.

17.7 From symptom to lever

17.8 Summary

`UnsafeRow` replaces one object per row with a packed byte buffer; the Tungsten sort works on prefix-plus-pointer arrays, and radix sort made a timestamp sort 1.7× faster. Whole-stage codegen fuses a chain of operators into one generated method whose loop runs over a columnar batch; the lab's listing shows scan, filter, projection and partial aggregate in a single method and the fusion worth 1.4–1.5× on CPU-bound stages and nothing on a shuffle-bound one. Fusion has a hard edge at 100 fields, and the stage right below that edge was the slowest in the experiment. A row-at-a-time Python UDF cost 32× a built-in expression, an Arrow-based one 8.6×. Off-heap memory did not reduce GC time on a stage where GC was already about 1% of task time.

Exercises

1. Run `EXPLAIN CODEGEN` on a filter-and-project query of your own. Identify the loop over the batch, each operator's inlined code, and the line where the next operator's `consume()` appears.

2. Build a `SELECT` with 99, 100 and 101 derived columns and compare wall-clock times and the `*(n)` markers. Does the 99–100 column slowdown reproduce on your cluster?

3. Replace a Python UDF in one of your queries with a built-in expression and with a pandas UDF. Record the plan node and the time of each.

Chapter 18

The Shuffle Framework and the Scheduler

Chapter 4 treated the shuffle as a wall. This chapter builds the wall: the shuffle manager and its writers, the map output tracker that tells reduce tasks where to fetch from, the DAG scheduler that cuts a plan into stages, the task scheduler that places tasks on executors, and the block transfer service that moves the bytes. It is the machinery behind every stage boundary and every straggler.

18.1 The shuffle manager and shuffle writers

Every shuffle on a modern Spark version goes through `SortShuffleManager` — the separate hash-based shuffle manager from early Spark versions was removed once sort-based shuffle's one-file-plus-index-per-task layout (Section 4.2) proved strictly better at scale. What varies is which *writer* a given shuffle uses underneath that one manager, chosen automatically — not configured — from the shuffle's own properties, in this order. One of those properties, referenced repeatedly below, is whether the shuffle needs *map-side aggregation*: combining values that share a key (a running `SUM`, say) on the map side, before the shuffle write, so only one partially-combined value per key crosses the network instead of every raw value — the same partial-then-final idea Section 2.5

traced for `HashAggregate`. A plain shuffle-then-join has nothing to combine, so it never needs this; a `GROUP BY` always does.

Table 18.1: Which shuffle writer a given shuffle gets, in the order Spark checks

Writer	**Fires when**	**Why it's cheaper**
`BypassMergeSortShuffleWriter`	no map-side aggregation, and partition count $\leq$ `spark.shuffle.sort.bypassMergeThreshold` (200)	writes one file per partition directly and concatenates them — skips sorting entirely, cheap only because so few files need merging
`UnsafeShuffleWriter`	serializer supports relocating serialized values without deserializing (Kryo, under most configs), and no map-side aggregation	sorts compact *(partition-id, pointer)* arrays (Section 17.3) over already-serialized bytes, never reconstructing an object mid-sort
`SortShuffleWriter`	neither precondition above holds (bypass threshold exceeded, or the shuffle aggregates)	general fallback: a real record-by-record sort by partition (and key, if aggregating)

The lab did not take the table on faith: it read the shuffle handle of every exchange in five query shapes at four partition counts, by walking the dependency graph of the executed plan (Figure 18.1).

Two findings correct earlier drafts of this book. At 200 partitions — the default and the specimen's count — the writer is the *bypass* writer, not the serialized one: the precondition is `partitions` $\leq$ `200`, and the check comes first. And a `GROUP BY` does not fall to the general sort writer: in Spark SQL the aggregation happens in `HashAggregate` operators on either side of the exchange, so the shuffle itself has no map-side combine

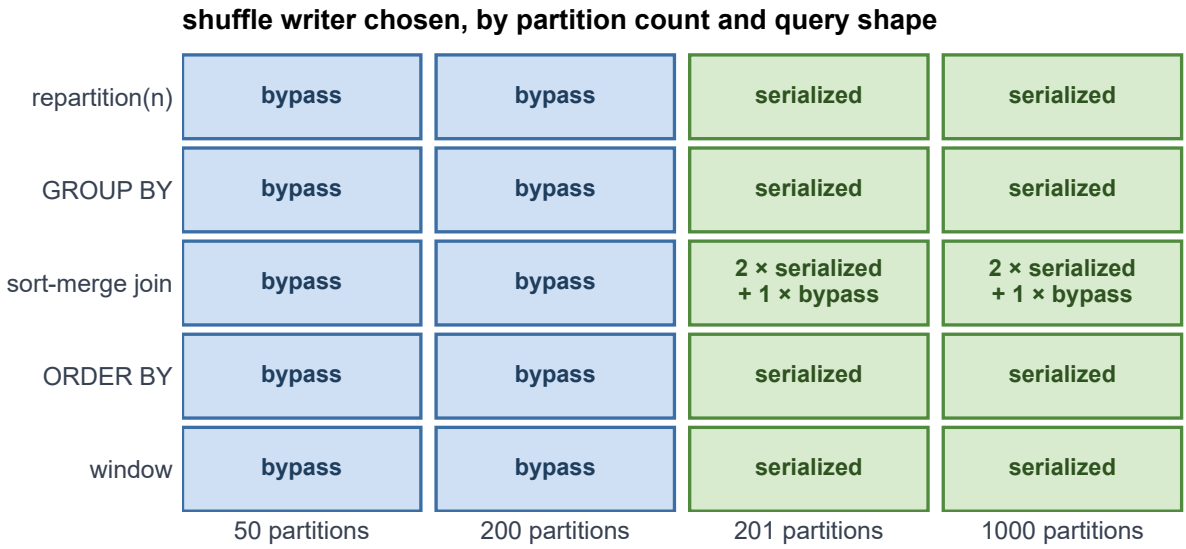


Figure 18.1: The writer each SQL shuffle got, in Spark 3.5.2. The choice flips at exactly 200 partitions: up to the bypass threshold every shuffle used the bypass writer; at 201 and above, the serialized (`UnsafeShuffleWriter`) path. The sort-merge join's third exchange, the single-partition one under the final count, stayed on bypass. No SQL shuffle used the general sort writer.

and qualifies for bypass or serialized. The sort-based fallback is reached by RDD code that asks for map-side aggregation, not by DataFrame queries. A shuffle you did not optimize is therefore not running the expensive path; the one knob that moves it between the two cheap ones is the partition count, and Figure 14.4 showed that count matters for other reasons.

18.2 The map output tracker

A reduce task needs to know which executors hold which map task's output before it can fetch anything — that registry is the *map output tracker*. The driver's copy, `MapOutputTrackerMaster`, is the single authoritative record, updated every time a map task finishes (with its output location) and every time an executor is lost (with that executor's outputs invalidated, which is precisely the trigger behind Section 15.4's stage-resubmission behavior). Executors hold a read-only, periodically-refreshed cache of the same information, `MapOutputTrackerWorker`, so that a reduce task can resolve fetch locations locally most of the time without an RPC to the driver for every single task.

18.3 The DAG scheduler and the task scheduler

Two schedulers do two different jobs, one layered on top of the other, and the split matters because they operate on different objects: the DAG scheduler never sees an individual task's placement, and the task scheduler never sees the dependency graph. The **DAG scheduler** builds the job/stage structure Chapter 1 and Section 1.5 already described (walking RDD lineage backward from an action, cutting a stage at every wide dependency) and owns the stage-resubmission decision behind Section 15.4's fetch-failure recovery — what is new here is what it hands off once a stage is ready: a *set of runnable tasks*, with no further opinion on execution order or placement. The **task scheduler** (`TaskSchedulerImpl`) owns everything past that handoff: given that set and the current executors, it decides *which* task runs on *which* executor, honoring the locality preferences below and retrying a task, up to `spark.task.maxFailures`, on a different executor before giving up on the task entirely. A stage resubmission from the DAG scheduler and a task retry from the task scheduler look similar from the UI but are two different components acting at two different granularities — one re-running a whole stage's missing map output, the other re-running one task that failed for a reason narrower than a lost executor.

Figure 15.6 in Chapter 15 is the DAG scheduler at work: after a lost output, the same stages appear with attempts 0, 1 and 2, each with only the tasks whose output was missing. The lab also measured what the task scheduler costs per task, by running a stage of empty tasks at four sizes on the 8-slot lab (Figure 18.2): about 0.04 ms per task from 5,000 to 20,000 tasks, 0.09 ms for the first 1,000 (start-up included), and 0.23 ms per task at 80,000 — 18 s for the stage. Scheduling is cheap until the task count is in the tens of thousands, then it is not; the cause was not isolated (the driver also records an event per task, Figure 15.2). This is the per-task cost behind the right-hand end of Figure 14.4.

18.4 RPC and block transfer

Coordination messages (task assignment, status updates, the map output tracker's queries) travel over Spark's internal RPC layer, built on Netty; the actual shuffle bytes travel over

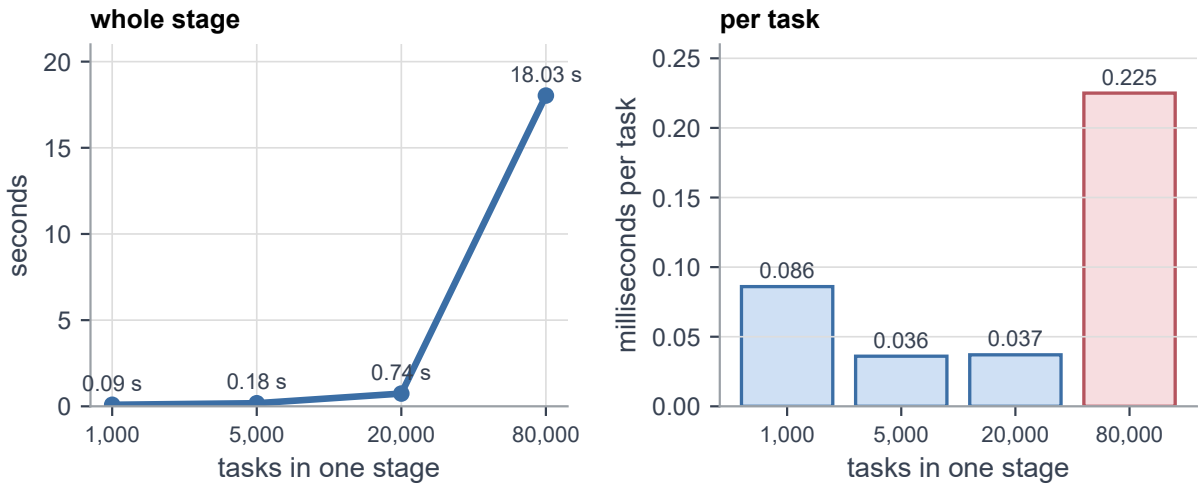


Figure 18.2: Driver-side cost of a stage of empty tasks. Per-task cost is flat at about 0.04 ms up to 20,000 tasks and rises sixfold at 80,000.

a separate path, the `NettyBlockTransferService`. The two are kept apart because they have opposite traffic shapes and opposite failure costs: an RPC message is small and latency-sensitive — a slow task-assignment reply stalls scheduling for every waiting task — while a shuffle fetch is large and throughput-sensitive, and can be retried or resumed without threatening the scheduler's own responsiveness. Running both over one channel would let a handful of multi-hundred-megabyte block transfers queue up behind (or starve out) the small control messages the scheduler depends on to keep moving; separating them means a busy shuffle fetch never blocks the RPC call that would otherwise reassign a freed task slot. This split is also what makes the external shuffle service possible at all: it only has to reimplement the `NettyBlockTransferService` half as a standalone process (Section 14.4), with no need to also speak the scheduler's RPC protocol, because block serving never depended on it. Figure 18.3 puts all five pieces from this chapter in one picture, with the RPC/block-transfer split drawn as the two separate paths it actually is.

18.5 Locality and speculation

The RPC and block-transfer split above is about *how* data moves once the scheduler has decided a task needs to fetch it; locality is the scheduler's other lever, deciding *where* to place a task in the first place so that as little of that movement is needed at all. Placing

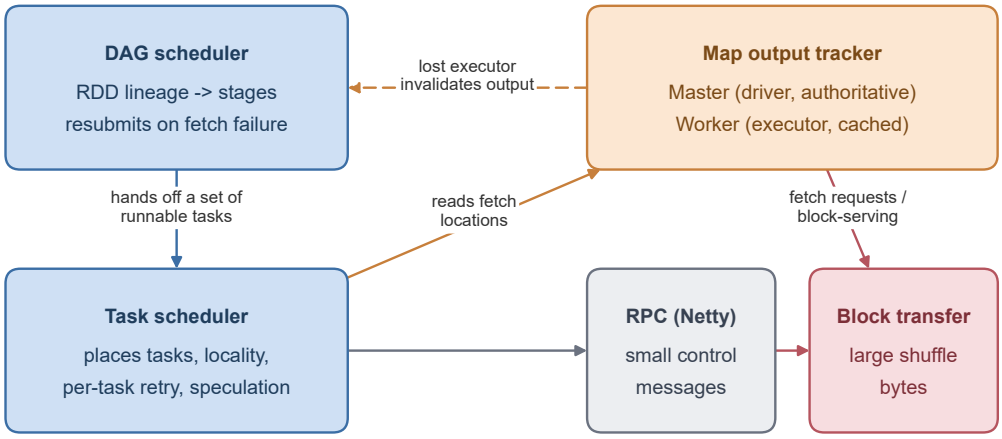


Figure 18.3: The scheduler's five components. The DAG scheduler hands the task scheduler a set of runnable tasks and is only reinvoked when the map output tracker reports a lost executor's output as gone; the task scheduler reads fetch locations from the tracker and issues its own small control messages over RPC, kept separate from the large shuffle bytes on the block-transfer path.

a task on the executor that already holds its input data avoids a network read entirely, and the task scheduler ranks candidate executors by four locality levels, each one hop further from the data than the last: `PROCESS_LOCAL` means the data is already in the same JVM, so no network or even inter-process copy is needed at all; `NODE_LOCAL` means a different executor process on the same physical machine, so any transfer stays on the machine's own loopback network rather than the cluster fabric; `RACK_LOCAL` means a different machine but one sharing the same network rack, typically a cheaper hop than a cross-rack one; and `ANY` means no locality preference at all, the level every task can always fall back to. The scheduler will wait briefly (`spark.locality.wait`, 3s by default) for a better-locality slot to free up before falling back to a worse one. The lab counted the level of every task in its event logs (Table 18.2).

Kind of task (lab, single JVM)	**PROCESS_LOCAL**	**NODE_LOCAL**
Iceberg scan (input bytes, no shuffle read)	7,073	0
shuffle read	30,616	6,610
shuffle read in cache-heavy applications	0	17,322
two-host cluster, fetch-failure run (35 tasks)	18	17

Table 18.2: Locality level of tasks recorded in the lab's event logs. No task was `RACK_LOCAL` or `ANY`: the lab has one machine, so the cross-node levels cannot occur.

The table shows what a one-machine lab can show and no more. Scan tasks were `PROCESS_LOCAL` because the only executor is the one process the data would be read into; the shuffle-read split between the two levels reflects blocks written by the same JVM or by a different task thread. Claim — not tested: on the cluster with table data in remote object storage, scan tasks have no preferred location (object storage has no "node holding this byte range") and so never wait for a better level, while a stage-2 task placed on the same node as a map-side executor reads over a local hop, cheaper than a cross-node one.

Speculation (already introduced as a symptom-level fix in Section 15.4) is that same task scheduler's mechanism underneath: once a task has run past `spark.speculatio`

`n.multiplier` times the median duration for its stage, it launches a second, speculative copy on a different executor and accepts whichever attempt's result lands first. What that earlier discussion left implicit is the layer it operates at: this is a decision the task scheduler makes about one task, with no visibility into or coordination from the DAG scheduler's stage-level view — consistent with the split above, since a straggler is a single task problem the task scheduler already owns, not a stage problem that would need the DAG scheduler involved at all. The lab confirmed the one placement rule the mechanism implies: a speculative copy is never placed on the host that runs the original. With four executors on a single host, `spark.speculation` changed nothing (41.6 s both ways); with executors under two host names it cut a 40-second straggler to 11.2 s (Figure 15.5).

18.6 From symptom to lever

Symptom	Evidence	Lever	Cost
want the cheapest shuffle writer	partitions above 200 use the serialized writer	nothing to do: both are cheap; do not size partitions for this	—
a stage with tens of thousands of tasks is slow to schedule	driver cost per task rises (0.04 to 0.23 ms at 80,000)	fewer, larger partitions (Figure 14.4)	less parallelism per stage
stage appears several times in the UI	attempts > 0 with `FetchFailed` tasks	protect or replicate shuffle output (Chapter 14)	held executors
speculation does nothing	all executors on one host	run executors on several hosts	placement constraints
a tail task keeps the stage open	one task of a stage long after its peers	`spark.speculation` for node stragglers	duplicate compute

Table 18.3: The scheduler levers: symptom, evidence, lever, cost.

18.7 Summary

One shuffle manager, three writers chosen by preconditions rather than configuration you set directly: bypass for few partitions, serialized-sort when the serializer allows it, plain sort otherwise. The map output tracker is the registry that makes fetching possible and that a lost executor invalidates, triggering the DAG scheduler's stage resubmission. The DAG scheduler owns stage-level lineage and boundaries; the task scheduler owns task placement within a stage, locality, and per-task retry, with speculation as its answer to a straggler that locality alone cannot explain. Both RPC coordination and the shuffle bytes themselves travel over the same underlying Netty-based transport, just on logically separate paths.

Exercises

1. Trace one wide query from `collect()` to task completion, naming each scheduler component it passes through.

2. Given a shuffle with 50 partitions and no aggregation, predict which shuffle writer handles it, then confirm from `spark.shuffle.sort.bypassMergeThreshold`.

3. Kill an executor mid-stage and, from the event log, find the map output tracker invalidation and the DAG scheduler's resubmitted stage attempt.

Chapter 19

DataSource V2 and Extensions

Iceberg is not built into Spark. It plugs in through the DataSource V2 API, and the planner negotiates with it: which filters can you apply, which columns do you need, can you answer this aggregate yourself? Understanding that negotiation explains why some predicates prune and others do not, and why some `COUNT(*)` queries take a tenth of a second on 30 million rows. This chapter reads the contract from the outside — through the interfaces Iceberg's classes implement, through what the plan shows for thirteen different `WHERE` clauses, through the statistics the source reports, and through what happens to a write when a task fails — and then shows how a jar adds Catalyst rules and SQL syntax to a session. Every output comes from the lab in Section A.3 (Spark 3.5.2, Iceberg 1.6.1).

19.1 The DataSource V2 read and write API

DataSource V2 (DSv2) is the interface contract between Catalyst and an external table source. Rather than describe it in the abstract, the lab asked the live objects which interfaces they implement (Table 19.1). A source is a stack of four objects — catalog, table, scan builder, scan — and each interface a class implements is a capability the planner may use.

For a read, a `TableProvider` resolves a table name to a `Table`, which hands back a `ScanBuilder` the planner negotiates with (Section 19.2) before it ever produces a phys-

Object	Interface (Spark 3.5)	What it lets the planner do, and where this book shows it
catalog	`TableCatalog`, `SupportsNamespaces`	resolve `lab.db.orders` to a table (Chapter 10)
	`ProcedureCatalog`	`CALL lab.system.rewrite_data_files(...)` (Section 19.4)
	`StagingTableCatalog`	stage a table so a failed `CREATE TABLE AS SELECT` leaves nothing (claim — not tested)
	`FunctionCatalog`	`bucket`, `days` and the other partition transforms as functions
table	`SupportsRead`, `SupportsWrite`	obtain a scan builder or a write builder (this section)
	`SupportsDeleteV2`	a `DELETE` on a whole partition resolved from metadata (Chapter 13)
	`SupportsRowLevelOperations`	`UPDATE`, `DELETE`, `MERGE` as copy-on-write or merge-on-read (Chapter 13)
	`SupportsMetadataColumns`	the hidden `_file` and `_pos` columns
scan builder	`SupportsPushDownV2Filters`	the filter negotiation (Section 19.2)
	`SupportsPushDownRequiredColumns`	column pruning (Section 7.6)
	`SupportsPushDownAggregates`	`COUNT`, `MIN`, `MAX` from metadata (Figure 19.4)
	`SupportsReportStatistics`	the size estimate behind join selection (Section 19.3)
scan	`SupportsReportPartitioning`	storage-partitioned joins (Section 5.8)
	`SupportsRuntimeV2Filtering`	dynamic partition pruning (Section 7.3)

Table 19.1: The DSv2 interfaces implemented by Iceberg 1.6.1's `SparkCatalog`, `SparkTable`, `SparkScanBuilder` and `SparkBatchQueryScan`, read by reflection from a running session. Each row is a capability the planner can use, and each chapter that shows its effect is cited.

ical scan. For a write, the same `Table` hands back a `WriteBuilder`, and the write is a fixed sequence of hops: the `WriteBuilder` produces a `BatchWrite` on the driver, which ships a `DataWriterFactory` to the executors; each task gets a `DataWriter`, writes its partition, and returns a small `WriterCommitMessage` (for Iceberg, the list of files it wrote); only when *every* message has arrived does the driver call `BatchWrite.commit()` once. That single call is the DSv2 hook behind Iceberg's atomic pointer swap (Section 10.3); Figure 19.1 draws the sequence and the failure path.

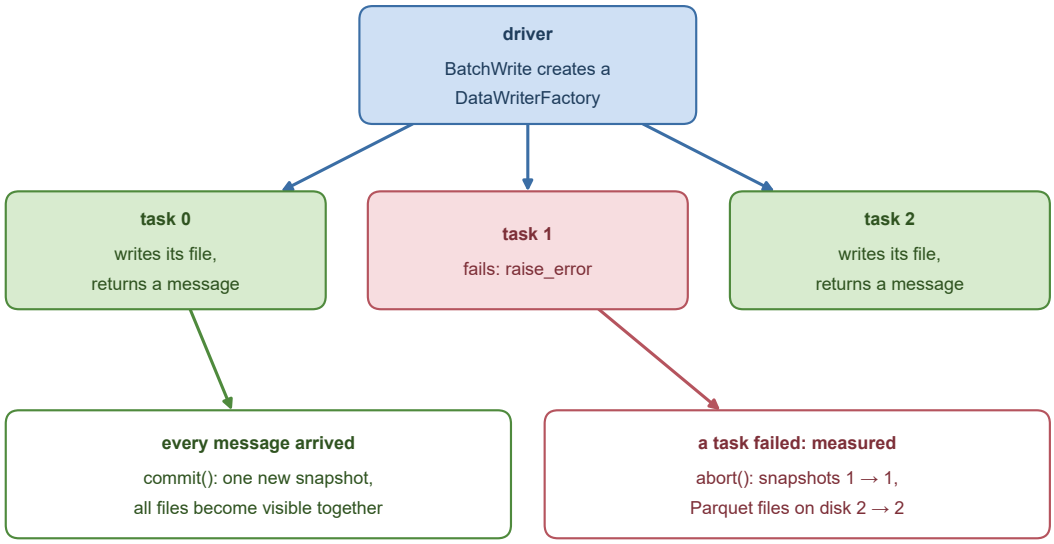


Figure 19.1: The write protocol, with the failure path measured. A task that raises an error stops the job before `commit()`: the table had one snapshot and two data files before the failed `INSERT` and the same afterwards.

The lab checked the failure path directly. An `INSERT` whose last tasks evaluate `raise_error('boom at the end')` aborted the job; the table had **1 snapshot and 2 Parquet files** on disk before and the same after. Tasks that had already finished writing their files did not leave them behind: the `abort` call cleans them up. A reader therefore never sees a partial write, and a failed job costs disk only until the abort runs. (When the JVM dies before it can abort — a killed driver — files can remain; those are the orphans of Section 13.6.)

19.2 Pushdown negotiation

Pushdown under DSv2 is a negotiation, not a guarantee: the planner offers a source a set of filters, columns, and (since Spark 3.2, for a narrow set of cases) aggregates, and the source reports back which it can apply. Iceberg implements the *V2* filter interface, `SupportsPushDownV2Filters`, and its reply has more than two outcomes. The lab ran thirteen `WHERE` shapes against the 30-million-row `orders` table and recorded, for each, how many rows left the scan and whether Spark kept a `Filter` node above it (Figure 19.2).

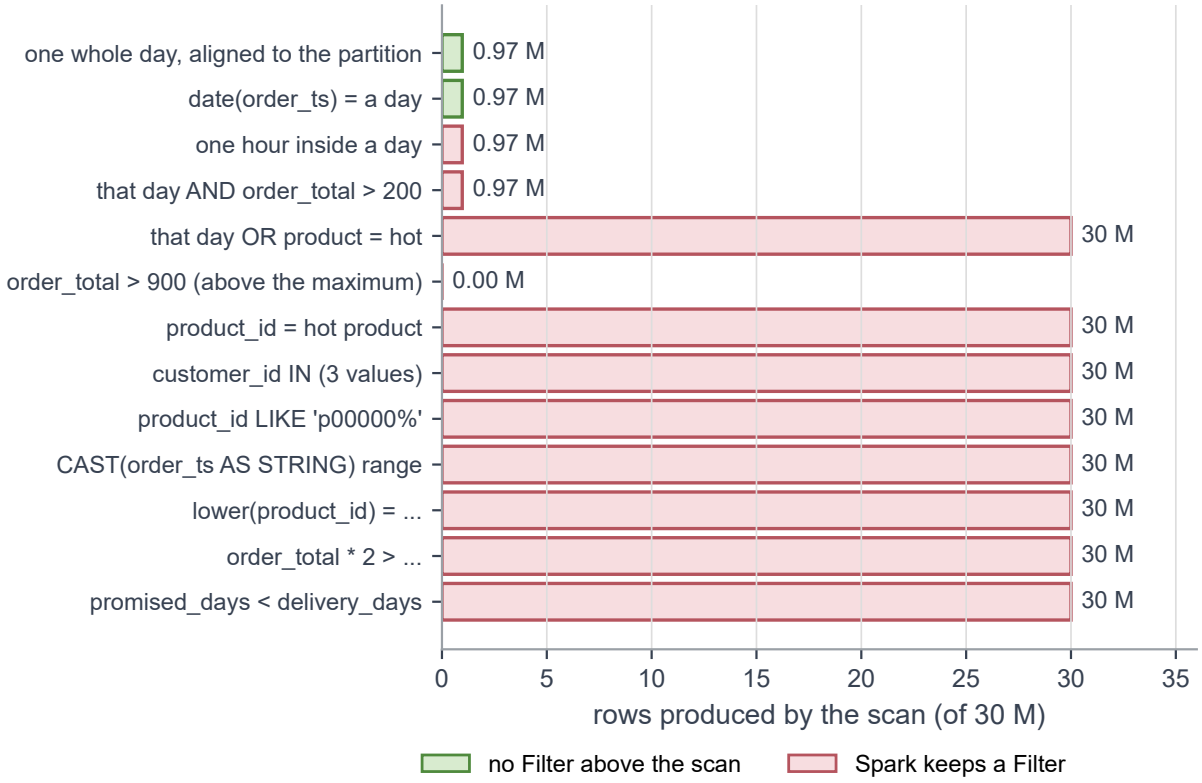


Figure 19.2: Rows produced by the scan for thirteen predicates. Only two — a range that equals exactly one day partition, and the `date()` form that Iceberg rewrites to that range — leave no `Filter` above the scan. Every other predicate keeps one.

Three things are visible. First, there are three outcomes, not two (Figure 19.3). A predicate that coincides with the partition boundary is *fully handled*: the scan yields exactly the day's 967,939 rows and no `Filter` remains. A predicate on a data column is *pushed to prune files and row groups* but Iceberg does not filter rows itself, so Spark keeps its `Filter`; whether the pushdown saved anything depends on the data layout.

And a predicate Iceberg cannot translate — a `CAST` to string, a function, arithmetic — is *not pushed* at all.

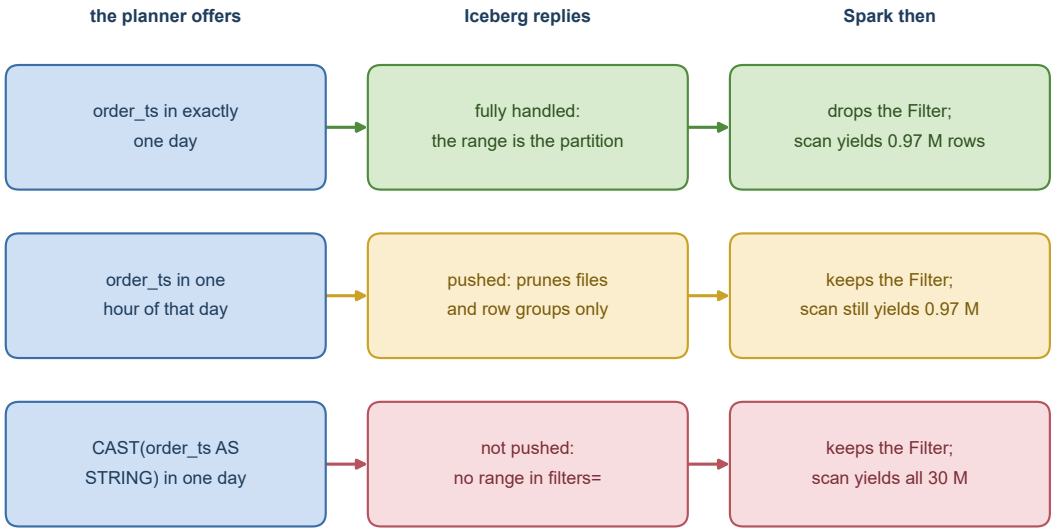


Figure 19.3: The same negotiation, three outcomes. Only a predicate that equals the partition boundary is fully handled; one hour inside the partition is pushed for pruning but re-checked by Spark; a `CAST` is not pushed.

Second, *pushed* does not mean *pruned*. `product_id = 'p0000042'` appears in the scan's `filters=` list, yet the scan produced all 30 million rows: the hot product is 30% of the data and appears in every file, so no file's statistics can exclude it. `order_total > 900` is the opposite case: the column's maximum is 305, so file statistics eliminated every file and the scan produced **zero** rows — while Spark still kept a `Filter` above it. The same pushed filter is worthless or decisive depending on whether the column's value ranges separate the files, which is the argument of Chapter 11.

Third, the way a predicate is written decides its outcome. The listing shows the scan nodes for the three cases of Figure 19.3, copied from the plans (the `filters=` list is where the negotiation's answer is visible):

```
-- one whole day (aligned)
BatchScan lab.db.orders[order_ts, order_total] [filters=order_ts IS NOT NULL,
  order_ts >= 1772668800000000, order_ts < 1772755200000000, groupedBy=]
-- one hour inside that day  (a Filter node remains above the scan)
BatchScan lab.db.orders[order_ts, order_total] [filters=order_ts IS NOT NULL,
  order_ts >= 1772704800000000, order_ts < 1772708400000000, groupedBy=]
-- CAST(order_ts AS STRING) in one day  (no range was pushed)
```

```
BatchScan lab.db.orders[order_ts, order_total] [filters=order_ts IS NOT NULL,
 groupedBy=]
```

Listing 19.1: The scan node shows what Iceberg accepted (`filters=`); lines shortened

`SupportsPushDownRequiredColumns` is the same negotiation for columns: the first line of each scan node above lists only `order_ts` and `order_total`, the two columns the query uses.

Aggregate pushdown. `SupportsPushDownAggregates` lets Iceberg answer some aggregates from manifest metadata without opening a data file. The lab ran six aggregates (Figure 19.4); the plan root tells which path was taken: a `LocalTableScan` means the answer was already computed.

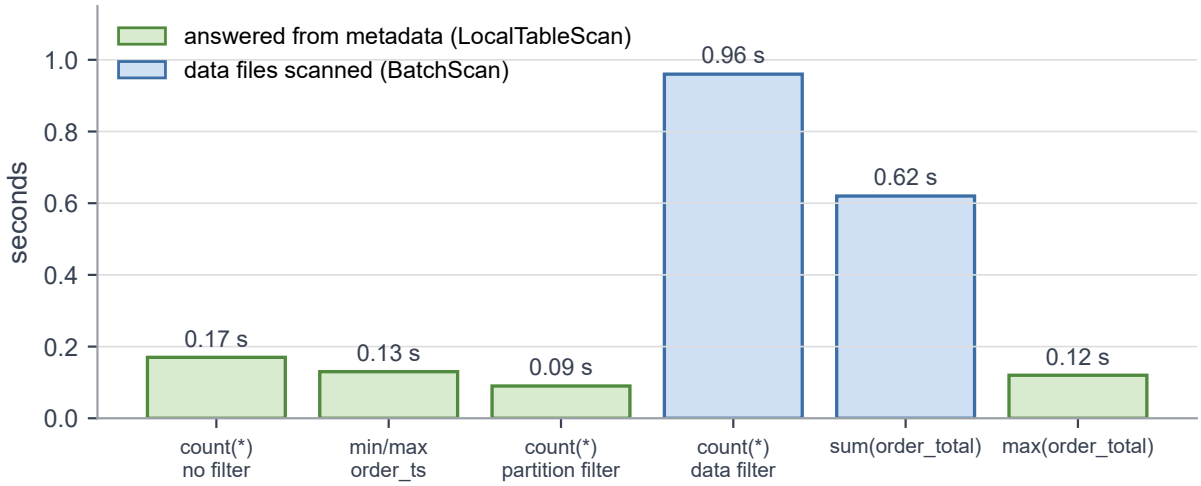


Figure 19.4: `COUNT(*)` on the whole table, on one day, `MIN`/`MAX` of `order_ts` and `MAX(order_total)` were answered from metadata in 0.09–0.17 s. `COUNT(*)` with a data-column filter and `SUM` had to scan the data files (0.6–1.0 s).

The rule follows from what the manifests record. Each data file's entry carries its record count and, per column, a lower and upper bound; a table-wide `COUNT`, `MIN` or `MAX` is a combination of those numbers. A `SUM` needs the values; a `COUNT` filtered on a data column needs to know which rows match. Pushing the aggregate is also the reason why a `COUNT(*)` on a 30-million-row table is not a useful benchmark of scan speed: it never scans.

19.3 How Iceberg reports splits and statistics

A DSv2 source does not hand the planner data directly. It hands back a set of `InputPartitions` (Iceberg's version of the split planning of Section 12.6) and a `PartitionReaderFactory` that reads each one. Separately, `SupportsReportStatistics` tells the planner an estimated size and row count *before* the scan runs. The lab printed what the optimizer saw (Table 19.2).

Scan	sizeInBytes	rowCount	bytes per row
`orders`, all 7 columns	2,520,000,000	30,000,000	84
`orders`, one day (partition-pruned)	81,306,876	967,939	84
`orders`, `product_id = hot`	2,520,000,000	none	—
`dim_product`, all 6 columns	216,000,000	2,000,000	108
`dim_date`, all 4 columns	11,680	365	32

Table 19.2: What `SupportsReportStatistics` reported to the optimizer. The per-row figure is the same for every row count, and the data-column filter changed nothing — the source has no selectivity information.

Three facts follow. The estimate is built from record counts, not from file bytes: a table of exactly the same bytes but fewer records would report less. It does respond to *partition pruning*: the one-day scan reports 81 MB because the manifest list narrowed it before the estimate was taken, which is why a tightly pruned scan of a huge table can legitimately report itself as small. And it ignores a data-column predicate: `product_id = hot` reports the full 2.52 GB with an unknown row count, because Iceberg has no histogram to estimate selectivity from. The per-row figures are not arbitrary: 84, 108 and 32 are the sums of Spark's default widths for the columns' types (strings 20 bytes, doubles and timestamps 8, integers and dates 4), so the estimate is *rows times a width derived from the schema*, not a measurement of Parquet bytes. That is why `spark.sql.autoBroadcastJoinThreshold` compares an uncompressed estimate (Chapter 6) and why `dim_product`, which occupies 174 MB on disk, is reported as 216 MB while

`orders`, 1.08 GB on disk, is reported as 2.52 GB.

The width is that of the *projected* columns, so column pruning shrinks the estimate. The lab asked for the same `dim_product` with different projections (Table 19.3): all six columns 216 MB, `product_id` and `category` 80 MB, `product_id` alone 40 MB, `list_price` alone 16 MB — exactly rows times the sum of the columns' default widths. A join that needs only a narrow slice of a wide dimension therefore looks smaller to the planner than the table does, and can cross under the broadcast threshold without any hint.

Projection of `dim_product` (2 M rows)	**Default widths**	**sizeInBytes**
all 6 columns	20+20+8+20+20+20 = 108	216,000,000
`product_id`, `category`	20+20 = 40	80,000,000
`product_id`	20	40,000,000
`list_price`	8	16,000,000

Table 19.3: The reported size is rows times the default width of the columns the scan projects.

19.4 Custom Catalyst rules and SQL extensions

Nothing about this negotiation requires modifying Spark itself. A `SparkSessionExtensions` entry point lets an external jar register additional Catalyst rules — injected into the analyzer, the optimizer, or physical planning, alongside Spark's own rules from Section 2.3 — as well as entirely new SQL syntax the parser would not otherwise recognize. Iceberg's `IcebergSparkSessionExtensions` is exactly this mechanism, and the lab shows what it adds by taking it away. The same four statements were run in a session with `spark.sql.extensions` empty (Table 19.4).

The split is the point. Reading, writing and metadata tables run on the catalog and the DSv2 interfaces alone. `CALL` and `ADD PARTITION FIELD` are grafted onto Spark's SQL grammar by the extension; none of it is native Spark SQL. The compaction statement this book has used throughout is one such addition:

Statement, no extension registered	Result
`CALL lab.system.rewrite_data_files(...)`	`PARSE_SYNTAX_ERROR: Syntax error at or near 'CALL'`
`ALTER TABLE orders ADD PARTITION FIELD ...`	`PARSE_SYNTAX_ERROR: Syntax error at or near 'FIELD'`
`SELECT count(*) FROM orders`	works
`SELECT count(*) FROM lab.db.orders.snapshots`	works

Table 19.4: With the extension removed, reads and metadata tables still work (they are catalog and DSv2 features); the Iceberg-specific *syntax* does not, because it is parser rules the extension injects.

```
CALL lakehouse.system.rewrite_data_files(
  table => 'analytics.orders',
  strategy => 'binpack'
);
```

Listing 19.2: A procedure call, available only with the Iceberg extension registered

19.5 Plugins

Registering a catalog (`spark.sql.catalog.<name>`, Section 10.5) and a session extension (`spark.sql.extensions`) are both configuration keys naming a class on the classpath. The lab's Iceberg installation is these two lines plus the jar:

```
spark.sql.catalog.lab=org.apache.iceberg.spark.SparkCatalog
spark.sql.extensions=org.apache.iceberg.spark.extensions.
   IcebergSparkSessionExtensions
```

Listing 19.3: The whole mechanism by which Iceberg is "installed" into a Spark session

The first key is what turns `lab.db.orders` into a resolvable name, by naming the class that implements `TableCatalog` (Table 19.1 lists the seven interfaces that class

implements); the second teaches the parser `CALL`. No Spark source change, no rebuild. Any table format or connector built to the same contracts plugs in the same way, which is the practical reason a table format can evolve on its own release cadence independent of Spark's.

> **Note** Claim — from the Spark 4 migration notes, not tested here: Spark 4.0 keeps the DataSource V2 contract used above, but reorganizes parts of the extension API and the default of `spark.sql.ansi.enabled`. Re-run the thirteen-predicate experiment on the Spark 4 Iceberg runtime before assuming the table in Figure 19.2 carries over.

19.6 From symptom to lever

Symptom	Evidence	Lever	Cost
a filter does not reduce the data read	a `Filter` node above the scan; `filters=` lacks the predicate	rewrite so the column appears bare (no `CAST`, function or arithmetic)	none
a pushed filter still scans everything	`filters=` lists it but scan rows equal the table	sort or Z-order so the column's ranges separate files (Chapter 11)	a rewrite
a time-range filter leaves a `Filter` node	range does not align with the partition transform	accept it, or align the range to the transform	none
join strategy surprises	`EXPLAIN COST`: estimate far from disk size	remember: rows × default width; use a hint or statistics (Chapter 5)	hint maintenance
`CALL` is a syntax error	`PARSE_SYNTAX_ERROR` at `CALL`	register `spark.sql.extensions`	none

Table 19.5: The DSv2 levers: symptom, evidence, lever, cost.

19.7 Summary

Iceberg is an ordinary DSv2 implementation from Spark's point of view: a catalog, a table, a scan builder and a scan, each implementing the interfaces that unlock a capability. The filter negotiation has three outcomes — fully handled, pushed for pruning only, not pushed — and only a predicate that equals the partition boundary is fully handled; the rest leave a `Filter` that re-checks every row. Aggregates `COUNT`, `MIN` and `MAX` are answered from manifest metadata in about a tenth of a second; `SUM` and data-filtered counts scan. The statistics the source reports are rows times schema-derived widths, respond to partition pruning, and ignore data-column predicates. A failed write leaves no snapshot. And `CALL` and the DDL extensions exist only because a jar named in `spar k.sql.extensions` added parser rules.

Exercises

1. Write a query whose filter Iceberg fully handles and one whose filter is pushed but still leaves a `Filter` node. From the plan and the scan's row count, explain the difference.

2. Run `SELECT max(order_total)` and `SELECT sum(order_total)` on an Iceberg table. Which plan root appears for each, and what in the manifests explains it?

3. Print the interfaces a catalog, a table and a scan builder implement in your own session (the lab used reflection through `loadTable`) and match each to a behaviour of this chapter.

Appendix A

The Reference Environment and the Running Example

Every version-specific claim in this book is written against one stack, one cluster shape, and one running example. This appendix fixes all three, plus the columnar-format background Part III assumes.

A.1 The stack

Table A.1 is the reference environment. A claim elsewhere in the book that depends on version-specific behavior — an AQE default, a config key introduced or renamed in a particular release — is written against exactly this stack; where behavior differs meaningfully on an older or newer version, the text says so explicitly rather than leaving it implicit.

A.2 The cluster shape

Worked numbers throughout the book — executor counts, task-slot arithmetic, memory sizing — assume one concrete shape unless a section says otherwise: executors with 4 cores and 16 GB of executor memory each (plus the memory overhead Section 14.1 derives separately), running as pods on Kubernetes with no co-located storage, on the assumption that table data sits in remote object storage rather than node-local disk (Sec-

Table A.1: The reference stack

Component	Version / detail
Apache Spark	3.5.2
Table format	Apache Iceberg, format version 2
Cluster manager	Kubernetes (executors as pods)
JVM	OpenJDK 17
File format	Apache Parquet

tion 18.5's locality discussion depends on exactly this separation). A cluster of 10 such executors gives 40 task slots — the number Section 4.4's shuffle-partition-count reasoning starts from.

A.3 The lab: where the captured output comes from

The plans, task counts, and timings printed in the book's listings were captured on a laptop-sized *lab*, not on the reference cluster. The lab is a single Docker container running Spark 3.5.2 on JDK 17 in local mode with eight cores, Apache Iceberg 1.6.1, and a file-based Iceberg catalog; it has no cluster manager, no object storage, and no catalog service. Its source lives in the `lab/` directory of the book's repository, including the data generator and every experiment script, so any number can be reproduced.

- **Data.** The generator builds the `orders` star schema of the next section at 30 million `orders` rows over March 2026, 2 million `dim_customer` rows, and 2 million `dim_product` rows, with one product taking 30% of all orders. `orders` is about 1 GB of Parquet.
- **Parallelism.** `spark.default.parallelism` is pinned to 40, so the quantity AQE's coalescing divides by matches the reference cluster's 40 task slots (Section A.2), even though only eight tasks run at once.
- **Scaling.** Production thresholds assume production data. Where an experiment needs a smaller advisory size or skew floor to trigger, the text says so and gives the

values used.

What transfers is the mechanism: which plan node appears, which task count follows from which formula. What does not is absolute time. Treat every duration in the book as a comparison between two runs on the same lab, never as a prediction for your cluster.

A.4 The running example: `orders`

The specimen dataset is an online marketplace's order history. The fact table, `orders`, is an Iceberg table partitioned by `day(order_ts)` (Section 11.1), holding a few hundred million rows per month, with the columns in Table A.2.

Table A.2: `orders` schema

Column	Type	Meaning
`order_id`	`string`	Unique identifier for one order
`order_ts`	`timestamp`	When the order was placed
`product_id`	`string`	Foreign key to `dim_product`
`customer_id`	`string`	Foreign key to `dim_customer`
`promised_days`	`int`	Delivery days promised at checkout
`delivery_days`	`int`	Actual days to delivery
`order_total`	`double`	Order value, in dollars

Three dimensions complete the star schema: `dim_product` (`product_id`, `category`, `list_price`, and other catalog attributes); `dim_customer` (`customer_id`, signup date, loyalty segment); and `dim_date` (`date`, `year`, `month`, `day_of_week`), used throughout Chapter 7's dynamic-partition-pruning examples to filter `orders` indirectly by calendar attributes that are not columns on the fact table itself. `dim_date` is small enough to broadcast under default settings (Section 5.5). `dim_product` is a catalog of millions of products with titles, brands, and descriptions: about 170 MB on the lab, far over the 10 MB broadcast threshold, so the specimen joins it with a sort-merge join and it becomes broadcastable only after a filter shrinks it (Section 6.4). `dim_customer` is larger

still. Neither needs column statistics to be classified: Iceberg reports a table's size from its snapshot metadata.

> **Note** One product, a heavily-discounted, briefly viral item, accounts for a disproportionate share of a given week's orders. This single hot key is the running skew example Section 1.6 first spots and Section 6.5 later fixes.

A.5 A columnar-format primer

Part III assumes familiarity with two ideas Parquet is built on, summarized here for a reader coming from row-oriented formats.

A **row-oriented** format (CSV, JSON, Avro used row-wise) stores a complete row contiguously — reading one column out of a wide table still means reading every other column's bytes for every row along the way. A **columnar** format stores one column's values contiguously across many rows instead, so reading a subset of columns reads only their bytes (Section 7.6's `ReadSchema` pruning depends on exactly this layout to be worth anything) and values of the same type sit next to each other, which compresses far better than mixed-type row bytes and is what a vectorized reader (Section 12.5) processes efficiently.

Parquet organizes a file as a set of **row groups** (a horizontal slice of rows, typically sized to match the input-partition target from Section 1.4), each holding one **column chunk** per column, itself split into **pages** that are individually encoded and compressed. Every row group's footer carries per-column **statistics** — min, max, null count — which is exactly what Section C.11's row-group-level pushdown filters against, and what Section 12.3's Iceberg-level metrics modes summarize one level up, at the file level, in the manifest.

> **Note** "Compressed" and "encoded" are distinct steps applied in sequence: encoding (dictionary encoding, run-length encoding) exploits repeated or low-cardinality values before general-purpose compression (Snappy, Zstd, Gzip) runs on the result. This is why Section 3.2's decompressed-size warning matters: a compression ratio of 3–5× on typical analytic data means an on-disk size is not a reliable proxy for

> the deserialized, in-memory size Spark actually has to hold.

Appendix B

Configuration Reference

Every configuration key the book relies on to explain a mechanism, grouped by subsystem. This is a ”which knob touches what, and what it costs” reference, not a recommended-values table — read the chapter cited before changing a value on a table or job you cannot afford to be wrong about.

B.1 Memory (Chapter 3)

Key	Default	Controls
`spark.executor.memory`	—	JVM heap size; the base the unified memory pool is computed from (Section 3.3).
`spark.executor.memoryOverhead`	max(10%, 384 MB)	Off-heap JVM needs, container-limit headroom (Section 14.1).
`spark.memory.fraction`	0.6	Share of (heap − reserved) given to the unified execution/storage pool.
`spark.memory.storageFraction`	0.5	Storage’s guaranteed share within the unified pool (Section 8.4).
`spark.memory.offHeap.enabled`	false	Route unified memory off-heap, outside GC’s view (Section 17.2).

Key	Default	Controls
`spark.serializer`	`JavaSerializer`	Set to Spark's Kryo serializer class as the practical choice on this stack (Section 3.5).

B.2 Shuffle (Chapter 4, Chapter 18)

Key	Default	Controls
`spark.sql.shuffle.partitions`	200	Shuffle partition count before AQE coalescing (Section 4.4).
`spark.shuffle.sort.bypassMergeThreshold`	200	Partition-count ceiling for the cheap bypass shuffle writer (Section 18.1).
`spark.shuffle.io.maxRetries`	3	Shuffle-client retries for a transient fetch failure against a still-live executor (Section 15.4).
`spark.shuffle.io.retryWait`	5s	Delay between those retries.
`spark.stage.maxConsecutiveAttempts`	4	Stage resubmission attempts after a lost-executor fetch failure.
`spark.task.maxFailures`	4	Per-task retry ceiling before the job fails.
`spark.shuffle.service.enabled`	false	External shuffle service — decouples shuffle data lifetime from the executor's own (Section 14.4).
`spark.shuffle.push.enabled`	false	Push-based shuffle merging on top of the external service.
`spark.excludeOnFailure.enabled`	false	Stop retries from landing back on a node already causing failures.

B.3 Adaptive Query Execution (Chapter 6)

Key	Default	Controls
`spark.sql.adaptive.enabled`	true (since 3.2)	Master switch for every mechanism in this section.
`spark.sql.adaptive.advisoryPartitionSizeInBytes`	64 MB	Coalescing target (Section 6.3).
`spark.sql.adaptive.coalescePartitions.minPartitionSize`	1 MB	Floor that stops over-merging.
`spark.sql.adaptive.coalescePartitions.parallelismFirst`	true	Lowers the coalescing target to the cluster parallelism when the shuffle is small (Section 6.3.1).
`spark.sql.adaptive.autoBroadcastJoinThreshold`	same as the planning-time key	Runtime broadcast-conversion threshold, measured against real post-shuffle bytes (Section 6.4.2).
`spark.sql.adaptive.skewJoin.skewedPartitionFactor`	5.0	Multiple of the median partition size that counts as skewed (Section 6.5).
`spark.sql.adaptive.skewJoin.skewedPartitionThresholdInBytes`	256 MB	Absolute-size floor, required alongside the factor above.

B.4 Joins (Chapter 5)

Key	Default	Controls
`spark.sql.autoBroadcastJoinThreshold`	10 MB	Planning-time broadcast decision, against an on-disk size estimate (Section 6.4.2).
`spark.sql.join.preferSortMergeJoin`	true	Prefer sort-merge over shuffle-hash when both are viable (Section 5.5).

B.5 Partitioning and pruning (Chapter 7)

Key	Default	Controls
`spark.sql.files.maxPartitionBytes`	128 MB	Input-partition target for Hive-catalogued file scans (Section 1.4).
`spark.sql.optimizer.dynamicPartitionPruning.enabled`	true	DPP for a star-schema join filtered from the dimension side (Section 7.3).
`spark.sql.optimizer.dynamicPartitionPruning.reuseBroadcastOnly`	true	Restricts DPP to dimension sides reused as a broadcast (Section 7.3).
`spark.sql.optimizer.dynamicPartitionPruning.fallbackFilterRatio`	0.5	Estimated filter ratio DPP needs without broadcast reuse (Section 7.3).
`spark.sql.optimizer.nestedSchemaPruning.enabled`	true	Prune struct fields down to the leaf the query reads (Section 7.6).
`spark.sql.adaptive.optimizeSkewsInRebalancePartitions.enabled`	true	Lets the `REBALANCE` hint split skewed partitions (Section 7.5).

B.6 Iceberg catalog and commits (Chapter 10)

Key	Default	Controls
`commit.retry.num-retries`	4	Retry attempts before a commit fails under contention (Section 10.4).
`commit.retry.min-wait-ms`	100	Initial backoff.
`commit.retry.max-wait-ms`	60,000	Backoff ceiling.
`commit.retry.total-timeout-ms`	1,800,000	Overall retry time budget.

B.7 Iceberg table properties (Chapter 11, Chapter 12, Chapter 13)

Key	Default	Controls
`write.target-file-size-bytes`	512 MB	Write-time file-size target (Section 11.4).
`write.distribution-mode`	`hash`	Shuffle-before-write strategy (Section 11.5).

`write.delete.mode`
`write.update.mode`
`write.merge.mode`
	`copy-on-write`	Sets copy-on-write vs. merge-on-read independently per operation type. Iceberg has changed this default across releases, so confirm with `SHOW TBLPROPERTIES` rather than assuming (Section 13.1).

`write.metadata.metrics.default`	`truncate(16)`	Manifest statistics granularity (Section 12.3).

Key	Default	Controls
`write.parquet.bloom-filter-enabled.column.<col>`	unset	Enable a bloom filter for point lookups (Section 12.4).
`read.split.target-size`	128 MB	Iceberg-side input-split target (Section 12.6).
`read.split.open-file-cost`	4 MB	Assumed per-file overhead when packing splits.
`read.parquet.vectorization.batch-size`	5,000	Rows per vectorized-read batch (Section 12.5).

B.8 Streaming (Chapter 16)

Key	Default	Controls
`spark.sql.streaming.stateStore.providerClass`	`HDFSBackedStateStoreProvider`	Switch to `RocksDBStateStoreProvider` to move state off-heap (Section 16.4).

B.9 Dynamic allocation and scheduling (Chapter 14, Chapter 18)

Key	Default	Controls
`spark.dynamicAllocation.enabled`	false	Master switch (Section 14.3).
`spark.dynamicAllocation.schedulerBacklogTimeout`	1s	Delay before requesting more executors under a pending-task backlog.

Key	Default	Controls
`spark.dynamicAllocation.executorIdleTimeout`	60s	Idle time before an executor is released.
`spark.dynamicAllocation.shuffleTracking.enabled`	true (since 3.0)	Keeps an executor alive while its shuffle output is still referenced, without needing the external shuffle service.
`spark.speculation`	false	Launch a duplicate copy of a straggling task (Section 18.5).
`spark.speculation.multiplier`	1.5	Multiple of median task duration that triggers a speculative copy.
`spark.locality.wait`	3s	Time to wait for a better-locality slot before falling back.

Appendix C

Reading Query Plans

A reference for the physical-plan operators the book refers to. Chapter 2 explains the compilation pipeline that produces these plans and how to read one quickly; this appendix is the catalogue you come back to. Everything here is as of the reference environment (Appendix A).

C.1 The six `EXPLAIN` modes

Form	Output
`EXPLAIN`	physical plan only
`EXPLAIN EXTENDED`	parsed, analyzed, optimized logical, and physical plans
`EXPLAIN FORMATTED`	physical plan as a tree + a numbered list expanding every node
`EXPLAIN COST`	optimized logical plan with the planner's row-count / size estimates
`EXPLAIN CODEGEN`	physical plan + the generated Java for each whole-stage stage
`df.explain(mode=)`	same five, as `"simple"`, `"extended"`, `"formatted"`, `"cost"`, `"codegen"`

C.2 Reading a physical plan

Read **bottom-up**: the leaves are scans, the root is the last operator to run. The tree connectors are `+-` for the child in line and `:-` plus a `:` spine for the first child of a binary operator (a join). A `*` prefix, written `*(n)`, marks an operator fused into whole-stage-generated Java stage `n` (Section C.9).

Here is that process actually applied, to the specimen's own `orders` ⋈ `dim_product` plan, captured on the lab with AQE turned off so that every operator carries its codegen marker (with AQE on, the same plan gains the query-stage nodes of Section C.10). Listing 2.2 shows the same plan before it runs.

```
*(7) Sort [on_time_rate ASC NULLS FIRST], true, 0
+- Exchange rangepartitioning(on_time_rate ASC NULLS FIRST, 200)
   +- *(6) HashAggregate(keys=[category], functions=[avg(...)])
      +- Exchange hashpartitioning(category, 200)
         +- *(5) HashAggregate(keys=[category], functions=[partial_avg(...)])
            +- *(5) Project [promised_days, delivery_days, category]
               +- *(5) SortMergeJoin [product_id], [product_id], Inner
                  :- *(2) Sort [product_id ASC NULLS FIRST]
                  :  +- Exchange hashpartitioning(product_id, 200)
                  :     +- *(1) Project [product_id, promised_days,
                  delivery_days]
                  :        +- *(1) Filter isnotnull(product_id)
                  :           +- *(1) ColumnarToRow
                  :              +- BatchScan lab.db.orders[order_ts,
                  product_id, ...]
                  +- *(4) Sort [product_id ASC NULLS FIRST]
                     +- Exchange hashpartitioning(product_id, 200)
                        +- *(3) Filter isnotnull(product_id)
                           +- *(3) ColumnarToRow
                              +- BatchScan lab.db.dim_product[product_id,
                              category]
```

Listing C.1: The specimen's physical plan with AQE off (`EXPLAIN`, trimmed)

Reading it bottom-up, one operator at a time: the first thing that runs is the left-hand `BatchScan` of `orders`, which reads the raw files. The `ColumnarToRow` above it

converts the columnar batches the scan produces into row form, the `Filter` drops rows with a null key, and the `Project` keeps the three columns the join and aggregate need. That is the entire left-hand side up to the first `Exchange`: the filtered rows are shuffled by `hash(product_id)` into 200 partitions, so that every row with a given `product_id` ends up in the same partition as every other row sharing it, and the `Sort` above it orders each partition by `product_id`. The right-hand branch does the same three things — scan, shuffle, sort — to `dim_product`, independently and in parallel with the left. Only once both branches have reached a sorted, co-partitioned state does the `SortMergeJoin` run: it walks both sorted streams together, matching rows with equal `product_id`. A partial aggregation then runs per partition, but it cannot finish the aggregate: the join partitioned by `product_id` and the aggregate groups by `category`, so a second `Exchange` regroups the partial results by `category` before the final `HashAggregate` (marker `*(6)`). A third `Exchange`, `rangepartitioning`, orders the twenty rows for the global `ORDER BY`, and the final `Sort` emits them. Four exchanges, so five stages; the codegen markers `*(1)` through `*(7)` count the fused pieces inside them. Every step in this reading is exactly "what does this node need before it can run, and has that already happened lower in the tree" — which is the whole discipline bottom-up reading is.

C.3 Scan operators

Where data enters the plan.

Operator	Source
`BatchScan`	a DataSource V2 table — Iceberg on the reference stack
`FileScan parquet` / `orc` / `csv`	a file-based table through the Hive catalog (DataSource V1)
`InMemoryTableScan`	a cached DataFrame or `CACHE TABLE`
`DataSourceV2ScanRelation`	the logical-plan form of a V2 scan

```
BatchScan orders
  PartitionFilters: [order_ts >= 2026-03-01, order_ts <
     2026-03-08]
```

```
PushedFilters:    [IsNotNull(product_id)]
DataFilters:      [IsNotNull(product_id)]
ReadSchema:       struct<product_id,promised_days,
   delivery_days,order_ts>
Batched:          true
```

Listing C.2: A scan node's fields

`PartitionFilters` filters resolved against partition values, so whole partitions (directories on object storage) are skipped without being opened. Empty on a partitioned table means the filter did not match the partition transform (Chapter 7).

`PushedFilters` filters handed to the file reader, which skips row groups using column min/max statistics in the footer and any bloom filters, without decompressing them.

`DataFilters` every filter on the scan, including the residue evaluated row-by-row after reading. `DataFilters` minus `PushedFilters` is what could not be pushed.

`ReadSchema` the columns actually read. Wider than the query needs means a `SELECT *` defeated column pruning.

`Batched: true` vectorized columnar reads are enabled (`spark.sql.parquet.enableVectorizedReader`); `false` falls back to row-at-a-time decoding.

C.4 Exchange operators

Every `Exchange` is a shuffle and a stage boundary: serialize, write to local disk, transfer over the network, re-read on the destination. Counting them is the fastest estimate of a query's cost.

Partitioning	In the plan	Created by
Hash	`hashpartitioning(col, n)`	joins, `GROUP BY`, `repartition(n, col)`
Range	`rangepartitioning(col ASC, n)`	global `ORDER BY`, `repartitionByRange`
Round-robin	`roundrobinpartitioning(n)`	`repartition(n)` with no column
Single	`SinglePartition`	global aggregate with no group key, `coalesce(1)`

```
Exchange hashpartitioning(product_id, 200), ENSURE_REQUIREMENTS
   , [plan_id=42]
```

Listing C.3: A shuffle exchange

`ENSURE_REQUIREMENTS` means the planner's `EnsureRequirements` rule inserted this exchange because the operator above it needs its input distributed that way. `ReuseExchange` in a plan means two identical exchanges were deduplicated — one shuffle feeds both consumers.

`BroadcastExchange` is not a shuffle. It collects one small side to the driver and ships it whole to every executor; it appears only below the build side of a `BroadcastHashJoin` or `BroadcastNestedLoopJoin`.

C.5 Sort operators

```
Sort [product_id ASC NULLS FIRST], false, 0
```

The boolean is `true` for a global sort (a single ordered output across all partitions, preceded by a `rangepartitioning` exchange) and `false` for a local per-partition sort. A `SortMergeJoin` always shows two local sorts, one per input, between the exchanges and the join. Sorts need a sort buffer in execution memory and spill when it is exhausted (Chapter 3).

C.6 Join operators

Chapter 5 covers how the planner chooses; this is how to recognize the choice in a plan.

BroadcastHashJoin. `BroadcastHashJoin […], BuildRight` (or `BuildLeft`) with a `BroadcastExchange` below the build side and *no* exchange on the probe (large) side. The cheapest join: no shuffle of the big table.

SortMergeJoin. `SortMergeJoin […], Inner` with two child branches, each a `Sort` over an `Exchange hashpartitioning`. The default for two large inputs, and the most expensive pattern — two full shuffles plus two sorts.

ShuffledHashJoin. `ShuffledHashJoin […], BuildRight` with two `Exchange` nodes but *no* sorts: one side is shuffled and built into a hash table in memory. Cheaper than sort-merge when the build side fits and the sort is the bottleneck; enabled by `spark.sql.join.preferSortMergeJoin=false` or a `SHUFFLE_HASH` hint.

BroadcastNestedLoopJoin. A non-equi join condition (`order_total > credit_limit`). $O(N \times M)$. Fine when one side is tiny; a warning sign on two large tables — usually a missing or non-equi join key.

```
BroadcastNestedLoopJoin BuildRight, Inner, (order_total#22 >
   credit_limit#41)
:- *(1) BatchScan orders
+- BroadcastExchange IdentityBroadcastMode
   +- *(2) Filter (segment#39 = VIP)
      +- *(2) BatchScan dim_customer
```

Listing C.4: BroadcastNestedLoopJoin on a non-equi condition

There is no `Exchange hashpartitioning` anywhere in this plan — the giveaway that the join key is not an equality. `dim_customer` itself is too large to broadcast by default (Section A.4), but here it is filtered down to its small `VIP` segment first, and it is *that* filtered result, small enough to broadcast, that the `BroadcastExchange` ships whole; every task on the `orders` side then compares each of its rows against every row

of that broadcast copy, checking `order_total > credit_limit` directly. This is only affordable because the broadcast side is small; the same plan shape with neither side small enough to broadcast falls to `CartesianProduct` instead.

CartesianProduct. Every row of one side with every row of the other. Unless it is a deliberate cross join, the `ON` clause is missing.

C.7 Aggregation operators

Aggregation is two-phase: a partial aggregate on each partition (map-side combine), a shuffle, then a final aggregate.

```
*(3) HashAggregate(keys=[product_id], functions=[avg(on_time)])
+- Exchange hashpartitioning(product_id, 200),
   ENSURE_REQUIREMENTS
   +- *(2) HashAggregate(keys=[product_id], functions=[
      partial_avg(on_time)])
      +- ...
```

Listing C.5: Two-phase aggregation

Reading bottom-up: the lower `partial_avg(on_time)` runs once per input partition, producing one partial running-sum-and-count per `product_id` it saw locally; the `Exchange` then redistributes those small partial results by `product_id`, so every partial result for the same product lands together; and the top `avg(on_time)` combines the partials that arrive for each key into the query's final per-product average. Nothing about the specimen's real values is shuffled directly — only the already-shrunk partial aggregates are.

Operator	Chosen when	Memory
`HashAggregate`	all aggregate buffers are fixed-width (numeric, date, timestamp)	off-heap hash map; falls back to sort-based under memory pressure
`ObjectHashAggregate`	a buffer holds a variable-width or complex type (`collect_list`, `collect_set`, some UDAFs)	on-heap objects; falls back to sort-based past `spark.sql.objectHashAggregate.sortBased.fallbackThreshold` (128) distinct keys
`SortAggregate`	hash aggregation is not feasible	needs input pre-sorted by the group keys; handles any size, slower

C.8 Filter and Project

`Filter` applies a predicate; `Project` selects columns and evaluates expressions. Both are usually fused into a neighbouring scan or join inside a `*(n)` block. A standalone `Filter` directly above a scan is worth a look: if its predicate is not also in the scan's `PushedFilters`, the pushdown failed and the predicate is being evaluated row-by-row after reading (Section C.11).

C.9 Whole-stage codegen markers

```
*(2) HashAggregate(keys=[city], functions=[sum(revenue)])
+- Exchange hashpartitioning(city, 200)
   +- *(1) HashAggregate(keys=[city], functions=[partial_sum(
      revenue)])
      +- *(1) Filter isnotnull(city)
         +- *(1) ColumnarToRow
            +- BatchScan sales [city, revenue]
```

`*(1)` is one generated Java method: scan, columnar-to-row, filter, and partial aggregate run without leaving it. The `Exchange` breaks the chain — you cannot fuse across a network boundary — so the final aggregate is `*(2)`, a second generated method on the

post-shuffle data. The number is a sequential id, not an order or a priority. Operators with no star ran through the interpreter; Section 2.6 lists why a stage loses codegen.

C.10 AQE nodes in a final plan

When Adaptive Query Execution is on (Chapter 6), the plan the SQL tab shows after the query finishes is rooted at `AdaptiveSparkPlan isFinalPlan=true`. `isFinalPlan=false` is the compile-time plan from `explain()` before an action — it may change substantially at runtime.

`AQEShuffleRead coalesced` AQE merged small post-shuffle partitions after seeing their real sizes.

`AQEShuffleRead skewed` AQE split one or more oversized partitions of a join input. When coalescing also applied, the argument reads `coalesced and skewed`.

`AQEShuffleRead local` a reduce task reads shuffle files from its own executor; used for the large side after a broadcast promotion.

`BroadcastQueryStage` a `SortMergeJoin` was converted to a broadcast at runtime because the shuffled side turned out small.

`SortMergeJoin(skew=true)` AQE detected and handled skew in this join.

Chapter 6 shows each of these in a final plan, with the plan annotated (Section 6.6). A promoted join no longer shows `SortMergeJoin`: AQE rewrites it to `BroadcastHashJoin`. The large side's shuffle write has already happened by then, so that side is read through `AQEShuffleRead local` rather than skipping its shuffle. A join cannot show both a skew split and a broadcast promotion on the same side, because a promoted side has no shuffle partitions left to measure.

C.11 Predicate-pushdown killers

Predicates that cannot be pushed to Parquet, ORC, or Iceberg, so they run row-by-row after the read (Chapter 7):

- a UDF: `WHERE my_udf(col) = v`
- a function or complex expression on the column: `WHERE length(city) > 5`, `WHERE a + b > 100`
- a `CAST` on the column side: `WHERE CAST(s AS INT) > 100`
- a non-deterministic function: `WHERE rand() > 0.5`
- a leading-wildcard `LIKE`: `WHERE name LIKE '%smith'` (a trailing wildcard, `'smith%'`, does push)
- an `OR` with any non-pushable branch — the whole `OR` fails

Verification. Run `EXPLAIN FORMATTED`, find the scan, and check that each partition-column filter is in `PartitionFilters` and each data-column filter is in `PushedFilters`. If one is missing, it hit a killer above.

Take `WHERE CAST(product_id AS INT) > 1000` against the specimen's `orders`, where `product_id` is a `string` (Section A.4). The scan node's detail block reads:

```
(1) BatchScan orders
    PushedFilters:    []
    DataFilters:      [(cast(product_id as int) > 1000)]
    ReadSchema:       struct<product_id,order_ts,...>
```

Listing C.6: A CAST silently killing pushdown

`PushedFilters` is empty — nothing made it to the storage layer at all — while the exact same condition sits in `DataFilters`, meaning every row of every file is read and decoded before this predicate is checked row-by-row in Spark itself. Compare that to the clean version of the same intent, `WHERE product_id > '1000'` (a plain string comparison, no `CAST`): the scan's `PushedFilters` would show `[IsNotNull(product_id), GreaterThan(product_id,1000)]` instead, and `DataFilters` would be empty — the same logical filter, but now resolved by Iceberg's own statistics before a row is ever decoded. The `CAST` is what moved it from one list to the other.

Appendix D

Glossary

Terms as this book uses them, in the order they are first introduced.

Driver The process holding the `SparkSession`, running the application's own code, and coordinating everything else (Chapter 1).

Executor A JVM process that runs tasks and holds data in memory or on disk for a running application (Chapter 1).

Job The unit of work triggered by one action; made of one or more stages (Chapter 1).

Stage A set of tasks that can run without a shuffle between them, bounded by wide-dependency shuffle boundaries (Chapter 1).

Task The smallest unit of scheduled work: one stage's operators applied to one partition (Chapter 1).

Narrow dependency A child partition depends on a known, fixed set of parent partitions — no shuffle needed (Chapter 1).

Wide dependency A child partition may depend on data from any parent partition — requires a shuffle (Chapter 1).

Catalyst Spark SQL's query optimizer: parse, analyze, optimize, plan, generate code (Chapter 2).

Whole-stage code generation Fusing a chain of operators into one generated Java method via produce/consume (Section 17.4).

Unified memory The single JVM memory pool execution and storage share and borrow from, with a boundary either side may cross into the other's currently-idle space, but which only execution can enforce by evicting the borrower back out (Section 3.3).

Spill Writing intermediate data (a sort buffer, a hash table) to disk because it does not fit in its memory budget (Chapter 3).

Shuffle Redistributing data across partitions between two stages, via a map-side write and a reduce-side fetch (Chapter 4).

Broadcast hash join A join where one side is small enough to be sent whole to every executor, avoiding a shuffle of the large side (Chapter 5).

Sort-merge join A join where both sides are shuffled and sorted by join key, then merged — the robust default for two large sides (Chapter 5).

Adaptive Query Execution (AQE) Re-planning after each shuffle using measured, rather than estimated, partition sizes (Chapter 6).

Skew One partition or key holding disproportionately more data than others, creating a straggler task (Section 6.5).

Partition pruning Skipping whole partitions (or files) based on a filter resolved before reading data (Chapter 7).

Dynamic partition pruning (DPP) Partition pruning using a value discovered at runtime from the other side of a join (Section 7.3).

Bucketing Dividing data within a partition into a fixed number of files by hashing a column, to avoid a shuffle on a repeated join (Section 7.4).

Storage-partitioned join A join with no shuffle because both sides are already co-located by the same bucket/partition transform (a partition transform — see Hidden partitioning, below) (Section 5.8).

Manifest An immutable file listing a set of data or delete files with their partition values and column statistics (Chapter 9).

Manifest list A per-snapshot file listing the manifests that make up that snapshot, with per-manifest summary statistics (Chapter 9).

Snapshot One committed version of a table's state, with its own sequence number and manifest list (Section 9.3).

Sequence number A monotonically increasing number recording a snapshot's relative age, used to decide which delete files apply to which data files (Section 9.3).

Position delete A delete recorded as an exact (file, row-position) pair — cheap to apply, requires the writer to know the row's position (Section 9.4).

Equality delete A delete recorded as a column-value match — cheap to write, progressively more expensive to apply as they accumulate, because every read must check each one against every candidate row (Section 9.4).

Copy-on-write (CoW) A row-level update strategy that rewrites whole affected data files (Section 13.1).

Merge-on-read (MoR) A row-level update strategy that records a delete file instead of rewriting data (Section 13.1).

Hidden partitioning Iceberg partitioning by a transform of a source column, resolved from a plain filter with no partition column in the query text (Section 11.1).

Catalog The component mapping a table name to its current metadata file location, atomically swappable on commit (Section 10.1).

Optimistic concurrency A commit protocol where writers do the expensive work assuming they will win, retrying only the cheap final compare-and-swap on conflict (Section 10.3).

UnsafeRow Tungsten's packed, off-heap-capable binary row format (Section 17.1).

DataSource V2 (DSv2) The Spark API contract an external table source (Iceberg included) implements to plug into the planner (Chapter 19).

Abbreviations

Abbreviation	Full form
AQE	Adaptive Query Execution
CBO	Cost-Based Optimization
CoW / MoR	Copy-on-Write / Merge-on-Read
DAG	Directed Acyclic Graph
DPP	Dynamic Partition Pruning
DSv2	DataSource V2 API
GC	Garbage Collection
JIT	Just-In-Time (compilation)
JVM	Java Virtual Machine
OLAP	Online Analytical Processing
OLTP	Online Transaction Processing
RDD	Resilient Distributed Dataset
RPC	Remote Procedure Call
SMJ	Sort-Merge Join
SQL	Structured Query Language
UDF	User-Defined Function

Technologies

Technology	What it is
Apache Spark	a distributed compute engine
Catalyst	Spark SQL’s extensible query optimizer
Tungsten	Spark’s binary in-memory execution back end
Apache Iceberg	an open table format for analytic datasets on object storage
Apache Parquet	a columnar file format
async-profiler	a low-overhead JVM sampling profiler

Appendix E

Further Reading

Full citations are in the bibliography. This appendix groups them, plus a few further pointers, by the part of the book they extend.

Foundations (Part I) and the engine's own papers

The original research papers behind Spark's execution model are short, readable, and still the clearest statement of *why* the engine is shaped the way Chapter 1 and Chapter 2 describe it: *Resilient Distributed Datasets* [14] (the RDD abstraction and lineage-based fault tolerance underlying every job/stage/task concept in this book), *Spark SQL: Relational Data Processing in Spark* [6] (Catalyst itself), and the retrospective *Apache Spark: A Unified Engine for Big Data Processing* [13]. *Spark: The Definitive Guide* [8] and *Learning Spark, 2nd ed.* [9] cover the same ground at tutorial pace, useful where this book moves quickly from mechanism to mechanism.

Making queries fast (Part II)

High Performance Spark, 2nd ed. [10] is the closest comp to this book's own emphasis — shuffle, joins, and partitioning as performance levers rather than API surface — and is the natural next stop after Chapter 4–Chapter 8. The official *Tuning Spark* guide [4] and *Spark SQL, DataFrames and Datasets Guide* [3], both pinned to this book's reference version (Appendix A), are worth checking directly whenever a config default is suspected

to have changed since.

The Iceberg lakehouse (Part III)

The *Lakehouse* paper [5] is the argument for why a table format over object storage exists at all, ahead of Iceberg specifically. For Iceberg itself: the official *Apache Iceberg Table Spec* [2] is the primary source for everything in Chapter 9 on the metadata tree and delete files, and the *Apache Iceberg Documentation* [1] covers catalogs, maintenance procedures, and Spark-side configuration in more day-to-day detail than this book's mechanism-first treatment. *Apache Iceberg: The Definitive Guide* [12] is the book-length comp, and the Cazpian engineering blog (`https://cazpian.ai/blog`) was a useful secondary source while drafting this part — cross-checked against the official spec and docs above wherever the two disagreed, most notably on maintenance-operation ordering (Section 13.6).

Operating Spark (Part IV)

The *Structured Streaming* paper [7] is the primary source behind Chapter 16's micro-batch and exactly-once model. *Designing Data-Intensive Applications* [11] is not Spark-specific but is the best available treatment of the distributed-systems ideas this book leans on throughout — partitioning, replication, consistency, fault tolerance — for a reader who wants the general theory behind a specific mechanism.

Internals (Part V)

Most of Chapter 17–Chapter 19 has no single canonical paper; the most reliable way to go deeper is the Spark source itself, read alongside *Spark: The Definitive Guide* [8]'s higher-level description of the same components and the official *Spark SQL, DataFrames and Datasets Guide* [3] for whichever configuration surface a given internal exposes:

Whole-stage codegen `sql.execution` package (Chapter 17).

The shuffle framework `shuffle` package (Chapter 18).

DataSource V2 `sql.connector` package (Chapter 19).

References

[1] Apache Iceberg. *Apache Iceberg Documentation*. `https://iceberg.apache.org/docs/latest/`. 2024.

[2] Apache Iceberg. *Apache Iceberg Table Spec*. `https://iceberg.apache.org/spec/`. Format version 2. 2024.

[3] Apache Spark. *Spark SQL, DataFrames and Datasets Guide (Spark 3.5.2)*. `https://spark.apache.org/docs/3.5.2/sql-programming-guide.html`. 2024.

[4] Apache Spark. *Tuning Spark (Spark 3.5.2)*. `https://spark.apache.org/docs/3.5.2/tuning.html`. 2024.

[5] Michael Armbrust et al. "Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics". In: *Proceedings of the 11th Conference on Innovative Data Systems Research (CIDR)*. 2021.

[6] Michael Armbrust et al. "Spark SQL: Relational Data Processing in Spark". In: *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*. ACM, 2015, pp. 1383–1394.

[7] Michael Armbrust et al. "Structured Streaming: A Declarative API for Real-Time Applications in Apache Spark". In: *Proceedings of the 2018 ACM SIGMOD International Conference on Management of Data*. ACM, 2018, pp. 601–613.

[8] Bill Chambers and Matei Zaharia. *Spark: The Definitive Guide*. O'Reilly Media, 2018.

[9] Jules S. Damji et al. *Learning Spark: Lightning-Fast Data Analytics*. 2nd ed. O'Reilly Media, 2020.

[10] Holden Karau, Adi Polak, and Rachel Warren. *High Performance Spark: Best Practices for Scaling and Optimizing Apache Spark*. 2nd ed. O'Reilly Media, 2026.

[11] Martin Kleppmann. *Designing Data-Intensive Applications*. O'Reilly Media, 2017.

[12] Tomer Shiran, Jason Hughes, and Alex Merced. *Apache Iceberg: The Definitive Guide*. O'Reilly Media, 2024.

[13] Matei Zaharia et al. "Apache Spark: A Unified Engine for Big Data Processing". In: *Communications of the ACM* 59.11 (2016), pp. 56–65.

[14] Matei Zaharia et al. "Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing". In: *Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association, 2012, pp. 15–28.